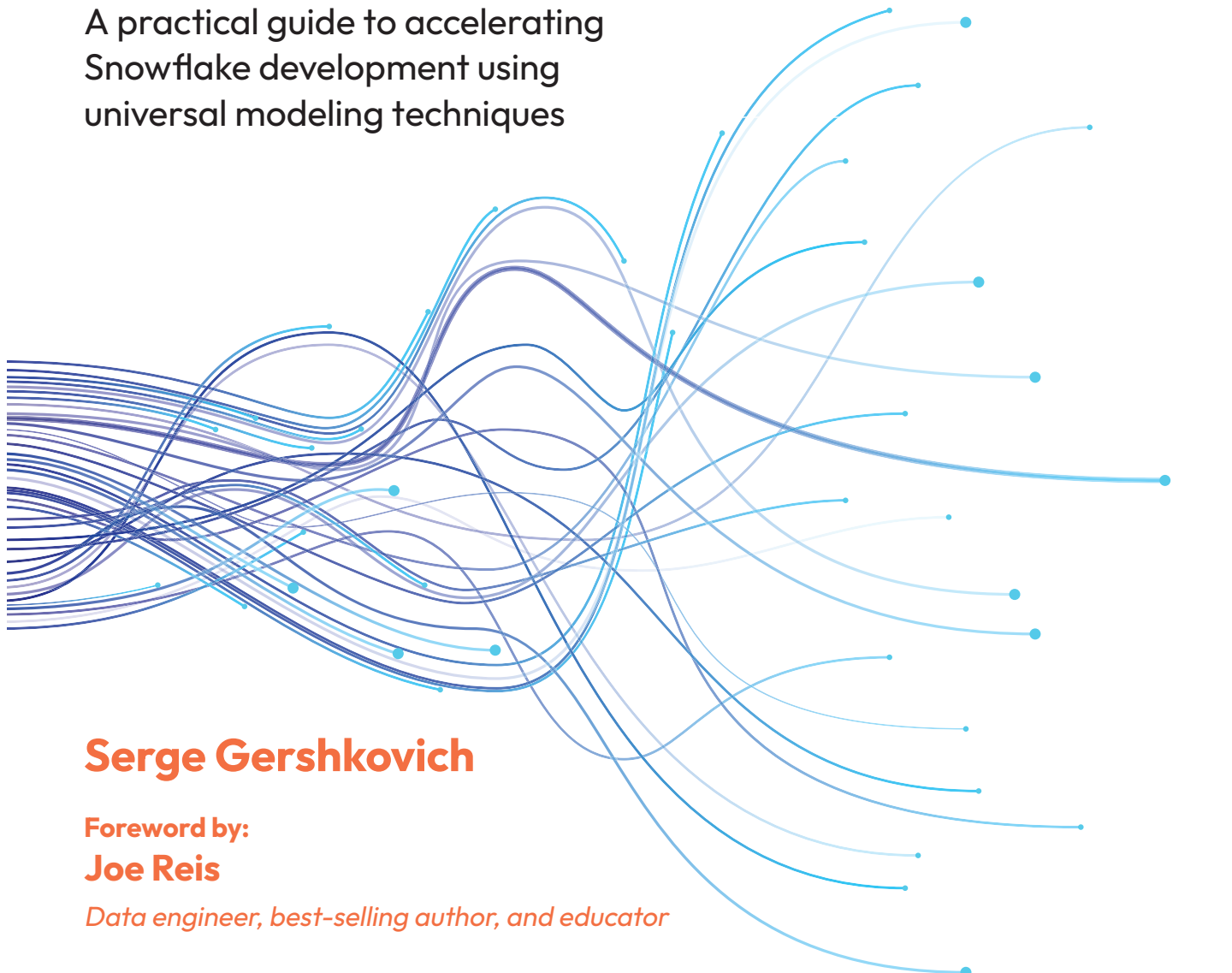


SECOND EDITION

<packt>

Data Modeling with Snowflake

A practical guide to accelerating
Snowflake development using
universal modeling techniques



Serge Gershkovich

Foreword by:
Joe Reis

Data engineer, best-selling author, and educator

Data Modeling with Snowflake

Second Edition

A practical guide to accelerating Snowflake development
using universal modeling techniques

Serge Gershkovich



Data Modeling with Snowflake

Second Edition

Copyright © 2025 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Sunith Shetty

Relationship Lead: Apeksha Shetty

Project Manager: Aniket Shetty

Content Engineer: Saba Umme Salma

Technical Editor: Aniket Shetty

Copy Editor: Safis Editing

Indexer: Manju Arasan

Proofreader: Safis Editing

Production Designer: Ganesh Bhadwalkar

First published: May 2023

Second edition: September 2025

Production reference: 1190825

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83702-803-0

www.packtpub.com

I want to thank Anna, Ed, and Ajay for recognizing the potential that even I didn't know I had. This book happened thanks to your guidance and encouragement. To my loving wife, Elena, thank you for your unwavering support throughout this process.

Foreword

A few years ago, Serge Gershkovich and I were in Verbier, Switzerland, surrounded by the stunning Swiss Alps. Sadly, I barely saw him, as he was glued to his laptop, racing to finish the first edition of this book. Now, with the second edition in your hands, it's clear that those long days paid off. Time flies!

And in that time, the data landscape has shifted dramatically. Data modeling today sits at a crossroads. All too often, the focus is on speed, scale, and quick wins. Proper data modeling is often an afterthought, if it's even considered at all. Tactical execution happens at the expense of strategic design. AI dominates every conversation. Some claim that modern cloud data platforms, with their seemingly limitless resources and AI capabilities, have made data modeling obsolete.

Nothing is further from the truth. Data modeling matters more than ever, and Serge's book stands as a firm rebuttal to that view. At its heart and soul, data modeling is *thinking*. In an era where AI thrives on high-quality data, the importance of modeling has only increased.

Serge begins by demystifying data modeling from first principles through to advanced, practical application in the Snowflake Data Cloud. He reminds us that modeling is not just a set of tactics, but also the art of selectively simplifying and mapping reality to data. From there, he guides readers through the four essential stages: conceptual, logical, physical, and transformational.

What sets this book apart is its deep integration with Snowflake's architecture. Serge shows you how to design data models that work, not just in theory, but in the real world. Starting from first principles, he moves through conceptual, logical, physical, and transformational stages, all deeply grounded in Snowflake's architecture. You'll see how to harness features like streams, tasks, and zero-copy cloning to meet almost any use case.

Serge's structured, business-first approach breaks the design process into manageable, collaborative stages, anchored in the idea that a data model must, first and foremost, be a business model. He emphasizes that models are not just technical artifacts, but consensus-driven blueprints that align technology with organizational reality.

This is more than a technical manual. It's a strategic guide for anyone in the data profession, from aspiring engineers to seasoned architects. Serge arms you with the verbal, visual, and technical skills to articulate, design, and implement robust, scalable, and cost-effective systems.

I'm glad Serge continues to share his expertise so generously. This edition is another step in his mission to elevate the craft of data modeling. You now have the blueprint. Use it well. And maybe next time, Serge and I will get a full day on the slopes.

Joe Reis

Data engineer, best-selling author, and educator.

Contributors

About the author

Serge Gershkovich is a seasoned data architect with decades of experience designing and maintaining enterprise-scale data warehouse platforms and reporting solutions. He is a leading subject matter expert, speaker, content creator, and Snowflake Data Superhero. Serge earned a bachelor of science degree in information systems from the **State University of New York (SUNY)** Stony Brook. Throughout his career, Serge has worked in model-driven development from SAP BW/HANA to dashboard design to cost-effective cloud analytics with Snowflake. He currently serves as Head of Product at SqlDBM, an online database modeling tool.

About the reviewers

Jason Bemont, a father of five, has spent 28 years in IT, specializing in data modeling and governance. He has led enterprise data modeling projects for Fortune 500 companies across financial, healthcare, and insurance sectors. Outside of work, he and his wife share a passion for breeding ball pythons. Honored to review *Unlocking Value for Enterprise Organizations*, he holds deep respect for Serge and even purchased *Data Modeling with Snowflake*, 1st Edition upon its release—long before they met—recognizing it as a benchmark for Snowflake data modeling.

Jason is the Data Modeling Lead at Beazley Insurance Company, where he oversees the enterprise data modeling team and governs federated data modeling efforts. Beazley is undertaking a major data modernization initiative to enhance seamless data sharing across its domains, driving rapid business growth and decision making. Many of these projects leverage Snowflake's capabilities.

Keith Belanger is a seasoned data strategist and architect with nearly three decades of experience designing and implementing modern data analytics platforms. He has held leadership roles in corporate America, including at a Fortune 100 company, as well as in consulting and, more recently, at product companies. Currently Field CTO at DataOps.live, he focuses on Snowflake-native applications that accelerate Data Product development. A recognized Snowflake Data Superhero, Keith is known for his thought leadership in data architecture, data modeling, and building data solutions aligned to business objectives. He is a frequent speaker at industry events.

Tim Spreitzer is a seasoned expert in cloud data management, specializing in Snowflake Cloud architectures and Azure technologies within the healthcare domain. They have a proven track record in designing secure RBAC frameworks and optimizing ELT processes, driving significant advancements in data operation performance and security. A dedicated mentor, they are committed to fostering team capabilities and establishing new industry benchmarks for efficient and secure healthcare data handling.

Table of Contents

Preface	xxiii
---------	-------

Part 1: Core Concepts in Data Modeling and Snowflake Architecture	1
--	----------

Chapter 1: Unlocking the Power of Modeling	3
---	----------

Technical requirements	4
------------------------------	---

Modeling with purpose	5
-----------------------------	---

Leveraging the modeling toolkit	6
---------------------------------------	---

The benefits of database modeling	8
---	---

Operational and analytical modeling scenarios	11
---	----

Is Snowflake limited to OLAP?	12
-------------------------------------	----

A look at relational and transformational modeling • 13

What modeling looks like in operational systems • 13

What modeling looks like in analytical systems • 14

Summary	17
---------------	----

Further reading	17
-----------------------	----

References	18
------------------	----

Chapter 2: An Introduction to the Four Modeling Types	19
--	-----------

Design and process	20
--------------------------	----

Ubiquitous modeling	21
---------------------------	----

Conceptual	24
What it is • 24	
What it looks like • 26	
Logical	28
What it is • 29	
What it looks like • 30	
Physical modeling	32
What it is • 32	
<i>What about views?</i> • 33	
What it looks like • 34	
Transformational	35
What it is • 35	
What it looks like • 36	
Summary	37
Further reading	38
 Chapter 3: Mastering Snowflake's Architecture	 39
Traditional architectures	40
Shared-disk architecture • 41	
Shared-nothing architecture • 42	
Snowflake's solution • 43	
Snowflake's three-tier architecture	43
The storage layer • 45	
The compute layer • 45	
The services layer • 46	
Snowflake's features	46
Zero-copy cloning • 46	
Time Travel • 47	
Hybrid tables • 47	
<i>Beyond structured data</i> • 47	

Costs to consider	48
Storage costs • 48	
Virtual warehouse compute cost • 49	
Serverless compute cost • 49	
Services layer compute cost • 49	
Data transfer costs • 50	
Saving cash by using cache • 50	
The services layer • 50	
<i>The metadata cache • 50</i>	
<i>The query results cache • 51</i>	
<i>The warehouse cache • 52</i>	
The storage layer • 52	
Summary	52
Further reading	53
 Chapter 4: Mastering Basic Snowflake Objects	 55
 Stages	 56
File formats	57
Tables	58
Physical tables • 59	
Permanent tables • 60	
Transient tables • 61	
Temporary tables • 61	
<i>Table types summary • 62</i>	
<i>Stage metadata tables • 62</i>	
Snowflake views	64
Caching • 65	
Security • 65	
Materialized views • 66	
Streams • 67	
Loading from streams • 67	

Change tracking • 68	
Tasks • 70	
<i>Combining tasks and streams • 72</i>	
Summary	72
References	73
 Chapter 5: From Logical Concepts to Snowflake Objects	 75
 Entities as tables	 76
How Snowflake stores data • 76	
Clustering • 80	
<i>Automatic clustering • 83</i>	
Attributes as columns	84
Snowflake data types • 84	
Storing semi-structured data • 85	
Constraints and enforcement	86
Identifiers as primary keys	87
Benefits of a PK • 89	
<i>Determining granularity • 89</i>	
<i>Ensuring correct join results • 89</i>	
<i>Avoiding duplicate values • 90</i>	
<i>Specifying a PK • 90</i>	
Keys taxonomy • 91	
<i>Business key • 91</i>	
<i>Surrogate key • 92</i>	
Sequences • 92	
Alternate keys as unique constraints	94
Relationships as foreign keys	95
Benefits of an FK • 98	
<i>Visualizing the data model • 98</i>	

<i>Informing joins</i> • 99	
<i>Automating functionality in BI tools</i> • 100	
Mandatory columns as NOT NULL constraints	101
Summary	102
 Chapter 6: Mastering Advanced Snowflake Objects	 105
Iceberg tables	106
Open table formats explained • 106	
ACID properties explained • 107	
How Iceberg tables work • 107	
Managed or unmanaged • 108	
Use cases • 109	
Dynamic tables	110
Declarative and imperative statements • 111	
Understanding target lag • 113	
Understanding refresh types • 114	
Monitoring, error handling, and failed refreshes • 115	
Tips, tricks, and cost considerations • 117	
<i>Orchestrating dynamic tables with precision</i> • 118	
Comparison and use cases • 120	
Hybrid tables	121
Referential integrity • 122	
Functionality and costs • 123	
<i>Working with indexes</i> • 123	
<i>Costs</i> • 125	
Working with hybrid tables and understanding limitations • 126	
Summary and comparison to standard tables • 127	
Event tables	128
Costs and benefits • 129	
Summary	130

Chapter 7: Seeing Snowflake’s Architecture Through Modeling Notation 133

A history of relational modeling	134
RM versus entity-relationship diagram	135
Visual modeling conventions	138
Depicting entities • 139	
Depicting relationships • 141	
<i>Crow’s foot</i> • 141	
<i>IDEFIX</i> • 142	
Adding conceptual context to Snowflake architecture • 143	
<i>Many-to-many</i> • 144	
<i>Representing subtypes and supertypes</i> • 145	
The benefit of synchronized modeling	147
Summary	148

Part 2: Applied Modeling from Idea to Deployment 149

Chapter 8: Putting Conceptual Modeling into Practice 151

Dimensional modeling	152
Understanding dimensional modeling • 152	
Setting the record straight on dimensional modeling • 152	
Starting a conceptual model in four easy steps • 153	
<i>Define the business process</i> • 155	
<i>Determine the grain</i> • 156	
<i>Determine the dimensions</i> • 157	
<i>Identify the facts</i> • 157	
From bus matrix to a conceptual model • 158	
Modeling in reverse	161
Identifying the facts and dimensions • 161	
Establishing the relationships • 162	
Proposing and validating the business processes • 164	

Summary	165
Further reading	165
Chapter 9: Putting Logical Modeling into Practice	167
Expanding from conceptual to logical modeling	168
Adding attributes	169
Cementing the relationships	170
Many-to-many relationships • 173	
Weak entities • 175	
Inheritance • 178	
Summary	181
Chapter 10: Database Normalization	183
An overview of database normalization	184
Data anomalies	185
Update anomaly • 186	
Insertion anomaly • 186	
Deletion anomaly • 187	
Domain anomaly • 188	
Database normalization through examples	188
1NF • 189	
2NF • 190	
3NF • 191	
BCNF • 192	
4NF • 193	
5NF • 195	
DKNF • 197	
6NF • 198	
Data models on a spectrum of normalization	200
Summary	201

Chapter 11: Database Naming and Structure	203
Naming conventions	204
Case • 204	
Object naming • 206	
Tables • 206	
Primary key columns • 207	
Foreign key constraints • 208	
Suggested conventions • 209	
Organizing a Snowflake database	210
Organization of databases and schemas • 210	
Managed schemas • 211	
OLTP versus OLAP database structures • 211	
Source layer • 212	
Staging layer • 213	
Normalized self-service analytics • 213	
Denormalized reporting analytics • 214	
Departmental data • 214	
Database environments • 215	
Summary	216
Chapter 12: Putting Physical Modeling into Practice	219
Technical requirements	220
Considerations before starting the implementation	220
Performance • 220	
Cost • 221	
Data quality and integrity • 221	
Data security • 222	
Backup and scalability considerations • 223	
Expanding from logical to physical modeling	223
Physicalizing the logical objects • 223	

Defining the tables • 226	
<i>Naming</i> • 226	
<i>Table properties</i> • 226	
<i>Declaring constraints</i> • 227	
Deploying a physical model	229
Creating an ERD from a physical model	229
Evolving from physical to semantic models	231
Working with a semantic model • 233	
Building a semantic model • 234	
Summary	240

Part 3: Solving Real-World Problems with Transformational Modeling 241

Chapter 13: Putting Transformational Modeling into Practice	243
Technical requirements	244
Separating the model from the object	244
Shaping transformations through relationships	244
Join elimination using constraints	247
When to use RELY for join elimination • 249	
When to be careful using RELY • 250	
Joins and set operators	252
Performance considerations and monitoring	255
Common query problems • 256	
Additional query considerations • 257	
Putting transformational modeling into practice	259
Gathering the business requirements • 259	
Reviewing the relational model • 260	
Building the transformational model • 261	
Summary	263

Chapter 14: Modeling Slowly Changing Dimensions	265
Technical requirements	266
Dimensions overview	266
SCD types • 266	
Example scenario • 267	
Type 0: Maintain original • 267	
Type 1: Overwrite • 268	
Type 2: Add a new row • 268	
Type 3: Add a new column • 269	
Type 4: Add a mini dimension • 270	
Type 5: Type 4 mini dimension + Type 1 • 271	
Type 6: The Type 1, 2, and 3 hybrid • 273	
Type 7: Complete as-at flexibility • 274	
Overview of SCD types • 276	
Recipes for maintaining SCDs in Snowflake	277
Setting the stage • 277	
Type 1: Merge • 279	
Type 2: Type 1-like performance using streams • 281	
Type 2 with a dynamic twist	284
The anatomy of a window function • 285	
Creating a dynamic Type 2 dimension • 286	
Creating a Type 2.1 dimension • 289	
Performance considerations • 291	
Type 3: One-time update • 292	
Summary	292
Chapter 15: Modeling Facts for Rapid Analysis	295
Technical requirements	296
Fact table types	296
Fact table measures	298

Getting the facts straight	298
The world's most versatile transactional fact table • 299	
The leading method for recovering deleted records • 301	
Type 2 slowly changing facts • 303	
Maintaining fact tables using Snowflake features	306
Building a reverse balance fact table with Streams • 306	
Recovering deleted records with leading load dates • 310	
Handling time intervals in a Type 2 fact table • 313	
Summary	317
 Chapter 16: Modeling Semi-Structured Data	 319
Technical requirements	320
The benefits of semi-structured data in Snowflake	320
Getting hands-on with semi-structured data	321
Schema-on-read != schema-no-need	324
Converting semi-structured data into relational data	328
Summary	334
 Chapter 17: Modeling Hierarchies	 337
Technical requirements	338
Understanding and distinguishing between hierarchies	338
A fixed-depth hierarchy • 338	
A slightly ragged hierarchy • 339	
A ragged hierarchy • 340	
Maintaining hierarchies in Snowflake	342
Recursively navigating a ragged hierarchy • 342	
Handling changes • 346	
Summary	347

Part 4: Data Modeling for Enterprise Teams **349**

Chapter 18: Scaling Data Models through Modern Techniques **351**

Technical requirements 352

Demystifying Data Vault 2.0 352

 Building the Raw Vault • 354

Hubs • 355

Links • 358

Satellites • 359

Reference tables • 361

 Loading with multi-table inserts • 361

Modeling the data marts 364

 Star schema • 365

 Snowflake schema • 365

Diving into data lakes 367

Discovering Data Mesh 369

 Start with the business • 370

 Adopt governance guidelines • 370

 Emphasize data quality • 371

 Encourage a culture of data sharing • 371

Data modeling in the dawn of AI 372

Summary 374

Chapter 19: Unlocking Value for Enterprise Organizations **377**

Misconceptions 378

 Data modeling is just for the data team • 378

 Data modeling is a one-time effort • 379

 Data modeling is only for large projects and organizations • 379

 Data modeling slows down project timelines • 380

Barriers	380
Demonstrating value with business metrics • 381	
Decentralized data ownership • 385	
Actions	386
Build momentum through quick wins • 386	
Link data modeling to business objectives • 387	
Data literacy discourages data siloes • 387	
Summary	389
 Chapter 20: Appendix	 391
Technical requirements	391
The exceptional time traveler	392
The secret column type Snowflake refuses to document	392
Read the functional manual (RTFM)	393
Summary	393
 Other Books You May Enjoy	 397
Index	401

Preface

Snowflake is one of the leading cloud data platforms, gaining popularity among organizations that are looking to migrate their data to the cloud. With its game-changing features, Snowflake is unlocking new possibilities for self-service analytics and collaboration. However, Snowflake's scalable consumption-based pricing model requires users to fully understand its revolutionary three-tier cloud architecture and pair it with universal modeling principles to ensure they are unlocking value and not letting money evaporate into the cloud.

Data modeling is essential for building scalable and cost-effective designs in data warehousing. Effective modeling techniques not only help businesses build efficient data models but also enable them to gain a deeper understanding of their business. Though modeling is largely data-base-agnostic, pairing modeling techniques with game-changing Snowflake features can help build Snowflake's most performant and cost-effective solutions.

Since the first edition of this book, the data landscape has changed significantly, and Snowflake has continued to innovate at an extraordinary pace. This second edition reflects these developments and addresses the increasing complexity of modern data architectures. It includes expanded coverage of semantic models, which have become an essential link between technical data structures, business understanding, and AI. This enables organizations to develop self-service analytics capabilities that genuinely meet the needs of their users.

The introduction of advanced Snowflake objects has changed how we approach data modeling in the cloud. Hybrid tables present new opportunities for transactional workloads; Iceberg tables offer open-standard compatibility, which improves data portability and interoperability, while dynamic tables allow organizations to create self-maintaining data pipelines.

However, having technical capabilities alone does not guarantee success. One critical addition to this second edition addresses a challenge that goes beyond database design: helping engineers communicate effectively with business stakeholders. Data modeling initiatives often fail not due to technical limitations but because of communication barriers between data teams and business leaders. Throughout this updated edition, we explore ways to translate technical concepts into business value, highlighting the **return on investment (ROI)** of data modeling efforts in terms that resonate with decision-makers and budget holders.

Being able to communicate the ROI of data modeling is essential for ensuring organizational buy-in and securing the resources needed for successful implementation. We've learned that even the most sophisticated data models may go unused if business teams do not understand their value or feel disconnected from their development. This edition provides practical frameworks for fostering collaborative relationships between technical and business teams, transforming data modeling from a technical task into a strategic business initiative, and making it a "team sport."

This book combines best practices in data modeling with Snowflake's powerful features to provide you with the most efficient and effective approach to data modeling in Snowflake. Using these techniques, you can optimize your data warehousing processes, improve your organization's data-driven decision-making capabilities, and save valuable time and resources. More importantly, you'll learn how to build bridges between technical implementation and business value, ensuring that your data modeling efforts translate into organizational success and competitive advantage.

Who this book is for

Database modeling is a simple yet foundational tool for enhancing communication and decision-making within enterprise teams and streamlining development. By pairing modeling-first principles with the specifics of Snowflake architecture, this book will serve as an effective tool for data engineers looking to build cost-effective Snowflake systems and for business users looking for an easy way to understand them.

The three main personas who are the target audience of this content are as follows:

- **Data engineers:** This book takes a Snowflake-centered approach to designing data models. It pairs universal modeling principles with unique architectural facets of the data cloud to help build performant and cost-effective solutions.
- **Data architects:** While familiar with modeling concepts, many architects may be new to the Snowflake platform and eager to learn and incorporate its best features into their designs for improved efficiency and maintenance.

- **Business analysts:** Many analysts transition from business or functional roles and are cast into the world of data without a formal introduction to database best practices and modeling conventions. This book will give them the tools to navigate their data landscape and confidently create their own models and analyses.

What this book covers

Chapter 1, Unlocking the Power of Modeling, explores the role that models play in simplifying and guiding our everyday experience. This chapter unpacks the concept of modeling into its constituents: natural language, technical, and visual semantics. This chapter also gives you a glimpse into how modeling differs across various types of databases.

Chapter 2, An Introduction to the Four Modeling Types, looks at the four types of modeling covered in this book: conceptual, logical, physical, and transformational. This chapter gives an overview of where and how each type of modeling is used and what it looks like. This foundation gives you a taste of where the upcoming chapters will lead.

Chapter 3, Mastering Snowflake's Architecture, provides a history of the evolution of database architectures and highlights the advances that make the data cloud a game changer in scalable computing. Understanding the underlying architecture will inform how Snowflake's three-tier architecture unlocks unique capabilities in the models we design in later chapters.

Chapter 4, Mastering Basic Snowflake Objects, explores the various Snowflake objects we will use in our modeling exercises throughout the book. This chapter looks at the memory footprints of the different table types, change tracking through streams, and the use of tasks to automate data transformations, among many other topics.

Chapter 5, From Logical Concepts to Snowflake Objects, bridges universal modeling concepts such as entities and relationships with accompanying Snowflake architecture, storage, and handling. This chapter breaks down the fundamentals of Snowflake data storage, detailing micro partitions and clustering so that you can make informed and cost-effective design decisions.

Chapter 6, Mastering Advanced Snowflake Objects, dives deep into the fundamentals and use cases of Snowflake's newer and more specialized table types. This chapter explores the mixed analytics/transactional potential of hybrid tables, building automated data pipelines with dynamic tables, and using Iceberg tables for interoperable workloads across cloud platforms.

Chapter 7, Seeing Snowflake's Architecture through Modeling Notation, explores why there are so many competing and overlapping visual notations in modeling and how to use the ones that work. This chapter zeroes in on the most concise and intuitive notations you can use to plan and design database models and make them accessible to business users simultaneously.

Chapter 8, Putting Conceptual Modeling into Practice, starts the journey of creating a conceptual model by engaging with domain experts from the business and understanding the elements of the underlying business. This chapter uses Kimball’s dimensional modeling method to identify the facts and dimensions, establish the bus matrix, and launch the design process. We also explore how to work backward using the same technique to align a physical model to a business model.

Chapter 9, Putting Logical Modeling into Practice, continues the modeling journey by expanding the conceptual model with attributes and business nuance. This chapter explores how to resolve many-to-many relationships, expand weak entities, and tackle inheritance in modeling entities.

Chapter 10, Database Normalization, demonstrates that normal doesn’t necessarily mean better—there are trade-offs. While most database models fall within the first to third normal forms, this chapter takes you all the way to the sixth, with detailed examples to illustrate the differences. This chapter also explores the various data anomalies that normalization aims to mitigate.

Chapter 11, Database Naming and Structure, takes the ambiguity out of database object naming and proposes a clear and consistent standard. This chapter focuses on the conventions that will enable you to scale and adjust your model and avoid breaking downstream processes. By considering how Snowflake handles cases and uniqueness, you can make confident and consistent design decisions for your physical objects.

Chapter 12, Putting Physical Modeling into Practice, translates the logical model from the previous chapter into a fully deployable physical model. In this process, we handle the security and governance concerns accompanying a physical model and its deployment. This chapter also explores physicalizing logical inheritance and demonstrates how to go from DDL to generating a visual diagram. Finally, we jump from physical models to semantic models and learn to “talk to our data” using Cortex AI.

Chapter 13, Putting Transformational Modeling into Practice, demonstrates how to use the physical model to drive transformational design and improve performance gains through join elimination in Snowflake. The chapter discusses the types of joins and set operators available in Snowflake and provides guidance on monitoring Snowflake queries to identify common issues. Using these techniques, you will practice creating transformational designs from business requirements.

Chapter 14, Modeling Slowly Changing Dimensions, delves into the concept of **slowly changing dimensions (SCDs)** and provides you with recipes for maintaining SCDs efficiently using Snowflake features. You will learn about the challenges of keeping record counts in dimension tables in check and how mini dimensions can help address this issue. The chapter also discusses creating multifunctional surrogate keys and compares them with hashing techniques.

Chapter 15, Modeling Facts for Rapid Analysis, focuses on fact tables and explains the different types of fact tables and measures. You will discover versatile reporting structures such as the reverse balance and range-based factless facts, and learn how to recover deleted records. This chapter also provides related Snowflake recipes for building and maintaining all the operations mentioned.

Chapter 16, Modeling Semi-Structured Data, explores techniques required to use and model semi-structured data in Snowflake. This chapter demonstrates that, while Snowflake makes querying semi-structured data easy, there is effort involved in transforming it into a relational format that users can understand. We explore the benefits of converting semi-structured data to a relational schema and review a rule-based method for doing so.

Chapter 17, Modeling Hierarchies, provides you with an understanding of the different types of hierarchies and their uses in data warehouses. The chapter distinguishes between hierarchy types and discusses modeling techniques for maintaining each of them. You will also learn about Snowflake features for traversing a recursive tree structure and techniques for handling changes in hierarchy dimensions.

Chapter 18, Scaling Data Models through Modern Techniques, discusses the utility of Data Vault methodology in modern data platforms and how it addresses the challenges of managing large, complex, and rapidly changing data environments. This chapter also discusses the efficient loading of Data Vault with multi-table inserts and creating Star and Snowflake schema models for reporting information marts. Additionally, you will be introduced to Data Mesh and its application in managing data in large, complex organizations. Of course, the conversation would not be complete without mentioning AI and the role data modeling plays in modern data initiatives. Finally, the chapter reviews modeling best practices mentioned throughout the book.

Chapter 19, Unlocking Value for Enterprise Organizations, tackles the biggest challenge facing data modeling initiatives: getting organizational buy-in from business stakeholders who don't understand the value. You will learn how to speak the language of business by demonstrating data modeling ROI using concrete metrics that show every dollar invested can generate nearly 30% returns. The chapter provides proven strategies for overcoming common misconceptions, building momentum through quick wins, and transforming data modeling from a technical burden into a strategic business advantage that drives collaboration and competitive edge.

Chapter 20, Appendix, collects all the fun and practical Snowflake recipes that couldn't fit into the structure of the main chapters. This chapter showcases useful techniques such as the exceptional time traveler, exposes the (secret) virtual column type, and more!

To get the most out of this book

This book will rely heavily on the design and use of visual modeling diagrams. While a diagram can be drawn by hand, maintained in Excel, or constructed in PowerPoint, a modeling tool with dedicated layouts and functions is recommended. As the exercises in this book will take you from conceptual database-agnostic diagrams to deployable and runnable Snowflake code, a tool that supports Snowflake syntax and can generate deployable DDL is recommended.

This book uses visual examples from SqlDBM, an online database modeling tool that supports Snowflake. A free trial is available on their website here: <https://sqldb.com/Home/>.

Another popular online diagramming solution is Lucidchart (<https://www.lucidchart.com/pages/>). Although Lucidchart does not support Snowflake as of this writing, it offers a free tier for designing ER diagrams as well as other models such as **Unified Modeling Language (UML)** and network diagrams.

Software/hardware covered in the book	Operating system requirements
Snowflake Data Cloud	Windows, macOS, or Linux
SQL	Windows, macOS, or Linux

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

Download the example code files

You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E>. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots/diagrams used in this book.

You can download it here: <https://packt.link/gbp/9781837028030>.

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: “Adding a discriminator between the CUSTOMER supertype and the LOYALTY_CUSTOMER subtype adds context that would otherwise be lost at the database level.”

A block of code is set as follows:

```
-- Query the change tracking metadata to observe
-- only inserts from the timestamp till now
select * from myTable
changes(information => append_only)
at(timestamp => $cDts);
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: “**Subtypes** share common characteristics with a **supertype** entity but have additional attributes that make them distinct.”



Warnings or important notes appear like this.



Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at customer@packt.com and mention the book’s title in the subject of your message.

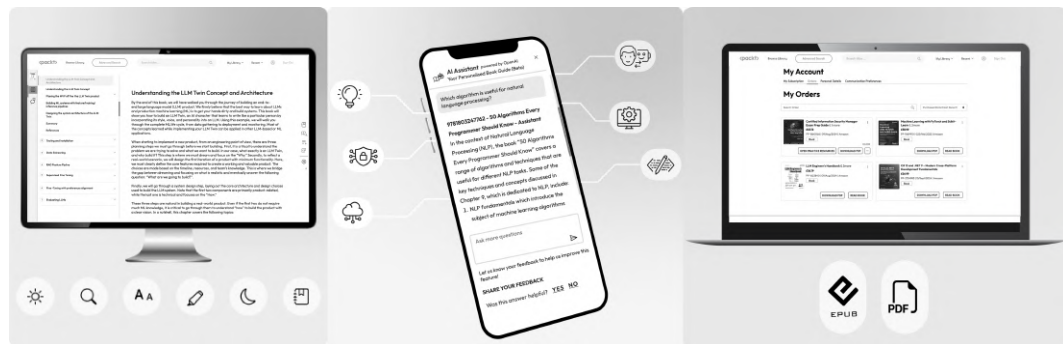
Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packt.com/submit-errata>, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packt.com/>.

Your Book Comes with Exclusive Perks - Here's How to Unlock Them

Unlock this book's exclusive benefits now Scan this QR code or go to packtpub.com/unlock, then search this book by name. Ensure it's the correct edition.	UNLOCK NOW 
Note: Keep your purchase invoice ready before you start.	



Enhanced reading experience with our Next-gen Reader:

-  **Multi-device progress sync:** Learn from any device with seamless progress sync.
-  **Highlighting and notetaking:** Turn your reading into lasting knowledge.
-  **Bookmarking:** Revisit your most important learnings anytime.
-  **Dark mode:** Focus with minimal eye strain by switching to dark or sepia mode.

Learn smarter using our AI assistant (Beta):

- ◆◆ **Summarize it:** Summarize key sections or an entire chapter.
- ◆◆ **AI code explainers:** In the next-gen Packt Reader, click the **Explain** button above each code block for AI-powered code explanations.

Note: The AI assistant is part of next-gen Packt Reader and is still in beta.

Learn anytime, anywhere:



Access your content offline with DRM-free PDF and ePub versions—compatible with your favorite e-readers.

Unlock Your Book's Exclusive Benefits

Your copy of this book comes with the following exclusive benefits:



Next-gen Packt Reader



AI assistant (beta)



DRM-free PDF/ePub downloads

Use the following guide to unlock them if you haven't already. The process takes just a few minutes and needs to be done only once.

How to unlock these benefits in three easy steps

Step 1

Keep your purchase invoice for this book ready, as you'll need it in *Step 3*. If you received a physical invoice, scan it on your phone and have it ready as either a PDF, JPG, or PNG.

For more help on finding your invoice, visit <https://www.packtpub.com/unlock-benefits/help>.



Note: Did you buy this book directly from Packt? You don't need an invoice. After completing Step 2, you can jump straight to your exclusive content.

Step 2

Scan this QR code or go to packtpub.com/unlock.



On the page that opens (which will look similar to Figure X.1 if you're on desktop), search for this book by name. Make sure you select the correct edition.

<packt> Search... Subscription

Explore Products Best Sellers New Releases Books Videos Audiobooks Learning Hub Newsletter Hub Free Learning

Discover and unlock your book's exclusive benefits

Bought a Packt book? Your purchase may come with free bonus benefits designed to maximise your learning. Discover and unlock them here

Discover Benefits Sign Up/In Upload Invoice

[Need Help?](#)

1. Discover your book's exclusive benefits

Search by title or ISBN

CONTINUE TO STEP 2

2. Login or sign up for free

3. Upload your invoice and unlock

Figure X.1: Packt unlock landing page on desktop

Step 3

Sign in to your Packt account or create a new one for free. Once you're logged in, upload your invoice. It can be in PDF, PNG, or JPG format and must be no larger than 10 MB. Follow the rest of the instructions on the screen to complete the process.

Need help?

If you get stuck and need help, visit <https://www.packtpub.com/unlock-benefits/help> for a detailed FAQ on how to find your invoices and more. The following QR code will take you to the help page directly:



Note: If you are still facing issues, reach out to customer care@packt.com.

Share your thoughts

Once you've read *Data Modeling with Snowflake, Second Edition*, we'd love to hear your thoughts! Please click [here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Part 1

Core Concepts in Data Modeling and Snowflake Architecture

This part provides you with a comprehensive overview of the power and potential of data modeling within the Snowflake cloud data platform. You will be introduced to the fundamental concepts and techniques that underpin effective modeling, including the importance of understanding data relationships and the role of modeling in driving better business outcomes. This part also includes a detailed examination of the four different types of modeling, highlighting their benefits and use cases. Finally, we focus specifically on Snowflake architecture and objects, exploring how to master this powerful platform and optimize it for maximum performance and value. Through a combination of theoretical insights and practical examples, you will gain a deep understanding of how to use modeling to unlock the full potential of Snowflake and transform your approach to data management and analysis.

This part has the following chapters:

- *Chapter 1, Unlocking the Power of Modeling*
- *Chapter 2, An Introduction to the Four Modeling Types*
- *Chapter 3, Mastering Snowflake's Architecture*
- *Chapter 4, Mastering Basic Snowflake Objects*
- *Chapter 5, From Logical Concepts to Snowflake Objects*
- *Chapter 6, Mastering Advanced Snowflake Objects*
- *Chapter 7, Seeing Snowflake's Architecture through Modeling Notation*

1

Unlocking the Power of Modeling

The word *modeling* has come to mean very different things in the half a century that it has been practiced in database systems. This opening chapter prefaces the book's overall aim to demystify modeling, along with its applications, methodologies, and benefits. Throughout this journey, the concept of modeling will unfold into a set of methods and terms that help organizations design and manage data and, more importantly, help them understand themselves.

In its broadest interpretation, modeling is a selective simplification that aids in navigating or designing something more complex. Any system can be broken down into smaller, more manageable pieces. Manipulating any piece individually may be straightforward, but doing so without regard to an overall strategy is a tenuous proposal that is sure to encumber scalability and maintenance down the line.

While modeling is generally considered database-agnostic, modern cloud data platforms, such as Snowflake, present their users with many unique features thanks to their innovative architecture and consumption-based pricing. A clear and forward-looking design that takes advantage of the native features of the platform that supports it is the key to building cost-effective solutions capable of meeting and anticipating business needs.

As the analytical requirements of a data-driven organization are notoriously complex and constantly evolving, modeling must keep pace and accompany data teams from idea to execution. To achieve this, modeling must go beyond the structure and relationships of database tables and embrace the transformational logic that moves and shapes the underlying data.

Only by leaning into the specifics of Snowflake features and architecture can a model be built efficiently from beginning to end.

In this chapter, we're going to cover the following main topics:

- Recognizing the utility of models in our daily lives
- Getting a glimpse of modeling conventions in action
- Getting acquainted with the tools in the modeling toolkit
- Uncovering the benefits of modeling for enterprise teams
- Incorporating modeling into strategic planning
- Understanding modeling applications for transactional and analytical systems

Technical requirements

This book focuses on data modeling specifically for the Snowflake Data Cloud. While modeling includes many system-agnostic terms and conventions, this book will leverage unique features of Snowflake architecture, data types, and functions when building physical models and **Structured Query Language (SQL)** transformations.

To follow along with the exercises in the following chapters, you will need a Snowflake account with access to a sandbox area for creating schemas, objects, and loading data.

You can sign up for a 30-day free trial of Snowflake (<https://signup.snowflake.com/>) if you do not already have access.

This book will frequently use visual modeling diagrams as part of the modeling process. While a diagram can be drawn by hand and constructed in PowerPoint or Lucidchart, a tool that supports common database modeling features is recommended. The exercises in this book will take the reader from conceptual database-agnostic diagrams to deployable and runnable Snowflake code. For this reason, a tool that supports various modeling types and can forward-engineer Snowflake syntax is recommended.

The diagrams in this book were generated using the SqlDBM online database modeling tool (<https://sqldb.com/Home/>), which supports the previously mentioned features and offers a 2-week free trial.

Modeling with purpose

Models are used to simplify complex systems. Take a modern city as an example; you will see that it consists of intricately linked systems such as highways, electrical grids, and transit systems. While these systems operate in the same physical territory, they require very different models to help us understand them. For example, a subway system snakes and curves below a city's varied terrain, but our model of it—a subway map—uses straight lines and places stations at nearly equidistant intervals. The subway map is not the city—it is a selective simplification of the city that makes it easier for passengers to visualize their journey. The transit map is a model so ubiquitous that it's hard to imagine doing it any other way—yet it took time to evolve.

The subway map, as we know it today, was invented by Harry Beck in 1931 while re-designing the map used by the London Underground. The old design was confusing to riders because it focused on the wrong goal—geographical exactness. Here's what it looked like before Beck:

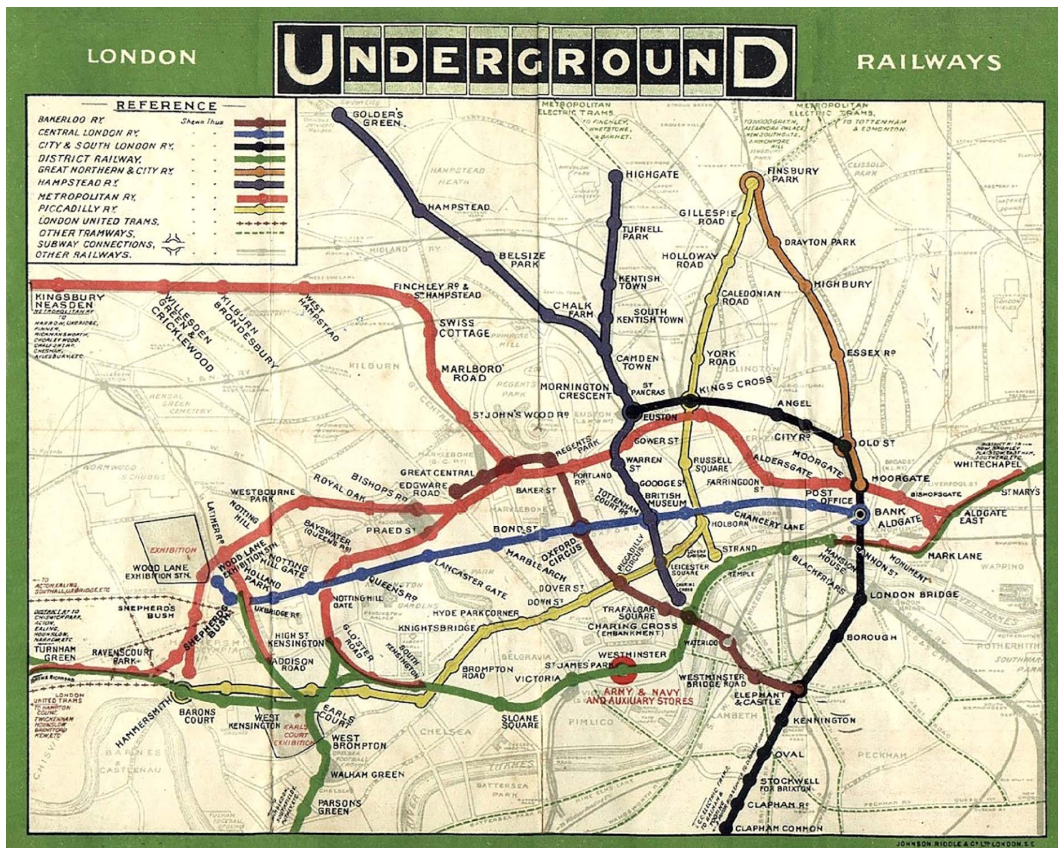


Figure 1.1: London Underground map, before Beck (legacy map)

Thankfully, Beck was not a cartographer—he was an engineer. By sacrificing topographical detail, Beck’s design allowed passengers to quickly count the number of stops required for their journey while retaining their overall sense of direction. This story reminds us (quite literally) of the refrain, *the map is not the territory*.

As with maps, various kinds of modeling exist to help teams within an organization make sense of the many layers that make up its operational landscape. Also, like maps, models help organizations prepare for the journey ahead. But how does one use a model to navigate a database, let alone plan its future?

Leveraging the modeling toolkit

Before we continue, we need to formally delineate three distinct concepts often used together in the service of modeling to make it simpler to refer to a specific tool in the modeling toolkit in later sections. By understanding where each piece fits in the broader domain of database design and management, diving into deeper technical concepts later in the book will become more meaningful and easier to digest.

The three components are listed here:

- Natural language semantics—words
- Technical semantics—SQL
- Visual semantics—diagrams

Let’s discuss each of these in detail, as follows:

- ***Natural language semantics:*** Terminology employed in communicating details of a model between people. These are agreed-upon words that employ pre-defined conventions to encapsulate more complex concepts in simpler terms. For example, when both parties involved in a verbal exchange understand the concept of a surrogate key, it saves them from having to explain that it is a unique identifier for a table record that holds no intrinsic business meaning, such as an integer sequence or a hash value.

To ensure effective technical conversations, it helps to be fluent in the semantics of modeling. Not only does it save time by succinctly communicating a complex concept, but it also saves even more time by not miscommunicating it. A waiter would return different foods when ordering chips in London rather than in Los Angeles. A properly modeled database would never return different records for the same surrogate key.

- **Technical semantics:** SQL is a domain-specific language used to manage data in a **Relational Database Management System (RDBMS)**. Unlike a general-purpose language (for example, YAML or Python), domain-specific languages have a much smaller application but offer much richer nuance and precision. While it can't format a website or send an email, SQL allows us to create the structure of our database and manipulate its contents.

SQL bridges modeling concepts (expressed in words or images) and what is physically defined in the database. Snowflake uses an **American National Standards Institute (ANSI)**-compliant SQL syntax, meaning its basic commands (such as SELECT, UPDATE, DELETE, INSERT, and WHERE) are compatible with other database vendors who use this standard. Snowflake also offers many extra functions, clauses, and conventions that go beyond ANSI-standard SQL and give users added flexibility to manage the database.

Unfortunately, due to its domain-specific nature, SQL presents a significant limitation: it can only express what the database explicitly understands. While SQL can define table structure and precisely manipulate data, it is too detailed to easily articulate the underlying business requirements.

- **Visual semantics:** Through their simplicity, images can convey a density of information that other forms of language simply cannot. In modeling, diagrams combine the domain-specific precision of SQL with the nuance of natural language. This gives diagrams a lot to work with to capture a data model's business meaning and technical specifics.

To start, diagrams vary in the level of detail they present—giving the observer exactly what they're looking for without overwhelming (or underwhelming) them with information. Most importantly, the semantic conventions used in diagrams are universal and can be understood by people besides data analysts and engineers. Yes—modeling diagrams are considered technical drawings; they represent strict technical concepts through agreed-upon visual conventions. However, in their simplest form, models can be understood almost intuitively with no prior knowledge. Even at the more advanced levels, such as logical and physical, learning to read a model is much simpler than learning SQL.

When all these semantics come together and are understood by the entire organization, they form a ubiquitous language, a concept first described by Eric Evans in *Domain-Driven Design*. Modeling, then, forms a part of the vocabulary that is understood universally throughout the organization to describe its business and store the data assets that support it. But that is just one of the many benefits that modeling provides.

The benefits of database modeling



Tactics without strategy is the noise before defeat. - Sun Tzu

For many people, database modeling brings to mind stale diagrams, arcane symbols, or extra work at the end of a project. Only a decade ago, fueled by the rise of distributed computing in the early 2000s—which popularized the concept of big data—the notion that modeling is dead gained notoriety. More precisely, it was thought that cheap and near-limitless computing power had made planning and designing a thing of the past. It was said that flexible semi-structured data formats and the ability to parse them on the fly—known as *schema-on-read*—had made modeling obsolete.

Eventually, operating and maintenance costs caught up with reality and revealed two great shortcomings of the schema-on-read approach. One is that no matter how data is structured, it must be functionally bound to the business that it helps support. In other words, semi-structured formats are neither a panacea nor an excuse to forgo the process of business validation. The second—and most important—is that a model is not simply the shape that data takes once uploaded to a database, but rather, the blueprint for business operations, without which it is impossible to build sustainable architectures.

Sustainable solutions require a long-term strategy to ensure their design matches the underlying business model. Without this, schema-on-read (discussed in *Chapter 16, Modeling Semi-Structured Data*), star schema (discussed in *Chapter 18, Scaling Data Models Through Modern Techniques*), or any other schema are narrow-sighted tactics that lead nowhere. But done right, modeling makes developing database architectures more agile and helps the project evolve from the idea phase to implementation. At every stage of development, the model serves as a guide for supporting the conversations necessary to propel the design into the next phase and provide additional business context. Once implemented, the model becomes a living document that helps users understand, navigate, and evolve the system it helped create.

While every organization models in the technical sense—creating tables and transforming data—not everyone models strategically, end to end, in the broad sense of the word—thereby foregoing the long-term benefits. Some of these benefits include the following:

- Consensus and visibility of the broader business model
- More productive conversations with business teams

- Better quality of requirements
- Higher signal, lower noise in technical conversations
- Cross-platform, cross-domain, and widely understood conventions
- Big-picture visual overview of the business and its database footprint
- Preliminary designs become implementation blueprints
- Accelerating onboarding of new team members
- Making data more accessible and unlocking self-service within organizations
- Keeping the database landscape manageable at scale
- Getting a handle on complex data pipelines

To demonstrate the difficulties of working without formal modeling, we can take a simple schema based on Snowflake's shared TPC-H dataset (available in the shared database called SNOWFLAKE_SAMPLE_DATA), which, at first glance, looks like this:

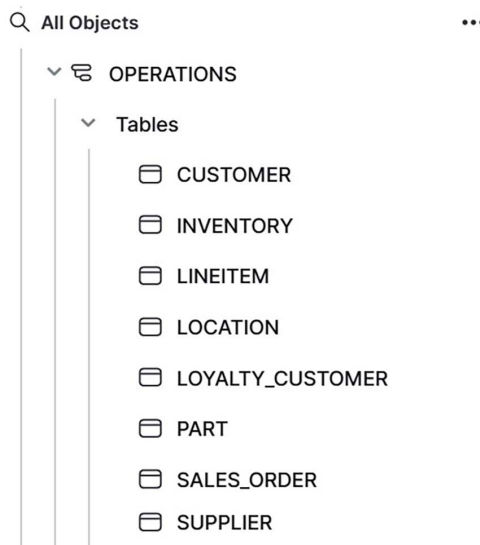


Figure 1.2: A list of tables in the Snowsight UI

While these tables have been modeled in the strict sense of the word and even contain data, we get very little information on what that data represents, how it relates to data in other tables, or where it fits in the broad context of business operations.

Intuition suggests that SALES_ORDER and CUSTOMER share a relationship, but this assertion needs to be tested. Even in this trivial example of only eight tables, it will take considerable time to thoroughly sift through the data to understand its context.

The irony is that many of the details we’re looking for are already baked into the design of the physical tables, having been modeled at some point in the past. We just can’t see them. Without a map, the terrain is lost from view.

Here is the same set of tables visualized through a modeling convention called an **Entity-Relationship Diagram (ERD)**:

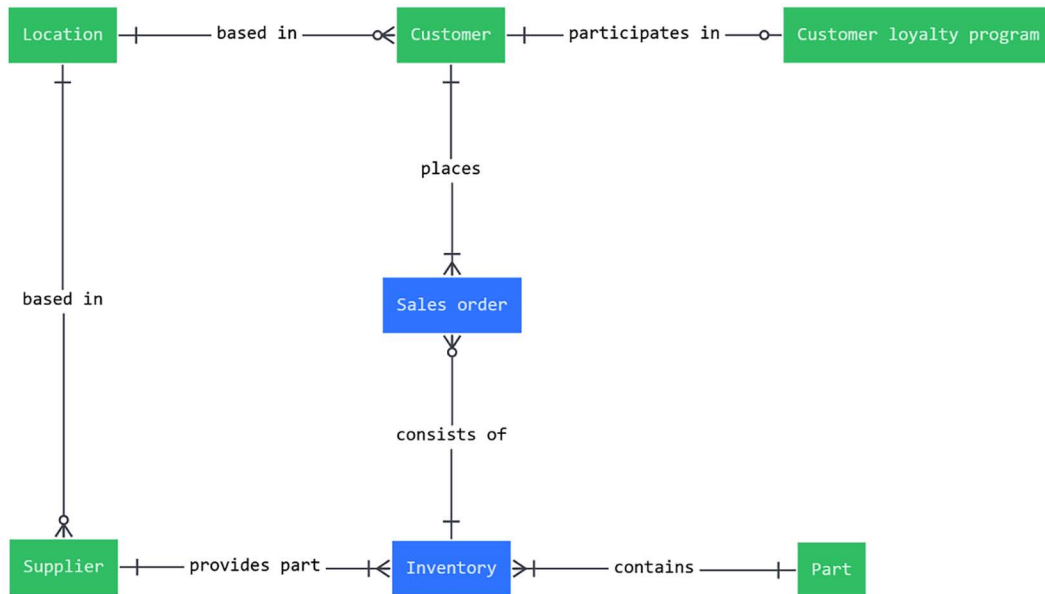


Figure 1.3: A conceptual model using crow's foot notation

At a glance, the big picture comes into focus. Diagrams such as this one allow us to understand the business concepts behind the data and ensure they are aligned. Having a visual model also lets us zoom out from individual tables and understand the semantics of our business: What are the individual pieces involved and how do they interact? This global perspective gives everyone in the organization a means of finding and making sense of data assets without requiring a technical background—thus, business analysts or new hires can unlock the value of the information without any help from the data team.

As the organization grows, expanding in personnel and data assets, it will inevitably become too big for any person, or even a team of people, to coordinate. Here, organizations that have embraced data modeling will stand out from those that did not. Modeling can be the thing that helps organizations scale their data landscape, or it can be the technical debt that holds them back.

Yet, for all its benefits, modeling is not a cookie-cutter solution that guarantees success. There are many approaches to modeling and various modeling methodologies that are suited for different workloads. Throughout this book, we will tackle the fundamentals of modeling that will allow you to understand these differences and apply the best solution using a first-principles approach. First, we will begin by breaking down the two main database use cases and observing the role modeling plays in each of them.

Operational and analytical modeling scenarios

The relational database as we know it today emerged in the 1970s—allowing organizations to store their data in a centralized repository instead of on individual tapes. Later that decade, **Online Transaction Processing (OLTP)** emerged, enabling faster access to data and unlocking new uses for databases such as booking and bank teller systems. This was a paradigm shift for databases, which evolved from data archives to operational systems.

Due to limited resources, data analysis could not be performed on the same database that ran operational processes. The need to analyze operational data gave rise, in the 1980s, to **Management Information Systems (MISs)**, or **Decision Support Systems (DSSs)** as they later became known. Data would be extracted from the operational database to the DSS, where it could be analyzed according to business needs. OLTP architecture is not best suited for the latter case, so **Online Analytical Processing (OLAP)** emerged to enable users to analyze multidimensional data from multiple perspectives using complex queries. This is the same paradigm used today by modern data platforms such as Snowflake.

The approach to storing and managing data in OLAP systems fundamentally differs from the operational or transactional database. Data in OLAP systems is generally stored in **data warehouses** (also known as **DWs** or **DWHs**)—centralized repositories that store structured data from various sources for the purpose of analysis and decision-making. While the transactional system keeps the up-to-date version of the truth and is generally concerned with individual records, the data warehouse snapshots many historical versions and aggregates volumes of data to satisfy various analytical needs.

Data originates in the transactional database when daily business operations (for example, bookings, sales, and withdrawals) are recorded. In contrast, the warehouse does not create but, rather, loads extracted information from one or various source systems. The functional differences between transactional databases and warehouses present different modeling challenges.

A transactional system must be modeled to fit the nature of the data it is expected to process. This means knowing the format, relationships, and attributes required for a transaction.

The main concern of a transactional database model is the structure and relationships between its tables.

By contrast, the data warehouse loads existing data from the source system. A data warehouse isn't concerned with defining a single transaction but with analyzing multitudes of transactions across various dimensions to answer business questions. To do this, a data warehouse must transform the source data to satisfy multiple business analyses, which often means creating copies with varying granularity and detail.

Modeling in a data warehouse builds upon the relational models of its source systems by conforming to common elements and transforming the data using logic.

Wait—if transformational logic is a core concept in data warehouse modeling, why is it so consistently absent from modeling discussions? In order to do transformational modeling justice, one must forgo the universality of general modeling principles and venture into the realm of platform specifics (that is, syntax, storage, and memory utilization). This book, in contrast, will embrace Snowflake specifics and go beyond physical modeling by diving into the transformation logic behind the physical tables. This approach provides a fuller understanding of the underlying modeling concepts and equips you with the required SQL recipes to not only build models but to load and automate them in the most efficient way possible. As we'll see in later chapters, this is where Snowflake truly shines and confers performance and cost-saving benefits.

Is Snowflake limited to OLAP?

Snowflake was originally envisioned as a data warehouse—with OLAP properties to enable multidimensional analysis at scale over massive datasets. However, over time, Snowflake has released innovations that push the bounds of what is possible and often defy attempts at classification. Hybrid tables, announced in 2022, present a unique hybrid storage architecture with rowstores for OLTP transactions and columnstores for OLAP analysis in one logical object. External and Iceberg tables further expand the platform's capabilities into the data lakehouse realm, which can't be categorized using traditional database terms.

These terms and objects are discussed in detail in later chapters.

Although OLAP and OLTP systems are optimized for different kinds of database operations, they are still databases at heart and operate on the same set of objects (such as tables, constraints, and views) using SQL. However, each use case requires very different approaches to modeling the data within. The following section demonstrates what modeling will typically look like in each scenario.

A look at relational and transformational modeling

The previous section described how modeling varies between operational and data warehouse scenarios. Before exploring the modeling process in detail, it's helpful to understand the look and feel of relational and transformational modeling and what we're working toward. Before proceeding, it would help to summarize the main differences between transactional databases and data warehouses. You can see what these are in the following table:

Transactional Database	Data Warehouse
Supports daily operations	Provides operational insight
Operates on single records	Summarizes many records
Accurate as of the present instant	Historical snapshots over time
Single source of truth (SSOT), non-redundant	Redundant to support different analyses
Data models defined by business operations	Data models generated by business questions
Static and structured data model	Inherited structure and dynamically transformed data model
Single-application data	Multiple sources of converging data

Table 1.1: Common differences between transactional databases and warehouses

Given these differences, the following sections demonstrate what modeling looks like in each system and what it aims to achieve.

What modeling looks like in operational systems

Completely ignoring the modeling workflow that got us here, which will be covered in later chapters, we can observe an example of the type of modeling most commonly seen in transactional systems. The physical diagram in *Figure 1.4* serves both as a blueprint for declaring the required tables and a guide to understanding their business context.

By following modeling conventions (don't worry if they are still unfamiliar—they will be covered thoroughly in the coming chapters), we can infer a lot of information from this simple diagram. For example, a person is uniquely identified by an eight-digit identifier (the primary key) and must have a **Social Security number (SSN)**, driver's license, name, and birth date.

The one-to-many relationship between the two tables establishes that while a person does not necessarily need to have an account created, an account must belong to just one person:

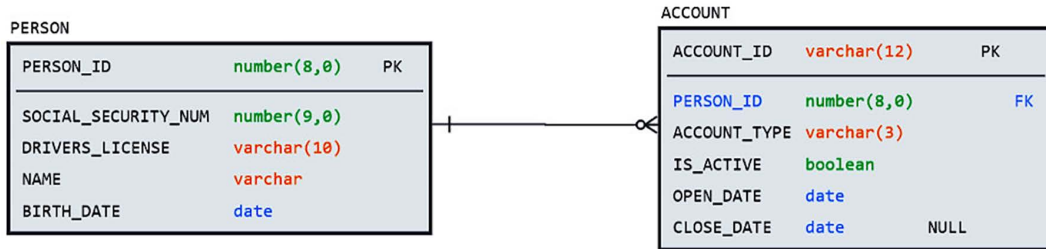


Figure 1.4: A physical model using crow's foot notation

These details, combined with the list of attributes, data types, and constraints, not only dictate what kinds of data can be written to these tables but also provide an idea of how the business operates. So, how does this differ in analytical databases?

What modeling looks like in analytical systems

In a data warehouse scenario, the PERSON and ACCOUNT tables would not be defined from scratch—they would be extracted from the source in which they exist and loaded—bringing both structure and data into the process. Then, the analytical transformations begin in answer to the organization's business questions. This is a process known as **Extract, Transform, Load (ETL)**. (Although ELT has become the preferred processing order, the original term stuck.)

Suppose the management team wanted to analyze which age groups (by decade) were opening which account types and they wanted to store the result in a separate table for an independent analysis.

The following diagram shows the resulting relational model of an object obtained through transformational analysis but provides no business context:

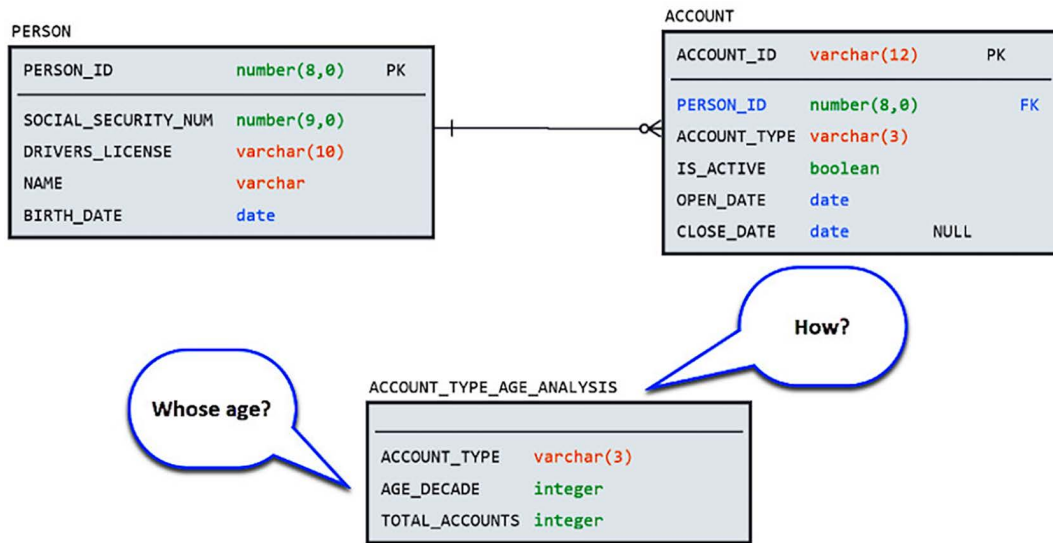


Figure 1.5: A relational model of a transformational requirement

Although physical modeling could describe such a table (as seen in *Figure 1.6*)—containing the account type with age and count of accounts as integers—such a model would fail to communicate the most relevant details, presented here:

- The logic used to perform the analysis
- The relationship between the source tables and the output

The business requirement for **ACCOUNT_TYPE_AGE_ANALYSIS** in this example purposely excludes the source key fields from the target table, preventing the possibility of establishing any relational links. However, the relational model still serves a vital role: it tells us how the sources are related and how to join them correctly to produce the required analysis.

The logic could then be constructed by joining **PERSON** and **ACCOUNT**, as shown here:

```
CREATE TABLE account_types_age_analysis AS
SELECT
  a.account_type,
  ROUND(DATEDIFF(years, p.birth_date, CURRENT_DATE()), -1
) AS age_decade,
  COUNT(a.account_id) AS total_accounts
```

```

FROM account AS a
  INNER JOIN person AS p
    ON a.person_id = p.person_id
GROUP BY 1, 2;

```

Although there is no relational connection between ACCOUNT_TYPE_AGE_ANALYSIS and its sources, there is still a clear dependency on them and their columns. Instead of using ERDs, which convey entities and relationships, transformational pipelines are visualized through a lineage diagram. This type of diagram gives a column-level mapping from source to target, including all intermediate steps, as shown here:

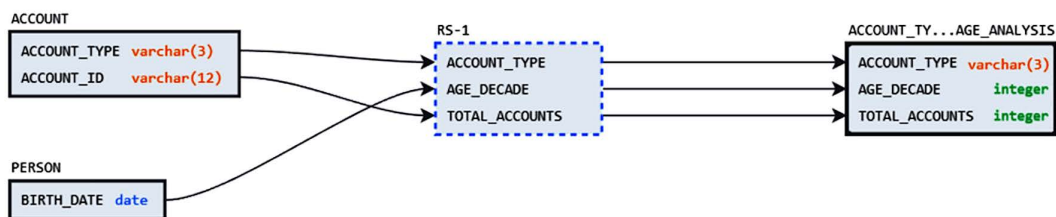


Figure 1.6: Transformational modeling seen visually



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Paired with the SQL logic used to construct it, the lineage graph gives a complete picture of the transformational relationship between sources and targets in an analytical/warehousing scenario.

Having witnessed both relational and analytical approaches to modeling, it is clear that both play a vital role in navigating the complex dynamic environments that one is liable to encounter in an enterprise-scale Snowflake environment.

Although we have only skimmed the surface of what modeling entails and the unique features of the Snowflake platform that can be leveraged to this end, this chapter has hopefully given you an idea of the vital role that modeling plays in building, maintaining, and documenting database systems. Before diving into the specifics of verbal, technical, and visual modeling semantics of modeling in the chapters to come, let's review what we learned.

Summary

There is no escaping modeling. We use it in our everyday lives to plan and navigate the complexities of the world around us—databases are no exception. For some readers, the modeling styles presented in this chapter may be a new way of conceptualizing their database landscape, while others may be getting reacquainted with its notation and use cases. Whether thinking about a company business model or sharing a finalized design with team members, we all engage in modeling to varying degrees.

Embracing database modeling and learning to speak in a commonly understood language unlocks many time-saving and collaborative benefits for the entire organization. Thinking long-term and modeling strategically, as opposed to reacting tactically, aligns database designs to the business that they underpin, ensuring their viability. Having seen the advantages that modeling uncovers and where it can be implemented, we can begin to analyze its components to understand precisely where they should be used and how they form a natural design progression.

In the next chapter, we will explore the four modeling types used in database design and discuss where they excel and how they build on one another to help take an idea and evolve it into a technical system design while generating living project artifacts to navigate and maintain the final product.

Further reading

Eric Evans' book, mentioned earlier in this chapter, explores how to create effective models by going beyond the surface and getting to the intention of the system itself. It is a recommended read for those wishing to go deeper into the realm of effective communication through models, unrestricted by specific technical domains, methods, or conventions:

Domain-Driven Design: Tackling Complexity in the Heart of Software, Eric Evans, Addison-Wesley Professional, 2004.

References

- *Legacy Tube map*, Wikimedia Commons: https://commons.wikimedia.org/wiki/File:Tube_map_1908-2.jpg. Accessed October 2, 2022.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

2

An Introduction to the Four Modeling Types

The previous chapter introduced the concept of a model as a selective simplification of reality. Like Harry Beck's Tube map, designed expressly to navigate London's subway system, other map variants—such as street and topographical—exist to describe different aspects of geography. The same applies to databases and the organizations that rely on them to enable their business operations and analytics.

Many people think of modeling as simply documenting a database—diagramming. But modeling goes way beyond tables and databases by not only helping the developers understand the business but also helping the business understand itself.

This chapter will explore conceptual, logical, physical, and transformational database modeling—each representing different stages of database design. Conceptual modeling focuses on defining high-level structures and relationships; logical modeling adds detail and nuance; physical modeling specifies how data will be stored, partitioned, and accessed on hardware; and transformational modeling logically defines how data is loaded as it moves across data warehouse layers.

While each of these modeling types has a dedicated chapter, the intention here is to understand where and why each process is used before proceeding to further detail.

In this chapter, we will cover the following main topics:

- Breaking modeling down to a flow
- How to use modeling elements to find a common language for communicating with business teams
- Aligning the business model to the data model with conceptual modeling
- Expanding on business concepts beyond database objects with logical modeling
- Creating the blueprint for database design by translating a logical model into a physical one
- Creating and managing **extract, load, and transform (ELT)** pipelines using transformational modeling

Design and process

Over the years, many books have been written about modeling and designing database systems. However, many of these books fail to make the connection between technicality and practicality—which are inextricably linked. Often, modeling books prioritize completeness—such as describing four different notations to express a relationship—and ignore usability (e.g., which notation would be most accessible to business users?).

The most significant complexity of a database system is not technical. It is the business model itself—the interactions within a business and the rules that govern them. If the business model is not aligned with the data model, the database will be repeatedly forced to adjust (while losing organizational trust and resources in the process).

What is needed is a business-readable language that developers use to build and document the database landscape.

Ubiquitous modeling

Before defining data structures, we need to understand the business model that generates them. Such a model will make it possible to build a database system aligned with business processes and able to anticipate change. Like seeing a forest for the trees, ubiquitous modeling allows us to see the business for the data.

A business has many layers: from a mission statement that defines a company ethos, to the sales transactions, to the logistics that support them, to the data, metadata, analysis, and more. However, when most people think of a model, they tend to focus only on the data layer—forgetting that, in isolation, the data only tells part of the story.

In reality, the modeling process involves teams across the entire organization, including management, business departments, data teams, and analysts. Everyone in the organization, no matter their technical background or domain expertise, will work with modeling in some capacity (even if it is in the role of a viewer). Never lose sight of the variance in skillsets when modeling—ensuring they are kept as simple as possible while including all the necessary information for the task at hand. The simplest and most effective strategy that appeals to modern enterprise teams is an agile and iterative approach that keeps business teams engaged while demonstrating continuous wins (e.g., improved delivery times, correctness, and data discovery). This is covered in greater detail in *Chapter 19, Unlocking Value for Enterprise Organizations*.

The following diagram depicts what is commonly thought of as data modeling within the broader business context:

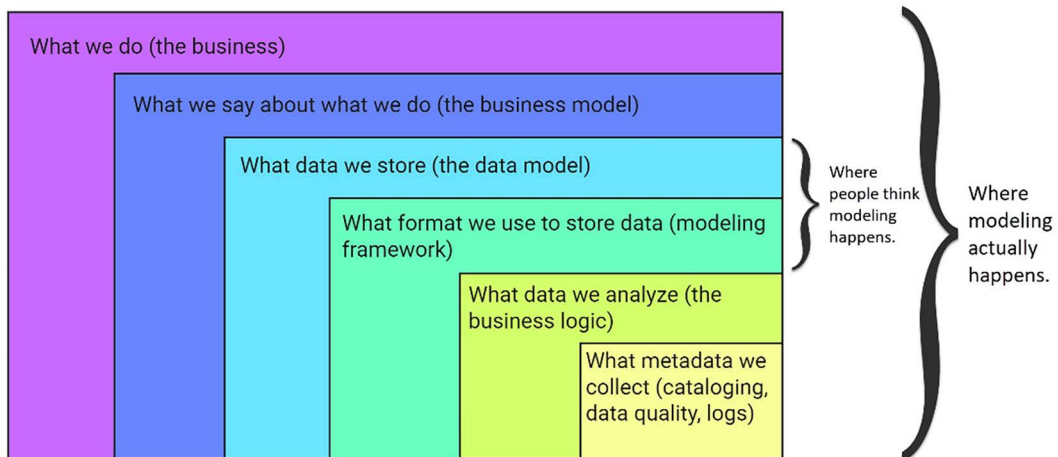


Figure 2.1: Business layers where modeling actually happens

***Relying exclusively on the data: A cautionary tale***

The management team asks you to build a model that handles all business data. Providing the sample data $\{0, 3, 6, 9\}$, they say that they can work with you to ensure you understand what you're building, and you can ask any (yes/no) questions you like.

Q. Is 12 in the set? A. Yes.

Q. Is 21 in the set? A. Yes.

Great, you understand the numbers and build a model to handle *multiples of three*.

The next day, a -1 arrives and breaks your model because the business set is *a set of all integers*.

Starting in the late 1960s and evolving to the present day, modeling and modeling notation have changed often: beginning with the first eponymous designs by Peter Chen, Charles Bachman, and Richard Barker, national standards such as Merise, and industry standards such as IDEF1X, **Information Engineering (IE)**, and **Unified Modeling Language (UML)**. Many notations, no clear winner, and no standard governing body to arbitrate—no wonder so many people are confused.

Here is a collage of data models using various standards:

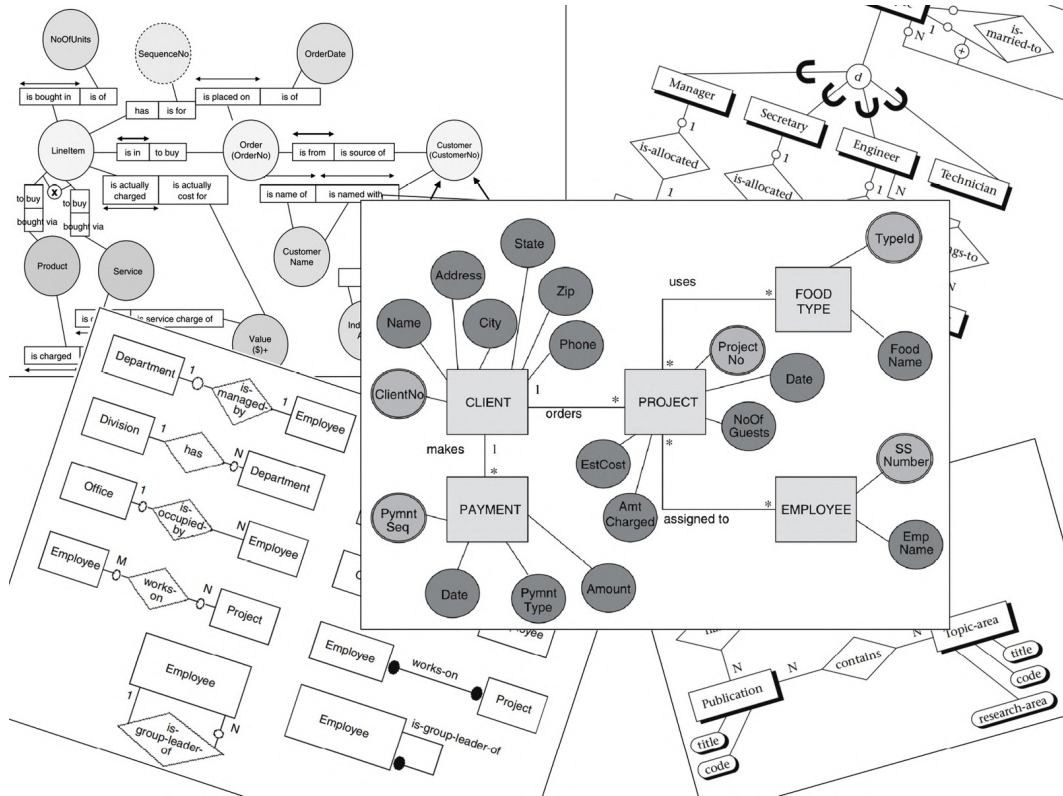


Figure 2.2: Examples of modeling notations

This book is not intended as an encyclopedia of modeling. It is meant to be a practical guide for applying modeling concepts in Snowflake to ensure efficiency, consistency, and scalability. This and future chapters will employ the most understandable and concise notation that makes modeling accessible to data engineers, developers, analysts, and business users alike by adhering to the following principles:

- **Keep it simple, stupid (KISS):** In modeling (as in life), the simplest approach is often the best approach
- **Be concise**
- **Be unambiguous:** Never give two meanings to the same convention
- **Embrace industry standards:** Industry standards should be followed when they facilitate understanding
- **Break industry standards:** Be flexible and eschew dogmatism in favor of usability

With these principles in mind, let's take a high-level look at the kinds of modeling that exist and where they are used. The following is meant to be an introduction to the concept of modeling and its implementation, so do not worry about the specifics or terminology. All of this will be covered in detail in later chapters.

Conceptual

Modeling begins long before databases—or even data—enter the picture; it starts with the business itself. A conceptual diagram should be as valid for describing the organization's operating model as it would be for laying the foundations of its database landscape.

What it is

Conceptual modeling is a process of identifying and visually mapping the moving pieces, or entities, of a business operation. Before going further, let's establish what an entity is.

An *entity* is a person, object, place, event, or concept relevant to the business for which an organization wants to maintain information. Examples of entities common to many companies include employee, customer, sale, and item. Entities are typically referenced in the singular and represent the class or type of an object (more on this and the singular versus plural naming debate in *Chapter 11, Database Naming and Structure*). *Entity instances* are occurrences of such a class or type:

- Superman is an instance within the SUPERHERO entity

Conceptual modeling is a collaborative process of discovery between the business and the data teams that produce visual project artifacts. Before data about entities can be collected and stored, the entities themselves must be uncovered and agreed upon. The goal of conceptual modeling is the synthesis of business and technical understanding between operational and data teams—the diagram is the by-product. The conversations that take place in the conceptual modeling service move the organization forward and add value in further stages of design and analysis.

Because business teams are unlikely to be familiar with model design, it is possible that working with the data team forces them to think in such terms for the first time. It is possible that when preliminary conceptual designs are initially drafted, business teams may be confronted with a reality that is not universally shared—and this is a good thing. When business processes are first set down in concrete, unambiguous diagrams, they bring to light the misalignments that individuals unknowingly carry, allowing them to converge on the truth.

For instance, are we in the business of serving meals to people in restaurants or selling ingredients wholesale to restaurant kitchens? If so, by phone, web, mobile sales force, or all the above? The data team, in isolation, is unlikely to know for sure.

Once the domain experts have established the entities, the analysis broadens to uncover their properties and relationships. Properties, or **attributes**, are descriptive pieces of information related to an entity:

- The Superhero entity has a Birthplace attribute with the value Krypton for the Superman instance

A customer might have a name, address, height, and driver's license—but which of these are relevant to our restaurant business? Are there other pertinent properties that were not included? The business teams can confirm this information.

Finally, there is the interaction between entities called a **relationship**. Relationships are expressed as verb phrases such as *is*, *has*, or *attends*. Relationship names are labeled in the diagram on the relationship line:

- SUPERHERO *has* a SUPER_POWER

Comic geeks know there are four types of superhero classes: alien savior, tech soldier, science-accident survivor, and vigilante. Superman, born under the red sun of Krypton, is of the first variety. So far, we have two entities (SUPERHERO and SUPERHERO_TYPE) and a relationship (SUPERHERO has a SUPERHERO_TYPE). The business teams must now confirm the granularity of both the entities and the relationship they share:

- **Entity granularity:** This is what a single instance of an entity represents:
 - The SUPERHERO entity is maintained at the grain of an individual hero. A single row in SUPERHERO represents one unique hero and their attributes.
 - The SUPERHERO_POWER entity is maintained at the grain of the hero superpowers. A single row in SUPERHERO_POWER represents just one of the many powers that a superhero can have.

- **Relationship granularity:** This is the granularity between entities. This consists of two parts—cardinality and optionality:
 - **Cardinality:** This is the maximum number of instances that two entities can share. This is either *one* or *many*:
 - A SUPERHERO entity has one SUPERHERO_TYPE
 - A SUPERHERO entity can have more than one SUPERHERO_POWER
 - **Optionality:** This signifies whether the relationship is *mandatory* or *optional* (e.g., *must*, *may*, or *can*). The word *zero* is often used as a synonym for *optional* in this context (e.g., *zero to many* instead of *optional many to many*) but should not be confused with cardinality. More details on the related nomenclature and notation are provided in *Chapter 7, Seeing Snowflake's Architecture through Modeling Notation*:
 - A SUPERHERO entity must have a SUPERHERO_TYPE
 - A SUPERHERO entity may or may not have a SUPERHERO_POWER

When describing the granularity, all this information is condensed into one term. Suppose the business teams confirm that superheroes must be classified into one of the four earlier types and can have many or no superpowers. In that case, we can describe the relationship between the entities as follows:

- A SUPERHERO must have one and only one SUPERHERO_TYPE
- A SUPERHERO can have zero or one or more SUPERHERO_POWER

Even in this primitive example, the word salad is becoming hard to swallow. But if we are to create tables in Snowflake to accurately store this information, we must be able to systematically capture these nuances, and a diagram is an ideal way to do so.

What it looks like

It is easier to define a conceptual model by what it is not expected to contain than what it does. Unlike a physical model, which must have strict database-specific precision to be deployable, conceptual models include as much or as little detail as needed to view and understand the details of the model at a high level.

At a minimum, a conceptual model must depict entities and their relationships. Beyond that, details such as attributes, granularity, and even some logical features may be included. The following are two examples of the superhero model we covered in the previous section.

Even at the highest level of abstraction, conceptual modeling is flexible in its details and presentation to fit the needs of the use case. Here, we see the model at its lowest resolution—as it might appear during the discovery phase of a development process—showing only entities and relationships:

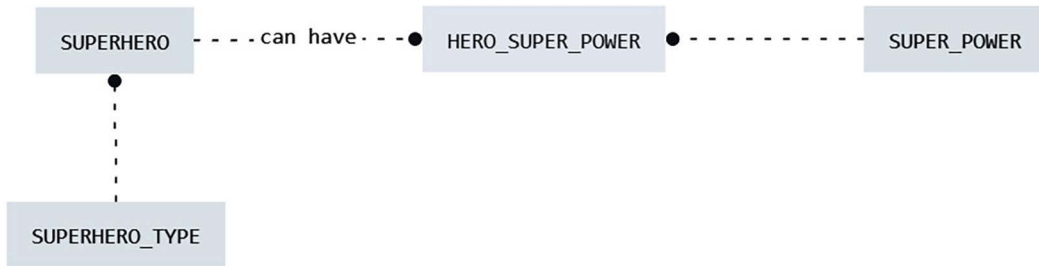


Figure 2.3: Conceptual entity-relationship (ER) modeling

The same design can be augmented with descriptions to give additional context to an audience who is not familiar with the domain in question:

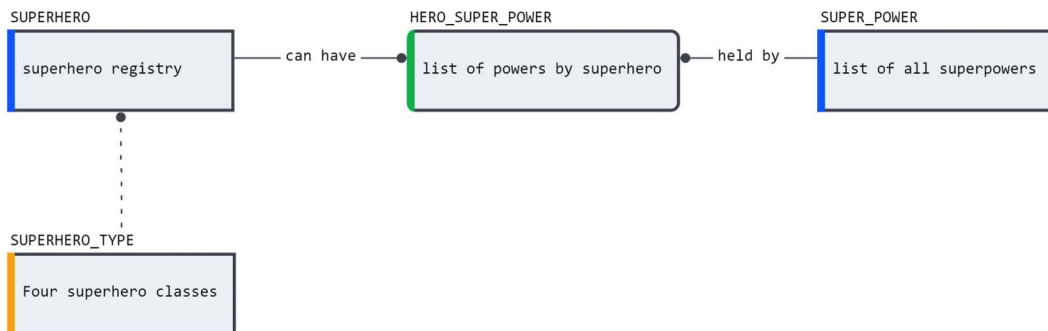


Figure 2.4: Conceptual ER modeling with descriptions

In this early stage, details such as data types or granular attributes are not required. For example, listing an address attribute and unpacking later into constituent parts, such as street, city, and postal code, may suffice:

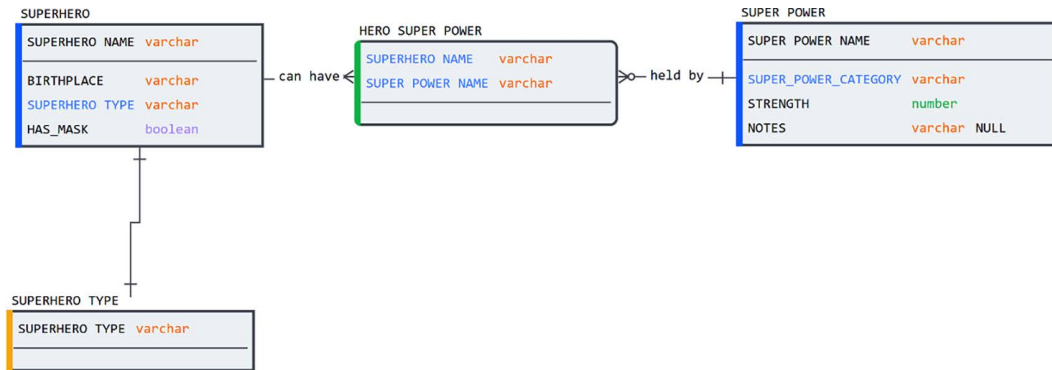


Figure 2.5: Conceptual ER modeling with attributes and cardinality

Conceptual models remain useful well after deployment. New employees might use a conceptual model to acquaint themselves quickly with the business model. A business user can likewise rely on the simplicity of the conceptual model to explore an adjacent domain to understand how it interacts with the one they are familiar with.

Conceptual modeling is also used to create mock-ups for planning new designs. Like a whiteboard, conceptual models capture early-stage ideas and allow them to evolve as changes and details are introduced. Unlike a whiteboard, a conceptual model is easy to modify and share with team members when maintained online in a collaborative modeling tool.

Now that we understand how conceptual models are used and what they look like, it's time to move on to the next phase of modeling: logical.

Logical

Once the building blocks of the business model have been identified through conceptual modeling (entities, attributes, and relationships), logical modeling begins. However, there is no strict distinction between the elements used in logical and conceptual designs. In fact, many database design textbooks do not differentiate between the two, combining them into a single style and tackling them in a single step.

This book takes a different approach—distinguishing between conceptual and logical modeling, not due to the elements or their notation but due to the natural flow of the design process. Because database textbooks are geared toward a technical audience, many lose sight of the less technical participants of database modeling: the business users.

Although a logical model can express everything that a conceptual one can, it also includes a great deal of technical detail, which may alienate those team members who lack the foundation to make sense of it. In the initial stages, domain knowledge is more important than technical details, so it may help to pivot between the two modeling styles, depending on the audience.

Unlike paper or a whiteboard, a modeling tool will allow you to switch between different modeling styles and notations seamlessly without affecting the underlying objects, allowing you to choose the appropriate level of abstraction, depending on the audience.

Let's try and understand what this means in practice.

What it is

Logical modeling is the bridge between the business's conceptual operating model and the physical structure of the database. Logical modeling uses conventions that give context to the nature of the data beyond what conceptual or physical models can express.

Logical modeling augments the conceptual by adding notation describing relationship metadata and incorporates elements of physical database design, such as data types and keys. Logical concepts such as subtypes and supertypes may already be familiar to those with a programming background but may appear arcane to business users.

In the logical modeling phase, technical details required for database implementation, such as data types and keys, are also introduced, albeit in a simplified form. What separates logical modeling from the physical is that the technical details are not specific to any database type.

Sitting between the physical and conceptual realms allows logical modeling to bridge both worlds and express nuance that would otherwise be lost in the rigidity of pure database objects.

In short, logical modeling helps teams ascertain business nuance on top of a model that closely resembles their existing (or soon-to-be existing) physical landscape.

Let's take a look at some of the logical-only elements that we can expect to find on such a diagram.

What it looks like

The logical modeling phase includes details such as data types, unique identifiers, and relationships. Contextual notation for subtypes and supertypes, as well as many-to-many relationships, are also included.

Subtypes are used on logical diagrams to represent inheritance. In the superhero example, SUPERHERO_TYPE can be used to create subtype tables based on the SUPERHERO supertype. This way, common attributes such as SUPERHERO_NAME can be shared, while attributes unique to each subtype can be maintained in the respective subtype tables (such as the IS_HUMANOID indicator for alien savior-type heroes).

Here's what it looks like for a subtype relationship that is both complete (double parallel lines) and exclusive (letter X in the center):

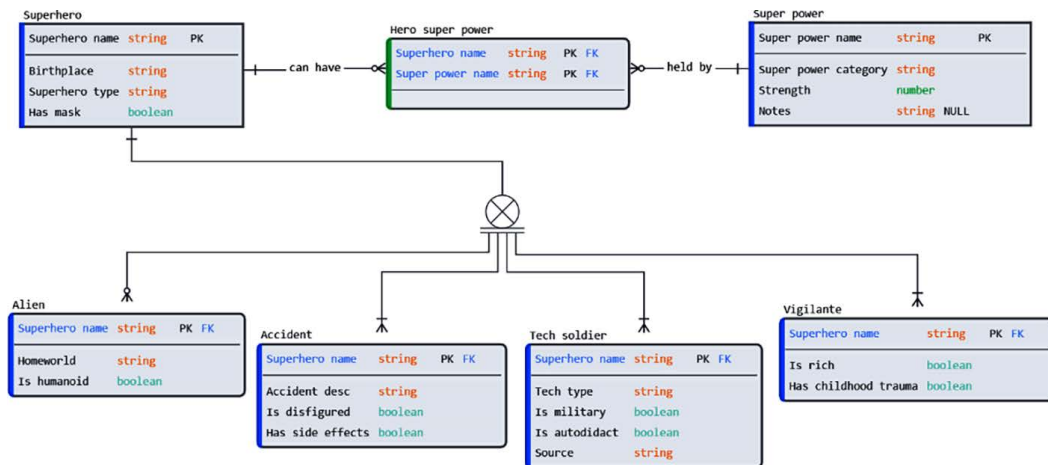


Figure 2.6: A logical subtype/supertype relationship

Many-to-many (M:M) relationships present a unique challenge for logical models. In our superhero example, we established that a superhero might have many powers (or, like Batman, none). Also, a superpower—let's say, flight—can be shared by many superheroes (or none if the power is *turning one's ears invisible*). In short, there are many superheroes with many superpowers.

The problem is that in a physical database, there is no way to store this information without duplicating the data in both tables:

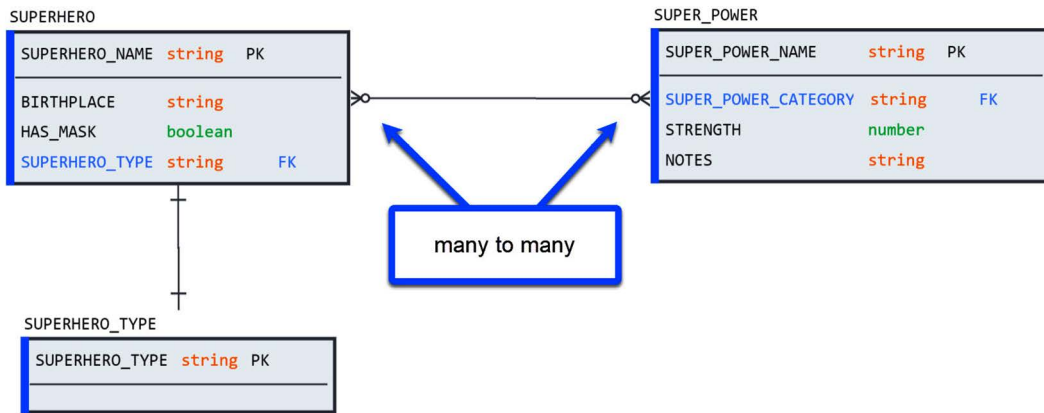


Figure 2.7: A logical M:M relationship between two tables



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



To represent an M:M relationship in a physical database, an intermediate associative table is required. Although M:M relationships between tables are allowed in logical designs, they violate the first two modeling principles outlined earlier in this chapter by using two tables to signify the existence of three.

To avoid such confusion, we borrow the diamond from the Chen notation to label the associative table in an M:M relationship:

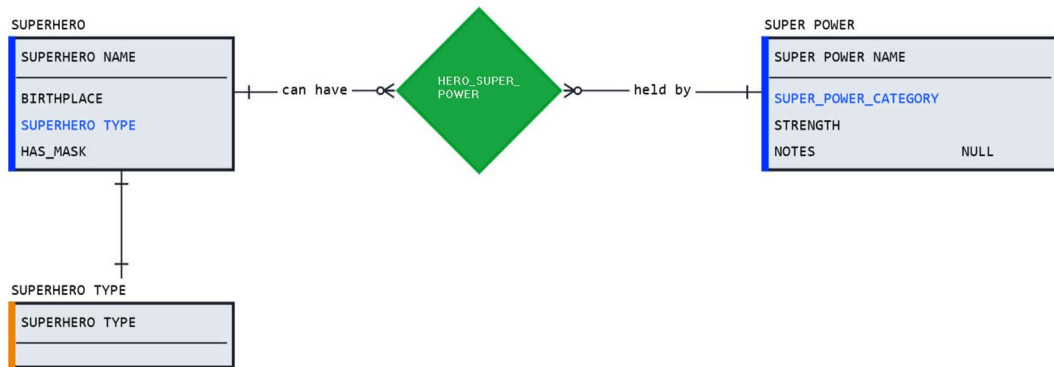


Figure 2.8: A logical M:M relationship as an associative table in Chen diamond notation

Here, it is clear to the observers that an M:M relationship exists and that a third table will be required (containing its own independent attributes) to capture the required information.

Once the logical context has been defined, work on the physical Snowflake model can begin.

Physical modeling

Chapter 1, Unlocking the Power of Modeling, separated the many meanings and applications of the word *model*, noting how it is used for the physical structure of database tables and the accompanying visual diagram. A **physical model** contains all the information necessary to construct and deploy the design to a specific database (e.g., Snowflake).

This also works in the opposite direction; many modeling tools and SQL **integrated development environments (IDEs)** can reverse engineer diagrams directly from the database **data definition language (DDL)**. **Reverse engineering** lets users generate visual representations of the underlying schema on the fly to help them explore relationships.

How exactly does this conversion work?

What it is

The physical model is a blueprint for deploying a database to Snowflake and the map that holds the technical and operational links that help users navigate it.

The transition from logical to physical modeling is largely systematic (relationships become constraints, data types are converted to database equivalents, and database properties are defined),

but some design decisions may still be required (e.g., rollups for subtypes, discussed in *Chapter 12, Putting Physical Modeling into Practice*). Such decisions will depend on business factors and future data consumption patterns.

Some modeling tools separate logical and physical models into different projects. Others, such as SQLDBM, keep conceptual, logical, and physical models in sync, allowing users to view a model at any level of detail without conversion.



Logical data types and Snowflake conversion

To facilitate migrations from other database platforms, Snowflake offers various compatible data types as synonyms for its own. For example, defining a column as either NUMERIC or INT is allowed but would result in the column being created as a Snowflake standard NUMBER(38, 0). This means that generic data types often used in logical models, such as STRING or INTEGER, can be used and automatically generate Snowflake equivalents upon deployment. More info can be found at <https://docs.snowflake.com/en/sql-reference/intro-summary-data-types.html>.

Physical modeling is the final step before Snowflake deployment. While the diagram displays data types and constraints, much more detail is embedded within the objects as part of this phase. Clustering keys, table types, and identity properties are set where required, and supporting objects such as sequences and file formats are also created and assigned. All this happens during a new development, but physical modeling is also used for existing schemas.

For organizations whose databases lack a formal model document, physical modeling—through reverse engineering—is the most direct way to create one. Many modeling tools can even suggest relationships in schemas where constraints have not been defined by detecting naming patterns. Although this lacks the metacontext of a logical model, it is a significant improvement over having no diagram at all.

What about views?

So far, we have been exploring database modeling in parallel with the business model—capturing and storing business-critical information as entities, attributes, and relationships. Views, however, do not store any data, as they are the SELECT statements stored as database objects. While it may be impossible to construct the CUSTOMER entity in a transactional database as a view without an underlying CUSTOMER table, data warehouses may use a view to logically unify several CUSTOMER sources into one dimension. In such a scenario, is the view not part of the CUSTOMER model? Of course, it is and should be visible in the physical model.

However, the *model* used by such a view is transformational, not physical, and will be covered in the corresponding sections. It's unfortunate that the word *logical* has already been taken by a class of modeling and can't be used to express logic-based transformations such as views and **CREATE TABLE AS SELECT (CTAS)** tables. Alas, we must adhere to the principle of unambiguity, which states *never give two meanings to the same convention*, so that's that.

So, how does a physical model differ visually from those discussed previously?

What it looks like

Physical diagrams look remarkably like their logical counterparts. However, they pack in much more information. Although the presentation options are configurable in most modeling tools, essential elements such as a complete column list, data types, and constraints must be defined. However, a physical model has many more details than can be displayed at a given time. Each element in the diagram (relationships, tables, and columns) holds its own set of physical attributes, which can be edited separately.

This is an example of the superhero database seen as a physical model:

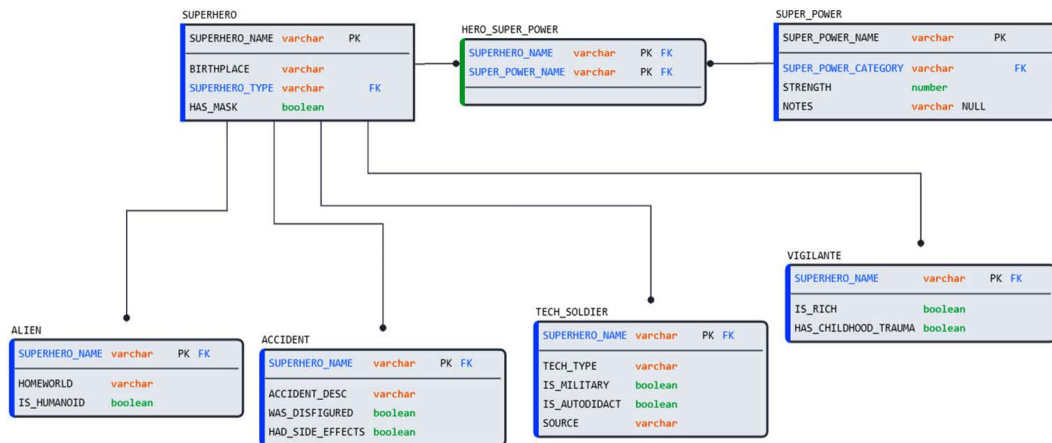


Figure 2.9: A physical model of the superhero database

A physical model, therefore, presents the viewer with only those properties that can be translated directly into the database. The relationships, as previously discussed, are as much a technical asset as they are clues to the operational nature of a business. Even in a data warehouse, where much of the landscape is defined through transformational modeling, the relationships outlined in the relational model help inform the connections and the joins that are made to make the transformations work.

Transformational

It all begins with `SELECT`. Modeling through transformational logic is a powerful and highly maneuverable method for modeling data that comes with one serious drawback: it needs existing data to `SELECT` from. Transformational modeling is rarely done in transactional databases because, in such systems, data is created and modified through the company's operational processes (e.g., purchases and profile updates)—with which expensive transformational processes should not compete for critical system resources. However, in data warehouses, where conformed datasets are extracted and duplicated with fresh timestamps for each load, transformational modeling becomes a necessity.

Because transformational modeling selects from existing structured data, the result set is already structured. Selecting the `SUPERHERO_NAME` and `HAS_MASK` columns and creating a table will preserve their structure (`VARCHAR` and `BOOLEAN`, respectively). However, as with all modeling, transformations must also be duly documented to allow database users to make sense of the complex data landscape that results.

What it is

Transformational modeling, unlike relational, isn't concerned with the structure of the data it is meant to hold because it uses existing data as part of the creation process. Instead, transformational modeling creates new objects by selecting data from existing sources. This is done through views using the `CTAS` command, or other DML statements.

But there is more to transformational modeling; it is also the `MERGE` and the bulk `INSERT` used to keep data in such tables up to date. When it comes to data, **online transactional processing (OLTP)** databases generally hold just one source of truth (the latest). But **online analytical processing (OLAP)** systems, by design, are disconnected from the latest data and must be loaded and synchronized on a schedule. The transformational approach is used to display the latest changes.

OLTP databases are generally used in operational use cases (such as sales, reservations, or application data) and operate on the **create, read, update, delete (CRUD)** of individual records. In contrast, warehouses rely on transformational modeling to unify data from disparate sources and create versions of the conformed data at various levels of detail and aggregation.

Transformational logic is also used by **business intelligence (BI)** and dashboarding tools when building visualizations and summarizing information. Such tools automate the work of writing queries by performing the required joins and aggregations when data is pulled onto the screen in the form of tables and charts.

Since transformations are expressed as code, what can we visualize with transformational modeling?

What it looks like

The best abstraction for the complexity of database logic is the SQL that produced it. At its core, transformational logic is SQL. It is the CTAS statement that creates and populates a table, but the INSERT or MERGE statement may also be used to update it or, VIEW may be called by these statements.

However, just as reverse engineering DDL can produce ER diagrams, it can also generate lineage diagrams from transformational SQL logic. Unlike relational diagrams, lineage graphs are low-res depictions of data flow from source to target in a SELECT statement. Without the accompanying SQL, the details that led to the creation of the table are lost:

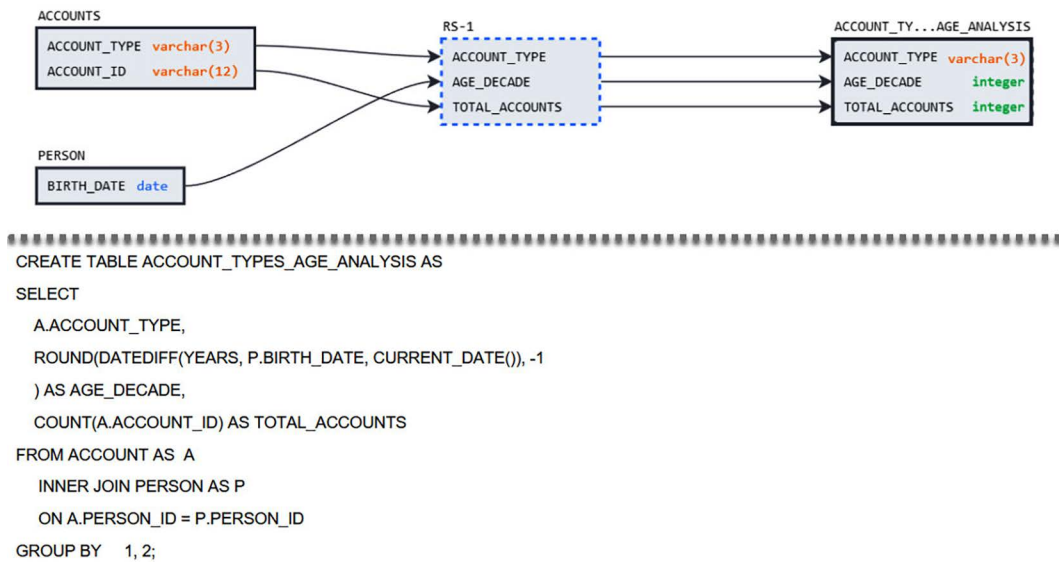


Figure 2.10: Transformational modeling expressed as code and on a lineage diagram

Unlike the other modeling diagrams—where pictures tell a clearer story than DDL—transformational modeling benefits from having code visible alongside the lineage graph. This allows users to acquaint themselves with the inputs and outputs of the transformation at a high level before diving into the details of the logic if they so choose.

After covering four different modeling styles with overviews, examples, and use cases, let's review what we've learned.

Summary

As we have seen throughout this chapter, modeling is a process used to agree on, plan, and develop a database design and a means to navigate and explore it to gain greater insight into the underlying business context. Every new project must pass through the four modeling stages, whether formally recognized or not. But even for existing databases that have not been adequately documented, reverse engineering is an effective mechanism to work backward from a database to uncover its business meaning.

The design journey starts with conceptual modeling—a collaborative process that involves business and data teams working together to understand the core elements that underpin business operations and how they interact. Conceptual models favor simplicity over detail—making them accessible to team members of all backgrounds and helping steer conversations to converge at a shared understanding of the business model.

After conceptual modeling, the data team can add further business context to the diagram using logical modeling elements. Logical models resemble physical designs but offer business nuances, such as many-to-many relationships, subtype classes, and relationship names—none of which can be represented in the database. This also makes logical models an ideal bridge for transitioning to physical modeling.

Physical modeling is concerned with database-specific details to ensure that the business rules are captured correctly and with the proper parameters to ensure efficient performance. Since the entities, attributes, and relationships have already been nailed down through conceptual and logical collaborations, the focus can turn to physical database details, where database-specific properties are defined.

Transformational modeling can begin by using the physical model as an orientative map—reshaping data into the format (or formats) best suited to answer business questions and drive decision-making. Although transformational modeling is performed purely through SQL logic, accompanying visuals (as in all types of modeling) can help accelerate the understanding and maintenance of complex logic.

Now that you have seen the types of modeling that exist and where they are used, the next step is learning how to apply them. However, before getting to that, we must make sure we understand the revolutionary cloud architecture Snowflake, which we'll be using as our canvas. In the next chapter, we will familiarize ourselves with the unique features that make Snowflake the fastest-growing cloud database platform on the market and how to use them to optimize costs and performance.

Further reading

Effective communication between technical and business teams is notoriously challenging. In *The Rosedata Stone*, Steve Hoberman presents a guide for creating a precise diagram of business terms within your projects. The business terms model is a simple yet powerful communication tool to help bridge the technical knowledge gap and meet business teams on their own “terms.”

Hoberman, Steve. *The Rosedata Stone: Achieving a Common Business Language Using the Business Terms Model*. Technics Publications, 2020.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

3

Mastering Snowflake's Architecture

For as long as databases have existed, they have faced recurring challenges in managing concurrency and scalability in the face of growing data volume and processing demands. Many innovative designs have been attempted over the years and have been met with varying degrees of success. However, that success often came with fresh drawbacks.

The Snowflake team saw that overcoming the age-old challenges of handling independent consumption demands of data storage and analysis required a radically new approach. The team decided to design a database that could operate natively on top of cloud computing platforms and thereby offer near-limitless scalability. Their efforts resulted in the creation of what Snowflake calls the Data Cloud—a platform that enables real-time data sharing and on-demand workload sizing through the separation of storage and compute.

In this chapter, we will cover the following topics:

- Exploring how databases have tried to achieve database scalability in the past
- Discovering how Snowflake built a scalable database to run natively in the cloud
- Seeing the unique features made possible by Snowflake's innovative architecture
- Understanding the costs associated with Snowflake's variable spend model
- Learning how to utilize various forms of cache to save on costs and improve performance

To truly appreciate what Snowflake has achieved, it is worth remembering the various traditional architectures that came before it and the limitations they tried to overcome.

Traditional architectures

To appreciate the innovation of the *Snowflake Data Cloud*, we have to take a step back and recall the designs and related limitations associated with its predecessors. Long before the advent of the cloud, databases started as physical on-premises appliances and, since their inception, have all faced the same challenge: scalability.

In the past, databases were confined to a physical server on which they relied for storage and processing power. As usage increased, memory would fill up, and CPU demand would reach the available limit, forcing the user to add more resources to the server or buy a new one altogether. As either response involved maintenance and downtime, hardware purchases had to be forward-looking, anticipating database growth several years into the future.

The following figure outlines the structure and key pieces of a traditional database. Although processing power, memory, and disk space were all customizable to a degree, they came packaged in a physical machine that could not scale.

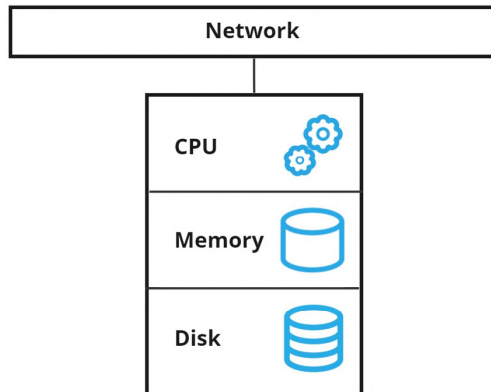


Figure 3.1: A diagram of a typical on-premises database server

Before cloud architecture unlocked the **software as a service (SaaS)** model and introduced variable spend pricing, hardware purchases required considerable **capital expenditure (CapEx)**. Thus, hardware sizing—estimating current and future computing needs—requires careful planning. A conservative estimate resulted in a sooner-than-expected upgrade, while overshooting meant paying top dollar for cutting-edge tech and not using it to its full potential for years to come. While data volume and CPU demands tended to grow synchronously with the business, unpredicted usage spikes in storage or compute were also common and led to similar headaches. That is when new architectures such as shared-disk and shared-nothing emerged to address the limits of scaling physical machines.

First, let us understand what changes shared-disk architecture introduced over traditional designs and the advantages that it offered.

Shared-disk architecture

The *shared-disk* approach uses a central storage location and makes it available to various compute clusters in a network. It is a simple design that, unfortunately, suffers from physical implementation issues when locking and accessing the data concurrently.

The following figure illustrates how shared-disk architecture externalized data storage away from individual machines and made it accessible to all computing clusters on the network.

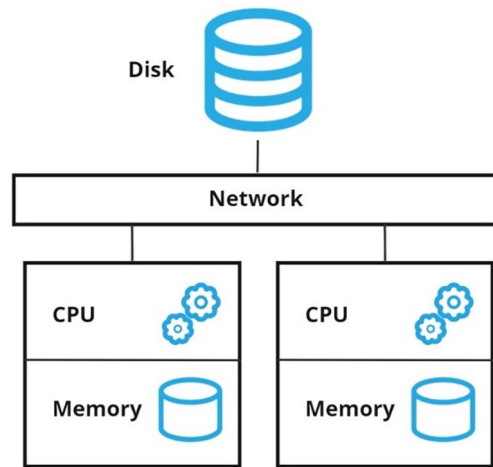


Figure 3.2: A diagram of a shared-disk architecture

The shared-disk design made it possible to add compute clusters of varying sizes to accommodate different workloads within the organization. However, this led to an inevitable bottleneck. The more clusters that are added to the network in a shared-disk architecture, the worse the contention for the central disk becomes. This design failed to deliver on the promise of scalability because it suffered from bottlenecks on the most important resource of a database: the data.

Where shared-disk failed, shared-nothing nearly succeeded.

Shared-nothing architecture

Shared-nothing architectures, like the ones used by AWS Redshift and Greenplum, avoided the issues of shared-disk by isolating it and making it part of the compute cluster. This design also addressed the varying consumption needs of teams across the organization, allowing database clusters to be sized based on demand. However, as we learn in grade school, sharing nothing is not the best strategy for collaboration.

The following figure shows how shared-nothing architecture allows self-sufficient but isolated database clusters.

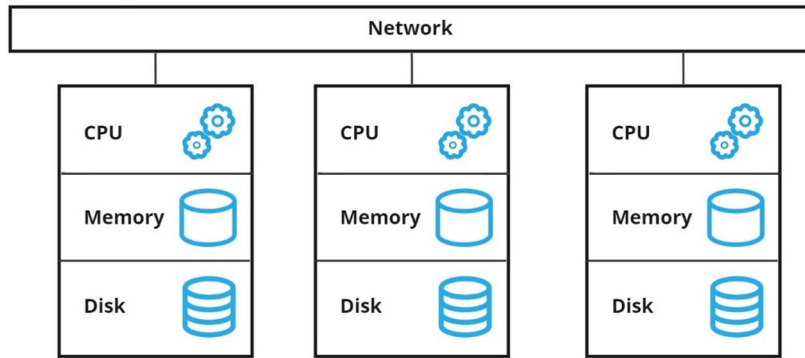


Figure 3.3: Components of a shared-nothing architecture

Creating database clusters in a shared-nothing architecture, especially one that runs on virtual cloud resources, solves the issue of investing up front in physical hardware. However, nodes in a shared-nothing architecture require data transfer to share information, which penalizes performance.

Tying the disk to the overall cluster also means striking the right balance between storage and processing patterns, which can vary independently. A heavy data science workload might require lots of compute to be directed at a relatively small dataset, while an ETL bulk-loading process might need the opposite. However, shared-nothing platforms did not offer many options for tuning each resource individually, nor were they simple to maintain and administer.

While shared-nothing architecture did not quite live up to the promise of seamless and easily managed scalability, it did pave the way for platforms such as Snowflake to tackle cloud computing challenges with cloud-native designs.

Snowflake's solution

To address the scalability issue that has plagued databases since their inception, the Snowflake team decided to formulate a new approach that would not be tied down by the limitations of past designs. They developed a modern platform built natively for the cloud that uses its unique features to enable concurrency, scalability, and real-time collaboration.

Snowflake's innovative cloud architecture still relies on physical disks, but it integrates them logically to make centralized storage available to its computing clusters without concurrency bottlenecks or data replication overhead. Finally, the best of what shared-disk and shared-nothing promised: separating the data from compute workloads, which can be independently provisioned and resized.

Snowflake runs entirely on virtually provisioned resources from cloud platforms (Amazon, Microsoft, and Google Cloud). Snowflake handles all interactions with the cloud provider transparently, abstracting the underlying virtual resources and letting the customer manage their data through a unified three-layer architecture.

Snowflake's three-tier architecture

Snowflake architecture consists of three layers: storage, compute, and cloud services. Snowflake manages all three layers so that interactions with the underlying cloud architecture are transparent to the users.

The following is an illustration of how Snowflake's architecture runs on top of cloud data platforms and separates disk from virtual compute clusters while managing a separate operational services layer (so you, the user, don't have to).

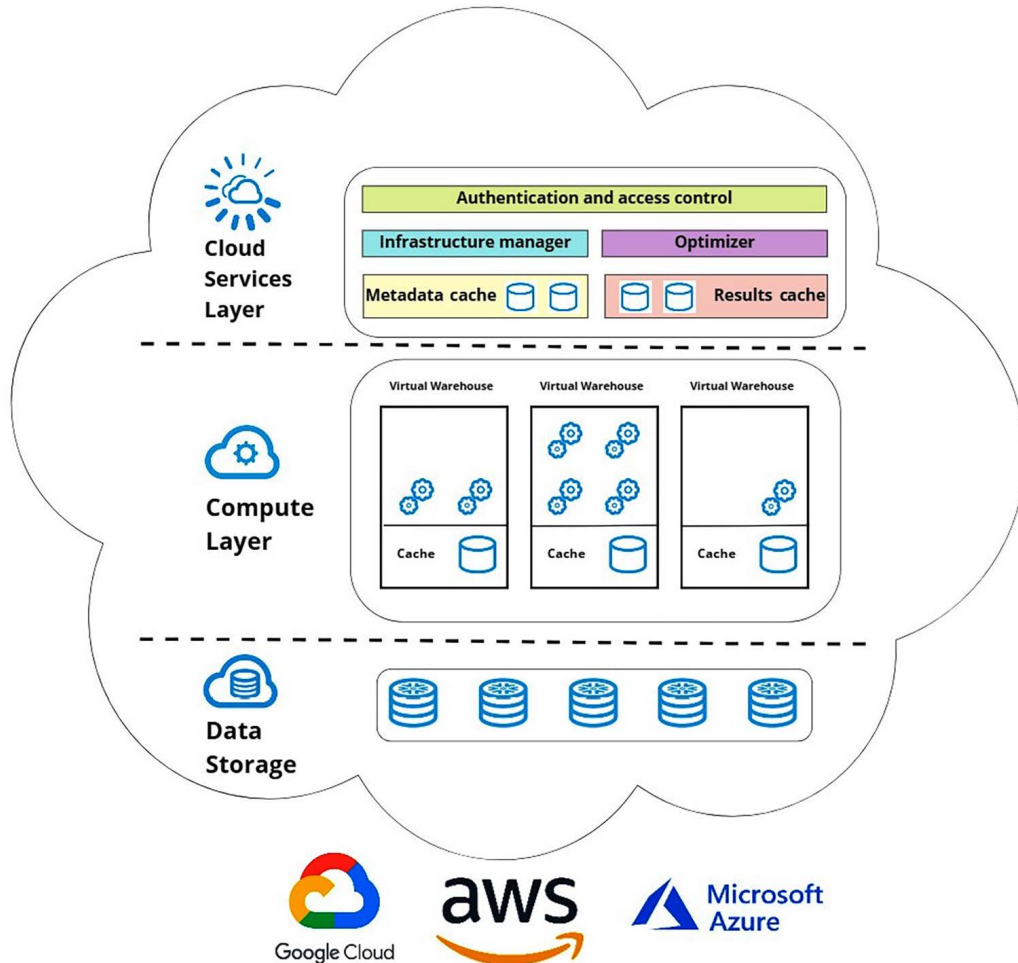


Figure 3.4: Snowflake hybrid cloud architecture



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now, let us get to know each of the three layers before explaining how they combine to enable Snowflake's innovative features, such as zero-copy cloning and Time Travel.

The storage layer

The *storage layer* physically stores data on disks in the cloud provider hosting the Snowflake account. As data is loaded into Snowflake, it is compressed, encrypted, and logically organized into tables, schemas, and databases. The users define the logical hierarchy of the database and its objects, while Snowflake takes care of the underlying partitioning and storage. The customer is billed only for the data they store, with no provisioning or sizing required.

Later chapters go into more detail on how Snowflake manages and optimizes data storage and the types of backup and redundancy options it offers. For now, let us understand how compute and virtual warehouse clusters work in Snowflake.

The compute layer

The *compute layer*, also known as the processing or virtual warehouse layer, provides a cluster of virtually provisioned CPU and temporary memory resources for executing queries. These clusters are called *warehouses* and are provisioned in t-shirt sizes—from XS to 6XL benchmarks. Through warehouses, Snowflake simplifies the consumption of virtual cloud resources using a simple formula. Each increase in warehouse size doubles the number of virtual servers in the cluster while also doubling the cost.

The packetization of virtual compute clusters into warehouses, with simple t-shirt sizes, abstracts the underlying cloud architecture and ensures simple and consistent utilization no matter the platform on which the Snowflake account is hosted. This way, warehouses can scale in a way that's simple to track. Scaling up, or resizing a warehouse to the next size in the t-shirt size, doubles the number of servers in the cluster, and scaling out adds warehouses of the same size to a compute cluster for increased concurrency (think of scaling up or out as compute *times two* or *plus one*, respectively).

The services layer

Snowflake's *services layer* coordinates all account activity in this hybrid-layer model and manages everything from security to encryption to metadata. The services layer handles operations such as query parsing and optimization, data sharing, and caching. From login, the services layer is there for the user through every operation they perform in the warehouse. The amount of time-saving work and automation that the services layer provides is reflected in Snowflake's marketing language: promising the user *near-zero maintenance*, along with low cost and exceptional performance.

Now, let us see how Snowflake's three-tier architecture comes together to unlock unprecedented possibilities for its users.

Snowflake's features

With its revolutionary cloud architecture, Snowflake continues to innovate and (pleasantly) surprise its users with game-changing performance enhancements beyond those that made it famous from its inception. While this is by no means a comprehensive list, the following sections highlight some of the most exciting and relevant features when it comes to data modeling.

Zero-copy cloning

Zero-copy cloning allows Snowflake users to clone data without physically duplicating it. Not having to move data means cloning happens nearly instantly—whether cloning a table or an entire database. Cloned objects are virtual copies of their source, so they do not incur storage costs. Once data changes occur in the clone or its source, the clone becomes a physical object and begins consuming storage resources.

Cloning is an ideal way to create system backups and testing environments—achieving in seconds what used to take days. At the object level, cloning is a convenient way to bring data across environments when developing or debugging.

Another enhancement made possible through innovative columnar storage techniques (which are explained in later chapters) is Time Travel.

Time Travel

Imagine the ability to go back in time, to a better moment like before you dropped that table in PROD, thinking it was TEST. Snowflake Time Travel makes that possible.

With *Time Travel*, users can query or clone data from a previous point in time, allowing them to recover from mistakes or compare changes from the present state. Time Travel is a built-in backup feature for Snowflake objects and, as such, uses extra storage and comes with the associated storage costs compared to objects that have this feature disabled. Later chapters will describe how to configure Time Travel to balance cost and flexibility.

Now, instead of table backups, let us turn our attention to an entirely different way of working with tables through one of Snowflake's latest innovations.

Hybrid tables

Data warehouses run **online analytical processing (OLAP)** architectures to enable them to efficiently perform the massive aggregations and calculations that analytic workloads require. However, use cases that depend on single-row operations or indexes perform better in **online transactional processing (OLTP)** databases. At Snowflake Summit 2022, a new feature was announced that gives users the best of both worlds: hybrid tables.

Hybrid tables are really two tables in one—an analytical and a transactional table working together under the hood to provide users with a unified solution that offers the best of both worlds. This architecture is known as **hybrid transactional/analytical processing (HTAP)**. Like all Snowflake objects, hybrid tables live up to the promise of near-zero maintenance by having the services layer automatically route workloads to the underlying table best suited for the task.

Hybrid tables are discussed in later chapters, so let us now go beyond structured data and discuss another of Snowflake's strong points.

Beyond structured data

Snowflake is known for its benchmark-setting performance abilities with relational tables and agility in working with semi-structured data. Even unstructured files are supported through external tables. Users can work with any of these file formats from a single place in a way that is as familiar and performant as querying a table.

Semi-structured files, such as logs and hierarchies, stored in JSON or similar formats can be loaded into Snowflake and queried like regular tables with almost no impact on performance. Snowflake provides an extensive library of functions that allow users to work with semi-structured records or flatten them into structured columns. Techniques for doing so are discussed in later chapters.

Unstructured data is also supported, and (its metadata) can be queried through external tables—metadata objects created on top of external stages. External tables are read-only but do support constraints for relational modeling. Additional information for working with external tables can be found in the Snowflake documentation here: <https://docs.snowflake.com/en/user-guide/tables-external-intro.html>.

All these great features and the agility of the cloud are bound to cost a fortune, right? Actually, with no upfront costs, competitive pricing, and the ability to size warehouses to user needs or suspend them entirely, users can often spend less on Snowflake than they did on legacy on-premises hardware. The caveat is that to do so, users must understand how Snowflake pricing works and how to manage it effectively.

Costs to consider

Unlike on-premises databases, which are purchased upfront and used for the duration of their life cycle, Snowflake employs a consumption-based model known as *variable spend* (commonly referred to as *pay-as-you-go*). Variable spend enables teams to do rapid prototyping or experiment with proofs of concept without any upfront investment and to control their costs by monitoring and adjusting usage patterns. Here, we will break down the types of costs associated with using the Snowflake platform so that we can make informed design decisions later on.

Let us begin with the cost of storing data in the cloud.

Storage costs

Snowflake bills its customers based on the daily average of the data stored in the platform. Since Snowflake's services layer automatically compresses data for optimal storage, customers enjoy lower storage costs without sacrificing performance. However, it is not just the raw data that counts toward storage quotas. Time Travel and fail-safe backups (discussed in detail in the next chapter) must also be considered.

Now, let us talk about the cost of processing data.

Virtual warehouse compute cost

Snowflake consumption is billed based on the number of virtual warehouses used, how long they run, and their size. An active warehouse is billed per second (with a 60-second minimum). Snowflake uses standard units called credits to simplify the math around warehouse pricing.

The exact cost of a credit varies based on factors such as the cloud provider, hosting region, and Snowflake edition (i.e., Standard, Enterprise, Business Critical, or VPS). Once these variables are fixed, *a credit equates to one XS warehouse running for one hour*. An important consideration is that warehouses consume credits for as long as they are active, regardless of whether they are running queries. An effective cost strategy should consider when warehouses are instantiated and shut down (auto-resume and auto-suspend options are available) and the size required for a given workload.

Serverless compute cost

Snowflake offers serverless features, such as Snowpipe and Query Acceleration, and automatically scales compute resources to meet their processing needs. These features are billed based on compute-hours and feature multipliers. A compute-hour is equivalent to running an XS Virtual Warehouse for one hour (i.e., one credit). Snowflake automatically adjusts the size of serverless resources based on workload demand while the multipliers are set per feature. For example, materialized view processing has a 2X multiplier, while Snowpipe is billed at 1.25X.

Finally, there is the cost of running and managing the platform.

Services layer compute cost

The services layer manages essential operations such as access control and query optimization. Even serverless tasks, such as clustering and Query Acceleration incur service layer costs. Many user-generated actions, such as querying table metadata (e.g., count, sum, or max), creating objects, and retrieving object metadata through commands such as show and describe are handled by the services layer and do not require an active warehouse.

The cost of the services layer is included in the warehouse credits as long as it does not exceed 10% of the daily warehouse consumption. When the services layer consumption does exceed 10% of warehouse spend, its operation is billed at standard credit rates (i.e., one credit per hour of XS or equivalent).

Data transfer costs

Snowflake does not charge fees for bringing data into your account (*data ingress*), but it does charge for data leaving your account (*data egress*). When you transfer data from your Snowflake account to a different region within the same cloud platform or to a completely different cloud platform, you will incur a per-byte fee. This fee varies depending on the region where your Snowflake account is hosted.

Having covered the costs of storage, compute, and data transfer, we should also discuss how we can save on these costs by learning to use Snowflake's various caches.

Saving cash by using cache

With on-premises databases, inefficient operations result in longer execution times. In Snowflake's variable spend model, that extra time is coupled with monetary penalties. Besides writing efficient SQL, Snowflake users should also understand the various caches associated with the service and virtual compute layers to understand where they can take advantage of pre-calculated results. A firm grasp of Snowflake caching will also inform decisions when modeling and building data pipelines and ultimately lead to dramatically faster processing.

Let us start with the services layer and familiarize ourselves with the caches it manages and offers its users.

The services layer

The services layer handles two types of cache: metadata and the query results cache.

The metadata cache

The services layer manages object metadata, such as structure, row counts, and distinct values by column. Reviewing this metadata through related SQL functions or the Snowflake UI will not require a running warehouse.

Snowflake stores metadata at the table level (e.g., size in bytes and date created) and keeps statistics (e.g., count, min/max, and null count) for columns in every micro-partition in the table.



Micro-partitions

Micro-partitions are groups of rows within tables organized in a columnar fashion. Micro-partitions allow for the granular pruning of large tables, which can comprise millions of micro-partitions. The Snowflake services layer automatically creates and manages micro-partitions as DML operations are performed in tables. Micro-partitions and caching are explained in detail in a later chapter, but for more information, see *Micro-partitions & Data Clustering* in the Snowflake documentation: <https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions.html>.

The services layer also stores metadata for the object definitions and structure. This includes the DDL for database objects and details such as the column list, data types (even for views), and constraints.

The other cache type managed by the services layer is the query results cache.

The query results cache

All query results in a Snowflake account are persisted for 24 hours by the services layer and can be reused by other users. Referencing a cached result resets the 24-hour clock for a maximum duration of 31 days, after which the result is purged. Aside from access privileges, there are other considerations to ensure that cached results can be reused. These include the following:

- The query must be syntactically equivalent
- Dynamic functions such as `current_date()` are not used
- The data in the underlying tables has not changed
- Users have the required permissions to access the underlying sources

Obtaining a result from the query cache is infinitely preferable to recalculating it because the operation happens instantly and consumes no compute credits. Unlike legacy databases, where the cache could be sacrificed to meet memory demands or lost altogether if the cluster was suspended, Snowflake's result cache is maintained until purged under the conditions previously mentioned.

Understanding the results cache is crucial for efficiently using Snowflake resources and making design decisions. However, that is not the only cache that users can take advantage of.

The warehouse cache

When a virtual warehouse accesses data from the central storage layer (referred to as the *remote disk* in underlying cloud storage such as AWS S3 buckets, Azure containers, or Google storage containers), the data is read into the warehouse cache (implemented using SSD storage). The amount of SSD storage depends on the warehouse size.

Reading from SSD storage is faster than reading from a remote disk, and the query optimizer (i.e., the services layer) will attempt to use the cache before defaulting to remote storage. However, unlike the services layer cache, the warehouse cache comes with two significant differences: the cache is specific to a given warehouse, and suspending the warehouse purges its cache.

With this in mind, users accessing the same data sources would benefit from sharing the same warehouse. However, trade-offs must be considered between the credits required to keep a warehouse active to preserve the cache and the I/O required to read the data from scratch from remote storage.

When a query is unable to hit any of the previously mentioned caches, Snowflake must access the storage layer to perform the operation.

The storage layer

Snowflake stores data on remote disks in the underlying cloud platform filesystem where the account is hosted. Reading data from a remote disk is less efficient than doing so from the warehouse cache (stored on SSD drives) and much less efficient than fetching a pre-calculated result. The storage layer has no cache to speak of. It is the last resort when no cache exists.

Let's summarize what we've learned in this chapter.

Summary

Snowflake's hybrid cloud-native design, built for the cloud from the ground up, enables real-time data sharing and on-demand workload sizing that gives its users unparalleled flexibility—overcoming many scalability limitations of previous database architectures. Snowflake's architecture allows secure data sharing between organizations across regions and cloud providers as quickly as it does between databases in the same account.

By understanding each of the layers that make up Snowflake’s cloud architecture (storage, compute, and services), we gained insight into how they enable powerful features such as zero-copy cloning, Time Travel, and hybrid *HTAP* tables. This chapter also outlined the costs of each of the three architecture layers and how to keep them in check. Furthermore, we discussed how various caching mechanisms across Snowflake’s architecture could work together to save costs and boost performance.

With so many revolutionary features, Snowflake does not think of itself as just a database but as a *Data Cloud*. When considering everything its features make possible—services such as a data and app marketplace—it is easy to see why new terminology was needed.

Now that we have had a glimpse of how Snowflake operates under the hood, it is time to dive deeper to understand the internal workings of its database objects. In the next chapter, we will look at Snowflake tables and explore their various types and parameters, as well as other object types that can be used as data sources and modeled as part of a database design. Understanding these objects and their unique features will allow you to model scalable and efficient database designs that take advantage of all the tools available in Snowflake’s arsenal.

Further reading

For the definitive guide (says so right there in the title!) on all of Snowflake’s features and object types, beyond the modeling-related content covered in this book, consider Joyce Avila’s excellent and complete reference:

Avila, Joyce. *Snowflake: The Definitive Guide: Architecting, Designing, and Deploying on the Snowflake Data Cloud*. O’Reilly Media, 2022.

Unlock this book’s exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

4

Mastering Basic Snowflake Objects

The previous chapter introduced the innovative architecture that powers the Data Cloud and unlocks possibilities that have never been possible in other databases. In this chapter, we will explore the database objects that Snowflake customers will use in their modeling journey. Objects such as tables and views will undoubtedly be familiar to most people who have previously worked with SQL. Still, even here, Snowflake's unique features unlock potential efficiencies in designing physical models, so users should be well acquainted with them.

Due to its variable pricing, Snowflake data modeling requires users to understand the cost/performance trade-offs of its supported objects and their parameters. For those who have never worked with Snowflake, exclusive features such as streams and external tables may still be unfamiliar. Making good design decisions demands that users have a firm grasp and a thorough understanding of Snowflake's complete modeling toolset.

This chapter will explore the fundamental Snowflake objects—from ingestion to storage to data pipeline orchestration—that users will rely on for most of their database modeling needs. Discussion of more advanced objects such as hybrid, Iceberg, and dynamic tables will follow in *Chapter 6*.

In this chapter, we're going to cover the following main topics:

- Stages and file format objects used for loading data into Snowflake
- The various physical table types used to store data
- Metadata tables for reviewing staged files
- Views and associated cost and performance benefits
- Materialized views and when to use them
- Streams for data change tracking
- Tasks for automating continuous ETL pipelines

Stages

Before data can make its way into tables, it must be loaded into Snowflake using a stage. **Stages** are logical objects that abstract cloud filesystems so they can be used in a standard manner to load data into Snowflake. There are two types of stages that can be defined: **external** and **internal**. External stages can be created on top of a cloud location (for the supported cloud storage services) outside Snowflake and are used to load data from external source systems. Internal stages are created within the Snowflake account, and therefore use the storage type of the hosting provider. Internal stages are used to stage files that originate from within the Snowflake account and cannot be used to load external data.

Supported cloud storage services for external stages are as follows:

- Amazon S3 buckets
- Google Cloud Storage buckets
- Microsoft Azure containers

The following figure shows the layout of external stages in relation to internal stage types.

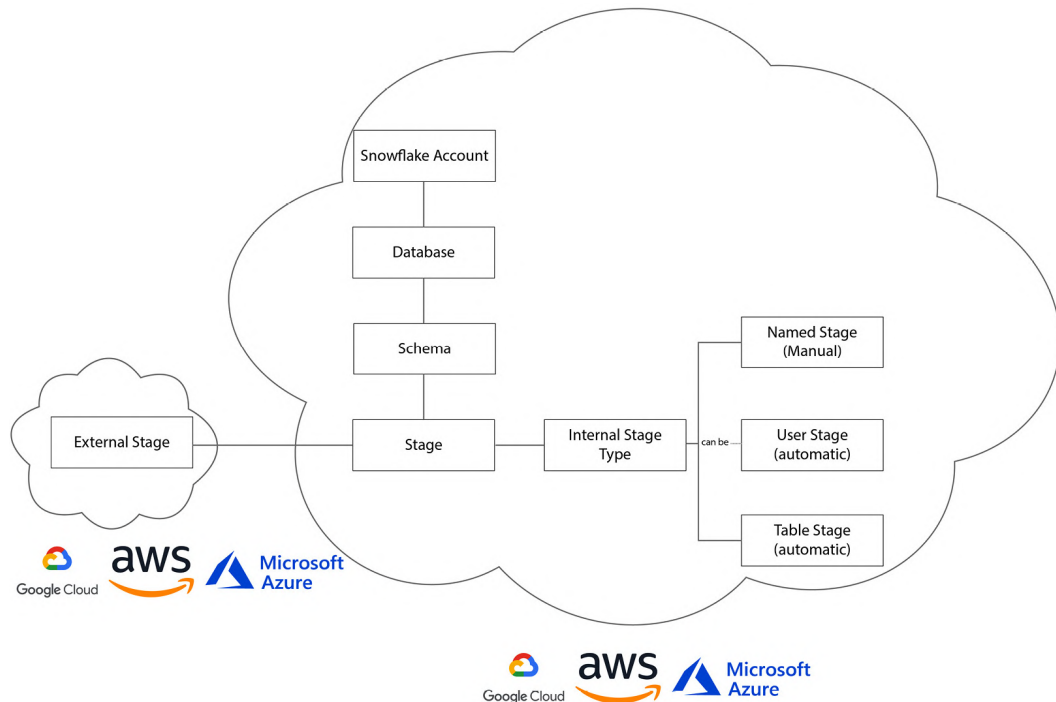


Figure 4.1: External and internal stages in Snowflake

Unlike the data stored in tables, which is micro-partitioned and uses an internal columnar format, stages can store any type of file. Although only certain structured and semi-structured file types are supported for loading data into Snowflake, the visibility of unstructured files in a stage is an important consideration when paired with external and directory tables (discussed later in this chapter).

Stages can store any variety of files and file types. However, a file format must be specified when accessing data from a stage for subsequent loading into a Snowflake table.

File formats

File formats can be thought of as templates describing the file types for reading or writing data to a file. A file format defines properties such as the delimiter type (e.g., comma, tab, or other), date/time formats, and encoding (for example, **Hex Base64**, and **UTF-8**).

Snowflake provides several structured and semi-structured file formats, such as **CSV**, **JSON**, and **Parquet**, with commonly used defaults. However, users are free to create their own named file formats when loading or unloading files of a specified type. File format properties may also be specified at runtime when using the `COPY INTO` command.

A description of supported file types and file formats for data loading is available in the Snowflake documentation: <https://docs.snowflake.com/en/user-guide/data-load-prepare.html>.

For a complete overview of data loading of all kinds, including streaming, see the guide on the Snowflake website: <https://docs.snowflake.com/en/user-guide-data-load.html>.

Having seen how data is loaded from stages into Snowflake using file formats, let's get to know the tables that will be used to store it.

Tables

Data in Snowflake is stored in tables, which, as discussed, are one of the fundamental components of data modeling. However, before exploring them in a modeling context, we should understand the various table types that exist in Snowflake and their costs.

The previous chapter described Snowflake's **Time Travel**, a feature that allows restoring dropped objects or querying data at a prior point in time. However, Time Travel comes with associated storage costs, and the number of available Time Travel days—known as the retention period—depends on the table type, as we'll shortly review in detail.

Snowflake also offers a managed type of Time Travel, known as **Fail-safe**. All permanent tables have a Fail-safe period of seven days. Unlike Time Travel, which the user can access, Fail-safe is managed by and accessible only to Snowflake to protect user data from disasters such as system failures and data breaches. To recover data stored in Fail-safe, users need to contact Snowflake directly.

For tables that offer Time Travel and Fail-safe, the seven-day Fail-safe period begins immediately after the Time Travel period ends. The following figure illustrates the types and duration of data protection that Snowflake tables offer:

Continuous Data Protection Lifecycle

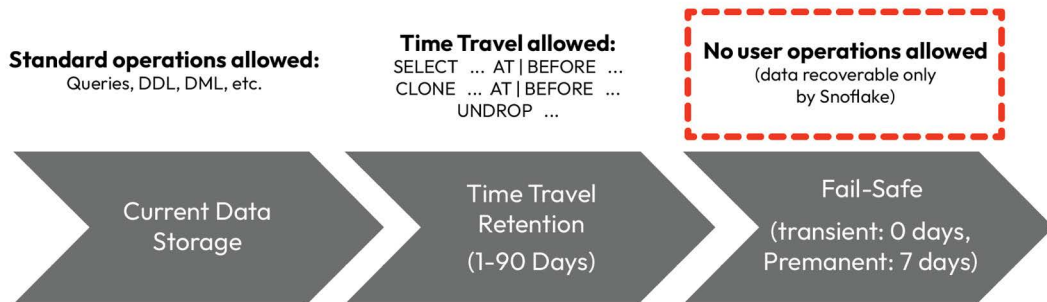


Figure 4.2: Continuous data protection lifecycle

Different use cases call for different backup strategies. Transactional data may require days of history, while a staging table may be needed only temporarily. Snowflake offers several physical table types with varying retention (and associated cost) options. These include the following:

- Permanent
- Transient
- Temporary

Snowflake also provides several metadata table constructs for stage contents. These are semantic objects that exist on top of stages to allow users to query file contents and metadata (such as filenames) just like they would a regular table. These objects are read-only and are offered in the following formats:

- External tables
- Directory tables

The following sections detail the unique properties of each table type.

Physical tables

In Snowflake, data is stored in physical tables logically grouped by schemas. Tables consist of columns with names, data types, and optional constraints and properties (for example, nullability, default value, and primary key).

The DDL command for creating a table is `CREATE TABLE` and is used in the relational modeling scenario to instantiate an empty table with a given structure.

Other variants of the `CREATE TABLE` command are used in the transformational context to create, pattern, or clone tables from existing objects with corresponding data, where applicable. These operations are referred to as *transformational* because they require a relational model/schema to exist beforehand (for example, creating a table by cloning an existing table). The supported variants are as follows:

- `CREATE TABLE AS SELECT`: Commonly referred to as **CTAS**, this creates a table containing the data returned from a query (and using the implied data types).
- `CREATE TABLE USING TEMPLATE`: This creates an empty table using column definitions derived from semi-structured files (currently supported for **Parquet**, **Avro**, and **ORC** formats).
- `CREATE TABLE LIKE`: This creates an empty table using the column definitions of an existing table. Column properties, defaults, and constraints are also copied.
- `CREATE TABLE CLONE`: This does the same as `LIKE` but uses zero-copy cloning to also include all the data from the source table (without actually copying it). The `CLONE` variant can be used with Time Travel to clone the data at a specific point in the past. Cloned tables do not consume storage until the structure or contents are modified in the clone or source objects.

Let's look at the different physical tables offered by Snowflake.

Permanent tables

Snowflake's default table type is a permanent table. This is a slight misnomer since all Snowflake tables except for **temporary** tables persist until explicitly dropped. Permanent tables come with built-in backup features. They include a Fail-safe period of seven days that cannot be modified or disabled and a default of one day of Time Travel. The Time Travel period for permanent tables can be disabled or extended on Snowflake's Enterprise plan to upward of 90 days.

Their extensive recoverability makes permanent tables an ideal candidate for storing business-critical information that would be costly or impossible to regenerate in the event of a catastrophe.

However, not all tables require backups or extended Time Travel periods. In such cases, transient tables should be considered instead.

Transient tables

Unlike permanent tables, transient tables have no Fail-safe period but do come with one day of Time Travel by default. Users can turn off Time Travel by setting it to 0 but cannot extend it beyond 1, even on Enterprise plans.

Transient tables are a good choice when backup and recovery are not a high priority—as in the case of staging tables. The same is true for development and test environments where transient defaults can be set at the schema and database level (described later in this chapter).

While transient tables are ideal for staging data and development environments, Snowflake also offers a temporary table type that persists only for the duration of a user session.

Temporary tables

The data retention properties of temporary tables are identical to those of transient tables, aside from one critical difference: temporary tables are dropped at the end of the user session. A session can be thought of as an active login. Although multiple sessions (such as when using multiple worksheets) can run in parallel, they all terminate along with the connection or login.

Due to their impermanent nature, temporary tables are meant to hold intermediate results for processes during the same session. Temporary tables can also be used for throw-away tests in development environments, saving the user the labor of dropping them explicitly.



Note on temporary table naming

Although temporary tables must be assigned to a schema like all database objects, their session-based persistence exempts them from uniqueness collisions with other table types. A temporary table can have the same name as a non-temporary table and will take precedence over it for the remainder of the session (thereby hiding the non-temporary table).

Let us recap the physical table types and their properties.

Table types summary

Snowflake offers its users many choices when it comes to tables and retention periods. Now that we are acquainted with the three physical tables, it is worth summarizing their properties in a single chart.

Type	Persistence	Fail-safe period (days)	Time-travel period (days)
Temporary	Remainder of the session	0	0 or 1 (default is 1)
Transient	Until explicitly dropped	0	0 or 1 (default is 1)
Permanent	Until explicitly dropped	7	0 or 1 (default is 1) Standard Ed. - or - 0 to 90 (default is configurable) Enterprise Ed.

Figure 4.3: Retention periods for permanent tables

We have gone into detail on Time Travel and recovery, but it is worth mentioning that another type of history can also be enabled for physical tables: change history. Unlike Time Travel, which allows users to access a historical snapshot of a table, change history records individual table changes and enables **change data capture (CDC)**. This functionality is covered in greater detail later in this chapter.

Physical tables are what Snowflake users most often interact with for storing and manipulating information, but other table types can be defined. External and directory tables—metadata tables over stage objects—can be created and modeled relationally. Let's explore these metadata objects in more detail.

Stage metadata tables

External and directory tables exist in Snowflake to allow users to access data in staged files as though selecting from regular tables. External tables are mentioned in this section as they allow primary and foreign keys and can be modeled in a standard relational context. Directory tables are similar to external tables but do not allow the creation of columns or constraints and are included for completeness.

External tables

External tables are metadata objects created on top of stages that allow users to query the data in externally staged files as if it were inside the database. The same file formats supported by a stage are supported for accessing the data through external tables.

The following columns are included for all external tables:

- **VALUE:** A **VARIANT**-type column representing a single row in the external file
- **METADATA\$FILENAME:** Identifies the name of each staged data file in the external table, including its path in the stage
- **METADATA\$FILE_ROW_NUMBER:** Shows the row number for each record in a staged data file

Although read-only, external tables allow users to create virtual columns as expressions over the **VALUE** or **METADATA** column. Querying data through external tables will likely be slower than native tables. It should be used to *peek* at file contents or metadata rather than relying on them as data sources.

External tables enable users to access staged file contents as if they were stored in regular tables. However, when only the file directory information is needed, directory tables can be used.

Directory tables

Conceptually, directory tables resemble external tables; however, there are several notable differences. As opposed to external tables, directory tables have the following differences:

- They are not separate database objects but can be enabled as part of a stage
- They provide file metadata such as path, size, and URL but not file contents
- They do not allow the creation of virtual columns (or constraints)
- They can be created on both internal and external stages

External tables permit users to access data stored in external cloud storage using the same conventions as regular physical tables, but directory tables only show file metadata. Directory tables are more like the **LIST** function—which returns a list of files from a stage—than an actual table. Using a directory table instead of the **LIST** function allows users to use familiar **SELECT** syntax to access the information.

The following command displays how to create a directory table:

```
CREATE STAGE <stage_name>
...
DIRECTORY = (ENABLE = TRUE);
```



Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button

- (1) to quickly copy code into your coding environment, or click the **Explain** button
- (2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {  
  return {sum: a + b};  
};
```

Copy**Explain**

1

2



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR visit packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now that we have covered Snowflake tables in all their incarnations, let's look at views.

Snowflake views

Snowflake views behave like most relational databases; they store a **SELECT** statement over physical objects as an object in a schema. Storing a **SELECT** statement as a shareable database object offers several advantages to users of the system:

- The **SELECT** statement does not need to be written from scratch each time it is required. This provides time savings and maintainability benefits through reusable modular code for data pipelines and analytics.
- It ensures consistency in filters and business logic for everyone with access to the view.
- Views are separate database objects to the data sources they reference and can therefore have different permissions that do not expose all the underlying data.

- It can select data from multiple sources at once.
- It always shows the latest data from source tables without having to refresh it (as opposed to materializing the results as a physical table).
- It has zero storage cost as data is not physically copied but read in real time when the view is called.

These benefits come at the expense of a single trade-off: performance. Each time a view is called, the data must be read from the underlying tables, and all the logic and formulas in the view are calculated using compute resources and consume credits. However, this is not as alarming as it sounds when we consider that Snowflake caching features, described in the previous chapter, also work for views.

Caching

Views attempt to take advantage of Snowflake's various caches to mitigate performance and cost issues when executing a user query.

The results cache, managed by the services layer, holds all query results in the system, including queries executed on views. As long as the underlying data has not changed, subsequent queries will return the results instantly from the cache if they are syntactically equivalent without using compute credits.

If the results cache cannot be used, Snowflake will attempt to take advantage of the warehouse cache when referencing table data through a view. Table data read from cloud storage will be available in memory through the warehouse cache for as long as the warehouse remains active—reducing the data retrieval times for the views that access it.

Security

As mentioned earlier, views can be used to enforce data security as they are standalone securable database objects. Tables containing sensitive data—at either the row or column level—can be restricted from public access through views. Views can limit what columns are selected and what filters are applied and have separate access roles assigned to them.

Snowflake users have access to an added level of security by using **secure views**. A secure view—declared like a regular view, but with the word `secure` in the `CREATE` statement—functions like a non-secure (regular) view except that the view definition (DDL) is not visible to the consumers, but only to the role that owns the view. Some internal optimizations are disabled for secure views to not expose the underlying data, but performance is generally comparable to non-secure views.

Now that we understand tables and views, let's get to know materialized views—a Snowflake object that offers benefits from both of these object types.

Materialized views

Not exactly views, yet not quite tables, materialized views—an Enterprise Edition feature—sit at the intersection of these two object types and offer interesting benefits with one important caveat.

As the name suggests, materialized views are actually physical tables that store the results of a view. When the underlying data changes, the result is re-materialized automatically. This means that materialized views offer the performance of a table combined with a cached query result while offering all the security benefits of a regular view.

The only trade-off is the cost. Since materialized views store query results in physical tables, they incur related storage costs. Additionally, materialized views will use credits when refreshing the data.

Materialized views have certain limitations in the logic they permit in the `SELECT` statement compared to regular views. Chief among them is that materialized views can only be built over a single data source (stages are allowed), so they cannot use joins. This limitation reduces the use case of materialized views to that of precalculated table results or aggregates. On the plus side, the Snowflake optimizer is smart enough to automatically redirect queries on the underlying table to materialized views whenever possible.

Snowflake users should consider materialized views if *all* of the following conditions are met but discount them if *any* are not:

- The query results do not change often
- The view results are used significantly more frequently than data changes occur
- The query consumes a lot of compute resources

To create a materialized view, use the following command:

```
CREATE MATERIALIZED VIEW <view_name>
```

Having covered Snowflake's various data source options, let us focus on detecting their data changes and automating transformational pipelines between them.

Streams

Streams are logical objects that capture data changes in underlying sources, including the previously mentioned objects (physical tables, views, and external and directory tables). Whenever a DML operation occurs in the source object, a stream tracks the changes (inserts, deletions, and the before/after images of updates). Streams achieve this through an *offset storage* technique—logically taking an initial snapshot of data and then tracking changes through metadata columns. Although a stream can be queried like a table, it is not a separate object and does not contain table data.

When a stream is created, metadata columns are tacked onto the source object and begin tracking changes. The following table describes the metadata fields and their contents:

Column	Allowed Values	Description
METADATA\$ACTION	INSERT, DELETE	Denotes the action performed on the row. New rows are marked as INSERT, while updates are separated into an INSERT row with new values and DELETE the row with old values.
METADATA\$ISUPDATE	TRUE, FALSE	A value of TRUE indicates that the INSERT and DELETE actions are the results of an update. Deleted (not updated) rows have a value of FALSE.
METADATA\$ROW_ID	40-character hash value	Snowflake assigns a unique ID to every table record, which can be seen here.

Figure 4.4: Stream metadata columns

The following command creates a stream on a table:

```
CREATE STREAM <stream_name> ON TABLE <table_name>
```

For every subsequent DML operation, the stream holds the before and after images of the changed records and, by pairing them with the relevant metadata that identifies the type of DML operation (insert, update, or delete), enables accurate CDC to downstream objects.

Loading from streams

Snowflake keeps streams manageable and prevents them from growing excessively through an elegant method of consumption. Consuming—or using stream contents to (successfully) load or update downstream sources—empties the stream, readying it to absorb future DML changes without requiring manual periodic pruning. Selects and failed consumption attempts will not clear the stream.

In the following diagram, let's look at how this works in practice across various DML operations. This example assumes that the stream is consumed (emptied) after each change to the source table.

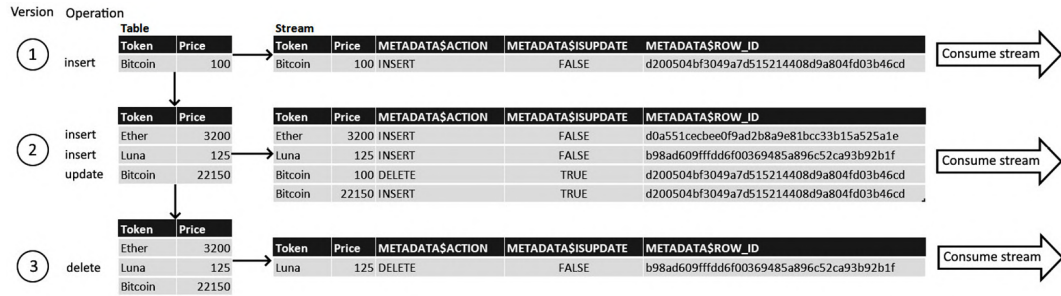


Figure 4.5: Consuming data from a stream after table changes

Depending on the nature of the data being tracked, streams are available in two varieties:

- **Standard** (or delta): Records inserts, deletes, and updates. This type of stream is supported for tables, directory tables, and views.
- **Append/insert-only**: Only tracks inserted records and ignores any updates or deletions. This stream type is supported for physical tables, directory tables, and views as *append-only* and, for external tables, as *insert-only*.

Being able to track changes without any processing overhead associated with manually tracking and comparing versions simplifies downstream DML operations by anticipating the type of change that needs to be updated. Due to this, streams are frequently used when modeling fact tables and **slowly changing dimensions (SCDs)** and keeping them up to date (these objects and the techniques used to maintain them are covered in later chapters).

Streams are intended to be consumed regularly. The default retention period for changes in a stream is 14 days. Although this can be extended, Snowflake cannot guarantee stream freshness beyond that period. However, there is a less volatile alternative, one that doesn't disappear when consumed.

The following section describes change tracking and how it can be used in conjunction with (or as an alternative to) streams for CDC.

Change tracking

While a stream is created as a standalone object in a schema, change tracking is enabled directly on tables, allowing Snowflake users to query CDC metadata. Change tracking uses the same metadata fields found in streams but appends them directly to a table. Unlike streams, the changes are not eliminated if they are used to update downstream objects; instead, the change tracking persists for the data retention time of the table.

Here is an example of how change tracking is enabled and queried for a table. In this example, three records are inserted into a change tracking-enabled table. Subsequently, one record is updated and another is deleted:

```
create or replace table myTable ( myTable_id varchar(10), my_date date );
-- Enable change tracking alter table myTable set change_tracking = true;
-- Initialize a session variable for the current date/time --create
timestamps set cDts = (select current_timestamp()); set cDate = (select
current_date());
--perform DML insert into myTable (myTable_id,my_date) values
('yesterday', $cDate-1 ), ('today' , $cDate+0 ), ('tomorrow' , $cDate+1 );
delete from myTable where myTable_id = 'yesterday';
update myTable set my_date = '1970-01-01' where myTable_id = 'today';
-- Query the change tracking metadata to observe the delta -- from the
timestamp till now select * from myTable changes(information => default)
at(timestamp => $cDts);
```

The result returned from querying the table's change log shows the cumulative effect (latest version) of the record (i.e., yesterday is absent, and today shows the updated value).

	 MYTABLE_ID	 MY_DATE	 METADATA\$ACTION	 METADATA\$ISUPDATE	 METADATA\$ROW_ID
1	today	1970-01-01	INSERT	FALSE	e160d63f9f840f270149
2	tomorrow	2025-04-29	INSERT	FALSE	3c9275e60380c089cbcf

Figure 4.6: Query results from default table change tracking



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



But watch what happens when the same change log is queried in append-only mode:

```
-- Query the change tracking metadata to observe
-- only inserts from the timestamp till now
select * from myTable
changes(information => append_only)
at(timestamp => $cDts);
```

The result only returns records that were inserted into the table, regardless of subsequent deletions or updates (i.e., all three values are present, and today shows the original value).

	MYTABLE_ID	MY_DATE	METADATA\$ACTION	0 1 METADATA\$ISUPDATE	METADATA\$ROW_ID
1	yesterday	2025-04-27	INSERT	FALSE	68b5491cf78900f2112c
2	today	2025-04-28	INSERT	FALSE	e160d63f9f840f270149
3	tomorrow	2025-04-29	INSERT	FALSE	3c9275e60380c089cbcf

Figure 4.7: Query results from table change tracking in append-only mode

Barring some limitations, change tracking can even be enabled for views, as long as the underlying tables have it enabled as well. Since change tracking metadata persists for the entire data retention period of a table and cannot be cleared when consumed, it is not suited for automated CDC pipelines but for analyzing the changes themselves.

Speaking of automated data pipelines, the final object that we will tackle in this chapter is designed to handle this crucial task and plays very nicely with streams and CDC detection.

Tasks

Snowflake uses tasks to schedule and automate data loading and transformation. Although data movement is not tracked in relational modeling, it is an integral part of transformational modeling and is covered here for completeness.

Tasks automate data pipelines by executing SQL in serial or parallel steps. Tasks can be combined with streams for continuous ETL workflows to process recently changed table rows. This can be done serverlessly (using auto-scalable Snowflake-managed compute clusters that do not require an active warehouse) or using a dedicated user-defined warehouse.

The code for creating a task is as follows:

```
CREATE TASK <task_name>
...
[ AFTER <parent_task_1> [ , <parent_task_2> , ... ] ]
[ WHEN <boolean_expr> ]
AS <sql>
```

Tasks are simple to understand—they run a SQL command (or execute a stored procedure) on a schedule or when called as part of a parent task. The following figure shows how tasks can be chained serially and in parallel to form dependencies for data pipelines.

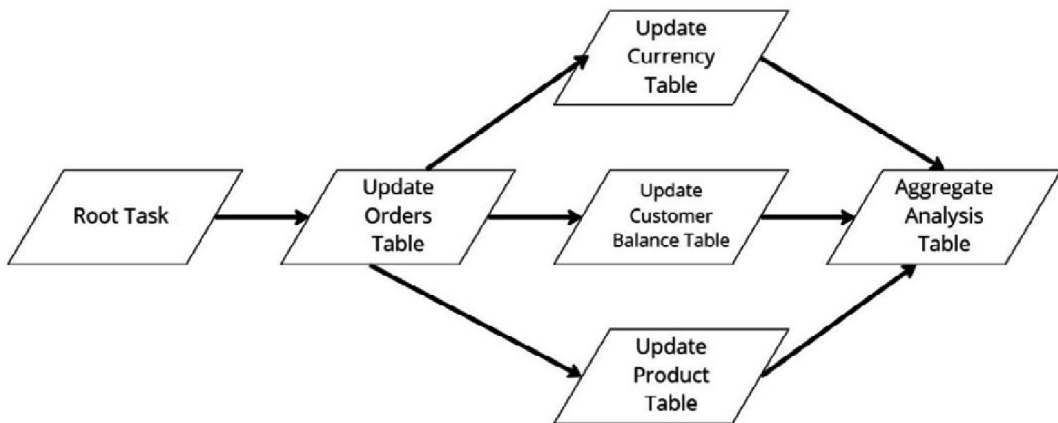


Figure 4.8: A task tree with serial and parallel dependencies

Stored procedures and scripting



Stored procedures and Snowflake scripting allow you to write procedural code using looping and branching constructs and fall outside the context of this book. For information on working with procedural logic, refer to the Snowflake documentation on stored procedures (<https://docs.snowflake.com/en/sql-reference/stored-procedures.html>) and Snowflake scripting (<https://docs.snowflake.com/en/developer-guide/snowflake-scripting/index.html>).

Now that we understand how tasks can be chained to form data pipelines within Snowflake, we can take them even further by combining them with streams.

Combining tasks and streams

Tasks pair nicely with streams and allow users to check for changes before executing. As part of their definition, tasks have a `WHEN <boolean_expr>` clause, which sets a logical condition that needs to be met before it can be executed. The condition must return a Boolean value but can run any function or sub-select that the user wishes. Fittingly, streams come with a system function that returns a value of `TRUE` whenever there are CDC records present and, otherwise, `FALSE`. The function can be called, referencing the stream name as follows:

```
SYSTEM$STREAM_HAS_DATA('<stream_name>')
```

Setting this condition at the start of a task means that it can be scheduled to run as frequently as changes are expected but will only be executed when there are records to process.

Let's summarize what we've learned in this chapter.

Summary

In this chapter, we demonstrated that Snowflake objects pack a lot of features, even behind familiar ones such as tables and views. A table in Snowflake can store more than just the data—depending on its settings, it can also hold months of historical and disaster recovery backups and offer offset change tracking for CDC. Views, likewise, exceed expectations by providing change tracking and automated re-materialization.

We saw how stages mark the entry point for data to make its way from external sources to Snowflake tables. Stages also provide helpful features, such as external table access for reading file contents without copying them to a table beforehand.

Finally, to coordinate incoming data, establish automated ELT pipelines, and streamline CDC, Snowflake pairs tasks with streams to give its users full serverless or managed control—tying stages, tables, views, and all the connective transformational logic together.

Having understood the strengths and capabilities of each object, we will open the next chapter by discussing the building blocks of data modeling and tying them to their Snowflake object counterparts: columns, tables, constraints, and others. These fundamentals will lay the groundwork to explore each of the four modeling types in detail later in the book.

References

You can refer to the following resource for additional information:

- *Understanding & using Time Travel* (Snowflake documentation): <https://docs.snowflake.com/en/user-guide/data-time-travel.html>

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

5

From Logical Concepts to Snowflake Objects

This chapter will bridge the gap between conceptual and logical database modeling concepts and physical Snowflake implementations. You'll discover how to translate conceptual designs into efficient data structures, ensuring your physical models are clear and optimized for performance. Understanding the nuances of Snowflake's underlying storage architecture, including micro-partitions and clustering, will enable you to design with scalability and efficiency in mind.

As part of physical design, we'll also explore Snowflake's features for handling complex data types, constraints, and relationships. Whether you're dealing with structured or semi-structured data, this chapter will equip you with the tools to harness Snowflake's full potential, making data management simpler, faster, and more effective.

In this chapter, we're going to cover the following main topics:

- Ensuring efficient designs by understanding how entities are represented as tables and how Snowflake manages the underlying storage
- Exploring the benefits of proper data typing for attributes
- Understanding database constraints and managing them in Snowflake
- Learning how to reap the benefits of identifiers and primary keys
- Using alternate keys when multiple primary key candidates exist
- Tying the model together through foreign key relationships
- Specifying and checking for mandatory columns

Let us start by revisiting the foundational modeling concepts covered in the previous chapters while tying them to their corresponding Snowflake objects.

Entities as tables

Before delving into database details, let's recall the concept of an entity at the business level: *a person, object, place, event, or concept relevant to the business for which an organization wants to maintain information*. In other words, an entity is a business-relevant concept with common properties. A rule of thumb for identifying and naming entities is that they conform to singular English nouns, for example, customer, item, and reservation.

The obvious candidate for storing and maintaining information in Snowflake is a table. Through SQL, tables give users a standard and familiar way to access and manipulate entity details. As we saw in the last chapter, Snowflake tables come in several flavors, offering different backup and recovery options. Besides selecting a table type that provides adequate Time Travel and Fail-safe, Snowflake tables live up to the company's claim of near-zero maintenance—there are no indexes, tablespaces, or partitions to maintain by hand.

The only two facets of storage over which users have control are the micro-partitions that Snowflake uses to store data in its internal columnar format and how they are clustered. Let's look at the two facets in more detail.

How Snowflake stores data

Data in Snowflake tables is stored in a columnar format using continuous storage units called **micro-partitions**. Typically, databases divide large tables into physical partitions to improve performance and scalability. However, physical partitions require maintenance and result in data skew (some partitions growing larger than others).

Snowflake escapes these limitations by keeping the micro-partitions small (around 50-500 MB uncompressed) and managing them transparently through the services layer. Micro-partitions are immutable, so **data manipulation language (DML)** operations are more efficient and always create new partitions. Due to their small size, granular statistics can be kept for individual columns within a micro-partition, and each one compresses optimally based on its data type. Unlike traditional partitioning, micro-partitions can overlap in their value ranges (but a single record can never be split across multiple partitions).

Micro-partitions are created by the natural ordering of data as it is loaded into a table. For example, if new transactions are loaded daily into a `SALES` table, `transaction_date` would form the natural partition of the table. If, on the other hand, the table was sorted by `client` instead, then partitions by `client` would be formed, and `transaction_date` values would overlap.

The main benefit of micro-partitions is that their metadata (ranges of values, distinct counts, and others) allows the Snowflake optimizer to prune user queries. Pruning avoids searching through micro-partitions where filter values are known not to exist. For example, even if the `SALES` table from the previous example were to grow to millions of records and span thousands of micro-partitions, query results could still be returned in milliseconds by pruning unnecessary partitions and only scanning those whose value ranges match the query filter conditions.

Using a simplified conceptual example, let us see micro-partitions in action. The following table contains data about superheroes and their creators. This is how it would appear to the user:

Superhero	Date Debuted	Creator	First Appearance
Dick Tracy	1931-10-04	Chester Gould	Dick Tracy
Conan the Barbarian	1932-12-01	Robert E. Howard	The Phoenix on the Sword Weird Tales
Flash Gordon	1934-01-07	Alex Raymond	Newspaper Comic
The Phantom	1936-02-17	Lee Falk	The Phantom
Sheena, Queen of the Jungle	1938-01-01	Will Eisner and S.M. "Jerry" Iger	Wags #46
Superman (Clark Kent)	1938-06-01	Jerry Siegel, Joe Shuster	Action Comics #1
Arrow	1938-09-01	Paul Gustavson	Funny Pages vol. 2 #10
Batman (Bruce Wayne)	1939-05-01	Bob Kane, Bill Finger	Detective Comics #27
The Spirit	1940-06-02	Will Eisner	The Spirit
Spider-Man (Peter Parker)	1962-08-01	Stan Lee, Steve Ditko	Amazing Fantasy #15

Figure 5.1: The logical view of Snowflake table data

Internally, two micro-partitions are created, and the data is compressed using a columnar format.

Micro Partition 1						Value Ranges (Example of Partition Metadata)
Superhero	Dick Tracy	Conan the Barbarian	Flash Gordon	The Phantom	Sheena, Queen of the Jungle	Dick Tracy / Sheena, Queen of the Jungle
Date Debuted	1931-10-04	1932-12-01	1934-01-07	1936-02-17	1938-01-01	1931-10-04 / 1938-01-01
Creator	Chester Gould	Robert E. Howard	Alex Raymond	Lee Falk	Will Eisner and S.M. "Jerry" Iger	Alex Raymond / Will Eisner and S.M. "Jerry" Iger
First Appearance	Dick Tracy	The Phoenix on the Sword Weird Tales	Newspaper Comic	The Phantom	Wags #46	Dick Tracy / Wags #46

Micro Partition 2						
Superhero	Superman (Clark Kent)	Arrow	Batman (Bruce Wayne)	The Spirit	Spider-Man (Peter Parker)	Arrow / The Spirit
Date Debuted	1938-06-01	1938-09-01	1939-05-01	1940-06-02	1962-08-01	1938-06-01 / 1962-08-01
Creator	Jerry Siegel, Joe Shuster	Paul Gustavson	Bob Kane, Bill Finger	Will Eisner	Stan Lee, Steve Ditko	Bob Kane, Bill Finger / Will Eisner
First Appearance	Action Comics #1	Funny Pages vol. 2 #10	Detective Comics #27	The Spirit	Amazing Fantasy #15	Action Comics #1 / The Spirit

Figure 5.2: A Snowflake table divided into two micro-partitions

Since the table is ordered by Date Debuted, when the services layer breaks the table up into micro-partitions, this column will have a natural clustering of values. While this example uses only two, Snowflake tables can be composed of millions of micro-partitions depending on their size.

Using the partition metadata, only some of which is shown in Figure 5.2 (column value ranges), Snowflake's query optimizer can prune or ignore partitions depending on the query filters.

In the following example, the optimizer can safely skip reading *partition 1* because its value range (known from the metadata) does not correspond to the filter condition.

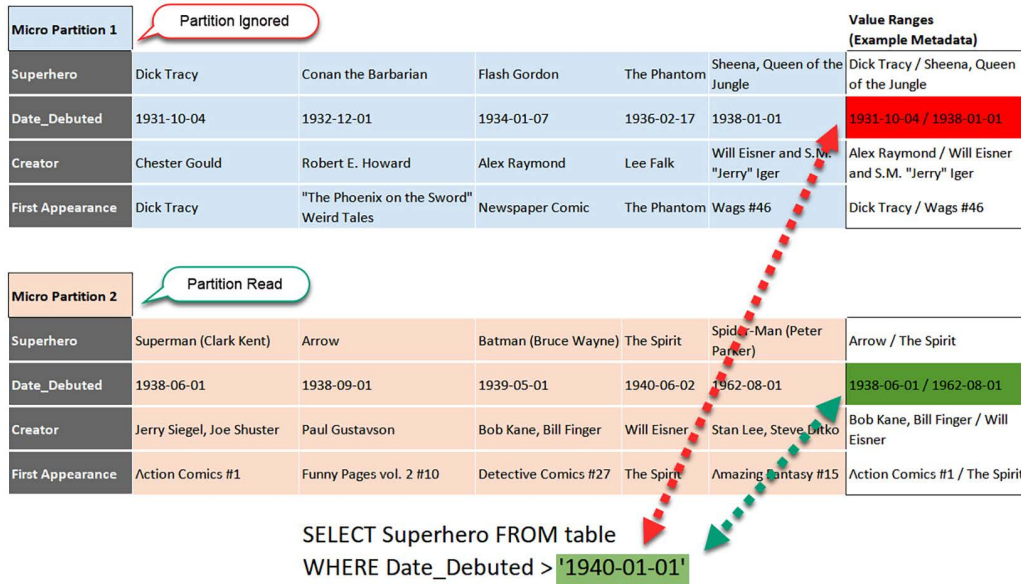


Figure 5.3: Successful query pruning

If the optimizer cannot eliminate any micro-partitions based on their value ranges, then all the partitions and their contents need to be read to find matching records.

In the following example, query pruning cannot be performed because both partitions match the search condition based on their value range, even though the creator Paul Gustavson only exists in *partition 2*:

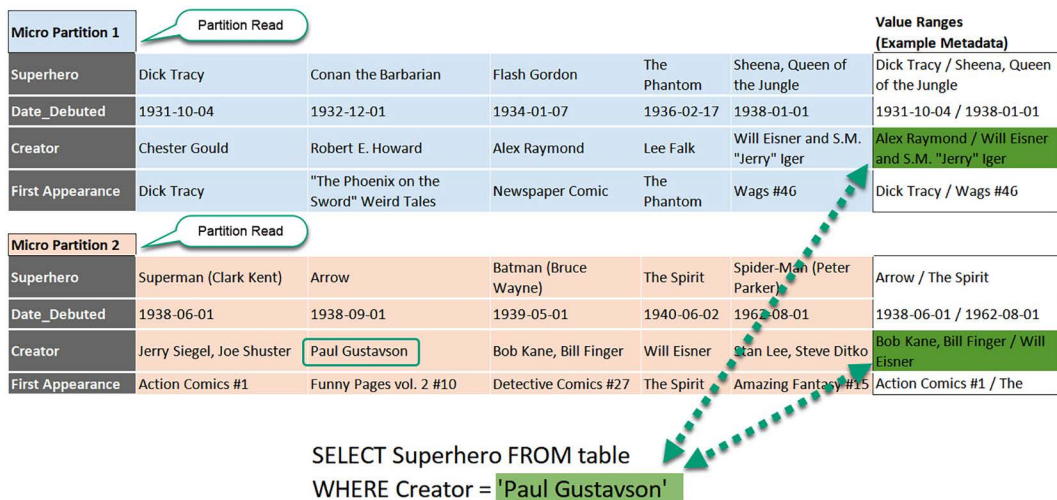


Figure 5.4: Query pruning cannot be performed with a given filter

How Snowflake manages micro-partitions depends on the order in which data is loaded into a table. Data can be ordered explicitly upon insertion using the `ORDER BY` clause or implicitly by the natural load pattern (for example, by `CREATED_DATE` or `LOAD_DATE`). When partitioning matches search filters, query performance is improved through pruning. For example, in a table with millions of rows, where data is loaded by `LOAD_DATE` and queried for `LOAD_DATE = CURRENT_DATE`, performance will be identical to a table that only held data for `CURRENT_DATE`. However, discounting this filter or searching for any other attribute would likely not benefit from query pruning.

As shown in the preceding examples, micro-partitioning creates multiple data segments with varying degrees of overlapping column values. This data segmentation, known as clustering, can be used strategically to organize the table structure and improve query performance. Let's understand how to use it to our advantage.

Clustering

As table data is broken up into micro-partitions, column values naturally group themselves into data segments. The degree of variance of column values and their overlap across micro-partitions is known as **clustering**. The less overlap is encountered between partitions, the higher the degree of query and column pruning that the optimizer can provide.

In addition to column metadata for micro-partitions, the Snowflake services layer also stores clustering statistics such as the following:

- The total number of micro-partitions in each table
- The number of micro-partitions containing values that overlap with each other (in a specified subset of table columns)
- The depth of the overlapping micro-partitions

The following (simplified) example shows how three micro-partitions can overlap in their clustering range. An actual table could have many more partitions and columns, and achieving a perfect separation is neither likely nor required for improving query performance.

Clustering Depth

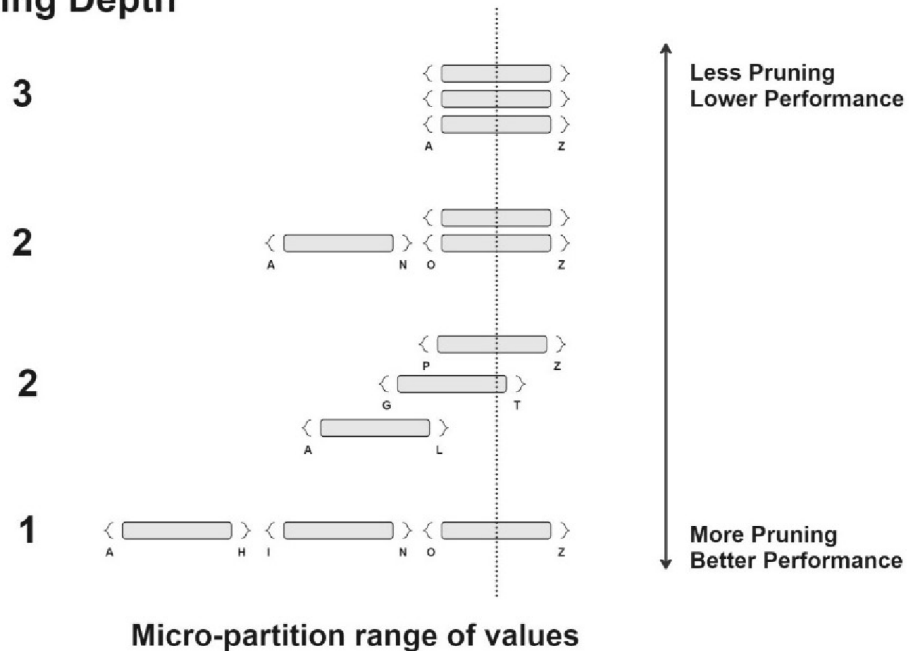


Figure 5.5: Partition overlap and clustering depth

When considering how data is loaded and queried from a table, the clustering depth or degree of overlap (less is better for pruning and performance) should be considered as tables reach a considerable size. Although ordering a table across one or multiple dimensions might improve query performance through pruning, the cost of the sorting operation may offset those gains.

Generally, even tables with several million rows would provide adequate performance without any sorting or ordering applied to the rows. However, query performance may degrade after a certain table size if the load pattern does not match the search pattern (for example, data is loaded by `TRANSACTION_DATE` but queried by `CLIENT`).

Users can assess query pruning information using Snowflake's visual query profiler. The total number of micro-partitions is displayed on the screen alongside the number of partitions accessed during a query.

The following screenshot shows that the number of scanned partitions equals the total—meaning the optimizer could not perform any pruning.

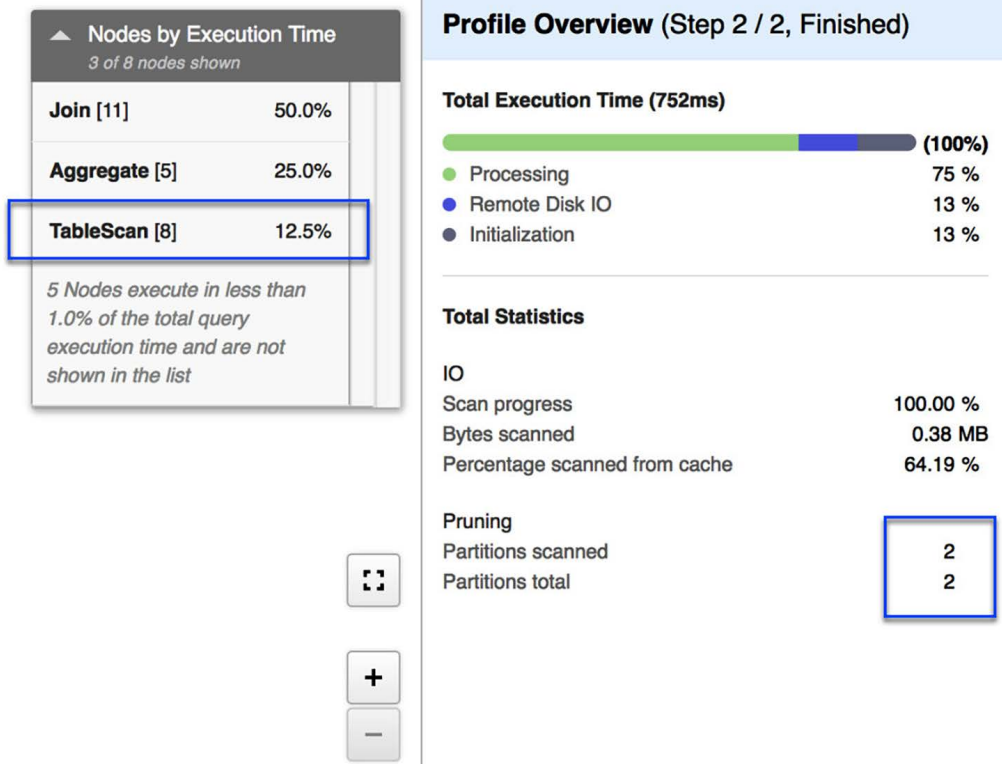


Figure 5.6: Evaluating pruning using the query plan

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



As stated earlier, a lack of pruning alone is not an indicator of poor query performance. Query duration is subjective due to factors such as caching, query complexity, and data volume. However, when query performance fails to meet expectations, factors such as a high table scan percentage, as shown in *Figure 5.6*, relative to other query operations and a lack of pruning, may indicate a need to recluster the table.

Reclustering can be performed manually by recreating the table while applying the required sorting or can be handled automatically by Snowflake.

Automatic clustering

Snowflake automatically handles clustering through the natural loading pattern of the incoming data. However, on large tables (multi-terabyte and above), DML operations may degrade the clustering quality on desired columns and impact query performance.

Due to the large table size, manual re-sorting of the table would be costly, so Snowflake provides the option of automated reclustering of a table through a specified clustering key. A **clustering key** is one or more columns (or an expression) that Snowflake uses to automatically sort the data in a given table and transparently re-sort it when necessary.

The benefits of defining a clustering key include the following:

- Improved query performance through pruning by skipping data that does not match query filters
- Better column compression and reduced storage costs
- Maintenance-free automated reclustering performed by Snowflake

However, remember that automated reclustering operations incur the same processing costs as those performed manually. Therefore, only consider setting a clustering key if performance is prioritized over cost or the clustering performance offsets the credits required to maintain it. The following considerations help determine when the latter is likely to occur:

- The table contains multiple **terabytes (TB)** of data and many micro-partitions.
- The queries can take advantage of clustering. Typically, this means that one or both of the following are true:
 - The queries need to read only a small percentage of rows in the table
 - The queries sort the data (for example, using an `ORDER BY` clause)
- A high percentage of queries can benefit from the same clustering key by selecting or sorting on the same few columns.

Further information on clustering depth can be found in the Snowflake documentation (<https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions.html>).

Entities are the main elements of a relational model. In this section, we saw how to translate an entity into a physical table in Snowflake and the design considerations to contemplate for optimal performance and storage. We will expand on the entity in the next section by adding properties.

Attributes as columns

Recall from the previous section that an entity is a business-relevant concept for which an organization wishes to maintain information. Recall that attributes—defined with the business team during conceptual modeling or loaded from existing source data during the ETL process—are properties that describe the entity and are stored as columns. Attributes can be descriptive (such as NAME, ADDRESS, and QUANTITY) or metadata (such as ETL_SOURCE and LOAD_DATE).

The nature of the attribute—whether numeric, string, date, or other—is an essential detail for understanding the business requirement at the conceptual level and selecting the right data type at the physical level. Snowflake offers basic data types found in other databases (such as VARCHAR, DATE, and INTEGER) and less-common ones (such as VARIANT and GEOGRAPHY), which offer exciting possibilities for modeling and working with table contents.

Let us get to know Snowflake data types and their unique properties so that we can adequately assign them to entity attributes.

Snowflake data types

Snowflake is an ANSI-compliant database and supports the common SQL data types for storing strings, dates, and numbers. In addition, Snowflake provides more advanced types for storing semi-structured and geospatial data. When we discussed micro-partitions, column compression (based on data type) was mentioned as a critical factor in performance and cost management. However, using data types correctly can also yield usability benefits when querying table contents.

Before continuing, please review the list of Snowflake data types in the documentation (<https://docs.snowflake.com/en/sql-reference/intro-summary-data-types.html>) unless you're already familiar with them.

Some data types offer properties that can be applied during table creation, such as collation and autoincrement. Collation lets the user set options such as case, accent sensitivity, and space-trimming for string comparisons. For numeric data types, identity and autoincrement options generate unique sequences that come in handy for surrogate keys (described in a later section).

Besides storage and compression, adequate typing of attribute columns offers advantages from a usability perspective by ensuring consistent formatting during data loading (for example, no stray characters in numeric columns or improper date formats in date). Beyond that, proper typing unlocks all relevant data type functions that Snowflake supports, such as `DATEDIFF` (calculates the difference between two dates) for dates and `STRTOK` (tokenizes a given string and returns the requested part) for strings. The complete SQL function reference can be found in the Snowflake documentation (<https://docs.snowflake.com/en/sql-reference-functions.html>).

Storing strings in Snowflake



Snowflake uses the `VARCHAR` data type to store string values. The maximum length of a `VARCHAR` column is 16 MB, or 16,777,216 characters/bytes. How much data fits in 16 MB? How about the contents of Tolstoy's *War and Peace* (587,287 words) four times over? Interestingly, the 16 MB length is also the default. But do not panic. Snowflake efficiently compresses column contents to their exact lengths and does not bill for the 16 MB maximum. However, sometimes it makes sense to explicitly limit a column to a fixed set of characters to inform database users that its contents are not meant to exceed the specified setting (for example, setting a column containing ISO currency codes to `VARCHAR(3)`).

Now, let us turn from structured columns to semi-structured data types.

Storing semi-structured data

Snowflake is widely recognized for its ease and agility in handling semi-structured data. There are three dedicated data types available for storing semi-structured information (`VARIANT`, `ARRAY`, and `OBJECT`), and aside from unique functions available to each (to be discussed shortly), they allow users to query their semi-structured contents using native SQL and without having to convert or flatten the data beforehand. Querying semi-structured data is discussed in later chapters and explained in the Snowflake documentation (<https://docs.snowflake.com/en/user-guide/querying-semistructured.html>).

Snowflake tables can consist of structured and semi-structured data types, which can be queried natively in the same `SELECT` statement. However, there are some differences between the three semi-structured data types and how they are used, so let us understand what they are:

- **VARIANT:** This is the recommended data type for hierarchical semi-structured data that allows users to load and query JSON, Avro, ORC, and Parquet data without converting or explicitly describing their structure. `VARIANT` has a maximum length of 16 MB and can store a value of any other data type (except `VARIANT`). Common uses of the `VARIANT` data type include email logs, web activity, and event data.

- **OBJECT:** A Snowflake OBJECT is analogous to a JSON object or dictionary, hash, or map, as it is known in other programming languages. An OBJECT is a key-value pair, where the key is VARCHAR text that identifies the value and the VARIANT-type value itself. An OBJECT is used when the keys (names) convey meaningful information (for example, a list of countries by ISO currency code used). An OBJECT has a maximum size of 16 MB and can store a value of any other data type, including OBJECT.
- **ARRAY:** A Snowflake ARRAY is similar to arrays in other programming languages. An array contains zero or more (the length is dynamic and not set explicitly) elements and references to their positions. Unlike an OBJECT, whose elements are accessed by (key) name, ARRAY elements are referenced by their positions. This makes an ARRAY data type suitable for looping or processing data in natural/chronological order. An OBJECT has a maximum length of 16 MB and can store a value of any other data type, including ARRAY. However, ARRAY positions contribute to the 16 MB limit, so their effective size is smaller.

We have only briefly touched on the available options and functions surrounding Snowflake's various structured and semi-structured data types. For further reading on Snowflake data types, the following documentation provides a sectioned reference for each of them: <https://docs.snowflake.com/en/sql-reference/data-types.html>.

Having understood data types for defining attribute columns, we can move on to learning about the various constraints that can be defined on them. However, before we can explore constraints in detail, we need to highlight one crucial factor regarding Snowflake's management of them.

Constraints and enforcement

The remainder of this chapter deals with table constraints, so it is necessary to understand what they are and to mention several important details regarding their use in Snowflake. In the ANSI-SQL standard, constraints define integrity and consistency rules for data stored in tables. Snowflake supports four constraint types:

- PRIMARY KEY
- UNIQUE
- FOREIGN KEY
- NOT NULL

Since the function of each of these constraints is covered later in this chapter, this section will be limited to their enforcement.

Enforcement, on the part of the database, means actively monitoring the integrity rules of a given constraint when DML operations are performed on a table. By enforcing a constraint, a database ensures that an error is raised when the constraint is violated, and the offending DML operation is not allowed to complete.

For example, a NOT NULL constraint on a column indicates that this column cannot contain NULL values. By enforcing this constraint, Snowflake raises an error if an operation tries to insert or update a NULL value in that column.

Before proceeding, it's important to distinguish between hybrid tables and other types of tables. Hybrid tables, which are discussed in more detail in *Chapter 6*, enforce all constraints—prioritizing data consistency at the expense of DML performance. In contrast, standard tables only enforce NOT NULL constraints, while the other three constraints serve as guidelines rather than strict rules. While these guidelines can be violated without causing an error, they help inform database users about the intended design and relationships of the table. More details on this will be covered later in this chapter.

With this in mind, let us look at the PRIMARY KEY constraint so we can understand what function this constraint serves both with and without enforcement.

Identifiers as primary keys

Tables store information for business entities using attributes of relevant data types. A row in the CUSTOMER table holds information for a given customer, and a row in the ORDERS table represents an order—or does it? Perhaps in this example, orders can contain multiple products and span just as many rows. To determine a unique instance of an entity, an **identifier**—or **primary key (PK)**, if referring to a physical database—is used.

A PK is a column or set of columns whose values uniquely determine an instance of an entity. Only one PK can be defined per table. From a business perspective, a PK represents a single entity instance. To return to the previous example—is an order a single row containing one product, or does our organization allow multiple products per order? PKs inform database users of what that reality looks like at the table level.

The following figure shows some sample data from a fictitious ORDERS table.

	O_ORDERSKEY	O_ORDERKEY	O_PRODKEY	O_ORDERSTATUS...	O_TOTALPRICE	O_ORDERDATE	O_ORDERPRIORITY
31	30	3,639,712	143,420	F	239,880.58	1993-07-18	3-MEDIUM
32	31	2,533,890	142,628	O	160,918.7	1998-06-26	1-URGENT
33	32	3,909,985	12,454	F	140,327.59	1993-06-07	3-MEDIUM
34	33	2,895,783	74,347	F	181,920.99	1993-08-30	2-HIGH
35	34	2,445,186	139,993	O	194,939.31	1997-09-02	2-HIGH
36	35	520,964	54,752	O	159,309.83	1997-05-26	5-LOW
37	36	419,684	11,422	F	112,659.1	1992-06-03	3-MEDIUM
38	37	1,644,164	21,691	O	247,340.72	1996-04-26	5-LOW
39	38	61,382	37,204	F	162,991.79	1992-05-11	4-NOT SPECIFIED

Figure 5.7: Sample data from an ORDERS table

There appear to be no duplicate values for the O_ORDERKEY column, but this is a small sample from a table with millions of records. It is impossible to tell by looking at the data whether an order is uniquely identified by O_ORDERKEY only, O_ORDERKEY in conjunction with O_PRODKEY, or some other combination of columns (when a PK consists of multiple columns, it is called a **composite** or **compound key**).

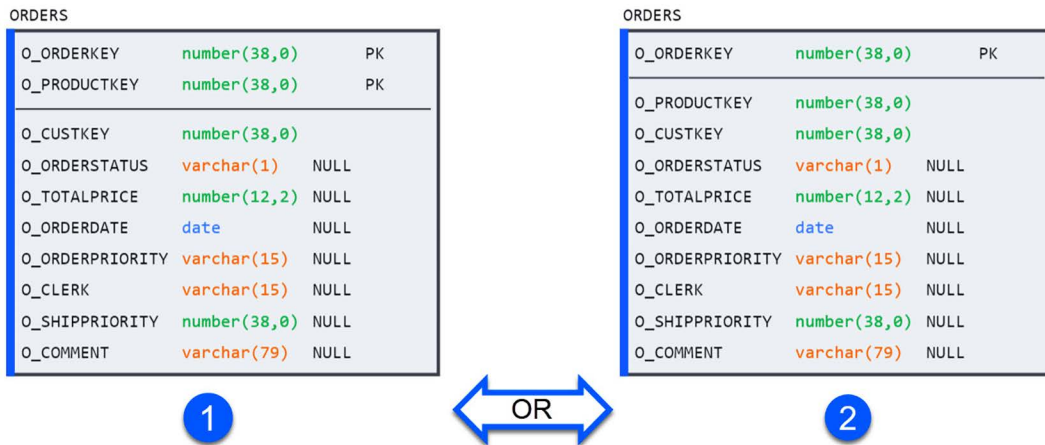


Figure 5.8: Single or compound PK defined on the ORDERS table

Even reviewing every record in the table and finding no duplicates does not guarantee that one won't arrive later if the business model and the corresponding data model are configured as per scenario 2 in Figure 5.8. Only by establishing a conceptual identifier and defining it physically in the corresponding Snowflake table can users definitively know how to isolate a unique entity instance within it.

Having understood how a PK is used to identify a unique record, let us discuss the practical advantages of using it.

Benefits of a PK

The most compelling argument in favor of PKs comes not from a textbook but from *Too Many Daves*, a children's poem by Dr. Seuss. The poem tells of Mrs. McCave and her poor lifestyle choices, such as having 23 children and naming all of them Dave. Henceforth, without a unique identifier, Mrs. McCave is unable to isolate a singular instance of Dave—an all-too-familiar problem for many database users (<https://allpoetry.com/poem/11575842-Too-Many-Daves-by-Theodor-Seuss-Geisel>).

Besides isolating the Dave you are looking for, there are other compelling reasons to define PKs.

Determining granularity

Once data has been populated in a table, the need to query individual entities or aggregate entity statistics immediately follows. However, how can the users tell how many rows constitute a single record? Is an order stored in a single line, or does it generate one line per product that it contains? In other words, what is the lowest level of detail?

The PK answers this question by defining the granularity of the table and telling the users the lowest level of data it contains and how to uniquely identify an entity instance.

Knowing the PK of a table allows database users to query the data precisely on the required column(s) and values and eliminate ambiguity (of having multiple Daves, for example).

The precision of isolating a unique entity also applies to aggregating table data for counting and summarizing its **metrics (numerical attributes)**. Snowflake offers various aggregating functions to summarize table data—such as COUNT, SUM, and MAX—but they require the user to specify the entity's unique identifier (or other grouping dimensions) in the GROUP BY statement. Incorrectly specifying (or assuming) a valid identifier of a table will result in queries that return erroneous counts and aggregates.

Ensuring correct join results

Queries often require data from one table to be joined and enriched by information from another. While there are many types of joins (LEFT, INNER, and FULL), users tend to avoid the Cartesian or cross join, which multiplies the rows from one table by the number of matching rows in another. Cartesian joins most often occur due to the inability to correctly identify a unique record in one or both tables involved in a join.

In the Dr. Seuss example, the ambiguity results in 23 rows instead of 1. In a database, where queries may operate on millions of records, unexpected record explosions could result in billions of extra rows and a meaningless final result. Such queries can run exponentially longer than expected under correct join conditions. In any database, this would be a total waste of time, but in Snowflake, this would also come with a hefty credit spend.

Avoiding duplicate values

A PK uniquely identifies an entity. Therefore, duplicate values in PK columns would violate this constraint and should be disallowed. However, only hybrid tables can enforce this rule. They can do so through an index that allows Snowflake to efficiently scan for existing values when performing a DML operation.

If using regular Snowflake tables, ad hoc validation can be performed before or after a DML operation to test for duplicates.

Dr. Seuss takes the poem to its logical conclusion that Mrs. McCave would have avoided much hardship by uniquely naming her children, saying: *“But she didn’t do it. And now it’s too late.”* But unlike naming children, it is never too late to assign a PK to a table. Let us look at the three methods Snowflake provides for doing so.

Specifying a PK

A PK can be specified when creating a Snowflake table in one of two ways—inline, next to the corresponding column(s) in the CREATE TABLE statement, or out of line, at its end:

- **New table inline:** A PK can be declared directly next to a corresponding column when creating a table. This method is only valid for single-column PKs and cannot be used to define composite keys:

```
CREATE TABLE ORDERS
(
  O_ORDERKEY number(38,0) CONSTRAINT my_pk_name PRIMARY KEY,
  O_PRODUCTKEY number(38,0) NOT NULL,
  O_CUSTKEY number(38,0) NOT NULL,
  -- < rest of columns>
  O_COMMENT varchar(200) COMMENT 'Order details'
);
```

- **New table out of line:** This method allows Snowflake users to specify a PK at the end of the CREATE statement and include multiple columns as part of the definition:

```
CREATE TABLE ORDERS
(
    O_ORDERKEY number(38,0),
    O_PRODUCTKEY number(38,0),
    O_CUSTKEY number(38,0) NOT NULL,
    -- < rest of columns>
    O_COMMENT varchar(200) COMMENT 'Order details',
    CONSTRAINT my_pk_name PRIMARY KEY ( O_ORDERKEY, O_PRODUCTKEY )
);
```

- **Existing table out of line:** A PK can still be defined for existing tables by performing an ALTER TABLE statement and specifying it like an out-of-line constraint:

```
ALTER TABLE ORDERS
ADD CONSTRAINT my_pk_name PRIMARY KEY (O_ORDERKEY, O_PRODUCTKEY);
```

Now that we have seen how to define a PK, let us discuss the different types of keys that exist and how they are used in modeling.

Keys taxonomy

Having seen the vital role that PKs play in defining and querying database tables, it is time to drill down into the various kinds of keys that exist to understand their taxonomy and the roles they play when designing tables.

Although the function of a PK remains unchanged—to uniquely identify a record in a table—the language of modeling allows for distinctions based on the nature of the key columns. Let us see what that looks like in practice, starting with the most common type: the business key.

Business key

A **business key** is a PK whose values hold meaning or significance within the organization. Any random group of characters can serve to identify an entity instance as long as they are unique. What sets the business key apart is that its values are significant in themselves.

All the examples from this chapter have been business keys. In the example in *Figure 5.7*, the values in O_ORDERKEY and O_PRODKEY are understood within the organization in operational teams and their systems, and the same codes are referenced, for example, in the CRM system and in other tables where they exist. If used as PKs, a Social Security number or an ISO country code would also qualify as business keys because of their broader significance in the outside world.

When modeling columns that contain business keys, a prefix or suffix of BKEY is often used to designate the distinction.

However, a key need not have business significance to be useful in table design. Meet the surrogate key.

Surrogate key

Unlike a business key—which holds business significance—a **surrogate key** is a PK whose value holds no special meaning. Surrogate keys are typically created using random characters, column hashes, or sequential integers. They satisfy the condition of uniqueness, but their values carry no intrinsic meaning.

Tables often have compound PKs with numerous member columns, which makes them tedious to type when, for example, needing to join one table to another. Surrogate keys get around this inconvenience because, by nature, they are single-column and have a standard value format (for example, column hash or sequence). Some data models—such as Data Vault 2.0, discussed in a later chapter—rely on surrogate keys to ensure strict naming and value patterns to make table design repeatable and consistent.

When modeling columns that contain surrogate keys, a prefix or suffix of SKEY is often used to designate the distinction.

Sequences

When creating a sequential surrogate key, Snowflake provides a mechanism for generating it called a sequence. A **sequence** is an independent database object that generates sequential (but not necessarily gapless) integers, ensuring unique column values. A start number and increment can be specified when creating a sequence.

As independent objects, a single sequence can be shared by multiple tables or manipulated through function calls.

Sequences can be used as surrogate keys by assigning them as the default column value when creating a table. To do so, use a sequence's `nextval` function as the column default, like so:

```
create or replace sequence seq1;  
create or replace table foo (k number default seq1.nextval, v number);
```

The following example demonstrates how new records would behave under various conditions of the `nextval` function:

```
-- insert rows with unique keys (generated by seq1) and explicit values
insert into foo (v) values (100);
insert into foo values (default, 101);
-- insert rows with unique keys (generated by seq1) and reused values.
-- new keys are distinct from preexisting keys.
insert into foo (v) select v from foo;
-- insert row with explicit values for both columns
insert into foo values (200, 202);
select * from foo;
```

K	V
1	100
2	101
3	100
4	101
200	202

Note that Snowflake does not guarantee contiguous sequences, and gaps in the numbering may exist. Alternatively, Snowflake can create and manage the sequence object transparently for a given table by using the `AUTOINCREMENT` or `IDENTITY` keyword when creating a column:

```
CREATE TABLE FOO
(
  k number DEFAULT seq1.nextval,
  v number
);
CREATE TABLE BAR
(
  k number NOT NULL AUTOINCREMENT START 10 INCREMENT 5,
  v number NOT NULL
);
```

For more information on using sequences, consult Snowflake's documentation: <https://docs.snowflake.com/en/user-guide/querying-sequences>.

We now understand the PK taxonomy and can distinguish a business key, which has business significance, from a surrogate key, which is not meaningful in itself and whose sole purpose is to provide a unique value. However, recall that a table can only have one PK defined. So, what happens when there are multiple columns that can uniquely identify an entity instance?

Alternate keys as unique constraints

Suppose we were modeling an EMPLOYEE table that contains an EMPLOYEE_ID column—a unique business identifier—and Social Security numbers—government-issued personal identifiers. Either column would satisfy the PK requirement of uniquely identifying a record in the EMPLOYEE table, but recall that a table may only be assigned one PK. To let database users know that another column (or columns) satisfies the conditions for a PK when a PK already exists, **alternate keys (AKs)** or UNIQUE constraints can also be defined.

In the previous example, the EMPLOYEE table had two valid PK candidates: EMPLOYEE_ID and SOCIAL_SECURITY_ID. In OLTP databases, the column or columns that act as the organizational business key should be made the primary. In a data warehouse, where business keys from multiple source systems may be loaded, a surrogate key would be used instead. By this convention, the EMPLOYEE table should be modeled with EMPLOYEE_ID as the PK and SOCIAL_SECURITY_ID as the AK (UNIQUE).

The following example shows what that would look like:

```
CREATE TABLE EMPLOYEE
(
    EMPLOYEE_ID varchar(10),
    SOCIAL_SECURITY_ID number(9),
    -- < rest of columns >
    NAME varchar,
    CONSTRAINT pk_employee PRIMARY KEY ( EMPLOYEE_ID ),
    CONSTRAINT ak_employee UNIQUE (SOCIAL_SECURITY_ID)
);
```

Unlike a PK, a table can have as many AKs as necessary. AKs give database users insight into the granularity of a table and its various columns and carry the same functional benefits as PKs. Like PKs, UNIQUE constraints are only enforced in hybrid Unistore tables and are merely orientative for standard ones.

So far, the chapter has focused on individual tables and their constraints, but modeling is concerned with defining entire database landscapes, which represent the interrelated nature of the many business entities. The following section will explain how to form relationships between entities and why this is a crucial detail in a database model.

Relationships as foreign keys

Any business can be broken down into a set of entities and their interactions. For example, customers place orders for items provided by suppliers while applying promotion codes from an active marketing campaign—which is still a very narrow slice of what goes on in a typical organization. So far, this chapter has focused on the entities themselves: orders, items, and suppliers, for example. Now it is time to focus on the interactions—or relationships, as they are called in modeling parlance—such as placing orders, providing items, and applying promotions.

When business entities are related, their corresponding tables must have a way to capture the details of the interaction. When a customer orders an item, the order details must capture who the customer is and what items they ordered. Remember, PKs identify a unique record in a table. Therefore, when two tables share a **relationship**, the PKs of one must be included in the other to store the details of the interaction. In a database, this relationship is established through a table constraint called a **foreign key (FK)**.

When PK columns from one table (known as the parent) are included in another (known as the child), an FK constraint can be declared in the child table to formalize the relationship. The FK constraint tells the database and its users that it is no coincidence that the two tables have some columns in common but that they share a relationship through some business context.

An FK is represented on a diagram by a line connecting the parent table to the child. Recall the example of PERSON and ACCOUNT from *Chapter 1, Unlocking the Power of Modeling*, where it was established that an account must be assigned to a person, and a person can open zero or many accounts. If it is established that PERSON_ID uniquely identifies a PERSON (its PK), this column must be included in the ACCOUNT table and declared an FK constraint. When creating the ACCOUNT table (a child table), the PK and the FK constraints must be defined.

The following code snippet shows the creation of the PERSON and ACCOUNT tables:

```
CREATE TABLE PERSON
(
    PERSON_ID number(8,0) NOT NULL,
    SOCIAL_SECURITY_NUM number(9,0) NOT NULL,
    DRIVERS_LICENSE varchar(10) NOT NULL,
    NAME varchar NOT NULL,
    BIRTH_DATE date NOT NULL,
    CONSTRAINT PK_1 PRIMARY KEY ( PERSON_ID )
);

CREATE TABLE ACCOUNT
(
    PERSON_ID number(8,0) NOT NULL,
    ACCOUNT_ID varchar(12) NOT NULL,
    ACCOUNT_TYPE varchar(3) NOT NULL,
    IS_ACTIVE boolean NOT NULL,
    OPEN_DATE date NOT NULL,
    CLOSE_DATE date,
    CONSTRAINT PK_2 PRIMARY KEY ( PERSON_ID, ACCOUNT_ID ),
    CONSTRAINT FK_1 FOREIGN KEY ( PERSON_ID ) REFERENCES PERSON (
PERSON_ID )
);
```

The resulting diagram, which makes this relationship much easier for a human to perceive, is the one initially shown in *Chapter 1, Unlocking the Power of Modeling*:

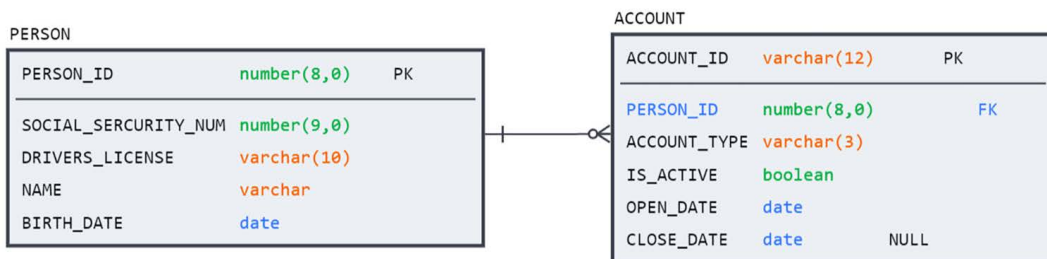


Figure 5.9: Two tables that share a relationship

The PKs and FKs defined in the preceding DDL and expressed in *Figure 5.10* ensure that the business rules regarding account opening are baked into the database design.

Sample values for the resulting tables are presented in the following diagram to illustrate the data such a model might contain.

(PK)		(AK)		(AK)	PERSON	
	PERSON_ID	SOCIAL_SECURITY_NUM	DRIVERS_LICENSE	NAME	BIRTH_DATE	
1	1	123,456,789	A12345BC67	Joe Person	2000-01-01	
2	2	359,437,832	Z12345DE34	Jane Person	2002-02-02	

(FK)	(PK)	ACCOUNT				
PERSON_ID	ACCOUNT_ID	ACCOUNT_TYPE	IS_ACTIVE	OPEN_DATE	CLOSE_DATE	
1	1	eav4a4tasgr	SAV	TRUE	2022-04-04	null
2	1	afm402mf03r3	CHK	TRUE	2022-03-03	null

Figure 5.10: Sample values for PERSON and ACCOUNT tables

In databases, FK-to-PK references are the most common form of relationship, but they are not the only form. An FK may also reference an AK. Often, a table might have a surrogate key as the PK and various business keys declared as AKs. Since the surrogate key is a technical artifact with no business value, related tables will instead refer to the alternate business key, as shown in the following example:

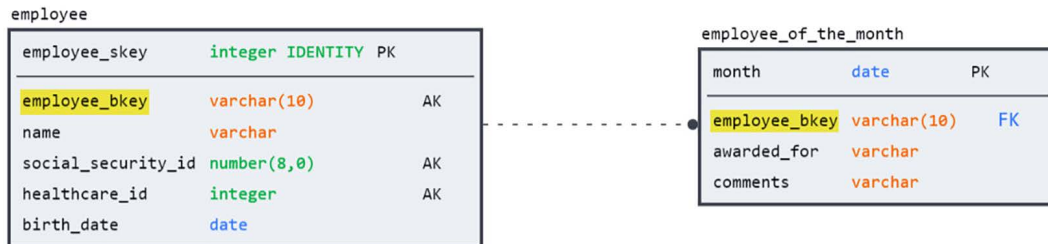


Figure 5.11: An FK referencing an AK

As you can see in the accompanying code, the syntax for referencing an AK for an FK is identical to that used to reference a PK:

```

CREATE OR REPLACE TABLE employee
(
    employee_skey      integer NOT NULL AUTOINCREMENT START 1
    INCREMENT 1,
    employee_bkey      varchar(10) NOT NULL,
    name               varchar NOT NULL,
    social_security_id number(8,0) NOT NULL,

```



```

healthcare_id      integer NOT NULL,
birth_date         date NOT NULL,
CONSTRAINT pk_employee_skey PRIMARY KEY ( employee_skey ),
CONSTRAINT ak_employee_bkey UNIQUE ( employee_bkey ),
CONSTRAINT ak_healthcare_id UNIQUE ( healthcare_id ),
CONSTRAINT ak_ss_id      UNIQUE ( social_security_id )
);
CREATE OR REPLACE TABLE employee_of_the_month
(
    month          date NOT NULL,
    employee_bkey  varchar(10) NOT NULL,
    awarded_for    varchar NOT NULL,
    comments       varchar NOT NULL,
    CONSTRAINT pk_employee_of_the_month_month PRIMARY KEY ( month ),
    CONSTRAINT fk_ref_employee FOREIGN KEY ( employee_bkey )
    REFERENCES employee ( employee_bkey )
);

```

Having understood what FKs are, let us get to know their benefits.

Benefits of an FK

Declaring an FK formalizes the functional relationship between two tables. This metadata now becomes part of the data model and is available for reference to anyone using the database. Here are some of the advantages this brings.

Visualizing the data model

Humans use visualizations to enhance cognition. The previous section demonstrated how a simple diagram could communicate the same information stored in code faster and in more detail. Now imagine going from two tables to an entire database schema. Without visual guidelines to help map out the entities and relationships, it would be challenging to make sense of all the tables and their contents without prior knowledge.

The following figure shows what going from 2 tables to 20 would look like on a diagram. Considering that data models in enterprise systems can span hundreds if not thousands of tables, the aid of a diagram becomes invaluable in navigating the business/data landscape.

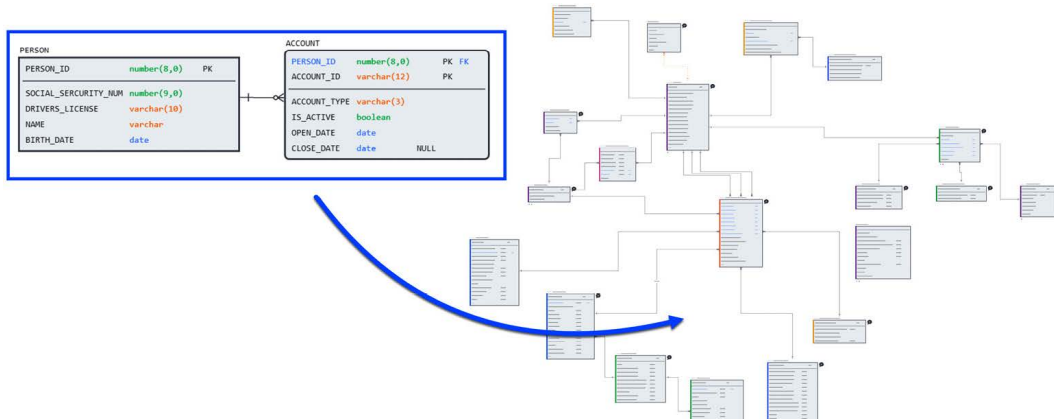


Figure 5.12: Going from two tables to an entire schema

However, relationships between tables are not just visual aids—they serve a practical purpose for writing queries.

Informing joins

Defining an FK requires the user to specify the reference between the parent PK column(s) and those in the corresponding child table. This means that the link between the two tables is now baked into the data model and can be referenced by users wishing to join the table data for analysis or transformation.

The following figure shows how the FK relationship highlights the common column (PERSON_ID) between the two tables.

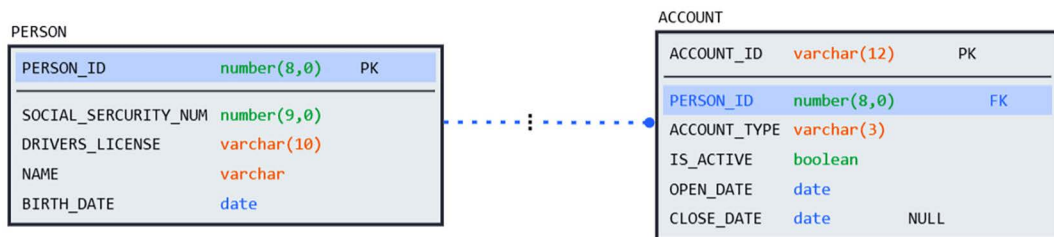


Figure 5.13: Highlighting FK columns to infer join conditions



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now, users can leverage this metadata to write queries that retrieve PERSON and ACCOUNT data by joining the two tables using PERSON_ID.

However, FKs are not just helpful to people—**business intelligence (BI)** tools also leverage them.

More importantly, Snowflake can leverage these details to avoid performing the join and improve performance when it is not required (more on this and the RELY property in *Chapter 13, Putting Transformational Modeling into Practice*).

Automating functionality in BI tools

Many BI tools use FKs to enhance usability and save time for their users. For example, dashboarding and analytics tools such as Tableau and Power BI detect FKs to use them to automatically create joins between tables without explicit help from the users.

Modeling tools and some SQL IDEs can generate an ER diagram by reading database DDL. While every database object carries a corresponding CREATE statement, the ER diagram will only be meaningful if the objects represented by the CREATE statement are also related through FKs.

The next benefit we'll discuss is only possible through enforcing the FK constraint and is therefore reserved for hybrid tables. However, it unlocks powerful new possibilities in the Snowflake platform that users should be aware of.



Enforcing constraints in hybrid tables

By using indexes, hybrid tables are able to rapidly locate individual records for spot-checking the existence of primary and foreign key values. This means that constraints are no longer merely orientative, but act as formal business rules embedded into the database architecture. We will dive deeper into hybrid tables, constraint enforcement, and referential integrity in *Chapter 6*.

The last constraint we will cover is the only one enforced on all Snowflake tables, so let us learn how it works.

Mandatory columns as NOT NULL constraints

When defining attributes for an entity, the question of which ones are mandatory and which are optional inevitably arises. As with most modeling decisions, the answer depends on the business context more than any technical database property. The same attribute, for example, the email address for CUSTOMER, may be mandatory for an online store but optional for a brick-and-mortar retailer. In the latter case, not having an email address means missing sales announcements, while in the former, it may mean being unable to access the website.

When moving from a conceptual model to a physical Snowflake design, mandatory columns can be defined through the NOT NULL constraint. The NOT NULL constraint is declared inline next to the corresponding column and does not need to be given a name. Due to this, it is not possible to declare NOT NULL constraints out of line.

The format for adding a NOT NULL constraint to a column is as follows:

```
<col1_name> <col1_type> [ NOT NULL ]
```

Previous examples, such as the ACCOUNT table, have demonstrated instances of NOT NULL constraints. Here, we can see that CLOSE_DATE is the only column in the ACCOUNT table that is not mandatory and is allowed to have a NULL value:

```
CREATE TABLE ACCOUNT
(
  PERSON_ID number(8,0) NOT NULL,
  ACCOUNT_ID varchar(12) NOT NULL,
  ACCOUNT_TYPE varchar(3) NOT NULL,
  IS_ACTIVE boolean NOT NULL,
```

```
OPEN_DATE date NOT NULL,  
CLOSE_DATE date,  
CONSTRAINT PK_2 PRIMARY KEY ( PERSON_ID, ACCOUNT_ID ),  
CONSTRAINT FK_1 FOREIGN KEY ( PERSON_ID ) REFERENCES PERSON ( PERSON_ID )  
);
```

Unlike the other constraints, enforcing `NOT NULL` does not require lookups or references to other tables—it can be done directly on the individual rows by checking for values in the designated columns. For this reason, `NOT NULL` is enforced for all Snowflake tables and does not incur any performance overhead during DML operations.

Summary

This chapter discussed how to transition from logical modeling concepts to physical Snowflake objects. During this process, we learned how Snowflake handles tables of near-infinite size by breaking them down into manageable micro-partitions, and how these partitions can be clustered to optimize query and DML performance.

After that, we learned how to define attributes by understanding Snowflake’s data types and their properties. Snowflake offers a variety of functions to make working with data types easier and more performant, to say nothing of the powerful options it offers for semi-structured data.

Before diving into individual constraint types, we understood what database constraints are and how Snowflake organizes and enforces them depending on the type of table on which they are applied.

We saw why unique identifiers are vital for defining tables and how Snowflake manages this through the PK constraint. PKs help make life easier for database users by helping identify the minimum level of detail in a table and ensuring accurate searches and joins. We also saw that sometimes, when multiple candidates exist for a PK, AKs can be defined through `UNIQUE` constraints and offer the same benefits.

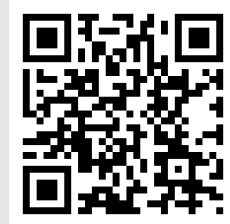
Then, we saw how relationships are formalized in a physical database by defining FK constraints. FKs make visualizing and navigating complex data landscapes possible by capturing the functional relationships between tables—information that human users of the database and BI tools frequently leverage.

Finally, we learned how to declare mandatory columns and enforce the rule using the `NOT NULL` constraint to ensure data quality.

This chapter and others have frequently relied on relational diagrams to clarify examples and explain abstract concepts. However, there has been no formal explanation of the modeling notation or an overview of the various elements in the visual modeling toolkit. Now that we understand the core elements of a data model at a logical and physical level, the next chapter will focus on modeling notation for designing or visualizing data models at any scale.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

6

Mastering Advanced Snowflake Objects

Modern data demands have transformed Snowflake into a comprehensive cloud data platform, providing users with tools and capabilities that extend well beyond traditional data warehousing. Imagine the potential if a single system could manage everything from high-volume bulk loading to high-concurrency transactional workloads and data lake architectures, all while supporting native features such as time travel and identity management.

By unifying these diverse scenarios, Snowflake offers a powerful opportunity to simplify data management. It replaces a complex array of integrations with a coherent strategy that ensures consistent data governance and security.

In this chapter, we will explore the features and advanced object types that enable Snowflake users to surpass the capabilities of simple tables and views. We will begin by examining Iceberg tables, which allow users to work with externally stored data while still delivering native table-like performance and facilitating interoperability across multiple cloud platforms. Additionally, we will discuss dynamic tables—Snowflake’s powerful feature that enables users to seamlessly build data pipelines without relying on third-party ETL and orchestration tools. We will also investigate objects that allow for high-concurrency transactional workloads while providing fast, synchronous analytical access.

By learning how to leverage these features in a unified manner within a single platform, users can benefit from a centralized set of security, analytics, and monitoring capabilities that simplify data management and reduce complexity. This unified approach minimizes the challenges associated with fragmented systems, brings analytics closer to the source of truth, and ensures that security and governance measures are applied uniformly across all workloads.

In this chapter, we'll cover the following topics:

- Working with external data with internal-like performance and cross-platform interoperability
- Understanding ACID properties and transaction management
- Creating dynamic data pipelines that require no orchestration
- The declarative and imperative approaches to data transformation
- How to manage transactional workloads and enforce data consistency
- Centralizing logs and telemetry data for standardized monitoring

Iceberg tables

Snowflake **Iceberg tables** seamlessly integrate the Apache Iceberg **Open Table Format (OTF)** within the Snowflake cloud platform. This integration enhances data management capabilities by providing robust features such as **Atomic Consistent Isolated Durable (ACID)** transactions, schema evolution, and time travel for externally stored and accessible files. As a result, Snowflake Iceberg tables deliver efficient query performance comparable to standard tables, ensuring consistent and scalable data operations. They also facilitate interoperability across multiple processing engines and cloud platforms, making them ideal for managing large and evolving datasets without vendor lock-in.

Open table formats explained

OTFs are open source logical tables designed to work with tabular data stored in existing file formats such as ORC and Parquet. In addition to the columnar data itself, OTFs store metadata (i.e., partition statistics and operational logs) in a separate file or directory. Unlike plain columnar file formats, OTF's metadata enables support for ACID transactions, schema evolution, time travel, and partitioning, ensuring that data remains consistent and accessible regardless of the underlying storage or processing engine.

Common standards for OTFs include Apache Iceberg, Delta Lake, and Apache Hudi. These formats promote interoperability across various platforms and query engines, enabling organizations to construct scalable and resilient data lakes that provide data as a single source to a wide range of analytic tools and workloads.

By adhering to open standards, OTFs facilitate seamless data sharing, enhance governance, and reduce vendor lock-in.

ACID properties explained

In computer science and database management, ACID properties are designed to guarantee data validity despite errors, physical failures, and other mishaps during transactions. A transaction is a single *logical* data operation, meaning it can be an update to a single record, or a sequence of changes such as debiting one account and crediting another as part of a funds transfer.

The properties of an ACID transaction are the following:

- **Atomicity:** Every transaction is treated as a single, indivisible unit that either fully completes or does not occur at all, ensuring that partial operations do not corrupt data.
- **Consistency:** Transactions must transition the database from one valid state to another, maintaining all defined rules, constraints, and data integrity.
- **Isolation:** Concurrent transactions operate independently without interference, so that the intermediate state of one transaction is invisible to others until it is completed.
- **Durability:** Once a transaction is committed, its changes persist even in the face of hardware or data management system failures.

Snowflake is fully ACID compliant and supports implicit and explicit (i.e., using BEGIN, COMMIT, and ROLLBACK commands) transactions. However, this is not true of data stored in files on cloud storage systems or in external tables (which are read-only). Using an OTF overcomes this limitation when working with physical files.

How Iceberg tables work

Snowflake Iceberg tables store their data in Parquet format outside of Snowflake on external cloud storage using one of the supported cloud storage providers (Amazon S3, Azure Storage, Google Cloud Storage). As a prerequisite, you will need access to one of these cloud storage providers and an appropriate **identity and access management (IAM)** entity for your storage location.

Since Iceberg data is stored outside of Snowflake, it cannot provide failsafe functionality; however, time travel is available as part of the OTF.

To create an Iceberg table, you must first have an external volume. An external volume is a Snowflake object that stores information about your cloud storage locations and IAM entities. Snowflake uses an external volume to connect with your cloud storage to access Iceberg metadata and Parquet data. You must use the ACCOUNTADMIN role to create external volumes.

Each external volume is associated with a single storage location and can support multiple Iceberg tables. However, the number of external volumes you need depends on how you want to store, organize, and secure your table data. You can use a single external volume if you want all the data and metadata for your Snowflake-Iceberg tables in subdirectories under the same storage location. Alternatively, you can create multiple external volumes to secure various storage locations differently. For example, you might create separate external volumes for Iceberg tables managed by Snowflake and those managed externally.

For more info on creating an external volume, refer to the Snowflake documentation: <https://docs.snowflake.com/en/user-guide/tables-iceberg-configure-external-volume>.

Managed or unmanaged

Iceberg tables require a catalog to manage their metadata. This function can be fulfilled by Snowflake (i.e., managed) or an external catalog (i.e., unmanaged). Unmanaged, or externally managed, Iceberg tables are read-only when used in Snowflake. The deciding factor between managed and unmanaged is whether you intend to maintain the contents (i.e., perform DML operations) of the table from Snowflake or simply read what is maintained by another system. When choosing the externally managed option, the following sources are supported:

- AWS Glue Data Catalog
- Iceberg metadata files in object storage
- Delta table files in object storage
- Open Catalog
- Remote Iceberg REST catalog

Unlike the managed approach, externally managed Iceberg tables require a catalog integration. A catalog integration is a named, account-level Snowflake object that stores information about how your table metadata is organized. Note that even unmanaged Iceberg tables offer native performance benefits over traditional external tables, such as query caching, pruning, and query parallelization, which, combined, result in a 2x performance improvement.

Users can convert from externally-managed to Snowflake-managed Iceberg using the `ALTER ICEBERG TABLE ... CONVERT TO MANAGED` command. As part of the conversion, Snowflake does not delete any metadata or rewrite any of the Parquet data files. After conversion, some data files may be rewritten as part of regular table maintenance. Note that Snowflake doesn't lock down or assume sole access to your external storage. To prevent table corruption after conversion, ensure that you stop any non-Snowflake writes to your external storage location.

For more information on creating a catalog integration, refer to this documentation and follow the steps for the desired catalog type: <https://docs.snowflake.com/en/user-guide/tables-iceberg-configure-catalog-integration>.

For more details on the limitations and trade-offs between managed and unmanaged Iceberg tables, refer to the following documentation: <https://docs.snowflake.com/en/user-guide/tables-iceberg-configure-catalog-integration>.

The following is a basic illustration of creating an Iceberg table with the required parameters and using Snowflake as the catalog:

```
CREATE ICEBERG TABLE test_db.icebergs.customer_iceberg (  
  <column list>  
)  
CATALOG = 'SNOWFLAKE'  
EXTERNAL_VOLUME = '<external volume name>'  
BASE_LOCATION='<relative path from the table's EXTERNAL_VOLUME location>'  
;
```

Once a managed Iceberg table has been created, users can insert and update records using the same DML commands as standard tables.

Use cases

Iceberg tables are an ideal solution for organizations that operate across multiple processing engines and cloud platforms and deal with large-scale, evolving datasets. Using an open table format such as Iceberg can help ensure interoperability and prevent vendor lock-in, allowing various teams to access one source of truth using their preferred technology stack.

For example, consider an e-commerce company that collects large volumes of clickstream and transactional data from its website and mobile app. The engineering team ingests the raw data from various sources into managed Iceberg tables. Once the data is in Iceberg, the data science team uses Apache Spark to run analytics and machine learning jobs, such as customer behavior analysis and predictive modeling, directly on the Iceberg tables without having to use Snowflake. Meanwhile, the data warehouse team builds reporting views on the same Iceberg tables to create dashboards and ad hoc queries, delivering timely business insights to stakeholders.

Recall that Iceberg tables store data in external storage outside of Snowflake, and (managed Iceberg) still offers schema evolution and performance comparable to standard tables. With so much overlap between existing functionality, such as external and standard tables, it is worth highlighting the intended use cases for each of these formats.

- **Snowflake managed Iceberg:** The organization operates in a multi-cloud and multi-team environment and wants interoperability with file formats along with DML operations, schema evolution, time travel, and the ability to report on data stored in Iceberg in Snowflake with performance demands on par with native tables.
- **Externally managed Iceberg:** The organization operates in a multi-cloud and multi-team environment, and another cloud provider or catalog is already established as the owner of the datasets being stored in Iceberg. The data team wants to report on data stored in Iceberg in Snowflake with performance demands that exceed those of external tables.
- **External tables:** The data team wants to provide read-only access to data stored in external stages in various supported formats, as if the data were actually stored in Snowflake.
- **Standard tables:** This is the best practice recommendation when none of the above conditions are met, which offers the greatest flexibility and performance when working with data in the Snowflake platform.

Now that we have understood Iceberg tables and their intended use cases, let's explore the possibilities of dynamic tables.

Dynamic tables

Snowflake **dynamic tables** integrate transformational logic with data pipelines into a single, self-orchestrating object. Dynamic tables behave similarly to materialized views but have much fewer limitations in their transformational logic and give users control over the latency of data refreshes. They are an alternative to external ETL/ELT processes requiring third-party tools to manage data movement and internally managed Snowflake tasks because dynamic tables encapsulate the transformation logic and data refresh behavior within a single object.

A dynamic table consists of a SQL transformation (like a view), a specified warehouse, and a target lag parameter, which defines the maximum threshold for downstream data freshness relative to upstream data changes. Snowflake automatically ensures that the table stays updated by incrementally processing changes in source data. This built-in automation reduces the complexity of managing data pipelines, as the services layer efficiently tracks dependencies, refresh schedules, and even manages full and incremental loads behind the scenes. Chaining together multiple dynamic tables creates a data pipeline fully managed by Snowflake, eliminating the need for orchestration and third-party software. Yet, such pipelines likely differ from the traditional (i.e., declarative) approaches you may be familiar with.

To understand how transformation pipelines built with dynamic tables differ from the traditional DML approach, we must first understand the difference between declarative and imperative logic.

Declarative and imperative statements

Imperative statements define *how* to achieve a result by providing explicit step-by-step instructions. DML statements, the backbone of most transformational pipelines, are an example of imperative commands, with operations like INSERT, UPDATE, and DELETE – for example, UPDATE target SET to_date = '9999-12-31'.

Declarative statements describe *what* the desired outcome is, leaving the *how* to the system or underlying engine – for example, SELECT '9999-12-31' AS to_date.

To illustrate the difference between the imperative and declarative approaches to problem-solving, let's use the classic question: How do you get to Carnegie Hall?

- **Imperative:** Take the N, Q, R, or W train to 57th Street, walk half a block north, and find the main entrance on the right.
- **Declarative:** Practice, practice, practice!

The takeaway is that, unlike regular tables in which data is maintained through (imperative) INSERT and MERGE statements, the only way to manipulate the result of a dynamic table is through the logic with which it was defined. As we will see in *Chapter 14*, this presents novel approaches to common challenges, such as changing data capture and slowly changing dimensions.

One of the most notable things about the declarative approach of dynamic tables is that it inherently results in fewer lines of code compared to the imperative alternative. In a declarative approach, the SELECT statement represents the entirety of the script—Snowflake handles the rest. In an imperative approach, the SELECT statement must also be accompanied by explicit DML instructions such as UPDATE and INSERT and scheduling parameters.

The following screenshot shows a direct comparison between the lines of code required to create a simple Type 2 dimension using an imperative versus declarative approach.

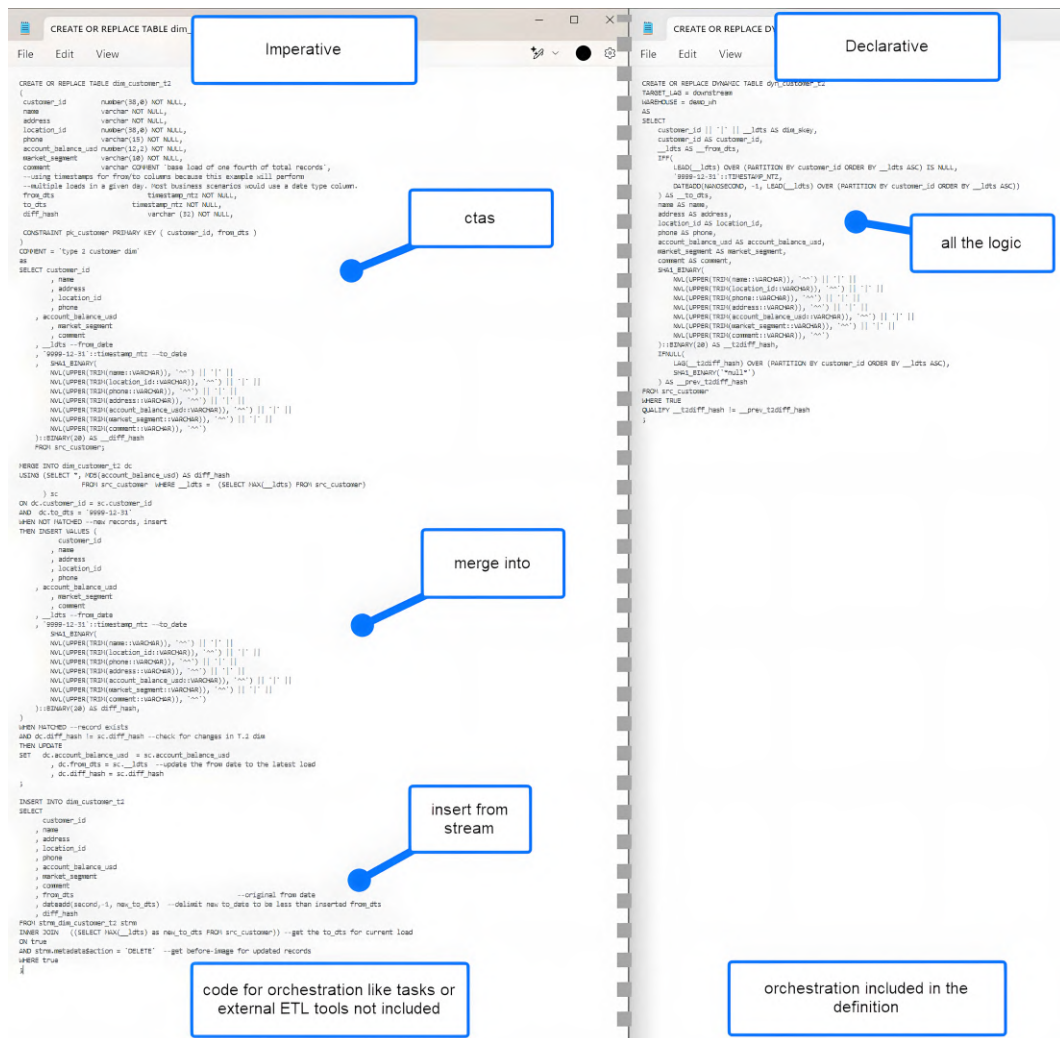


Figure 6.1: Comparing lines of code in an imperative and declarative Type-2 dimension

Regarding maintenance and debugging, the simplicity of fewer lines of code and fewer statements in a process chain should not be overlooked.

Understanding target lag

A data pipeline can be established by defining dynamic tables that reference other dynamic tables. Unlike Snowflake tasks that can be orchestrated to run at a specific time or set interval, dynamic tables update automatically when source data changes. How fast those changes propagate downstream depends on the target lag parameter.

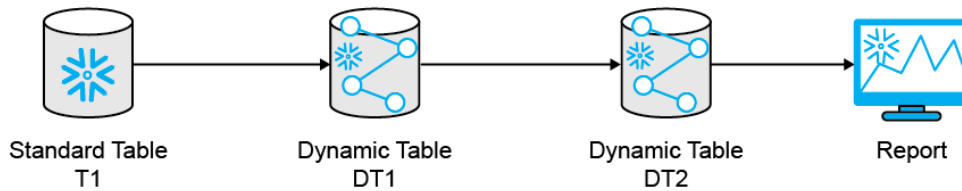


Figure 6.2: A data pipeline using dynamic tables

The **target lag** for a dynamic table is measured in relation to the base tables at the root of the graph rather than the dynamic tables directly upstream. Snowflake schedules refreshes to ensure that the actual lag of a dynamic table remains below its target lag. However, this is not guaranteed and may be affected by the performance of the warehouse. To simplify the management of data pipeline latency, the target lag provides the following specification options:

- **Measured lag:** Defines the maximum time that the dynamic table's content should lag behind updates to the base tables.
 - **Example:** `TARGET LAG = '30 minutes'`
- **Downstream:** Delegates the target lag setting to dynamic tables downstream in the data pipeline and refreshes on their demand.
 - **Example:** `TARGET LAG = DOWNSTREAM`

Let's use the example in *Figure 6.2* to demonstrate a sample dynamic table pipeline configuration. Our goal is to make information loaded to table T1 available in an hour or less in dynamic table DT2, which feeds information directly to a report. If DT1 only serves as an intermediary step to getting data into DT2, then its target lag can be delegated to DT2.

For a one-hour max refresh interval to the report, set the following:

- `DT1 TARGET LAG = DOWNSTREAM`
- `DT2 TARGET LAG = '1 hour'`

To keep data consistent when one dynamic table depends on another, Snowflake refreshes all dynamic tables in an account at compatible times. The timing of less frequent refreshes coincides with the timing of more frequent refreshes. So, at any given time, when you query a set of dynamic tables that depend on each other, you are querying the same snapshot of the data across these tables.

Let's say, in the previous example, we set target lags as follows:

- `DT1 TARGET LAG = '30 minutes'`
- `DT2 TARGET LAG = '1 hour'`

Snowflake might determine that DT1 should run every 25 minutes and DT2 every 50 minutes. But their refreshes will coincide where their timings overlap – for example:

- `00:00` - DT1, DT2
- `00:25` - DT1
- `00:50` - DT1, DT2

To make this possible, the target lag of a dynamic table cannot be shorter than that of dynamic tables upstream.

Note that a downstream error (e.g., in DT2) will not roll back successful updates on upstream dynamic tables (e.g., in DT1), and will result in different snapshots being displayed across the pipeline. More on this in the section on monitoring and error handling.

Understanding refresh types

Dynamic tables can be updated through full or incremental refreshes.

Consider a full refresh akin to a `CREATE TABLE AS (CTAS)` statement, while an incremental update resembles a `MERGE`. The refresh type is a crucial factor that impacts both the cost and performance of a dynamic table. Once a dynamic table is created, the refresh type cannot be changed, so it's essential to understand the differences and limitations associated with it. The refresh type for a dynamic table can be set to the following:

- **Incremental:** Uses Snowflake automated processing to analyze the dynamic table's query and calculate changes since the last update. Unfortunately, not all operations are supported (more on this later), so an incremental refresh might not be possible in all cases.
- **Full:** Snowflake executes the query for the dynamic table and completely replaces the previous materialized results.

- **Auto (default):** Snowflake chooses the refresh mode that's likely to perform best depending on your query definition.

An incremental refresh typically results in faster and more cost-effective loading. To take full advantage, Snowflake recommends that incremental refreshes limit change volume to about 5% of the source data and cluster it by the grouping keys to reduce processing overhead. Bear in mind that not all operations are supported by an incremental refresh, and as of this writing, the list of known limitations includes subqueries, set operators, and certain types of outer joins, among others.

However, enhancements are being made frequently so please consult Snowflake's documentation for the latest information on incremental refresh limitations:

<https://docs.snowflake.com/en/user-guide/dynamic-tables-limitations#label-dynamic-tables-limits-incremental-refresh>

Monitoring, error handling, and failed refreshes

Dynamic tables automate the data transformation process from root sources to downstream targets. Anywhere along this chain, schema changes or data inconsistencies can disrupt the normal flow and interrupt the process. This section will describe how to monitor the operation of dynamic tables and manually control the refresh process for debugging and error handling.

Snowflake offers a convenient way to monitor and control dynamic table refreshes and performance through the Snowsight UI. Log in to Snowflake, and select **Monitoring -> Dynamic Tables** from the left-hand menu. To view this information, a role with the MONITOR privilege is required and further operations such as suspending and resuming require OPERATE.

NAME	STATE	LAST REFRESH STATUS	CURRENT LAG	TARGET LAG	DATABASE	SCHEMA
DYN_TABLE_W_ERROR	Active	Failed	2m 24s	2m	SNOWFLAKE	SNOWFLAKE
DYN_TABLE	Active	Succeeded	12m 37s	20m	SNOWFLAKE	SNOWFLAKE
DAILY_SALES	Suspended	Succeeded	95d 1h 16m 37s	5m	SNOWFLAKE	SNOWFLAKE
CUSTOMER_DYN	Suspended	Succeeded	433d 1h 28m 37s	1m	SNOWFLAKE	SNOWFLAKE
DYN_CUSTOMER	Suspended	Succeeded	92d 3h 24m 37s	1m	SNOWFLAKE	SNOWFLAKE
CUSTOMER_DYN	Suspended	Succeeded	379d 17h 11m 49s	1m	SNOWFLAKE	SNOWFLAKE
CUSTOMER_DYN	Suspended	---	---	1m	SNOWFLAKE	SNOWFLAKE

Figure 6.3: Monitoring dynamic tables

From this screen, users can monitor and control the dynamic tables they have access to and review refresh details. Clicking on any dynamic table in the list will open its latest refresh details. The graph shows the entire data pipeline for the selected table and allows users to confirm details such as refresh mode and whether a warehouse was used during a run. Historic runs can be seen on the **Refresh History** tab.

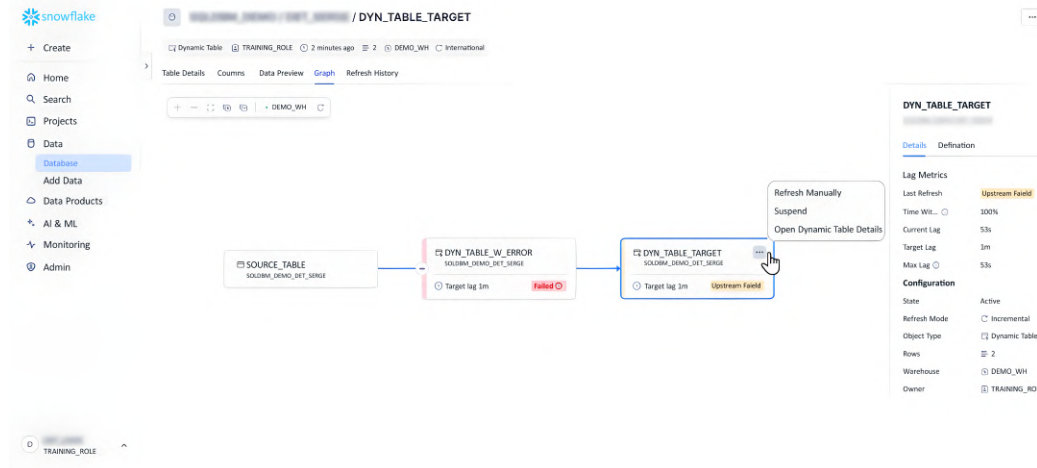


Figure 6.4: Dynamic tables monitoring graph

When an error is encountered during a refresh, the dynamic table does not update and reflects the latest successful state. From the monitoring screen or the query history, users can debug the incident to find the offending records and address the issue at the root level. The following operations can help control the flow of data during this process. These are available from the Snowsight UI or programmatically as ALTER statements:

- **Suspend:** Pause subsequent refreshes for a dynamic table. Since a dynamic pipeline must maintain the same snapshot state across all downstream dynamic tables, any downstream dependencies are likewise suspended. A paused dynamic table does not incur cloud services or virtual warehouse costs but does consume standard storage costs.
- **Resume:** Set the refresh schedule to active and resume refreshes. Resuming a suspended dynamic table is possible if its upstream dependencies are set to active.
- **Refresh:** Manually refresh a dynamic table. This is possible for active dynamic tables (i.e., in between scheduled refreshes) and suspended tables (they will not auto-resume). To keep the snapshot state consistent, upstream dynamic tables are also refreshed.

Note that a table can be suspended manually or automatically due to an upstream suspension. Although both cases are identically labeled in the UI, they are treated differently when resuming to make working with dynamic tables easier. When a dynamic table is upstream suspended (i.e., a predecessor was manually suspended), it will auto-resume once the upstream table is resumed. If a dynamic table was suspended manually, resuming a predecessor will not auto-resume it.

Also, note that five consecutive failed runs will auto-suspend a dynamic table (and any downstream successors).

For more information on monitoring dynamic tables using the Snowsight UI and SQL statements, please consult the Snowflake documentation:

<https://docs.snowflake.com/en/user-guide/dynamic-tables-monitor>

Now that we know how to control dynamic tables, let's discuss strategies to reduce them.

Tips, tricks, and cost considerations

To get the best performance from your dynamic tables, it's important to understand their cost structure. Dynamic tables incur three types of costs:

- **Storage:** Data in dynamic tables incurs the same storage costs as regular tables. They support fail-safe, time travel, and transient options—the latter being preferable for intermediary dynamic tables in a data pipeline. Dynamic tables also maintain an additional internal metadata column to identify each row when using incremental refresh, augmenting storage costs.
- **Virtual warehouse:** During a refresh, dynamic tables use their declared warehouse to transform data and are billed at standard warehouse compute rates. Snowflake recommends using a dedicated warehouse for dynamic tables.
- **Cloud services:** During a refresh, dynamic tables incur cloud service costs to identify changes in underlying base objects and whether the virtual warehouse needs to be invoked.

There are also indirect costs associated with such factors as complexity and maintenance that dynamic tables help reduce. Depending on the use case (e.g., slowly changing dimensions, incremental loading using streams and tasks), the reduction in the number of steps and lines of code required to orchestrate such operations versus the automated refresh process built into dynamic tables can be considerable, and should not be overlooked. We will look at concrete examples and compare operations using dynamic and standard tables in *Chapter 14, Modeling Slowly Changing Dimensions*.

If we examine the allowable operations on existing dynamic pipelines, we'll discover ways to get the benefits of dynamic table simplicity with the precision of task orchestration.

Orchestrating dynamic tables with precision

Compared to tasks, dynamic tables trade scheduled refreshes for simpler declarative logic and automated refreshes based on freshness lag. However, due to business requirements or cost considerations, precise control over when dynamic tables refresh is often required. Luckily, there is a way to get the best of both worlds and combine the versatility of dynamic tables with the fine-grained control of tasks.

Suppose we wanted to orchestrate a data pipeline that takes data from several source tables and transforms it through several dynamic tables, which then feed business-facing dashboards. A key requirement is that while the source tables may receive periodic updates, the dashboards should only refresh once per day, before the start of business hours.

To do this, we can use a “controller” pattern, which can then be orchestrated using a scheduled task. The controller pattern involves adding a dummy dynamic table to the end of the data pipeline (i.e., all leaf nodes are upstream predecessors of the controller). As the scheduling will be handled by the task, all dynamic tables can set `TARGET_LAG = DOWNSTREAM`. Since there is no active target lag, these tables will not auto-refresh. Still, consider suspending them to differentiate them from the auto-updating dynamic tables in Snowsight.

A controller dynamic table can be created as follows, ensuring it does not consume any compute resources:

```
CREATE DYNAMIC TABLE controller
  TARGET_LAG = DOWNSTREAM
  WAREHOUSE = <my dedicated warehouse>
AS
  SELECT 1 AS dummy FROM <leaf1>, ..., <leafN> LIMIT 0;
```

This setup results in the following sample scenario:

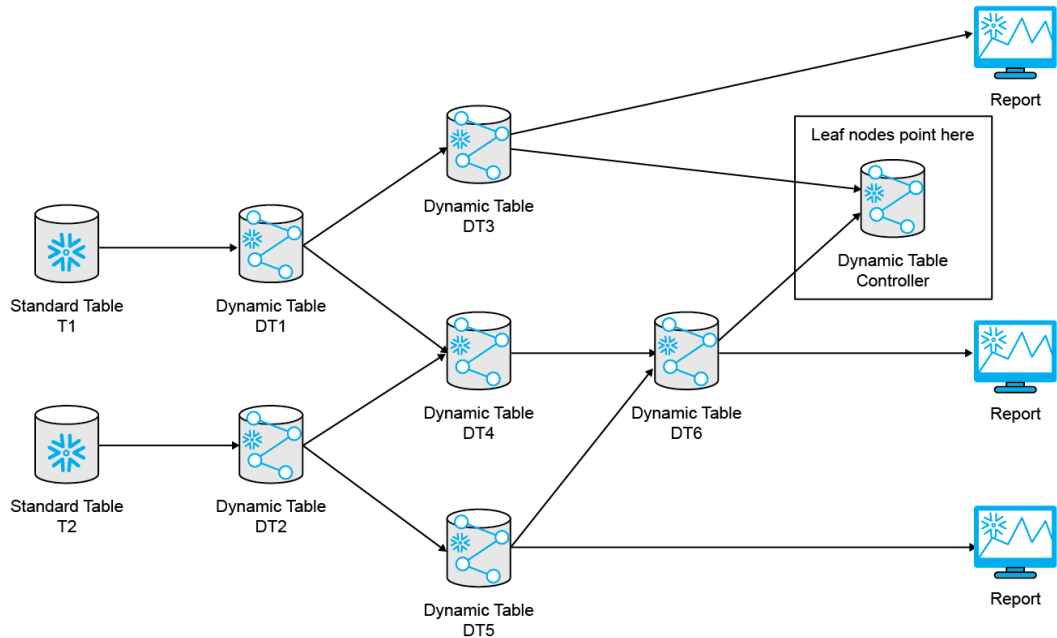


Figure 6.5: Dynamic pipeline controller pattern

Let's review the necessary parameters while implementing some of the best practices that we have already outlined. For the data pipeline in *Figure 6.5*, we ensure the following are met:

- All dynamic tables use a target lag of “downstream”
- All dynamic tables use a dedicated warehouse
- All dynamic tables besides those that are reporting-relevant are created as transient
- For consistent behavior across Snowflake releases, explicitly set the refresh mode on all production tables

The final step is to create the task that will orchestrate the daily refresh of the dynamic pipeline. Since the controller node will remain suspended, we only need to issue a REFRESH command to start the entire upstream refresh process:

```

CREATE OR REPLACE TASK daily_refresh
  WAREHOUSE = <same as dynamic table warehouse>
  SCHEDULE = <required schedule> --ex. 'USING CRON 0 6 * * * UTC'
AS
  ALTER DYNAMIC TABLE controller REFRESH;

```

Note that by using the controller pattern, users can transition the entire data pipeline seamlessly from batch to streaming simply by altering the target lag of the controller and unscheduling the task.

Now let's review how dynamic tables compare to materialized views and the streams and tasks approach and understand the best-intended use cases for each of them.

Comparison and use cases

By now, you should have a solid grasp of the behavior and application of the various options Snowflake offers for transforming data and orchestrating refresh schedules. As a summary, it is helpful to compare the three options that we've previously covered to understand their unique strengths and applications.

The following table provides a comparison between dynamic tables, streams and tasks, and materialized views:

	Dynamic Tables	Streams and Tasks	Materialized Views
Logical style	Declarative	Imperative	Declarative
SQL limitations	Supports most SQL operators but cannot call stored procedures, tasks, or external functions	Can contain calls to non-deterministic code, stored procedures, and other tasks. Also supports UDFs and external functions	Only allow one base table and many SQL operators, such as joins and unions are not supported
Latency	Automatically refreshed to be current up to the target lag	Can be scheduled to run at precise times and intervals	Always current with respect to the base table
Complexity	Low compared to comparable stream and task alternative	Allow for added complexity thanks to procedural code and scripting	Lowest because of their inherent limitations
Unique advantage	Snowflake automatically manages incremental refreshes and abstracts the complexities of managing change data capture	Can run serverlessly and handle complex logic including scripting and external functions	Snowflake automatically reroutes queries from the base table to materialized views to increase performance

Figure 6.6: Comparison of dynamic tables, streams and tasks, and materialized views

With this in mind, let's summarize what we've learned about these three transformational approaches by reviewing their recommended use cases.

- Dynamic tables:** Provide the simplest way of transforming data in your pipeline without having to write code to track data dependencies, manage data refresh, or operate streams and tasks. When the aim is to create easy-to-manage, Snowflake native data pipelines, dynamic tables provide a compelling option.

- **Streams and tasks:** A native way to build complex data pipelines that offer precise controls for scheduling and advanced scripting and external functions.
- **Materialized views:** Used to transparently optimize costs and improve performance by automatically routing queries from their base table.

Now that we have seen how to work with dynamic tables let's explore an exciting addition to Snowflake's object arsenal that bridges the gap between transactional and analytical use cases.

Hybrid tables

Snowflake **hybrid tables** offer an innovative solution to the distinct but conflicting demands of transactional and analytical workloads. While transactional tasks require small, random, and high-concurrency reads and writes, analytics on the same dataset call for large-scale in-memory processing. The dual-storage architecture of hybrid tables addresses these needs by employing row-based storage for fast transactional operations and parallel columnar storage for analytical queries—seamlessly switching according to the workload type and providing optimized performance for each case.

Row-based storage allows hybrid tables to offer features such as row-locking and indexing and employ constraints such as primary and foreign keys while enforcing **referential integrity (RI)**. This unlocks workloads that would otherwise be impractical in a column store architecture such as transactional applications that must maintain state and perform atomic transactions across tables. Best of all, the dual-storage architecture means that transactional and analytic workloads can run concurrently in the same system—eliminating the need for data movement and collapsing the tech stack into one data platform.

Traditionally, database systems have tended to excel at one or the other, thanks to their underlying architecture. Systems that need to perform massive aggregations and calculations for analytic workloads are called data warehouses and run on **online analytical processing (OLAP)** architectures. Transactional use cases that depend on single-row operations or indexes perform better in **online transactional processing (OLTP)** databases. The combination of OLAP and OLTP is known as **hybrid transactional and analytical processing (HTAP)**, which offers the best of both.

Before getting into hybrid tables' costs, performance, and limitations, let's take some time to understand RI—a key feature of hybrid tables—and its vital role in ensuring data consistency.

Referential integrity

In database systems, RI helps maintain data accuracy and consistency by ensuring that each foreign key in a table points to a unique primary key value in another table. By enforcing RI, Snowflake ensures data consistency by preventing the following actions:

- Loading an FK value into the child table that does not exist in the parent table
- Removing a PK value from the parent table that exists as an FK in the child table

The following figure shows an example of these two scenarios being enforced:

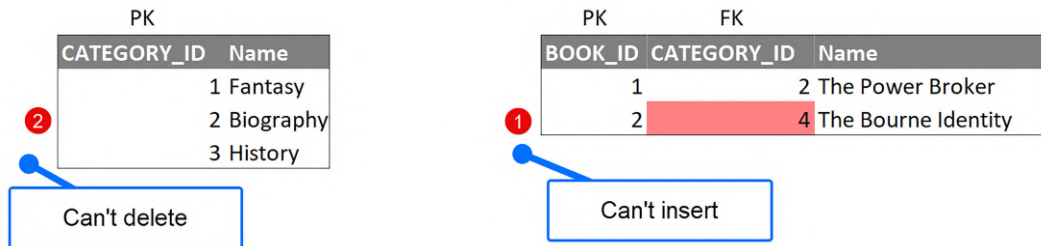


Figure 6.7: Referential integrity violations



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Hybrid tables enforce all constraints, including PKs, FKs, UNIQUE, and NOT NULL. Recall that standard tables can only enforce NOT NULL constraints, so when strict data consistency is required, hybrid tables present a compelling solution.

However, it's important to note that while hybrid tables offer unique advantages, they come at a significant cost premium compared to standard tables, and there are limitations to consider before choosing this option.

Functionality and costs

Due to their row store architecture and indexing, hybrid tables facilitate fast, high-concurrency operations for single-record transactional tasks. Before proceeding, let's clarify what indexes are, how they enhance query performance, and their inherent trade-offs.

Working with indexes

As a requirement, a hybrid table must specify a primary key and will generate an index and order rows based on the PK columns automatically. **Indexes** enhance query performance by providing quick lookup and retrieval of rows based on specified columns. Snowflake allows users to define additional indexes on attributes (i.e., non-PK columns), accelerate lookups along those columns, and increase performance on query predicates (i.e., =, >, >=, <, <=, and IN). Indexes on non-primary are known as **secondary indexes**.

If you have common, repeated queries with filters on a specific attribute or group of attributes, consider adding a secondary index to that attribute or group of attributes to improve performance.

Indexes can be specified when creating a hybrid table or using the `CREATE INDEX` command afterward. Note that `UNIQUE` and `FK` constraints automatically create their own indexes:

```
CREATE OR REPLACE INDEX indexName ON tableName (col1, [col2]);
```

Additionally, users can specify an `INCLUDE` column on a secondary index. If an index allows the database to quickly locate a group of rows, `INCLUDE` helps locate data within that group – for example:

```
CREATE HYBRID TABLE books (  
  book_id INT PRIMARY KEY,  
  book_title STRING,  
  book_category STRING,  
  INDEX idx_category (book_category) INCLUDE (book_title)  
);  
  
SELECT book_title FROM books WHERE book_category = 'Fiction';
```

While INCLUDE increases the size of the secondary index it is created on, it is more efficient than creating a separate secondary index on that column. When considering employing indexes, it's important to take these cost and performance trade-offs into account:

- Indexes increase storage costs by saving additional copies of a subset of the data.
- While they accelerate data retrieval, indexes add overhead to DML operations because they need to be updated synchronously as changes occur.
- Understanding the criticality and frequency of commonly run queries is important to determine whether an index justifies the added costs and DML overhead. As the ancient koan ponders: if an index is created on a table and no query is around to use it, does it make a sound (investment)?

When an index includes (or “covers”) all the columns required to satisfy a particular query—allowing the query to be fulfilled entirely from the index without needing to read data from the main table—it is called a covering index. To verify whether an index covers a given query and confirm whether the row store or column store is being used to retrieve the data, refer to the TableScan or IndexScan details in the query profile in Snowsight. The scan mode indicator will confirm whether the ROW_BASED or COLUMN_BASED mode is being used to retrieve the data from a row or column store, respectively. The following combinations will indicate whether an index is being hit and whether the row store is being used as the data source for a query. A visual from Snowsight accompanies this example:

- **A primary index is used:** TableScan with Scan Mode = ROW_BASED is visible in the query profile.
- **A secondary index is used:** IndexScan with Scan Mode = ROW_BASED is visible in the query profile. The index name and access predicate details are also available.
- **Column store is used:** The query reads from a column store copy of the dataset. Index scans can also access object storage for Time Travel queries.

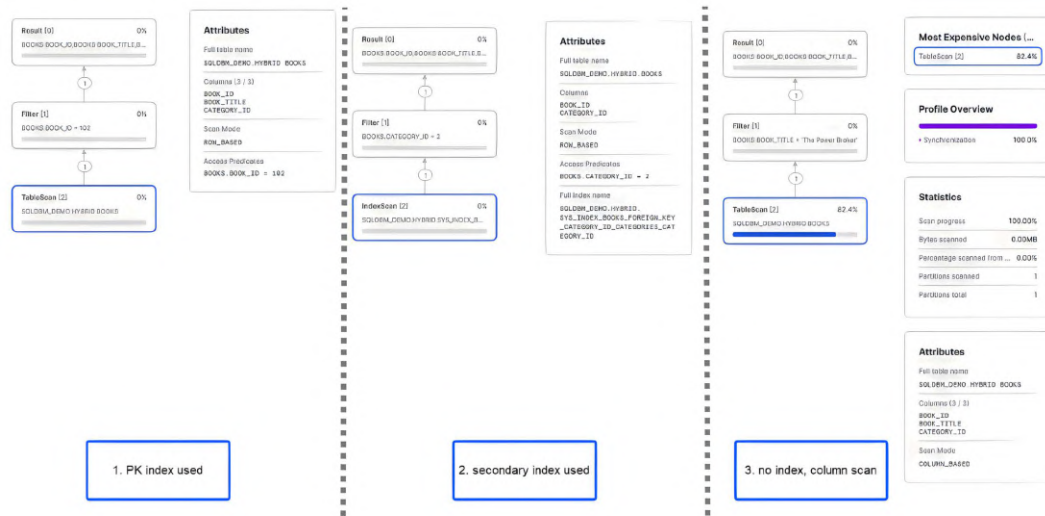


Figure 6.8: Monitoring index coverage and the source data store in hybrid tables

Note that, where possible, queries on hybrid tables can read data from a standard columnar warehouse cache. However, queries on hybrid tables cannot use the results cache.

Now, let's focus on the costs of hybrid tables and better understand the operating costs and total cost of ownership for specific use cases.

Costs

Given their performance benefits and unique architecture, hybrid tables incur a considerable premium compared to what it costs to operate standard tables in Snowflake. Hybrid tables are charged based on three types of consumption:

- Storage:** Hybrid tables' storage costs depend on the amount of stored data, charged at a flat monthly rate per gigabyte. However, the rate is considerably higher than standard Snowflake storage. The latter varies based on the type of contract (i.e., on-demand or capacity), cloud provider and region, and consumption tier, but, on average, expect hybrid tables' storage to be about 15 times more expensive than standard storage. Exact pricing can be found in the consumption table mentioned shortly. Also note that hybrid tables typically have a larger storage footprint compared to standard tables due to row stores being unable to achieve the same compression efficiency as column stores in most cases.
- Virtual warehouse compute:** Queries against hybrid tables are executed using virtual warehouses and charged at standard warehouse credit rates.

- **Serverless compute:** Hybrid tables incur additional credits due to their use of serverless resources on the underlying row storage clusters. The consumption is measured based on the amount of data read from or written to these clusters.

For more information on hybrid tables' cost and Snowflake's service consumption table, refer to their official documentation at <https://docs.snowflake.com/en/user-guide/tables-hybrid-cost>.

While hybrid tables clearly come at a cost premium compared to standard tables, they still offer a compelling option for transactional workloads that benefit from real-time analytics. To understand whether a unified HTAP approach justifies the investment, weighing the total cost of ownership of a separate OLTP datastore (e.g., ETL, maintenance, and data transformation) is important.

Working with hybrid tables and understanding limitations

Working with hybrid tables means first understanding how to create and populate them with data efficiently. Hybrid tables allow two kinds of loading:

- **Bulk optimized load:** Can only be performed the first time a hybrid table is created
- **Incremental batch load:** All subsequent loads use the non-optimized batch method, typically allowing 1 million records per minute.

You can check the loading method by viewing the statistics info in the query plan in the Snowsight UI. “Number of rows inserted” is displayed as “Number of rows bulk loaded” when bulk loading is used.



Figure 6.9: Bulk optimized and batch loads to a hybrid table

When creating a hybrid table from a large existing dataset, make sure to take bulk-optimized loading into account. Also note that although indexes can be added later, doing so is more efficient with a bulk load.

It's worth noting that hybrid tables are a relatively recent addition to Snowflake, debuting in general availability in November 2024. Although Snowflake continues to release new features and remove restrictions, users should be aware of certain limitations as of this writing.

To begin with, hybrid tables are currently supported only for accounts using AWS hosting. Snowflake also places size and throughput restrictions on hybrid table usage and may throttle or restrict their functionality if any of the following are exceeded:

- **Storage quota:** 2 terabytes per database
- **Operations quota:** Approximately 8,000 hybrid table requests per second per database
- **Table quota:** 100 total per Snowflake account, and no more than 10 databases added within a one-hour window

Hybrid tables also restrict the types of time travel users can perform, do not support undropping, and cannot be created as transient. As enhancements happen frequently, be sure to check the official documentation on currently standing limitations: <https://docs.snowflake.com/en/user-guide/tables-hybrid-limitations>.

With this in mind, let's summarize hybrid tables' main benefits and use cases and compare them to standard tables.

Summary and comparison to standard tables

With their unique architecture, hybrid tables make it possible to run transactional workloads concurrently with analytics all in a single platform. Although users should be aware of the costs and limitations described earlier, the following unique features make hybrid tables worth considering where applicable:

- **Mixed workloads and simplified data stack:** Dynamic tables allow transactional workloads and analytics to be run in a single platform. A unified HTAP approach removes the need for data federation and its inherent latency, complexity, and costs by enabling native querying and joining of data in hybrid tables with information stored in standard tables.
- **Row locking for concurrency and atomic transactions:** Row-level locking allows for higher concurrency by enabling multiple transactions to operate simultaneously on different rows of the same table, reducing contention and improving performance. This also simplifies transaction management by ensuring atomic commits across multiple tables.

- **Data quality through enforced referential integrity:** Dynamic tables enforce primary and foreign key constraints (in addition to UNIQUE and NOT NULL), guaranteeing referential integrity and trusted data between related tables.
- **Native data governance:** Dynamic tables benefit from Snowflake's native governance features such as RBAC and row- and column-level security through tagging and masking policies. By combining data governance and security into one platform and one object, hybrid tables help reduce the complexity and overhead inherent in federated data management.

Let's summarize this section by comparing the key aspects and applications of hybrid tables to those of standard tables.

Feature	Hybrid Tables	Standard Tables
Primary use case	Optimized for mixed-use HTAP workloads that combine OLTP and OLAP	Primarily designed for OLAP
Data storage format	Dual architecture with row store with parallel column storage	Columnar storage using micro-partitions
Locking	Row-level locking	Partition or table-level locking
Primary key constraints	Required, enforced	Optional, not enforced
Foreign key constraints	Optional, enforced, providing referential integrity	Optional, not enforced
Unique constraints	Optional, enforced	Optional, not enforced
NOT NULL Constraints	Optional, enforced	Optional, enforced
Time Travel	Limited support	Full support
Zero-Copy Cloning	Not supported	Supported
Fail-Safe	Not supported	Supported
Clouds and regions	AWS	AWS, GCP, Azure
Clustering keys	Not supported. Rows are ordered by PK	Supported
Data size and throughput	2 Tb per database, 8k requests per second	No limit
Read performance	Optimized for small random point reads and writes	Optimized for analytics and large aggregations
Write performance	Optimized for random writes in small batches	Optimized for bulk loading
Point lookups	Very fast when indexes cover the query	Acceptable performance using search optimization service
Storage size	Larger due to dual row and column based storage	Smaller due to efficient columnar compression
Cost	Higher due to elevated storage costs and serverless resources	Comparatively smaller without considering related costs for a dedicated OLTP system and data federation

Figure 6.10: Comparison of hybrid and standard tables

Now that we are familiar with hybrid tables and their features, let's explore event tables—Snowflake's solution to collecting and managing telemetry data.

Event tables

Modern cloud data platforms have evolved to handle diverse use cases that go beyond traditional data warehousing. These days, Snowflake powers everything from transactional workloads to native apps and microservices, often working together as part of a unified system. Monitoring, troubleshooting, and analyzing the operation of such diverse workloads requires gathering and organizing logs and telemetry data about their operation. **Telemetry** is the information collected and transmitted by a system regarding its performance or status, such as logs, resource usage,

interactions, and sensor readings. With so much operating data being constantly generated, users need a comprehensive and consistent way to monitor and alert on trends and anomalies.

Snowflake event tables offer an automated solution for capturing and analyzing all that telemetry data using native and industry-standard logging and tracing features in a single place. Event tables use a predefined set of columns (following the OpenTelemetry framework) to capture telemetry data and allow users to interrogate it using standard SQL.

By default, all Snowflake accounts include an event table called `SNOWFLAKE.TELEMETRY.EVENTS`, which automatically collects telemetry data. Users can also create their own event tables and associate them with an account. The Enterprise Edition allows users to associate event tables with specific databases—for example:

```
CREATE EVENT TABLE myDB.mySchema.myEvents;  
  
ALTER DATABASE myDB SET EVENT_TABLE = myDB.mySchema.myEvents;
```

Since all event tables work the same way and have the same columns, users benefit from a consistent and scalable repository of telemetry data. This data is automatically captured from the following Snowflake objects:

- Stored procedures
- Streamlit apps
- User-defined functions
- User-defined table functions

Although logging levels and methods are beyond the scope of this book, we should highlight the costs and benefits that event tables offer in this regard.

Costs and benefits

Event tables are like a digital diary for the events and telemetry data generated in a Snowflake account. Collecting this information in an automated, centralized, and consistent manner allows developers to simplify their approach to analyzing, alerting, and reacting to irregularities or trends happening across the entire system. The key facets of event tables that make this possible are as follows:

- **Centralization and standardization:** Event tables provide a central and standard repository for logs and traces generated by various Snowflake objects. This eliminates the need to coordinate between disparate file and log formats and gives developers a one-stop shop for collection and analysis.

- **Automatic logging:** Event tables automatically capture logs generated by Snowflake objects and Snowpark applications. Users can easily manage logging and trace levels system-wide for specific types of telemetry data for simple and direct control. For more info on logging, tracing, and their severity levels, see the Snowflake documentation: <https://docs.snowflake.com/en/developer-guide/logging-tracing/telemetry-levels>.
- **Tabular data:** Event tables store telemetry information in tables instead of files, which makes them easier to analyze using SQL. Like all Snowflake tables, event tables can natively store semi-structured data for in-depth analysis.
- **Security:** Unlike flat files, event tables and the default `EVENTS_VIEW` can be secured using native Snowflake RBAC and row access policies.

Since event logging is handled automatically for all on-demand and serverless workloads, it is billed as a serverless feature (currently 1.25 credits for serverless compute and 0.28 credits per 1,000 file batches).

Storage for event table data and warehouse utilization for querying it is billed at standard rates. For up-to-date pricing on all Snowflake features, please refer to the official documentation: <https://www.snowflake.com/legal-files/CreditConsumptionTable.pdf>.

Having seen the benefits of event tables as well as the other advanced Snowflake objects, let's recap what we have covered.

Summary

Snowflake is a true cloud data platform that enables users to run a variety of workloads beyond what a traditional data warehouse can support. It provides essential tools for exploring diverse data architectures, including transactional systems and data lakes. Additionally, Snowflake features support the development of online applications and the management of dynamic data pipelines. Notably, Snowflake allows all these workloads to operate as one unified system, offering centralized analytics, monitoring, and security—effectively collapsing the data stack and reducing the need for data federation.

This chapter delved into several Snowflake objects that extend beyond traditional tables and views. We began with Iceberg tables, which seamlessly integrate an open table format into Snowflake. This integration enables ACID transactions, schema evolution, and time travel for externally stored data. As a result, teams can work on the same authoritative dataset from multiple tools and cloud providers without compromising performance or facing vendor lock-in. By treating physical files like Snowflake-managed storage, Iceberg tables combine the best features of external and native tables.

Next, we explored dynamic tables, which further enhance Snowflake's capabilities by introducing a declarative approach that automatically manages data transformations and dependencies. Dynamic tables reduce the need for manual orchestration and can be paired with tasks to provide precise control over data refresh schedules and warehouse costs.

We then dove in to explore hybrid tables. We saw how they incorporate transactional indexing and row-based storage, making it possible to execute high-concurrency OLTP-like operations alongside analytic queries on a single platform. By supporting real-time inserts, lookups, and enforced referential integrity, hybrid tables eliminate the necessity for separate databases to manage transactional workloads. They offer a powerful dual architecture that integrates Snowflake's columnar store for analytical queries, bridging the gap between OLTP and OLAP in a unified HTAP solution.

Finally, we examined how event tables deliver consistent and automated log capture and system-wide control over logging thresholds and severity levels. By centralizing and standardizing telemetry data in a tabular format, users can query logs using SQL, apply native RBAC and row access policies, and streamline troubleshooting and anomaly detection.

In the next chapter, we will dive deeper into modeling notation to enhance our ability to convey the nuances of conceptual, logical, and physical models. Understanding modeling notation is crucial for effectively communicating database designs and ensuring that stakeholders and technical teams can articulate requirements and align on solutions for current and future data projects.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

7

Seeing Snowflake's Architecture Through Modeling Notation

Throughout this book, relational diagrams have been used to support examples and illustrate ideas in ways that words could not. Although various modeling styles and notations have been introduced, a thorough overview of the visual semantics of modeling and the various elements and their properties has not been covered.

In this chapter, we will run through the complete visual toolkit that will enable users to accelerate the design and understanding of a physical Snowflake database and add functional context through conceptual conventions. Using a simplified and pragmatic approach, users of any background can view and explore a database schema at the level of detail that best suits their needs.

In this chapter, we're going to cover the following main topics:

- Recalling the history of modeling styles and notations
- Understanding the connection between the relational model and its visual depictions
- Learning how to depict the various types of entities
- Exploring how to represent relationships at the physical and conceptual levels
- Understanding how to add functional context to go beyond physical tables
- Discovering how to simplify various modeling conventions to arrive at a synchronized modeling practice
- Doing it all in a manner that is accessible to technical and business users alike

A history of relational modeling

The diagrams and examples throughout this book rely on relational modeling to illustrate database concepts and constructs. Learning how to parse and communicate through relational diagrams provides a dual-faceted benefit to database users by allowing them to rapidly visualize and bring to life complex database landscapes and design them from scratch using visual cognitive aids. Yet, despite these benefits, many database users consider modeling an arcane practice without relevance in the modern warehouse and mistake it for a chore instead of the time-saving boon it really is.

The practice of data modeling and its associated notations have passed through many changes and diverging trends without a final standard ever emerging, encumbering universal adoption. The conceptual data model dates back to the 1960s when Charles Bachman first used rectangles to denote record types and arrows to form relationships between them. Toward the end of the decade, Edgar F. Codd formalized the **relational model (RM)**, which gave rise to the relational database as we know it today. Using the RM, Codd managed to tie mathematical theory, consistent with the structure and language of first-order predicate logic, to database systems. Unfortunately, Codd was less concerned with the visual aspect of the model and offered no diagramming guidelines.

In the 1970s, computer scientists such as Peter Chen and Richard Barker expanded Bachman's simplistic modeling conventions to include more context for the underlying entities and relationships.

However, none of these designs were convincing enough to warrant widespread adoption. While singular elements of these designs gained prevalence—such as the use of crow's foot notation to describe cardinality—wholesale adherence to any one diagramming style proved too restrictive for widespread adoption by failing to strike a balance between readability and technical detail. Some, including Chen's original ER format, arguably, achieve neither.

The following diagram in Chen notation attempts to describe a simple RM consisting of four entities and their attributes. Yet, it fails to do so effectively due to an overwhelming number of visual elements.

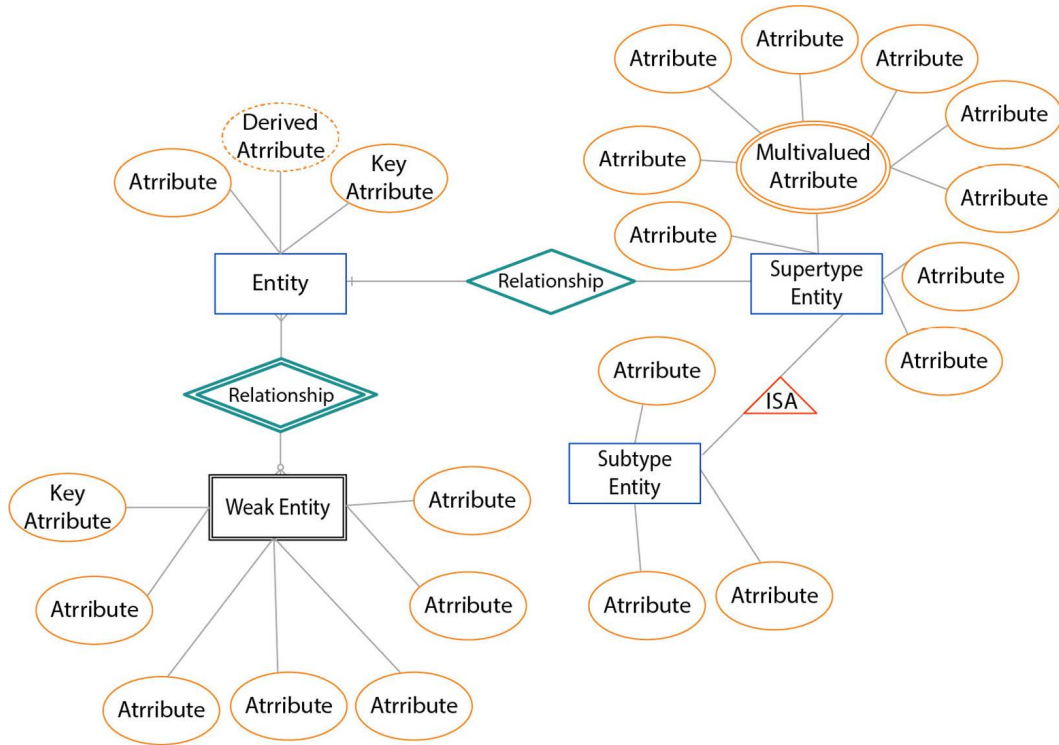


Figure 7.1: An ER diagram in Chen notation

To effectively use visual diagrams, it is essential to separate the model (structure) from its presentation (diagram). A data model's visual representation may vary depending on the level of abstraction, but the elements it represents correspond to real-world business entities that are not subject to reinterpretation.

RM versus entity-relationship diagram

Recall that an RM can be identified from business-relevant concepts and mapped onto a database schema that includes their attributes, identifiers, and relationships. An **entity-relationship diagram** (ERD) is a visual rendering of the RM using simple, easy-to-understand semantics.

In the following examples, we will revisit a set of relational entities from *Chapter 1, Unlocking the Power of Modeling*. In the RM, each of the three entities corresponds to a table identified by a primary key and related to other entities through foreign keys. Just observing the table names or even a data preview does not make it easy to determine these relationships, as can be seen in the following RM:

```

SUPERHERO
-----
SUPERHERO_NAME|BIRTHPLACE    |SUPERHERO_TYPE|HAS_MASK|
-----+-----+-----+-----+
Ratman         |Gotham Gutter|Accident      |true    |
Razor Tooth    |Amazonia     |Accident      |false   |
Galactoid      |Krab Nebula  |Alien         |true    |
SUPERHERO_TYPE
-----
SUPERHERO_TYPE|
-----+
Accident      |
Tech Soldier  |
Vigilante     |
Alien         |
COMIC_BOOK
-----
COMIC_BOOK_NAME          |ISSUE_DATE|
-----+-----+
BC Comics v4             |1978-04-03|
Tales from the Underground v3 |1985-12-23|
Tales from the Underground v24|1995-07-15|
BC Comics v103           |2015-06-19|
APPEARED_IN
-----
COMIC_BOOK_NAME          |SUPERHERO_NAME|
-----+-----+
BC Comics v4             |Razor Tooth    |
Tales from the Underground v3 |Ratman         |
Tales from the Underground v24|Ratman         |
Tales from the Underground v24|Razor Tooth    |
BC Comics v103           |Galactoid      |

```



Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button

- (1) to quickly copy code into your coding environment, or click the **Explain** button
- (2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {  
  return {sum: a + b};  
};
```

Copy**Explain**

1

2



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR visit packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



However, an ERD brings the associative context of the tables into rapid focus by highlighting the entities and relationships behind the data being presented.

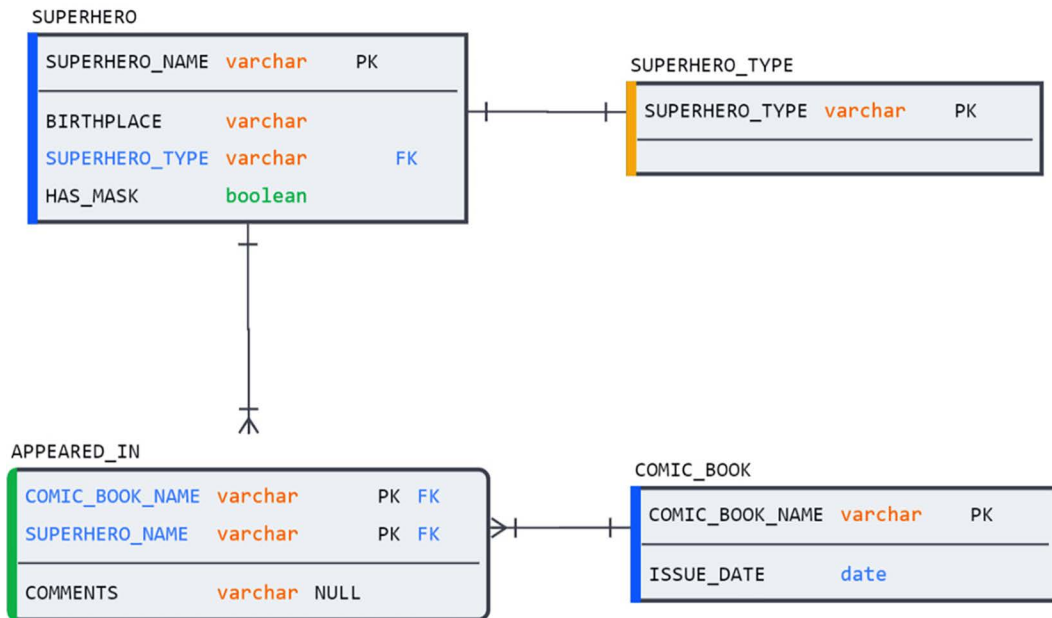


Figure 7.2: A physical model using crow's foot relationship notation

Thanks to the visual clues of the ERD in *Figure 7.2*, the context behind the data, such as the **many-to-many (M:M)** relationship between `COMIC_BOOK` and `SUPERHERO`, instantly comes into focus—that is, if the viewer is familiar with the visual semantics that the ERD is using. So, let us review these elements in detail and demonstrate how we can represent a wide array of technical and conceptual modeling conventions using a few visual conventions.

Visual modeling conventions

This chapter will highlight all the semantic elements that will be used throughout the rest of the book. As described earlier, no fixed and universally prescribed standard for drawing modeling diagrams exists. The conventions used here, along with their accompanying visual semantics, are the author's attempt at simplifying the broad spectrum of modeling concepts and notations into a core set of elements, using the most widely used symbols to represent them. The visual semantics in this book do not conform to any single modeling notation but borrow freely from several leading industry standards. The aim is to provide a modeling language that is both precise enough to accurately reflect technical database details and simple enough for business users to understand the business context that lies beyond the technical specifications.

Unlike textbooks that separate conceptual, logical, and physical models and force users to convert and adapt one to the other, this book will use conventions that make it possible to keep all three in sync throughout the modeling process. This approach minimizes the effort involved in maintaining the models consistently as development unfolds and offers a consistent view at various levels of abstraction.

With that in mind, let us begin with the notation used to describe the central element of the data model: the entity.

Depicting entities

Since the early days of modeling, entities have been depicted using rectangles. Whether they are displayed on a conceptual diagram with no additional detail besides the entity name or on a physical diagram containing a list of attributes, data types, and other information, entities are always represented as rectangles.

However, modeling semantics do distinguish between strong and weak entities. When we think of an entity, we generally think of a **strong entity**—identified by its primary key. However, **weak entities** cannot stand independently and depend on attributes from another table to help identify them. Weak entities must use a foreign key as part of their primary key because the information they contain does not make sense in isolation. This dynamic can be observed in the SIDEKICK weak entity table in *Figure 7.3*, which can only exist in relation to the strong entity, SUPERHERO.

The following diagram shows the strong entity, SUPERHERO, alongside a weak entity, APPEARED_IN:

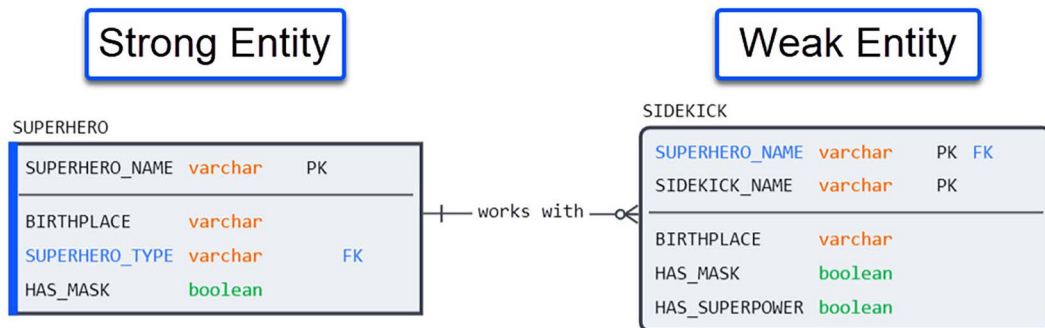


Figure 7.3: Strong and weak entity types in a diagram

In a diagram, entities are depicted as rectangles, with their corners depicting their strong or weak nature. Strong entities are shown with straight, perpendicular corners, while weak entities use rounded corners.

Now, let's zoom in on the details inside the entity. In logical and physical modeling, entities include details about their attributes, constraints, and data types. The following figure explains what the details inside the entity represent.

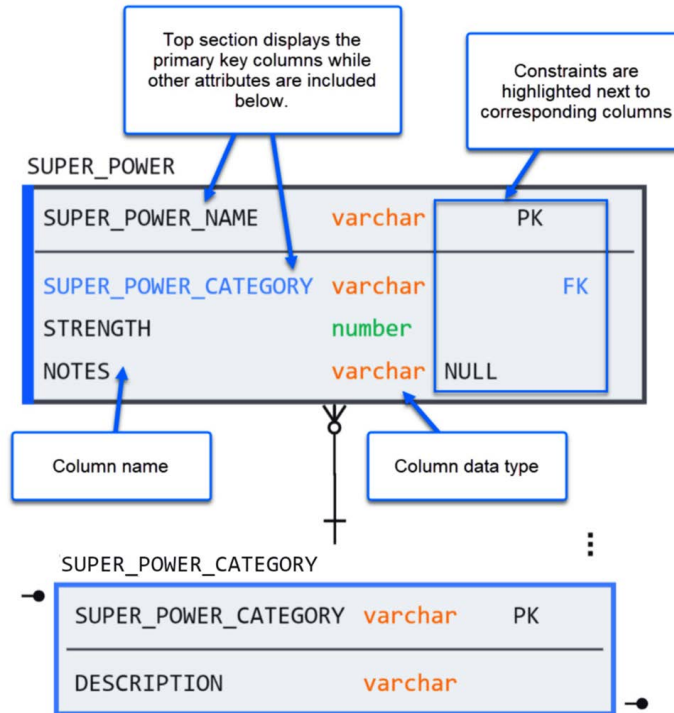


Figure 7.4: Entity details illustrated

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now that we know how to draw entities, let us learn how to represent their relationships.

Depicting relationships

Relationships between entities are represented as connecting lines. However, a relationship can convey more context than a solid line can capture. Relationship lines can also relay information about cardinality and dependency between the associated entities. Some of these details map perfectly onto the physical database, while others are conceptual and orientative. This book uses two notations to represent relationships depending on the context. To understand them, let us start with crow's foot notation, the more common of the two, and discover how it depicts granularity.

Crow's foot

Crow's foot notation derives its name from the three-pronged *many* symbol that resembles a crow's foot. Each side of a crow's foot connector displays information about the cardinality and optionality of the relationship.

There are two cardinality options (*one* or *many*) and two optionality indicators (*required* or *optional*). This gives us four possible combinations, as described in the following figure:

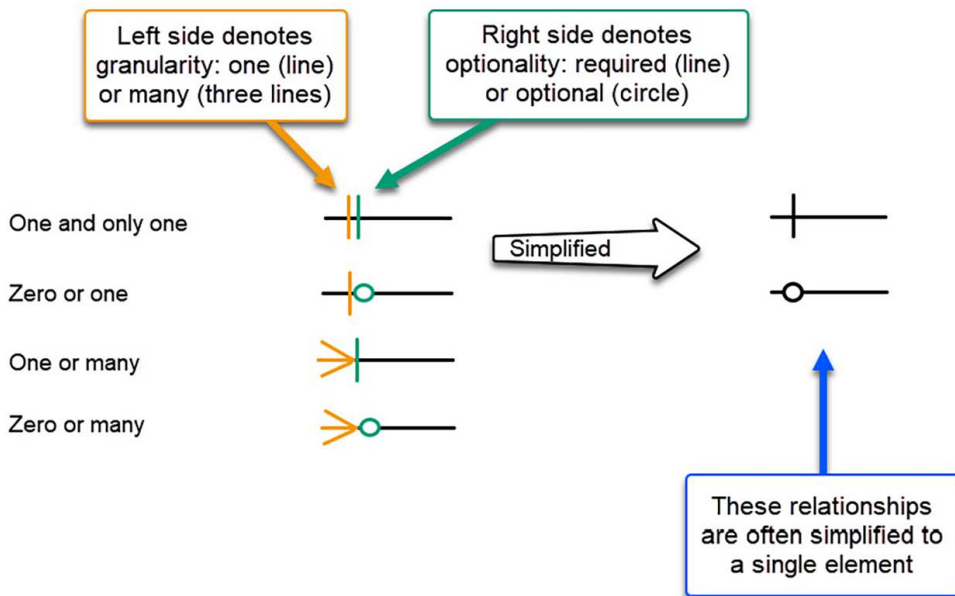


Figure 7.5: Crow's foot relationship types

In logical/conceptual modeling, a relationship name (also known as a role name) can be specified to add context. When specifying cardinality, remember that it applies to both edges of the relationship—meaning, a relationship can be read in either direction, as shown in the following example:

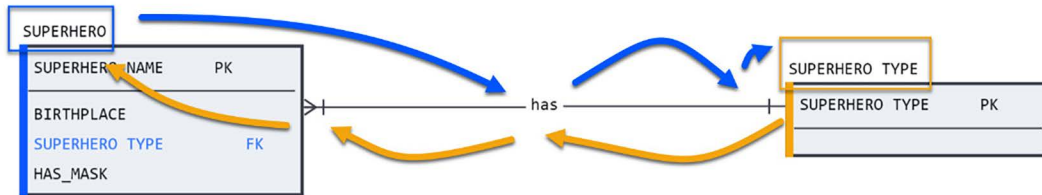


Figure 7.6: Reading a crow's foot relationship

The diagram is read from the entity to the role name, to the connector on its far side, and to the other entity. Take the following example:

- A superhero has one and only one superhero type
- A superhero type is had by one or many superheroes

Using crow's foot notation, database modelers can capture functional details of the minimum and maximum granularities in entity relationships. However, crow's foot cardinalities do not map one to one onto physical designs. When technical precision is needed for physical models, IDEF1X notation is preferred.

IDEF1X

IDEF1X (correctly pronounced *eye-def-one-ex* but often shortened to *eye-dee-fix*) notation was invented in the 1970s by the US Air Force. The appeal of IDEF1X is how closely it maps onto physical database structures. The entities and relationships defined in IDEF1X notation can be unambiguously translated into SQL, and vice versa.



While the IDEF1X standard does include a specific connection relationship syntax (e.g., P, Z, n, n-m), this syntax is not easily readable and is not widely used.

Instead of cardinality, IDEF1X notation focuses on the strong or weak entity nature of the relationship. Strong entity relationships are depicted by dotted lines and are referred to as *non-identifying*. Relationships to weak entities use a solid line and are called *identifying* because they require a primary key from a strong entity to identify a unique instance.

Many-to-many

Although crow's foot notation allows for M:M relationships in theory, in practice, the approach suffers from one significant limitation: it prevents modelers from keeping their conceptual/logical model in sync with their physical model.

Recall that the following relationship is impossible to create in a physical database without using an associative table.

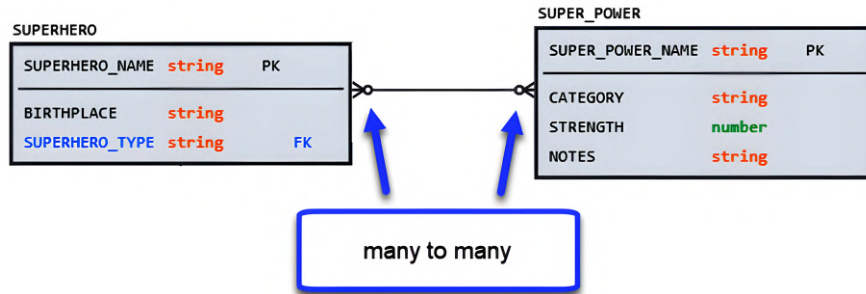


Figure 7.8: An M:M relationship between two tables

However, by coopting the relationship diamond from Chen notation to represent the associative table, we can keep the physical and conceptual models in sync while keeping the necessary context within the model.

The following figure shows the same M:M relationship in both physical and logical models.

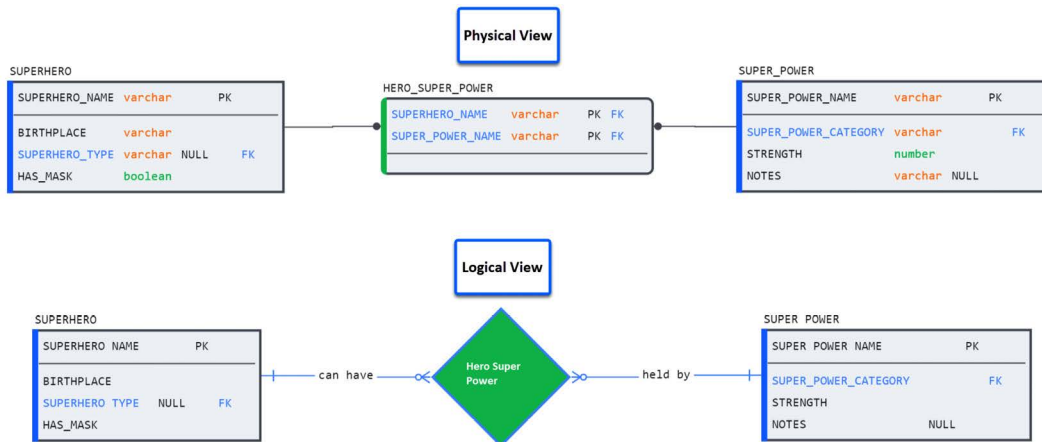


Figure 7.9: An M:M relationship synchronized between physical and logical models

Now that we know how to retain the context of an M:M relationship and keep it in sync across logical and physical models, let's break down the notation for subtypes.

Representing subtypes and supertypes

In *Chapter 2, An Introduction to the Four Modeling Types*, we saw how subtypes were used on logical diagrams to represent inheritance. Recall the example in the following figure showing how a SUPERHERO entity can have four subtypes.

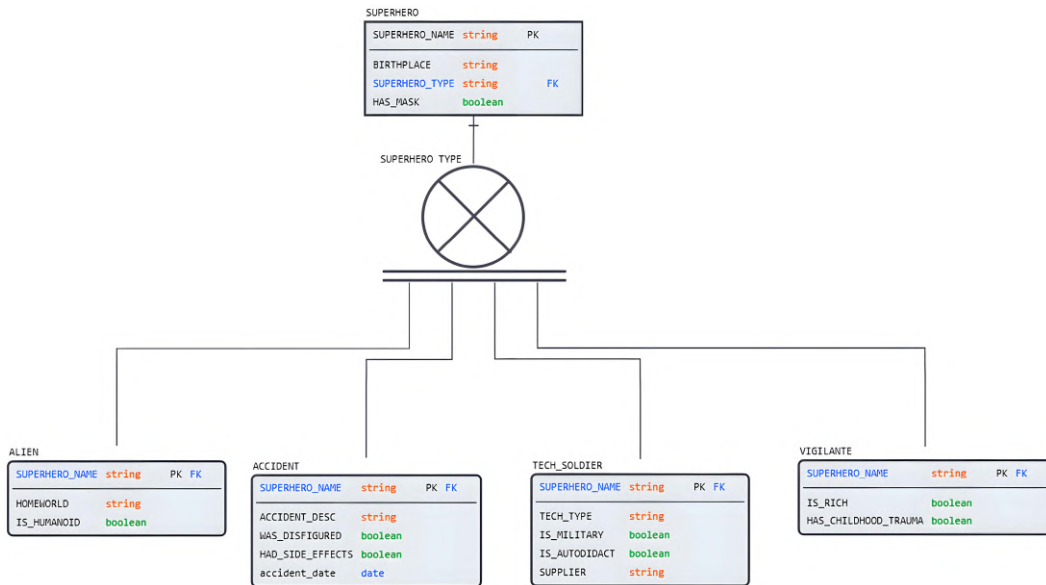


Figure 7.10: Four superhero subtypes are displayed using a discriminator symbol

The figure not only identifies the four tables as subtypes of **SUPERHERO** but also gives us context into the nature of the subtype—that it is both complete (double parallel lines) and exclusive (letter X in the center). Let's understand these properties and how to denote them.

Subtype relationships have two binary properties to describe their nature:

- **Complete/incomplete:** Indicates whether all the subtypes are known or whether more can be added in the future, respectively. The previous diagram is an example of a complete relationship, as all four superhero types are present.
- **Inclusive/exclusive:** Indicates whether the supertype can exist as multiple subtypes. Since a superhero can only belong to one of the four subtypes, the relationship is exclusive.

These properties can be displayed using a mix of IE and IDEF1X notation. The central discriminator icon can be displayed as a circle or the letter D rotated 90 degrees counterclockwise. The defining elements are the letter X through the middle to denote an exclusive relationship and single or double lines at the bottom to describe complete and incomplete, respectively. The four configurations can be seen in the following figure:

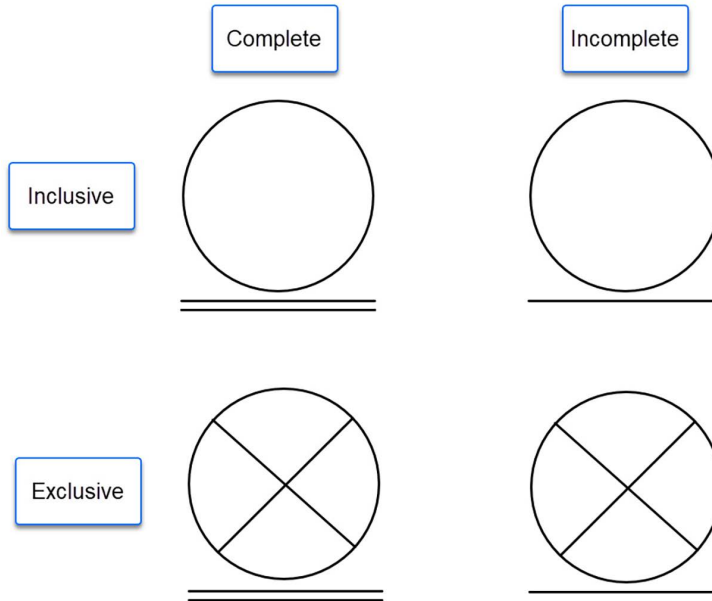


Figure 7.11: Four possible configurations of a discriminator symbol



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now that we have covered how to depict the physical database on a diagram and extend the visual context beyond what the physical database is capable of storing, let's consider how we can take advantage of the primary challenge we laid out at the start of the chapter: keeping the models in sync.

The benefit of synchronized modeling

Throughout this chapter, we have seen examples of diagrams describing data models and capturing details that may not have a strict corollary in SQL. By simplifying modeling to its core components—many of which are likely to be understood even by people without a formal background in database design—we ensure that modeling is not confined solely to the database team. As you recall, a model describes not only the database but also the business itself.

Keeping the physical and conceptual models in sync provides another benefit: the model can be viewed at any time at various levels of abstraction because the underlying physical elements remain the same. Users can view the model at a physical level when more detail is needed and flip to a conceptual view if they wish to see it more abstractly.

To illustrate this, the various diagrams in *Figure 7.12* are displayed side by side, demonstrating how physical and conceptual elements can be mixed and matched as necessary to perform an analysis.

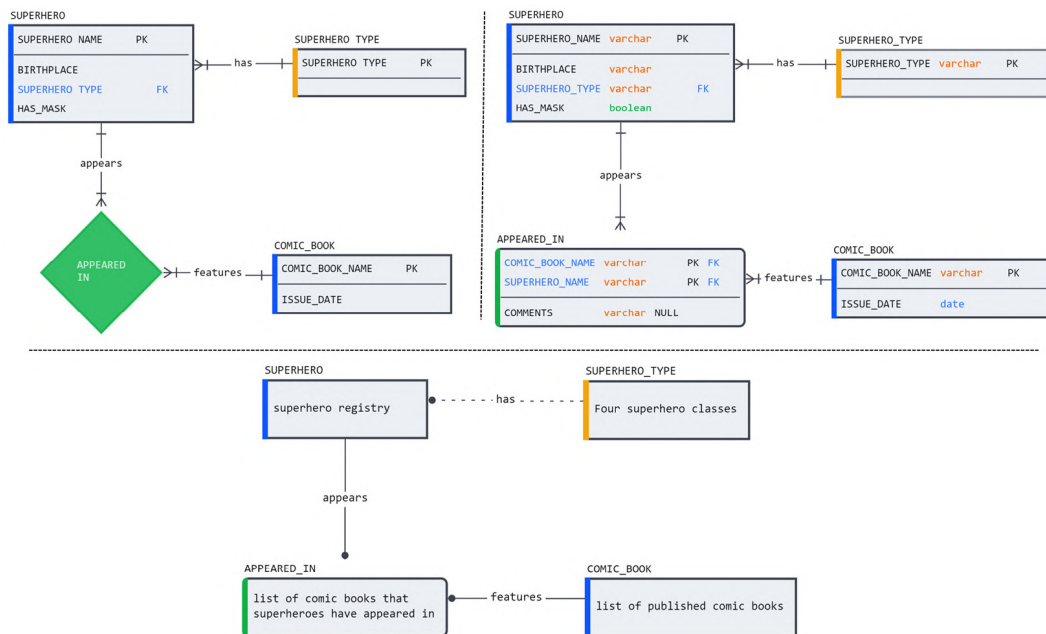


Figure 7.12: Elements from conceptual, logical, and physical models displayed side by side

Keeping the entities consistent across models carries an additional benefit in that no other maintenance is required to keep the models consistent. By changing the detail level, no conversion or rework is needed to adapt one style to the other.

Now that we've seen how modeling notation has evolved and how to harness its most relevant and valuable elements in a simple-to-use and simple-to-read style, let's review the practical takeaways of this chapter.

Summary

In this chapter, we looked back at the history of relational modeling and the various notation styles that have been used to help visualize the designs. Focusing on the most straightforward and practical semantics of these modeling styles, we could zero in on a pragmatic approach that preserves technical precision while being simple enough to be understood even by people without a formal background in database design. The modeling technique proposed in this book keeps the number of entities synchronized across conceptual, logical, and physical models, thus saving manual effort and ensuring that anyone at any level of detail can view the same model.

Using the consolidated list of modeling conventions, we looked at how to display an entity on a relational diagram in the simplest way possible. Next, we covered relationship notation and the additional context that can be conveyed beyond a simple line using two of the most popular conventions in database modeling: crow's foot and IDEF1X. The former is ideal for communicating granularity details between entities, and the latter uses a simplified style that maps exactly into the physical model.

We also learned how to go beyond what the physical model can display and added functional context through conceptual/logical modeling conventions such as subtypes and M:M relationships.

Having learned how to use the semantic elements in the modeling toolkit and mastered Snowflake architecture and objects in previous chapters, we are ready to begin the modeling journey in earnest, starting from conceptual modeling.

The next chapter will break down the conceptual modeling process into simple, repeatable steps that bridge the gap between functional and technical teams, allowing them to communicate effectively to identify and translate business processes into database designs.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

Part 2

Applied Modeling from Idea to Deployment

This part focuses on the practical application of data modeling techniques across different stages of the database design process. You will learn how to put conceptual modeling into practice by gaining a solid understanding of the data entities, relationships, and business rules that define the underlying data structure. You will also learn how to enrich these details and add nuance using a logical model that cannot be captured in a physical database. This part also delves into the critical concept of database normalization, essential for minimizing data redundancy and ensuring accurate and consistent data storage. After covering database naming and structure and exploring the best practices for creating intuitive, meaningful, and scalable database schemas, you will learn how to put physical modeling into practice while optimizing database performance.

This part has the following chapters:

- *Chapter 8, Putting Conceptual Modeling into Practice*
- *Chapter 9, Putting Logical Modeling into Practice*
- *Chapter 10, Database Normalization*
- *Chapter 11, Database Naming and Structure*
- *Chapter 12, Putting Physical Modeling into Practice*

8

Putting Conceptual Modeling into Practice

Conceptual database modeling is a high-level approach to designing a database that focuses on capturing business entities and their relationships. This approach allows designers to develop a deeper understanding of the data, making it easier to identify potential issues or inconsistencies in the design. It can also make the database more flexible and adaptable to future changes and make it easier to understand and use for people who are unfamiliar with it. As a result, conceptual database modeling can help make a database more effective and efficient at supporting the needs of an organization.

In this chapter, we will generate a conceptual model from scratch using Kimball's **dimensional modeling (DM)** technique. This approach brings the data team together with business experts to ensure that database designs reflect the reality of business processes from inception, thereby saving costly rework down the line.

However, conceptual models are not solely used in initiating new designs—they carry all the same benefits for existing physical schemas. This chapter will also explain a simple method for adapting a physical design to a conceptual model and generating an easy-to-read diagram that can be used by business and technical teams alike.

In this chapter, we will cover the following main topics:

- Discovering DM and its applications
- Constructing a bus matrix using Kimball's four-step method
- Creating a conceptual model using the bus matrix

- Reverse engineering a conceptual model from a physical schema
- Embarking on conceptual design

Out of all the modeling types, conceptual captures and displays the least amount of detail. This makes conceptual modeling ideal for getting acquainted with a database landscape at a high level and for designing one from scratch. Designing data models is an art honed over many iterations, but where do you begin if you are new to modeling?

Dimensional modeling

In the early 2000s, Ralph Kimball and Margy Ross published the groundbreaking book, *The Data Warehouse Toolkit*, which has persisted for decades as the authoritative blueprint for constructing database designs. Many of the terms, concepts, and techniques described in later chapters of this book trace their origins to *The Data Warehouse Toolkit*—whose latest edition fittingly carries the subtitle, *The definitive guide to dimensional modeling*.

To be clear, Kimball's approach is not the only way to go about creating a conceptual model. The Agile-based **Business Event Analysis and Modeling (BEAM)** method (see the *Further reading* section at the end of this chapter) and other techniques are also worth exploring. However, Kimball's methodology is widely used and universally recognized and will serve as a springboard for our modeling journey.

Understanding dimensional modeling

DM is a longstanding technique for making databases simple by aligning them to business processes—a theme that should now be familiar to you. As simple as that sounds, DM covers many wide-ranging topics, from business requirement workshops to types of fact and dimension tables, to industry-specific modeling best practices.

This chapter will focus on the early stages of DM and discuss how data teams can engage with business experts to uncover the dimensions and business processes that lay the groundwork for a conceptual model. However, one point needs to be addressed before getting started.

Setting the record straight on dimensional modeling

Critics of DM have argued that platforms such as Snowflake, which provides cheap and easily scalable computing resources, have made the effort and rigor involved in DM obsolete. Nonsense. A variable-spend platform such as Snowflake can get very expensive if left unchecked. Grounding designs in time-tested frameworks is a good way to keep costs under control.

Another frequent argument is that analytics engineering through tools such as dbt has quickened development to such a pace that it allows you to build and fix a design if need be, in less time than what DM calls for. While analytics engineering does accelerate the delivery cycle, not having a solid grounding in the fundamentals only enables you to make mistakes faster. For example, dbt *snapshots* use Kimball's **Type-2 slowly changing dimensions (SCDs)** under the hood and require a good understanding of their fundamentals to use them effectively.

The third criticism of the Kimball methodology goes something like this: *Kimball is no longer relevant because Data Vault or <some other modeling> is easier to scale*. Ensemble methodologies, such as **Data Vault 2.0** (covered in *Chapter 18, Scaling Data Models Through Modern Techniques*, of this book) are indeed highly scalable and effective modeling frameworks. However, they do not suit all scenarios and still require knowing and implementing DM concepts (a satellite is also a Type-2 SCD).

Finally, despite having debunked the main arguments against DM, it remains to be said that this book is not advocating for DM to be used as the principal framework for database or warehouse design. Rather, since DM objects and language permeate database design, it is essential to know the rules and fundamentals and to understand when and where to deviate.

With this in mind, let's dive in and see how we can use DM principles to kick-start a conceptual model or validate an existing physical design through the same means.

Starting a conceptual model in four easy steps

This section will review the four-step DM method for creating a conceptual model by identifying the business processes and dimensions involved. Recall that the aim of conceptual modeling and database modeling as a whole is to align as closely as possible to the reality of the business operations. For this reason, this stage of the modeling process will require the most collaboration from the business teams to identify the conceptual building blocks for future technical developments.

To witness this process in action, we will use the tables in the `SNOWFLAKE_SAMPLE_DATA.TPCH_SF1` schema to represent a real-world retail organization. So that you understand what we are working toward, you can explore the sample tables in the Snowflake **user interface (UI)** or see the following diagram resulting from this exercise:

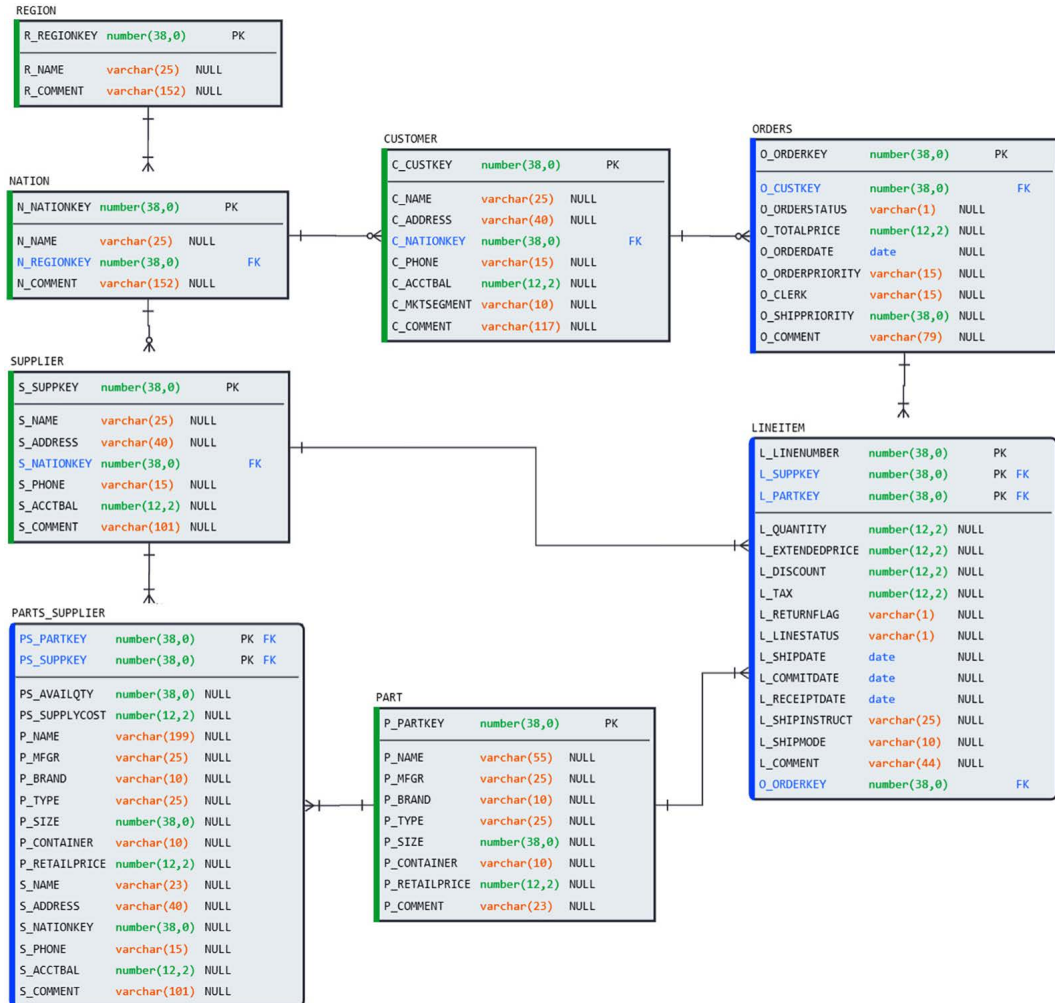


Figure 8.1: A physical model of Snowflake's sample TPC-H schema

Before reviewing each step individually, here are the four steps that Kimball suggests for starting DM at a conceptual level:

- Define the business process
- Declare the grain
- Identify the dimensions
- Identify the facts

Modeling begins by organizing workshops with business domain experts and department stakeholders to determine the answers to these four points. This is where the reality of the business is discussed, formally defined, and agreed upon by the stakeholders so that its relevant data can be modeled and captured. Everyone must understand that although the concepts are being discussed at a high level, they will later be used to determine design decisions regarding database details, business rules, and their governing logic—details that can be extremely difficult to alter in later stages of development.

The outcomes and decisions established during the collaborative workshops should be considered binding contracts between the business and technical teams, who will later carry them forward to implementation.

Before making its way to an **entity-relationship (ER)** diagram, the information captured during these discovery sessions is captured in what Kimball calls the **bus matrix**. In a bus matrix, each row represents a business process or subject area, and each column represents a dimension or fact table—their intersections form the relationships or transactions within an organization.

As we work through each of the four steps, we will see how the bus matrix unfolds and helps guide the development of the model. As the first step in the process, let us begin by working with the domain experts to identify the existing business processes in the TPC-H organization.

Define the business process

As the consultants in the film *Office Space* famously inquired: *what would you say you do here?* While the answer for a given organization might sound simple, such as *selling merchandise* or *taking reservations*, the reality is usually much more complex when additional activities, such as procurement, inventory, promotions, and cancelations, are considered.

Recall that the primary goal of modeling is to capture the relevant metrics and properties we want to track, measure, and analyze. Think of business processes as the transactions that occur naturally as the business operates. In DM, transactions are referred to as **facts**, and tables that record them and (nearly always) pair them with metrics are called **fact tables** (more on fact tables in a bit).

Starting with the company's primary function, such as selling, registering, or manufacturing, extend the discussion to the supporting functions, such as sourcing inventory, stocking warehouses, and contracting with suppliers, until every department has been heard from and has relevant operational processes cataloged.

The resulting list generated from these meetings will form the left side of the bus matrix. In the TPC-H example, the list may look something like this:

Purchase Parts
Track Inventory
Sell Parts
Customer Returns
Supplier Returns

Figure 8.2: The business processes that will later form the bus matrix

Once the business processes have been identified, it's time to go into more detail and determine their lowest grain of analysis.

Determine the grain

In the previous step, we listed the operational transactions that make up our business processes. These transactions will become fact tables in the database that we are designing. But what does a row for each transaction represent? For example, the primary function of our business may be to sell parts, but do we record and analyze such a transaction at the part or customer level? Or can it be both or either, depending on the business unit?

In answer to this question, we must now determine the **grain**—the lowest level of detail required for meaningful analysis of each business process. Generally, the recommended grain is the lowest level of detail captured by the business processes themselves—this is referred to as the **atomic grain**. Grounding our design at the atomic grain ensures that it can accommodate any subsequent rollups and aggregations to higher levels of analysis. With this approach, changes in business requirements will not require changing the model because the model already contains the maximum granularity recorded by the business processes.

When selecting the grain, we must also ensure that different grains are not mixed in the same fact table. For example, a sales fact table maintained at the grain of daily transactions must not include monthly or yearly totals. Doing so would result in costly and inefficient database update operations on the table every time a transaction was recorded.

Conversations about grain will naturally bring to light the entities involved in the operational functions: customers, suppliers, and parts. This information will help us determine the other half of the bus matrix.

Determine the dimensions

The entities that came to light during the conversations surrounding business processes help give context to the nature of our business. Entities help answer questions such as *Who?*, *What?*, and *Where?* and will often contain attributes that offer further descriptive context.

In our example, the conversation may have started with sales and moved through the logical paces of the part being sold, the supplier who provided the part, and the warehouse used to store the inventory of available parts until all the relevant entities were identified. These entities, which often correspond to singular nouns in spoken language, will become dimension tables in the database we are building and contain attributes that need to be defined and modeled.

Having identified several key dimensions, such as Customer, Order, and Part, we can draw the other axis of the bus matrix. The following screenshot shows the resulting grid when business processes are placed next to their component dimensions:

Business Processes (facts)	Entities (dimensions)						
	Date	Order	Customer	Location	Supplier	Part	Inventory
Purchase Parts							
Track Inventory							
Sell Parts							
Customer Returns							
Supplier Returns							

Figure 8.3: The bus matrix with business processes and dimensions

Notice that the Date dimension is the first one named. Facts, such as sales, streams, and logins, have at least one relevant time dimension that needs to be recorded (although factless fact tables also exist and are covered in later chapters).

When all dimensions have been identified, it's time to finalize the bus matrix by nailing down the facts.

Identify the facts

DM defines facts as tables containing transactions among business dimensions paired with relevant metrics. The metrics record information is pertinent to the grain of the fact table, which comprises relevant dimensions. This relationship can be mapped directly in the bus matrix by putting an X at the intersection of business processes and their required dimensions.

In our example, the finished bus matrix may look something like this:

Business Processes (facts)	Entities (dimensions)						
	Date	Order	Customer	Location	Supplier	Part	Inventory
Purchase Parts	X			X	X	X	X
Track Inventory	X				X	X	X
Sell Parts	X	X	X		X	X	
Customer Returns	X	X	X		X	X	X
Supplier Returns	X			X	X	X	X

Figure 8.4: The bus matrix that maps business processes to the dimensions involved

Using the resulting matrix, we can revisit the earlier question of sales grain and see whether we've answered it correctly. While the problem may seem technical, the answer is always business-driven. Perhaps the atomic grain of a sale consists of its date and time, the details of the customer and product being purchased, and the metrics surrounding the transaction, such as price, quantity, and discount. Our business teams may want to track total sales and cancelations at the customer level while reviewing product margins and discounts at the part level. We can construct tables to accommodate business needs by capturing this atomic grain in the design.

Performance and maintenance decisions at further stages of the modeling process may result in multiple rollup tables being created from the granular atomic facts. Still, the analysis at the conceptual level is sufficient to accommodate any such decisions.

Having laid the groundwork for uncovering the facts and dimensions, let's visualize the bus matrix in a conceptual model.

From bus matrix to a conceptual model

Now that we have a bus matrix worked out and agreed upon with the business teams, we can begin to transfer the details onto a conceptual diagram. The first step in the process is to create the dimensions as entities. Usually, the most relevant attributes would have also been identified as part of the business workshops and could also be included in the model. However, to split the exercise into cleanly separated steps, this will be done in the next chapter, where we discuss logical modeling.

Here is what the diagram would look like with only the dimensions included:

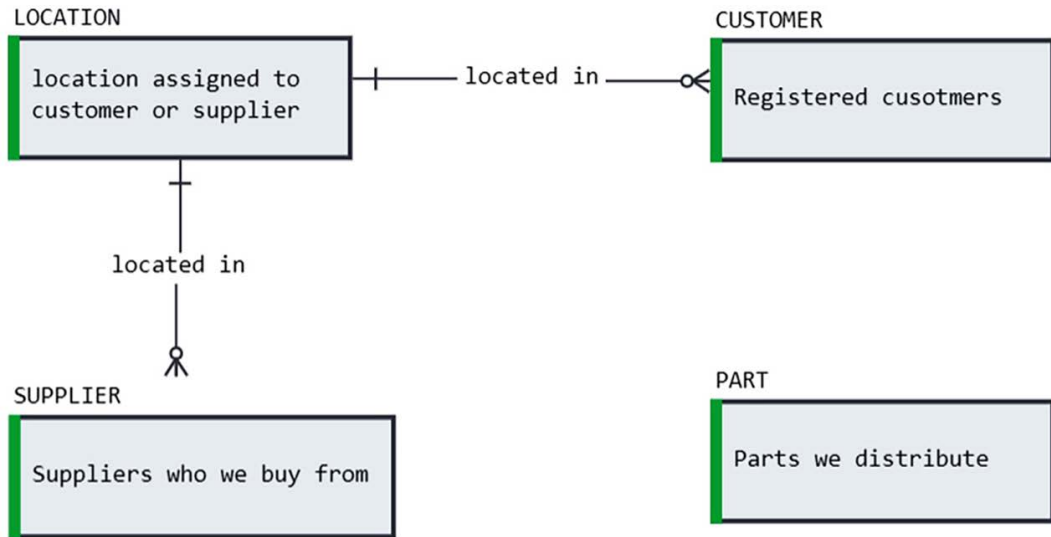


Figure 8.5: A conceptual model showing only dimensions

With the entities in place, we can now create the fact tables and connect them using the relationships and business rules defined in the workshops. The model now contains all the facts and dimensions listed in the bus matrix, as shown in the following diagram:

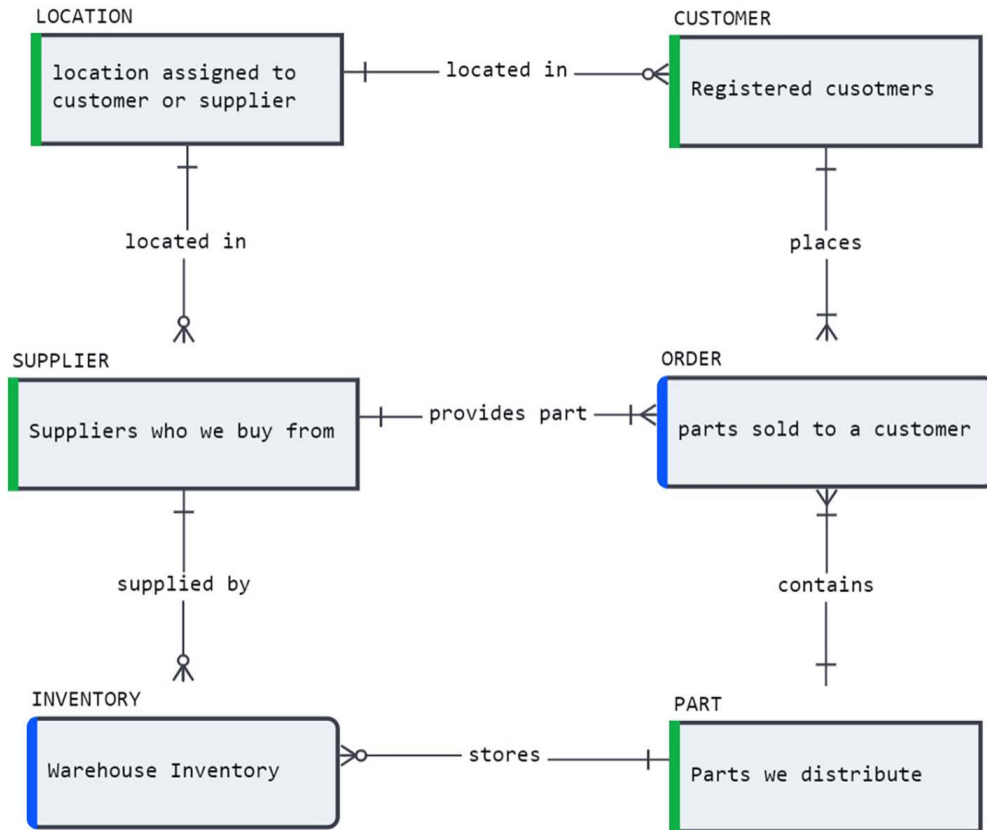


Figure 8.6: The conceptual diagram resulting from the bus matrix



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Notice that while five business processes are listed in the matrix, only two fact tables have been modeled in the diagram. Often, the same fact table can include enough details to cover multiple business processes—for example, customer sales and returns, because they occur at the same grain.

By asking a few business-oriented questions, we have seen how to determine the grain of the various operations of our company and identify the related facts and dimensions, putting it all together into the bus matrix. A conceptual diagram can be easily generated and used as a foundation for later steps in the modeling process using the information in the bus matrix. But just as a conceptual model can be used to initiate database design by formalizing existing business processes, it can also work in reverse to help sense-check existing physical designs.

Modeling in reverse

Real-world enterprise operational systems and data warehouses can reach heights of great complexity, especially if they have evolved over many years and absorbed untold numbers of dubious design decisions. Even the standing members of the data team may not be able to explain every detail, let alone a new hire or a business analyst. Fortunately, conceptual modeling can help simplify and visualize existing database landscapes as quickly as it does when starting new ones.

Database modeling tools allow users to take schema definitions as DDL and ingest them to generate a corresponding diagram—a process known as **reverse engineering**. Since all the table structures and constraints are already defined in the database, generating an ER diagram and visualizing the database landscape can be done in seconds. Because reverse engineering uses Snowflake object DDL, it produces physical models. As physical models contain the highest level of detail of all modeling types, they can be easily abstracted into conceptual. While modeling tools can automate most of the steps of going from physical to conceptual, the task can also be performed by hand without much effort by following the four-step method in reverse.

We are embarking on an exercise of going from *what is* to *what ought to be*. The latter part cannot be known from the database but verified with the business teams through workshops and discussions. This exercise aims to generate a best-guess conceptual model from *what is* so that it can facilitate those discussions. Let's begin by labeling the existing tables.

Identifying the facts and dimensions

Start by taking the existing tables in the physical schema and classifying them into facts and dimensions using previously discussed rules of thumb. Dimension tables will typically be named using singular nouns and contain descriptive attributes but not quantitative measurements. Facts are usually named using transitive verbs, including measures that quantify each transaction.

In our example, the result of this exercise would look something like the following diagram:

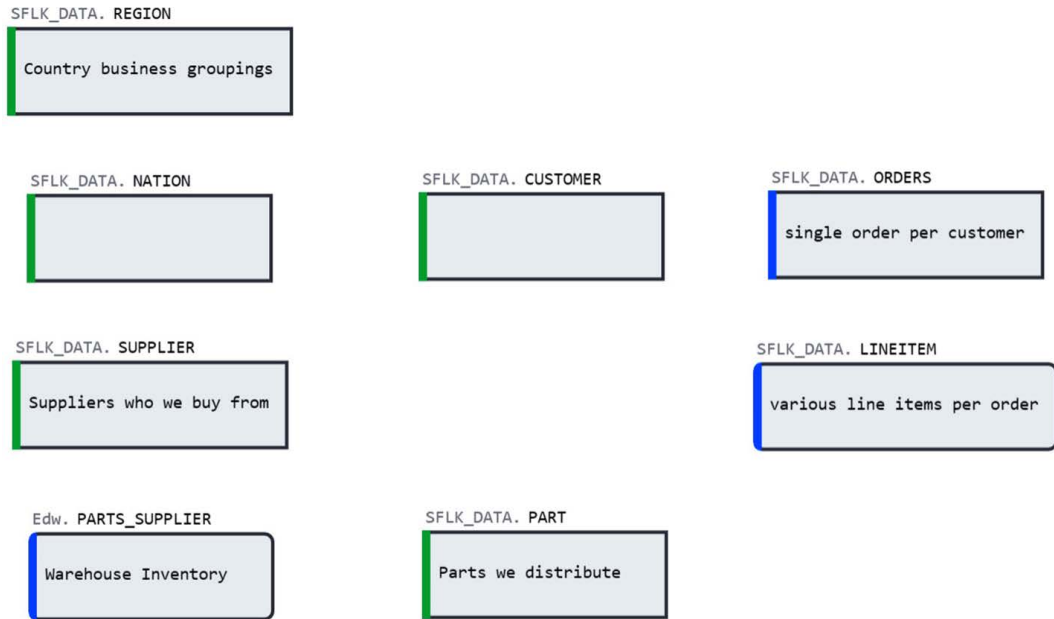


Figure 8.7: Facts and dimensions identified from the physical model

Now that the entities have been mapped, we must identify the relationships.

Establishing the relationships

Identifying the relationships between tables can be swift or arduous depending on whether standards and best practices have been observed in the physical schema. For example, if primary and **foreign keys (FKs)** have been declared, modeling tools will use this information to automatically establish and visualize all the relationships in the schema.

Absent these constraints, we can rely on column naming conventions to intuit the relationships. Starting with the dimensions, look for columns that resemble the table name and include something that hints at their being a **primary key (PK)** (keywords such as *id*, *key*, or *code*). For example, the **NATION** dimension contains a column called **N_NATIONKEY** and is, therefore, likely to be the PK. Test the assertion with **GROUP BY HAVING**, which should return no values greater than one if the column is a PK, and the underlying data is clean.

The following query should return no results if **N_NATIONKEY** is the PK for **NATION**:

```
SELECT N_NATIONKEY , COUNT(*) AS CNT FROM NATION
GROUP BY 1
HAVING CNT > 1
```

Remember that even the data cannot conclusively prove what the PK should be—it can only point to a highly likely candidate. Only the business users can conclusively confirm the grain of a business dimension.

Once PK candidates have been proposed for dimensions, look for instances of the PK columns in facts and other dimension tables. In DM, columns with the same structure and meaning across multiple tables are known as *conformed dimensions*. For example, an ID column in the CUSTOMER table is a conformed dimension of CUSTOMER_ID in SALES if both columns share the same data type and values. When individual attributes within a dimension are consistent across multiple tables, they are considered *conformed attributes*. For example, STAR_RATING may be a conformed attribute in both PRODUCT and SUPPORT_SURVEY. Consistent naming of columns across tables is immensely helpful in this type of analysis. Modeling tools rely on the naming of conformed attributes to automatically propose FK relationships.

Once the relationships have been identified, attempt to establish their cardinality and optionality details. While the table constraints give some clues about the nature of the relationship, only the business teams can confirm the reality (although the data can be an indicator). In the following example, the physical constraints can have multiple conceptual interpretations:

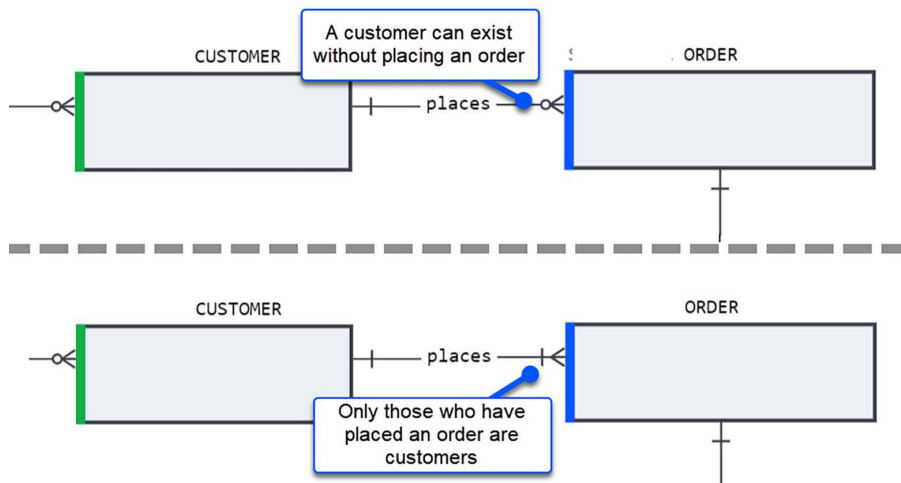


Figure 8.8: Multiple conceptual representations of a physical relationship

Once we have established all the entities and relationships and made an informed guess as to their cardinality, we have completed the draft of a conceptual model, which might look something like the following diagram:

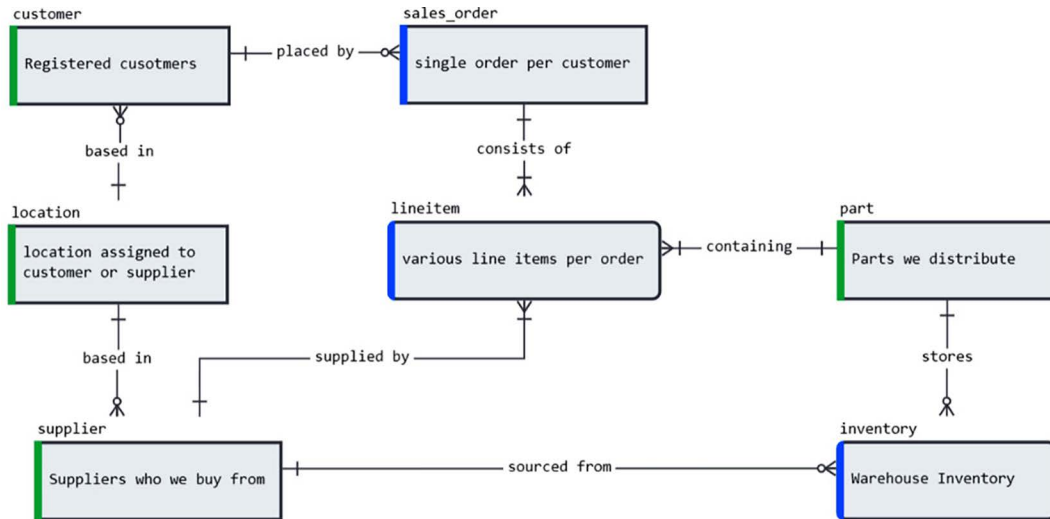


Figure 8.9: A conceptual model of a physical schema

Using the conceptual model, which provides a clear visual representation of the existing database design, we can open the dialogue with the various domain teams responsible for each business process and their respective dimensions to validate what has been built.

Proposing and validating the business processes

Even though we are working with a relatively small schema, classifying and identifying the tables and relationships has not been easy. We have generated a simplified vision of the business model using a conceptual diagram that must now be validated with the business teams and product owners. Recall that the same fact table can capture multiple business processes and identifying them cannot be done without input from the business teams.

Based on the input of business owners and domain experts, the conceptual model can be adjusted and refined to reflect the true reality of business operations. The conceptual review may even result in changes to the physical model if the business and database models have deviated over time.

Whatever the scenario, we now have a clear and accurate conceptual database model that fully aligns with the business it supports. The conceptual model can help analysts and new hires quickly get acquainted with the business. After having put in the work to validate it, the conceptual model should be maintained going forward by repeating the process described in this chapter.

Now that we have seen how to generate a conceptual model using Kimball's methods as well as working backward from a physical database, let's review what we've learned.

Summary

In this chapter, we have seen how a conceptual model and its accompanying diagram make it easy to visualize an organizational business model and validate it with domain experts against the intended business operations that it is meant to support.

To get started in conceptual modeling, we used Kimball's DM methodology, which has been used for decades to guide the design of database and warehouse architectures. Kimball uses a four-step method to initiate discussions between the data team, business teams, and domain experts to identify the business processes that an organization engages in.

Once business processes have been identified, we determine their grain or lowest level of detail pertinent to our business. Describing the grain will also help us discover the core dimensions of our business. Plotting dimensions and business processes on a chart, known as the bus matrix, produces an elegant cross-section of our business and lets us transfer its details easily to a conceptual diagram.

It is not only new designs that benefit from the aid of conceptual models. A conceptual model can also be used to visualize and validate a physical database. Modeling tools can automatically generate physical diagrams from existing DDL through reverse engineering. Otherwise, modelers can reverse the four-step Kimball method to arrive at a conceptual model starting from a physical database.

Depending on its initial design or changes that have occurred over time, a physical database may deviate from the actual business model of the organization. Conceptual diagrams are an important tool in opening the conversation with business teams to align the physical and business models by giving non-technical users a simple way to understand what has been built.

However, the collaboration between the data and business teams does not stop at the conceptual model.

In the next chapter, we will expand on what we have built by adding functional details and preparing our model for physical deployment.

Further reading

Kimball and company popularized and significantly contributed to DM and data warehousing. Their books are still relevant today and offer great overviews of concepts and fundamental terminology.

For an Agile alternative to Kimball, you should look at the BEAM method, described in *Agile Data Warehouse Design*:

- Kimball, Ralph, and Ross, Margy. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. John Wiley and Sons, 2013.
- Kimball, Ralph, et al. *The Kimball Group Reader: Relentlessly Practical Tools for Data Warehousing and Business Intelligence Remastered Collection*. John Wiley and Sons, 2015.
- Corr, Lawrence, and Stagnitto, Jim. *Agile Data Warehouse Design: Collaborative Dimensional Modeling, From Whiteboard to Star Schema*. DecisionOne Consulting, 2011.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

9

Putting Logical Modeling into Practice

In the previous chapter, we observed data teams working with business teams to create a high-level conceptual model representing an organization's main entities and relationships. While a conceptual model helps to understand the overall structure and requirements of the data without going into excessive detail, the next stage in the modeling process requires us to go further and develop a detailed model to be used as a blueprint for moving to a physical database design.

To complete the logical model, the data team will have to collaborate with domain experts from the business once again to expand the list of entities, attributes, and relationships that will be used in the database, as well as the data types and constraints for each element. Just as the conceptual model held bidirectional benefits in both developing a fresh design and simplifying an existing one, a logical model is not merely a stepping stone in the modeling process.

Logical models are much more detailed and granular than conceptual ones. Although they lack the database-specific information of a physical model, they make up for it by providing contextual information that physical databases cannot capture. Once developed, a logical model provides a basis for taking a design forward and adapting it for deployment on a specific database.

In this chapter, we will cover the following main topics:

- Identifying and adding attributes and data types
- Adding structural and technical details to entity relationships
- Resolving many-to-many relationships
- Expanding weak entities
- Understanding inheritance in database models

Let's continue where we left off with the conceptual model developed in the previous chapter and evolve it into a logical one.

Expanding from conceptual to logical modeling

In the previous chapter, we used Kimball's four-step methodology to develop a bus matrix and create a conceptual model based on the recorded information. Details that informed the bus matrix were gathered through workshops and discussions between the data team and experts on the business side, who could elucidate the business's operational model and create a functional artifact—the conceptual diagram.

The following diagram shows the conceptual model as it looked at the end of the exercise:

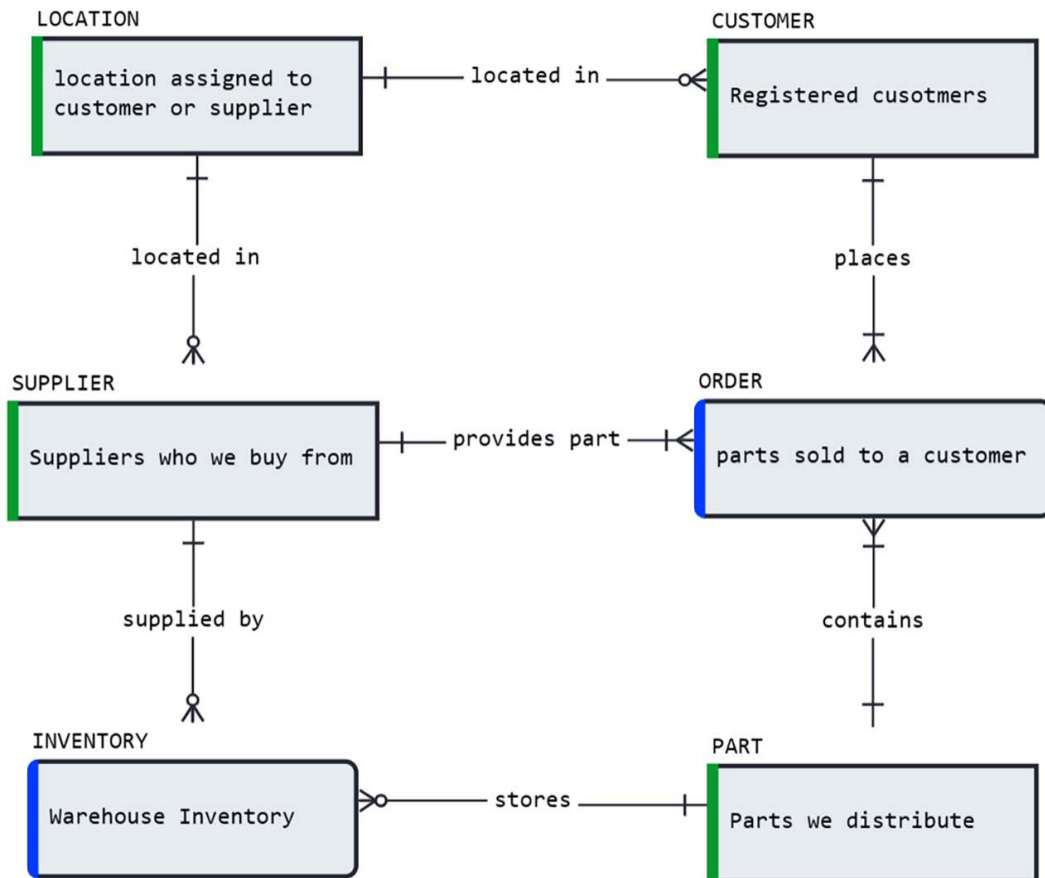


Figure 9.1: The final conceptual diagram from Chapter 8

Despite the obvious deficiencies, such as missing attributes and Snowflake-specific object properties, the conceptual diagram did not attempt to add any contextual detail on the functional relationships between the entities, such as subtypes and many-to-many associations.

To uncover the relevant fields that should be added to our model's entities, we should again engage business experts specializing in the related business domains.

Adding attributes

The most important detail in a dimension is its unique identifier. This determines what constitutes a unique instance of each entity in our business. Examples of unique identifiers include things such as serial numbers for parts and employee IDs for a company's human capital. Domain experts from each business area can confirm these and other necessary details. We will use the `CUSTOMER` dimension as an example to identify the relevant details and incorporate them into the model.

Suppose we sat down with the head of the sales team to learn about how our organization identifies customers and their relevant attributes. The domain expert explains that, besides typical customer attributes such as name and address, our organization is also interested in tracking the account balance and identifying its market segment. The sales team also explains how customers are grouped into regions based on their `NATION` attribute, maintained in a separate `LOCATION` dimension. Besides identifying the attributes and their data types, we also learn that all of them are required (cannot be `NULL`) besides the optional `Comments` field.

Snowflake supports most basic SQL data types in table columns and semi-structured and geo-spatial data types. For compliance with the **American National Standards Institute (ANSI)** SQL standard, Snowflake also supports synonyms for many common data types found in other databases and logical modeling. For example, although Snowflake stores string data as `VARCHAR`, a column could be modeled as `STRING` or `CHAR`, and upon deployment, such a column would be created as `VARCHAR`. For a summary of supported data types and aliases, refer to the following documentation: <https://docs.snowflake.com/en/sql-reference/intro-summary-data-types.html>.

With the feedback from the sales team, we now have a logical model of the customer dimension, complete with details about its **primary key (PK)**, **foreign key (FK)**, data types, and their mandatory or optional nature. The transition from conceptual to a logical level of detail is shown in the following figure:

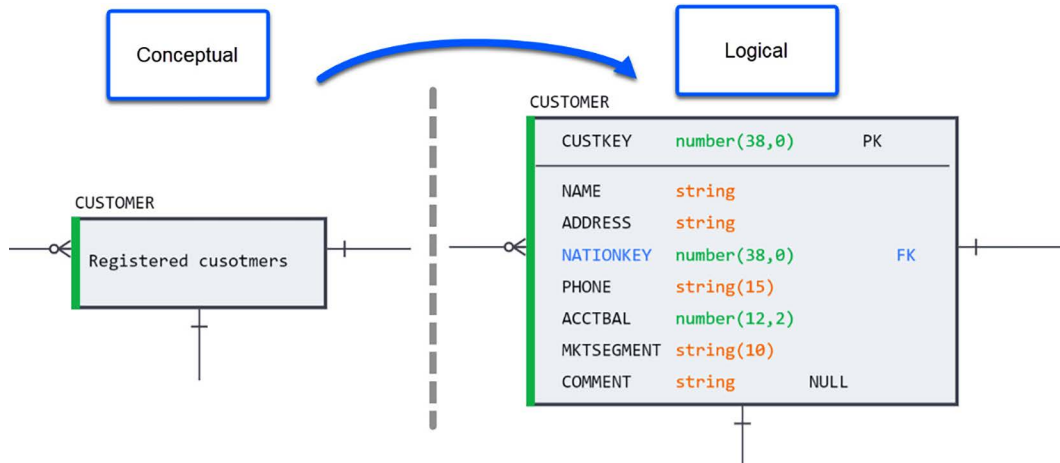


Figure 9.2: Going from conceptual to logical modeling

After completing the exercise for all existing dimensions, we focus on how they come together to record the facts.

Cementing the relationships

With attributes and PK and FK relationships established in the dimensions, we again turn to the business experts to help flesh out the fact tables and determine their grain.

Building from the conceptual model, we ensure that all relationships have a defined granularity and optionality and that the data teams understand their business context. Mistakes made in the fact table definition are the costliest to reconcile and could result in costly readjustment, so extra care should be taken. Work with domain experts to ensure that the fact tables capture the true atomic grain of the information recorded by business operations and that the optionality is correctly understood.

In the following example, the logistics team confirms that our warehouse separates parts by supplier to facilitate returns and inventory tracking. In short, the warehouse can store many parts from many suppliers. Role names are also verified and documented at this point for non-obvious relationships. The resulting logical model now looks like this:

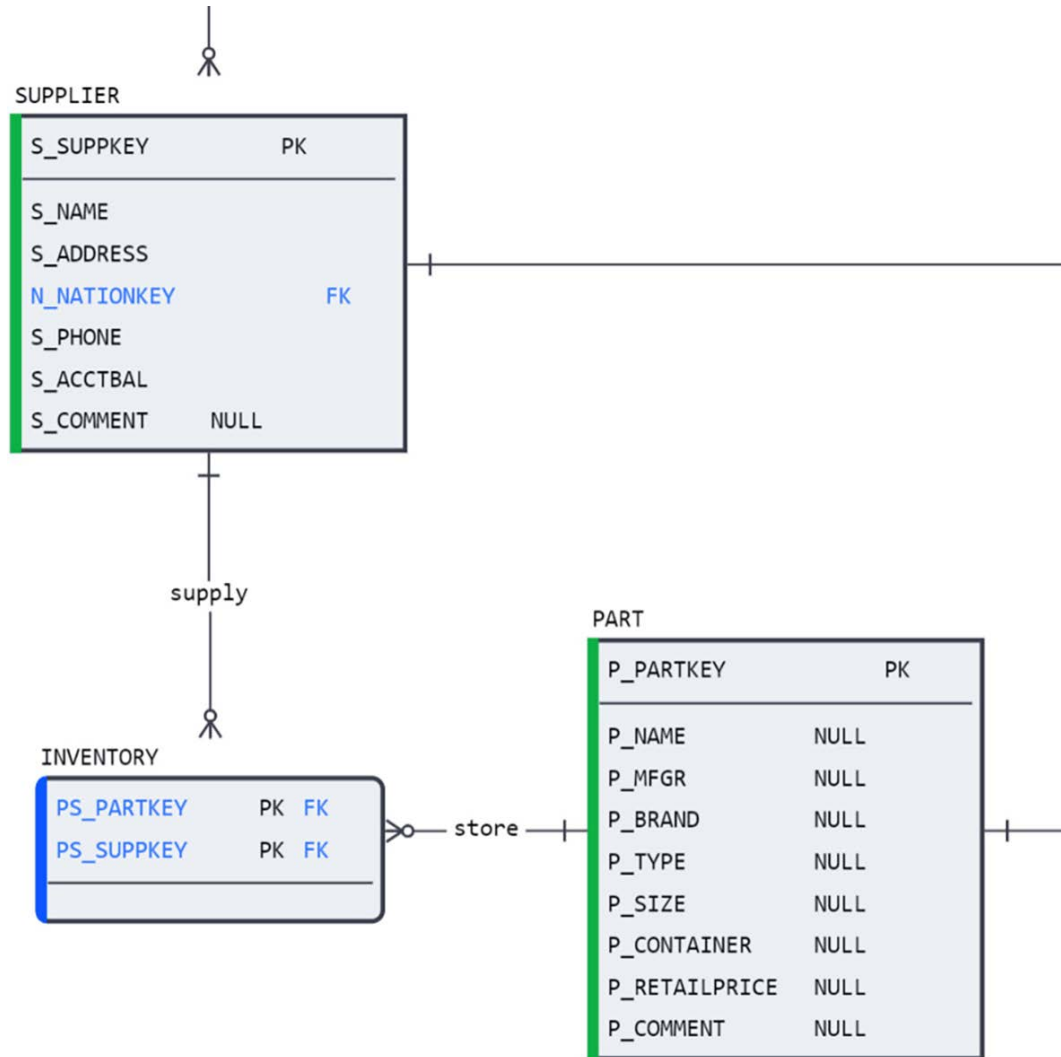


Figure 9.3: Determining the fact table grain

However, we have yet to define the measures describing the facts stored in the table. For warehouse inventory, this can be as simple as the quantity of the parts on hand and the cost for which each part was obtained (since this can fluctuate by supplier).

The following diagram shows the final state of the INVENTORY table, complete with cardinality, granularity, role names, and required measures:

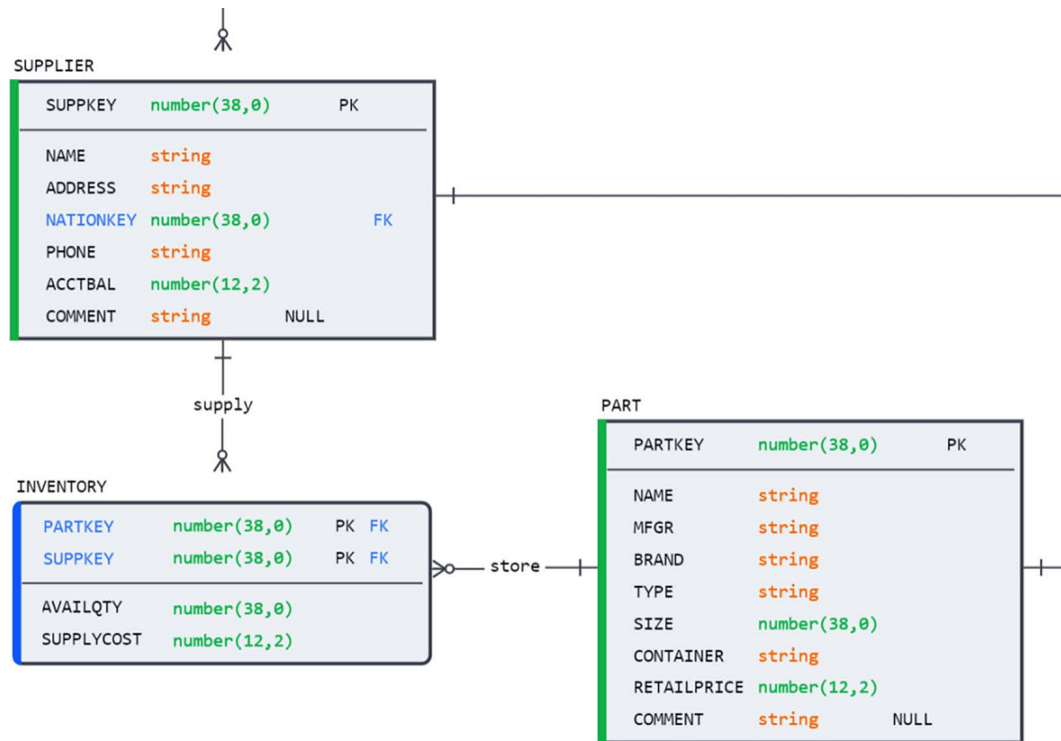


Figure 9.4: Fact table with grain and measures

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Repeating the exercise for all the facts in our model gives us a complete vision for proceeding to subsequent design phases. The next step is to identify the many-to-many relationships.

Many-to-many relationships

A **many-to-many (M:M)** relationship exists between two entities when each can have many instances of the other. For example, in our organization, suppliers can supply many parts, and parts can be supplied by many different suppliers. Therefore, a third entity (Inventory), called an **associative** or **junction entity**, is required to store and track which part came from which supplier.

Recall the example from *Chapter 2, An Introduction to the Four Modeling Types*, where a list of all available superpowers required an associative table to represent the possible superpowers of a superhero:

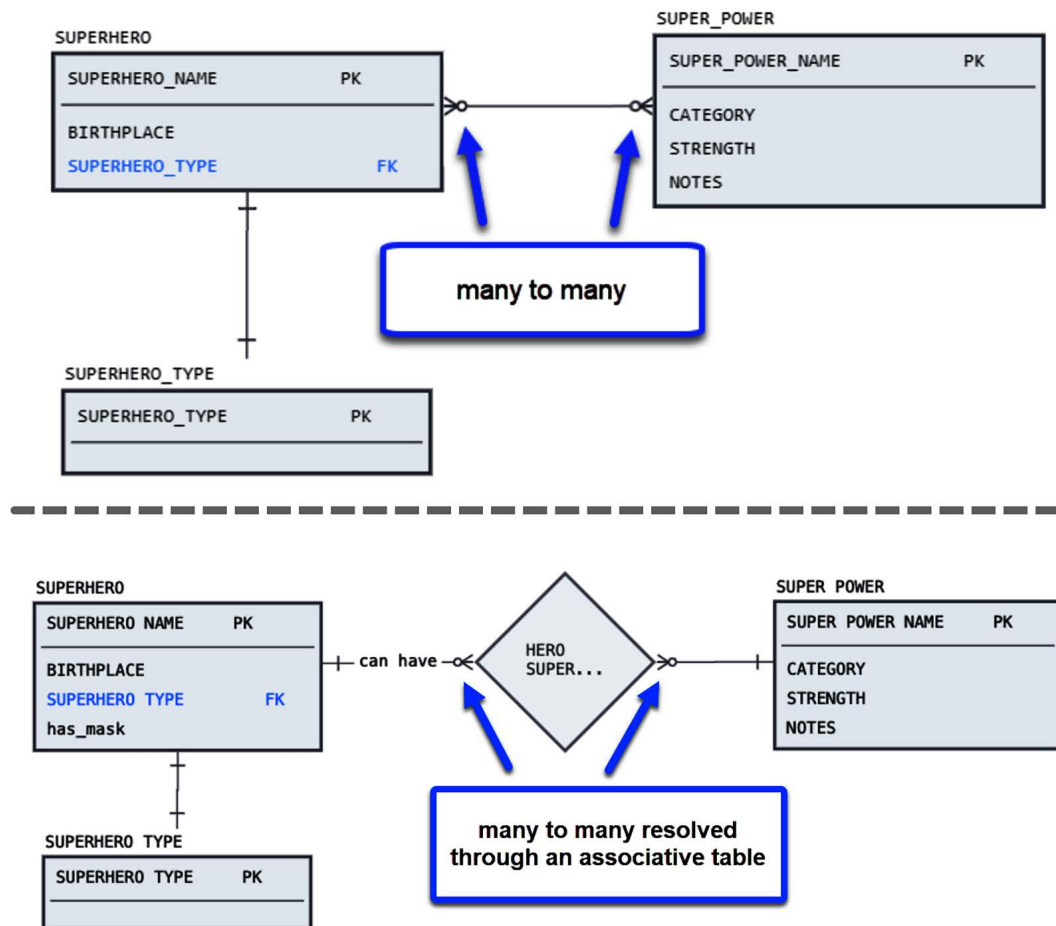


Figure 9.5: Resolving an M:M relationship through an associative table

In the current example, there are no unresolved M:M relationships because the real-world operation of our business necessitates a physical entity (the warehouse) to store an inventory of many parts provided by many suppliers. However, by using the diamond notation to capture the business context of this relationship, we communicate beyond the database level the idea that inventory cannot exist on its own. Rather, it only exists due to an interaction (procurement) of a part from a supplier:

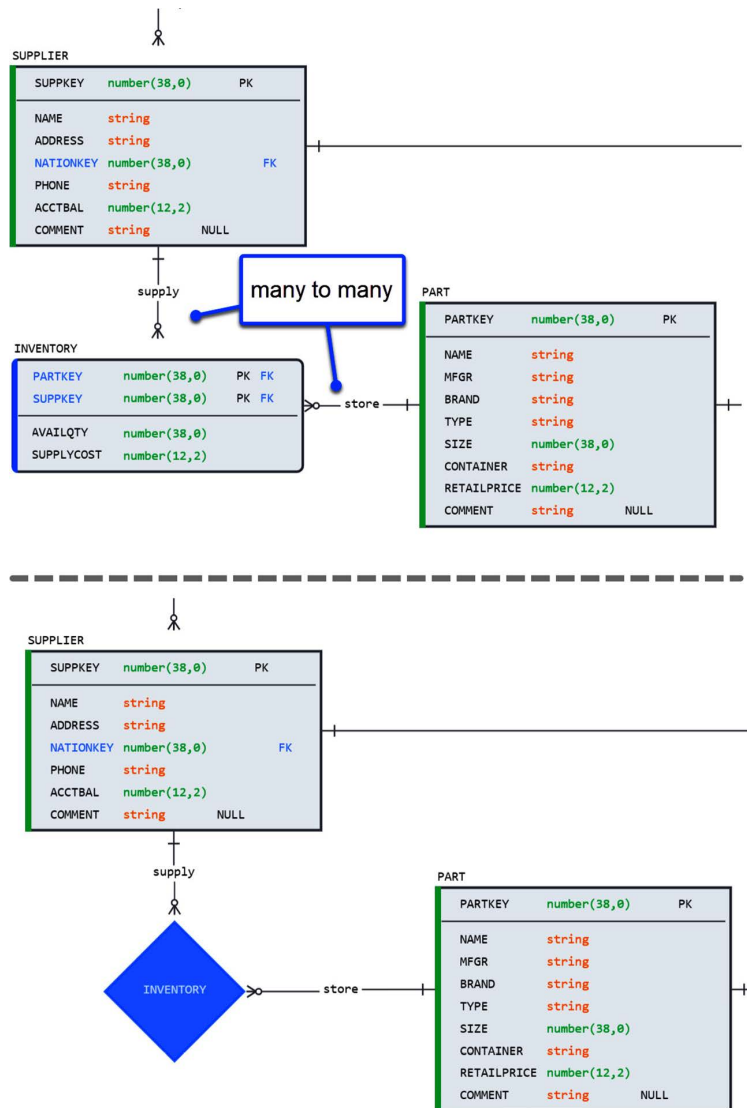


Figure 9.6: Highlighting an existing M:M relationship on a logical diagram

An M:M relationship is one example of an entity that cannot exist in isolation—weak entities are another.

Weak entities

A **weak entity** is an entity that cannot be uniquely identified by its own attributes; it must use an FK in conjunction with its own to create a PK. In other words, a weak entity has an FK as part of its PK instead of a strong entity that depends entirely on its unique identifier.

In a diagram, weak entities are identified by their rounded corners instead of the perpendicular edges of strong entities. However, their implications extend beyond semantics and impact how the model handles changes in the data. Let's look at an example using the ORDERS table.

At first glance, ORDERS has multiple crow's feet connectors, which make it seem like an associative M:M table. Notice the similarities between ORDERS and INVENTORY in the following diagram:

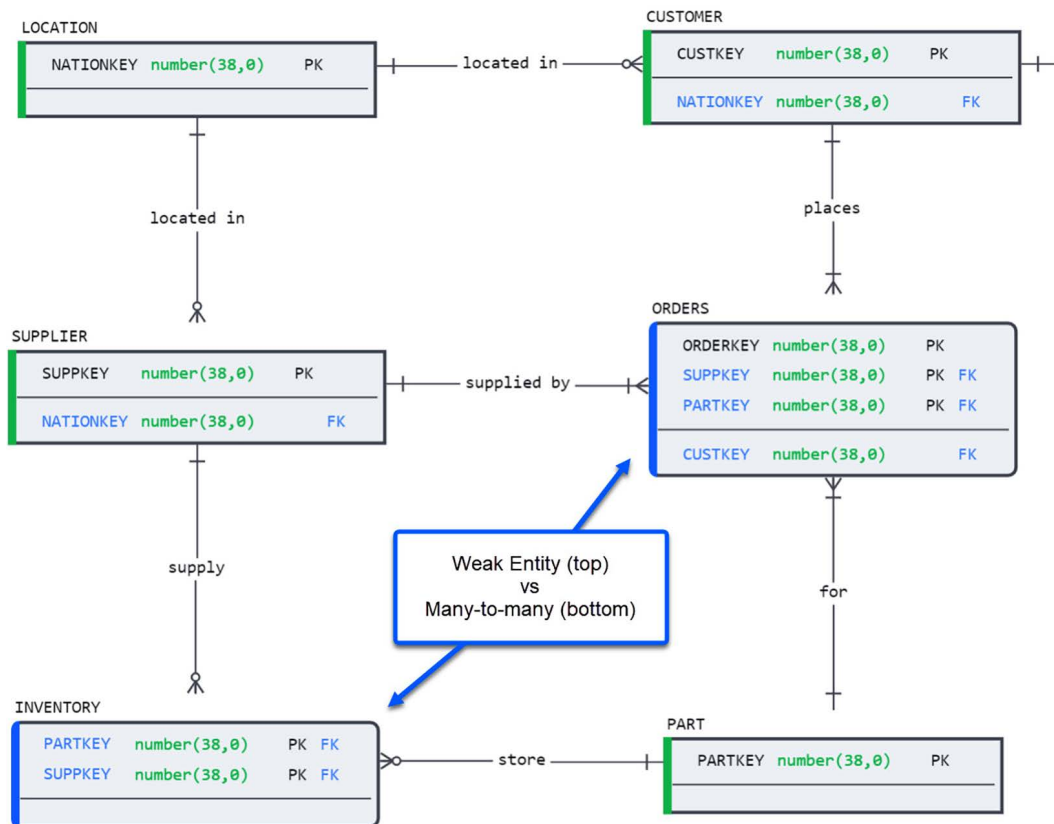


Figure 9.7: Weak entities distinguished from M:M by their PKs

Orders are business entities with unique codes that must be tracked and audited. On the other hand, INVENTORY has no unique identifier, nor does it make business sense to give it one—recording the PKs of the part, supplier, and quantity on hand is sufficient.

After speaking to the sales team, we learn that an order is viewed differently depending on the business context. An order is a single transaction between a customer and the company viewed from a sales perspective. However, from a logistics perspective, an order consists of many records to reflect individual items purchased, returned, or discounted. Our business teams need to reference and analyze individual orders, but keeping everything in one table makes that difficult.

The solution is to separate orders into a strong entity (ORDERS) and a weak entity (LINEITEM), thereby facilitating storage, analysis, and maintenance. This means using the order identifier (ORDERKEY) as the PK of the ORDERS entity and creating a weak entity called LINEITEM, with its own unique identifier and previously existing FK columns as part of the PK and ORDERKEY as a non-PK, FK attribute.

The transition from a weak entity to separate strong and weak entities can be seen in the following diagram:

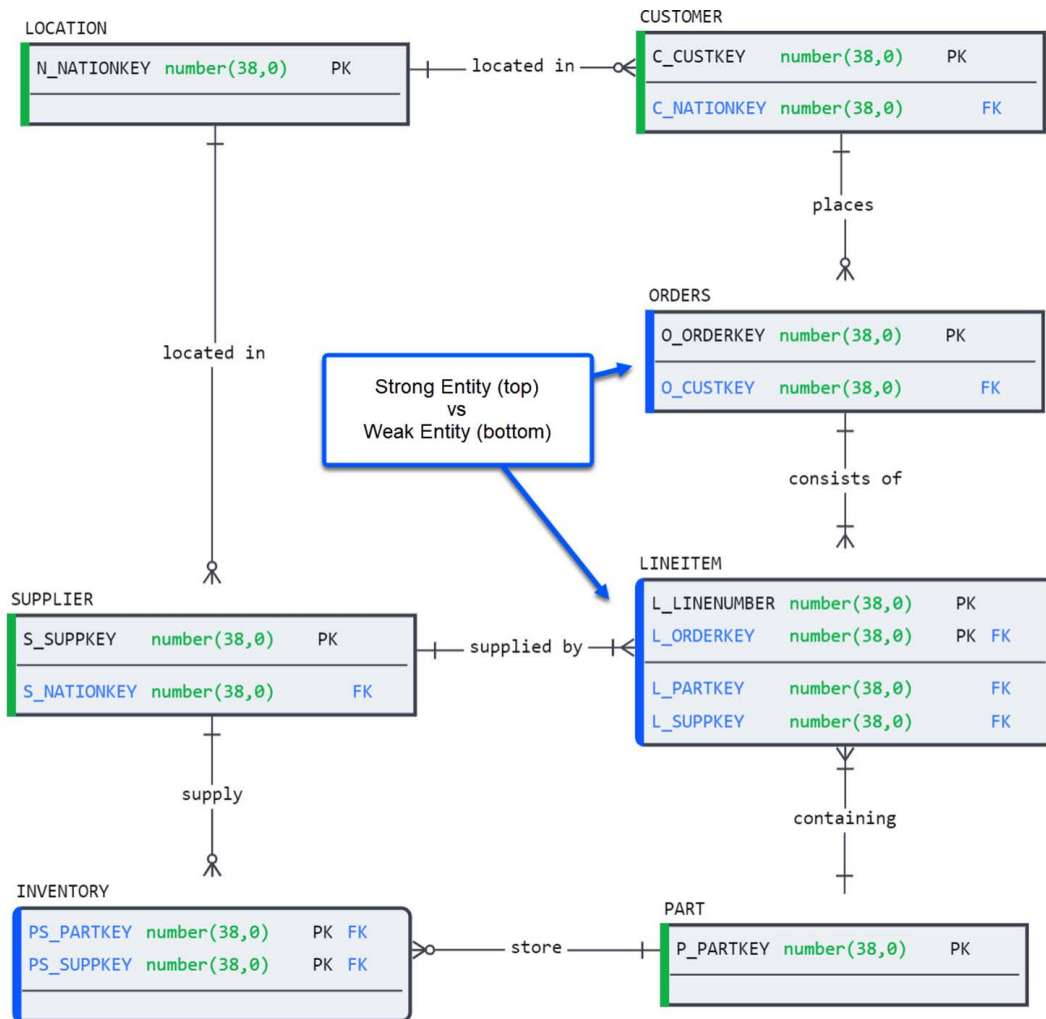


Figure 9.8: Separating a weak entity into strong and weak entities

In this scenario, orders can be quickly reviewed and counted because each is stored in one individual record, and line items can be changed and updated in a separate table.

While weak entities rely on strong entities to identify them, they are inherently different elements. Let's explore what happens when we want to model entities that are similar in nature and share common characteristics.

Inheritance

Those with a background in object-oriented programming will be familiar with inheritance—the passing of properties from a parent to a child class. The same is true in data modeling, except that parent and child classes are called supertypes and subtypes, respectively. **Subtypes** share common characteristics with a **supertype** entity but have additional attributes that make them distinct.

Suppose that our sales team maintains a particular category of loyal customers based on the volume of their repeat business. A loyal customer has the same attributes as a regular customer, even sharing the same identifier, and contains attributes that describe their loyalty level and reward points.

At the physical level, the resulting table, `LOYALTY_CUSTOMER`, may appear as an ordinary child table or weak entity when viewed on a diagram:

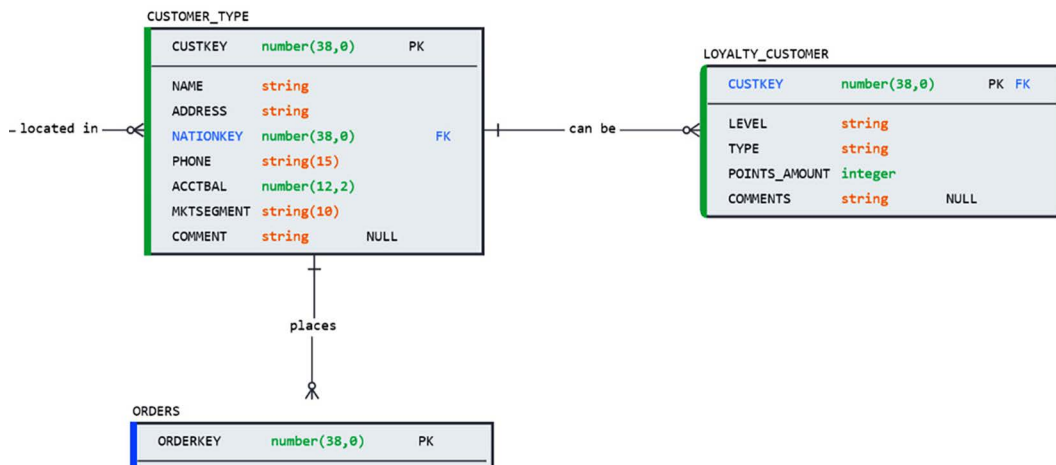


Figure 9.9: The physical view of `CUSTOMER` and `LOYALTY_CUSTOMER`

However, this would not capture the true nature of its relationship to CUSTOMER as subtype notation would. Adding a discriminator between the CUSTOMER supertype and the LOYALTY_CUSTOMER subtype adds context that would otherwise be lost at the database level:

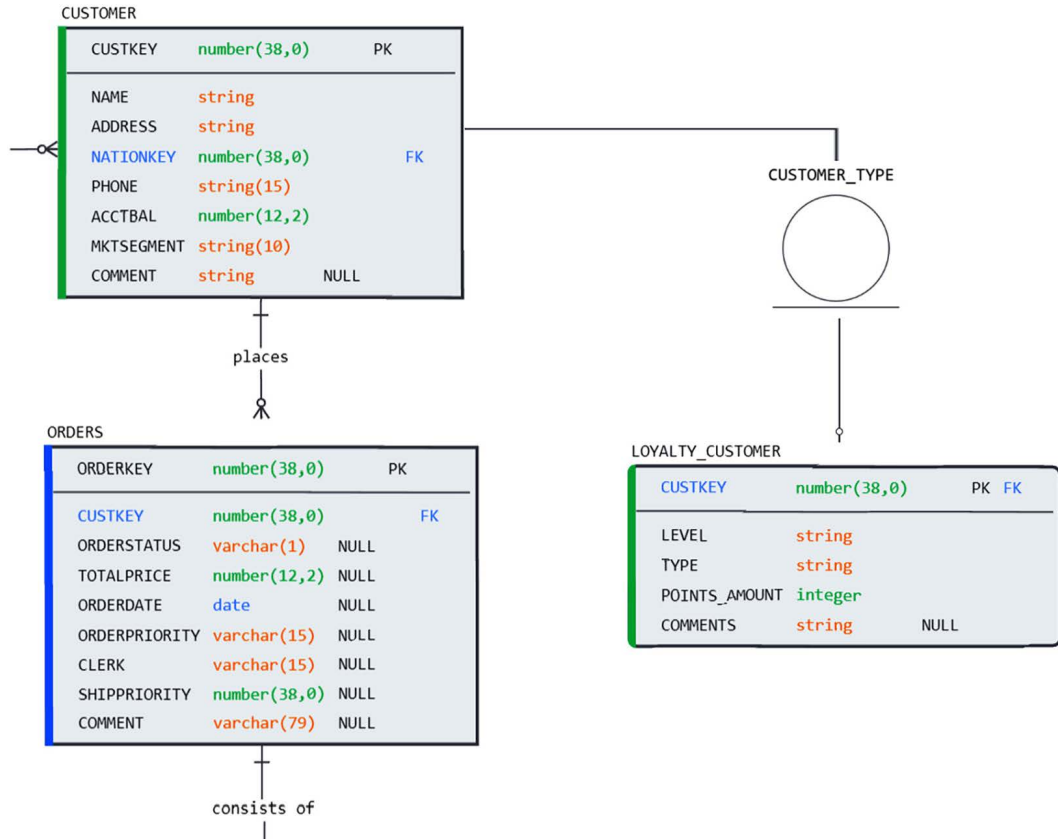


Figure 9.10: A logical view of a CUSTOMER supertype with LOYALTY_CUSTOMER subtype

Compared to Figure 9.9, Figure 9.10 transmits the following business context:

- CUSTOMER is the parent/supertype entity whose attributes can be applied to its child/subtypes (defined by the discriminator symbol)
- The LOYALTY_CUSTOMER subtype is incomplete (single, as opposed to double—the line below the discriminator), meaning more subtypes may be added in the future
- The LOYALTY_CUSTOMER subtype is inclusive (the empty circle without an X mark), so CUSTOMER can be a LOYALTY_CUSTOMER type, and other possible types that are yet to be defined

This completes the transition from a conceptual to a logical model. Let's review the completed model in the following diagram:

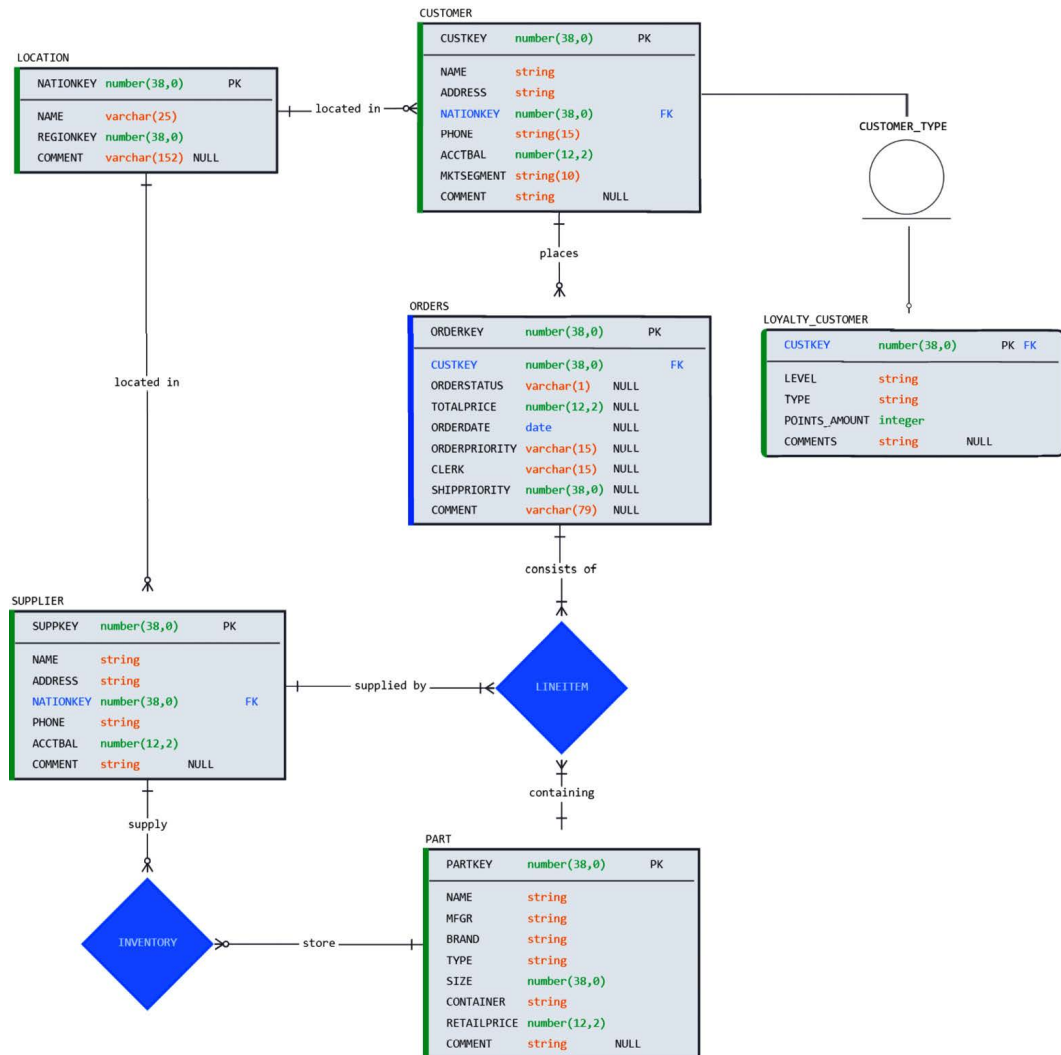


Figure 9.11: A completed logical model



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



The conceptual model that we started with in *Figure 9.1* is a representation of the data requirements of our organization that is easy to understand and communicate to stakeholders. It does not focus on the specific implementation details of the database but rather on the overall structure and relationships between different data entities.

On the other hand, logical database modeling creates a detailed model of the (pending) database that adds structural details and expresses the existing entities and relationships in greater technical detail. The goal of logical modeling is to lay the groundwork for designing the actual database while capturing additional nuances that cannot be represented using physical database objects.

Summary

In this chapter, we continued the modeling journey by taking the conceptual design from the previous chapter and expanding it to a logical design by adding structural details and technical details that closely resemble the physical model we will create and deploy in later chapters.

Continuing to work with business experts from our organization, we identified the attributes that will be used to capture crucial business master data and transactions. We defined each business entity's identifiers, attributes, measures, and data types to do this.

Once attributes have been set in place, we reviewed the relationships between entities to understand the nuances of their associations to architect them in such a way as to fit the needs of the business and simplify their maintenance.

We started by identifying the M:M relationships. These relationships require an associative table to capture the interactions between the entities involved but are not considered entities themselves. Associative tables, therefore, do not have a business key but rely on the FKs of their related entities.

Next, we checked for weak entities that cannot be uniquely identified by their attributes alone. Depending on the business needs, weak entities may encumber the maintenance and analysis of the underlying entity, which can be split into separate, strong entities to overcome the problem.

Finally, we covered the concept of inheritance and how it manifests in database design through subtype and supertype relationships. Identifying subtypes and capturing their business context through a discriminator symbol helps set them apart from regular child tables.

The next step in the modeling journey is converting the logical model to a physical Snowflake database. However, before diving into the physical design, we will need to get comfortable with the concept of normalization and cover the best practices for structuring and maintaining physical objects in Snowflake, which we will do in the coming chapters.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

10

Database Normalization

Previous chapters have explored the method of capturing the real-world business workings of an organization and modeling them using visual semantics. The resulting model and accompanying diagrams make it easy for the domain and data teams to reach a consensus about the business's fundamental entities and the interactions between them. However, as the modeling process approaches the physical stage, we should understand that many ways exist to structure the data at the database level. The process of dividing the data into smaller, modularized segments is known as normalization. In this chapter, we will understand how it works and the advantages and disadvantages that go along with it.

Normalization is not a binary classification but a spectrum of ever-increasing rules a design must satisfy to achieve a stated level. While normalization contains the root word *normal*, along with the obvious positive connotations, more normalization does not necessarily imply a *better* design. As we will see in this chapter, greater levels of normalization impose more rigor on the database's structure—reducing data anomalies at the cost of increased maintenance and analytical complexity.

This chapter will explore normalization through its various levels to understand the rules, advantages, and trade-offs. After a thorough overview, we will focus on the most common types of normalization and understand the aspects that make them popular choices for database design. This chapter aims to provide a complete overview of normalization, its benefits, and its costs to allow users to find the right fit for the task at hand.

This chapter will cover the following main topics:

- Understanding normalization on a spectrum
- The pros and cons of normalization
- How much is too much? Finding the sweet spot
- Reviewing the data anomalies that normalization protects from
- Taking an in-depth look at normalization from the first to the sixth normal form

An overview of database normalization

Database **normalization** is the process of organizing a database in a way that reduces redundancy and dependency within its tables. This is achieved by breaking a large table into smaller ones and linking them through foreign key relationships. Doing so leads to fewer data inconsistencies and improved data integrity. A normalized database results in a modular design that is easy to scale and modify.

Normalization occurs through escalating stages of formal rules called **normal forms**, ranging from the **first normal form (1NF)** to the **sixth normal form (6NF)**—although the first through third are most commonly used and are sufficient for most use cases.

Each normal form builds on the requirements of its predecessor and adds additional criteria that every database table must satisfy. A normal form is considered satisfied when every table in the database meets the criteria laid out for it (and, by extension, its predecessors).

Neglecting normalization rules may result in erroneous self-contradicting data that is difficult to manage. The following screenshot shows a table that violates nearly every consistency principle there is:

Album										
Album	Artist	Artist Country	Label	Genre	Length	Format	Price	Certified Sales	RIAA Certification	RIAA Cert. ID
Aquemini	OutKast	USA	LaF	Progressive Rap, Soul, G-funk	74:47	CD	10.99	2,000,000	2× Platinum	2
Aquemini	OutKast	America	LaFace	Progressive Rap, Soul, G-funk	74:47	CD	9.99	2,000,000	3× Platinum	2

Figure 10.1: An example of data that does not follow any normalization rules

How many data issues could you spot in just two rows?

- No unique identifier makes knowing what a single row represents difficult
- Multiple values stored in the Genre column hinder the analysis
- The two rows appear to be duplicated but there is no way to tell without a primary key
- USA and America—these two values describe the same entity

- Duplicate data entry in the Label column results in misspelled values
- The Price value is different between the two records, but there is no way to know which is correct
- The RIAA Certification value is a function of Certified Sales, so 3x Platinum with 2,000,000 copies sold is impossible
- RIAA Certification is at odds with RIAA Cert. ID
- Adding a new Format value would require duplicating all Album values
- If the store in question wants to stop selling the CD format, deleting both rows would also remove album information

Normalization helps prevent data anomalies such as the preceding ones through schema design and database rules. As more normalization is applied, the possibility of anomalous information drops while the overall number of tables grows. Data quality safeguards obtained through normalization are offset by the complexity of querying information across multiple tables instead of having it in a single source.

Before we review the progression of normal forms, we need to better understand the data anomalies from which they help protect.

Data anomalies

Bad data comes in many flavors. From misspellings to improper encoding, some data quality issues can not be avoided. However, denormalized designs make it possible to walk headlong into several well-known and preventable blunders.

To understand how normalization prevents data anomalies, we need to unpack the dual dangers it mitigates—redundancy and dependency:

- **Redundancy:** Repeated data, whether within one or across multiple tables. When data values are duplicated, synchronizing everything through DML operations becomes harder.
- **Dependency:** When the value of one attribute depends on the value of another. Dependencies can be **functional** (such as a person's Age attribute depending on their name) or **multivalued** (such as Name, Age, and Hobby stored in a single table would make it impossible to delete a hobby without deleting all the people who practice it).

With these dangers in mind, let's review the kinds of data anomalies that have the potential to creep into a denormalized database design.

Update anomaly

An **update anomaly** results from a partial update of redundant information. For example, when information is duplicated over multiple records or tables, it becomes possible to update some but not all occurrences, resulting in conflicting or contradictory instances.

In the following example, the birth year, an attribute of the *Pirate* dimension, is included in the *Pirate Prizes* fact table. The birth year detail provides extra context for consumers of the fact table but results in duplicate data and opens the door to an update anomaly, as displayed in *Figure 10.2*:

Pirate Prizes		
Pirate Name	Born	Captured
Francis Drake	1540	Cacafuego
Francis Drake	1680	Nuestra Señora del Rosario
Henry Every	1659	Charles II
Henry Every	1659	Ganj-i-Sawai

Figure 10.2: Update anomaly

Because the birth year value is repeated instead of stored once as an attribute of the pirate entity, anomalous, contradicting records can occur. However, that is not the only anomaly that can result from placing details from various dimensions in a single table.

Insertion anomaly

An **insertion anomaly** occurs when it is difficult or impossible to insert new data into the database consistently and accurately. When too many details of varying granularity are bunched into a single table, inserting information that does not simultaneously tick all the boxes becomes difficult.

In the following example, pirates and ships are stored in the same table, and both values are mandated through NOT NULL constraints. Here, *Anne Bonny* presents a challenge since she hung around *Calico Jack* and never captained a ship of her own:

Pirates and their ships			
Pirate Id	Name	Born	Ship
1	Francis Drake	1540	Golden Hind
2	Blackbeard	1680	Queen Anne's Revenge
3	Calico Jack	1682	Kingston

4	Anne Bonny	1697	
---	------------	------	--



Figure 10.3: Insertion anomaly

In this example, the table constraints make it impossible to insert a pirate's information without specifying a ship. This dependency does not just encumber insertions, it affects deletions as well.

Deletion anomaly

A **deletion anomaly** occurs when deleting data from the database results in the unintended loss of related information. Like an insertion anomaly, a deletion anomaly results from too many dimensions being bunched into one table (such as `Pirate Id` and `Ship`, from the previous example).

Calico Jack never could hold on to a ship for very long. The *Kingston* was repossessed (with cargo intact) less than two months after being taken.

However, as we can see in *Figure 10.4*, deleting *Kingston* from our table also means wiping away all mention of *Calico Jack*:

Pirates and their ships			
Pirate Id	Name	Born	Ship
1	Francis Drake	1540	Golden Hind
2	Blackbeard	1680	Queen Anne's Revenge
3	Calico Jack	1682	Kingston

Figure 10.4: Deletion anomaly

One thing all these anomalies have in common is that they are self-evident. However, some anomalies only reveal themselves through business knowledge.

Domain anomaly

A **domain anomaly** occurs when a database fails to incorporate business rules into its design, thereby making it possible to create values that violate them. This anomaly is harder to catch because it requires functional knowledge of the data in question.

The following example shows a list of Pirates attributes:

Pirates			
Name	Born	Held Letter of Marque	Designation
Francis Drake	1540	Y	Privateer
Henry Every	1659	N	Pirate
Blackbeard	1680	N	Privateer
Calico Jack	1682	N	Pirate

Figure 10.5: Domain anomaly

Those unaware of the distinction between *pirate* and *privateer* (the latter performs piracy on behalf of a sovereign who authorizes the activity through a *letter of marque*) may miss the discrepancy in Blackbeard's record (he was as *piratey* as pirates come).

Normalization can prevent data anomalies by subdividing tables to reduce dependency and redundancy and encoding business rules into the design. In the following sections, we will follow the application of normal forms to see how this is achieved.

Database normalization through examples

Normalization was first proposed by Edgar F. Codd, the inventor of the relational model for database management. Codd introduced normalization in 1NF form and later extended it to 2NF and 3NF. Later, Codd, working with Raymond F. Boyce, developed **Boyce-Codd Normal Form (BCNF)**, also called **3.5NF**.

As database theory continues to develop, subsequent normal forms have been proposed up to 6NF, following the progression from least to most restrictive. While it is important to understand normalization in all its forms, it is also essential to recognize that typical business scenarios do not require going past 3NF.

To understand how normal forms organize a database and prevent data anomalies, we will run through an exercise of taking a wholly denormalized dataset and running it through the normalization rules required to satisfy each form. The exercises that follow use examples of data from the music industry. But, unlike the wide world of music, the examples assume that an album name is unique and that no two artists can have identically named albums.

With this in mind, let's begin cleaning up the messy data from *Figure 10.1* by converting it to 1NF.

1NF

1NF is satisfied when the following are in place:

- Every record is unique
- Every cell contains a single value

Ensuring unique records is one of the fundamental requirements of a relational database design. Working with the premise that *Album* is a unique identifier, we begin by eliminating the duplicate second row and assigning a primary key constraint on the *Album* column.

Next, we must ensure that every cell contains single (atomic) values. Storing multivalued data within a single cell presents a challenge for data analysis looking for meaningful insights. For example, without atomic values, calculations such as the Genre count or checking for the existence of a specific Genre value require parsing the contents of each cell before they can be analyzed.

In our example, multiple values are present in the Genre column:

Genre
Progressive Rap, Soul, G-funk

Figure 10.6: Multivalued data

Pivoting these values into individual rows and creating a separate dimension for Genres satisfies 1NF. The resulting design allows efficient analysis of Genres without violating the *Album* table's primary key or duplicating its attributes:

Album									
Album	Artist	Artist Country	Label	Length	Format	Price	Certified Sales	RIAA Certification	RIAA Cert. ID
Aquemini	OutKast	America	LaFace	74:47	CD	10.99	2,000,000	2× Platinum	4

Album Genres	
Album	Genre
Aquemini	Progressive Rap
Aquemini	Soul
Aquemini	G-funk

Figure 10.7: A design that satisfies 1NF



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now that we have unique identifiers and no multivalued data, let's move on to the second normal form.

2NF

2NF is satisfied when the following are in place:

- 1NF rules are satisfied
- Every non-candidate key attribute must depend on the whole candidate key (no partial dependencies)

Let's unpack that. A **candidate key** is a unique identifier that cannot be further simplified and remain unique. The following example shows an instance of a partial dependency:

Artist Grammys by Year				
Artist 	Year 	Genre	Nominated	Won
Metallica	2009	Metal	4	2
Jay Z	2009	Rap	6	1
Lady Antebellum	2009	Country	2	0

Figure 10.8: An example of a partial dependency

While Artist, Year, and Genre together identify a unique record, only Artist and Year are required (this is the candidate key). The number of Grammys nominated and won depends entirely on the candidate key (Artist and Year) and would not make sense if all or part of the key were missing. However, Genre depends on Artist only (not Year)—forming a partial dependency and violating 2NF.

In our original example, the Album and Format columns form the candidate key in the following table. However, nearly all the attributes besides Price (such as Artist and Length) depend only on Album. That is to say, they partially depend on the candidate key and do not satisfy 2NF:

Album									
Album 	Artist	Artist Country	Label	Length	Format	Price	Certified Sales	RIAA Certification	RIAA Cert. ID
Aquemini	OutKast	America	LaFace	74:47	CD	10.99	2,000,000	2× Platinum	4
Aquemini	OutKast	America	LaFace	74:47	Vinyl	24.99	2,000,000	2× Platinum	4
Birth of the Cool	Miles Davis	America	Capitol Records	35:29	CD	7.99	280,000		

Figure 10.9: A table with partial dependencies

The present design requires us to duplicate all Album attributes for every available Format. In 2NF, we can achieve a much cleaner design by decomposing the preceding table into two separate tables with no partial dependencies. Every attribute now depends entirely on the complete candidate key (Album and Album + Format, respectively):

Album								
Album 	Artist	Artist Country	Label	Length	Certified Sales	RIAA Certification	RIAA Cert. ID	
Aquemini	OutKast	America	LaFace	74:47	2,000,000	2× Platinum	4	
Birth of the Cool	Miles Davis	America	Capitol Records	35:29	280,000	None	1	

Album Formats		
Album 	Format 	Price
Aquemini	CD	10.99
Aquemini	Vinyl	24.99
Birth of the Cool	CD	7.99

Figure 10.10: A design that satisfies 2NF

Now that every attribute in our tables depends on a candidate key, let's see what's required to satisfy the third normal form.

3NF

3NF is satisfied when the following are in place:

- 2NF rules are satisfied
- There are no transitive functional dependencies

What is the transitive world? A **transitive functional dependency (TFD)** is formed when a non-primary key column depends on another non-primary column—for example, Label Country, which depends on Label. TFDs are a problem because they allow update anomalies, such as those in the following table, to occur:

Album								
Album 	Artist	Artist Country	Label	Label Country	Length	Certified Sales	RIAA Certification	RIAA Cert. ID
Aquemini	OutKast	America	LaFace	USA	74:47	2,000,000	2× Platinum	4
Birth of the Cool	Miles Davis	America	Capitol Records	USA	35:29	280,000	None	1
Hello Nasty	Beastie Boys	America	Capitol Records	Uzbekistan	67:28	3,000,000	3× Platinum	5

Figure 10.11: An update anomaly due to a transitive dependency

To get around this issue, we must resolve all the TFDs by creating separate dimension tables for each of them, where all the attributes depend entirely on the complete primary key. The resulting schema satisfies 3NF:

Album						Label	
Album 	Artist	Release Year	Length	Certified Sales	RIAA Certification	Label 	Label Country
Aquemini	OutKast	1998	74:47	2,000,000	2× Platinum	LaFace	USA
Birth of the Cool	Miles Davis	1957	35:29	280,000	None	Capitol Records	USA
Hello Nasty	Beastie Boys	1998	67:28	3,000,000	3× Platinum		

Album Price			RIAA Cert. Requirements			Artist	
Album 	Format	Price	RIAA Cert. Req. ID 	Minimum Sales	Name	Artist 	Artist Country
Aquemini	CD	10.99	1	0	None	OutKast	America
Aquemini	Vinyl	24.99	2	500,000	Gold	Miles Davis	America
Birth of the Cool	CD	10.99	3	1,000,000	Platinum	Beastie Boys	America
Hello Nasty	Digital	8.99	4	2,000,000	2× Platinum		
			5	3,000,000	3× Platinum		

Figure 10.12: A design that satisfies 3NF

A mnemonic for remembering the first three normal forms

In court, sworn testimony is often accompanied by a commitment to tell the truth, the whole truth, and nothing but the truth. In database design, normalization rules stipulate that records should depend on the key (1NF), the whole key (2NF), and nothing but the key (3NF).

We have removed TFDs from our design, but before moving on to the fourth normal form, let's examine a slightly more restrictive version of 3NF.

BCNF

BCNF (also referred to as 3.5NF) is satisfied when the following are in place:

- 3NF rules are satisfied
- Every non-trivial functional dependency is itself a key

In 1974, Raymond Boyce and Edgar Codd got together to invent a stricter version of 3NF to address additional potential anomalies. They aimed to reduce redundancy while having the structure reflect the underlying business rules. Before going into detail, it should be said that it is rare to encounter scenarios that satisfy 3NF but not BCNF. To invent such a scenario, let us abandon music for a moment and look at restaurants.

Take the following table of dining reservations from Nuovo Vesuvio, Artie Bucco's upscale Italian eatery:

Today's Diners		
Table 	Time 	Reservation Type
1	6:30	walk-in
2	7:00	phone
2	8:00	phone
3	6:30	phone
3	8:00	walk-in
4	11:30	Tony Soprano

Figure 10.13: A 3NF-compliant table with hidden business logic

Although the table is in 3NF and free from data anomalies today, the design fails to reflect or enforce some important business restrictions. Namely, table 2 is available only to those who have phoned to reserve in advance, table 1 is for *walk-ins* only, and, most importantly, table 4 is exclusively reserved for Artie's childhood chum, *Tony Soprano*.

To satisfy Boyce and Codd (and Soprano), a *Reservation Types* table must be created to maintain the set of allowable possibilities (including the unlikely event of Tony calling ahead to reserve table 4—never witnessed but allowed). The resulting layout would be as follows:

Today's Diners		
Table 	Time 	Res. Type Key
1	6:30	A
2	7:00	B
2	8:00	B
3	6:30	C
3	8:00	D
4	11:30	E
1	8:00	A

Reservation Types				
Res. Type Key 	Table	Called Ahead?	Is Tony Soprano?	Reservation Type Name
A	1	N	N	walk-in
B	2	Y	N	phone
C	3	Y	N	phone
D	3	N	N	walk-in
E	4	N	Y	Tony Soprano
F	4	Y	Y	Tony Soprano

Figure 10.14: A design that satisfies BCNF (and Soprano)

Nuovo Vesuvio, understandably, follows stringent rules about reservations that vary from table to table. But what if every table accepted every known reservation method? Could the existing design be improved to reduce redundancy? Ronald Fagin certainly thinks so—it's why he created the fourth normal form.

4NF

The **fourth normal form (4NF)** is satisfied when the following are in place:

- BCNF rules are satisfied
- Every non-trivial functional and multivalued functional dependency begins with a superkey

Recall that a **superkey** is a combination of columns that uniquely identifies a row (but, unlike a candidate key, it can also include extra columns). A **multivalued dependency** is a relationship between two or more attributes in a database, where the value of one attribute depends on the values of multiple other attributes. To understand this in practice, observe the following table, which shows album inventory by format across three stores:

Store Album Formats			
Store 	Album 	Format 	Price
High Fidelity	Aquemini	Vinyl	26.99
High Fidelity	Birth of the Cool	Vinyl	26.99
High Fidelity	Hello Nasty	Vinyl	26.99
Empire Records	Aquemini	CD	10.99
Empire Records	Birth of the Cool	CD	10.99
Empire Records	Hello Nasty	CD	10.99
Empire Records	Aquemini	Vinyl	24.99
Empire Records	Birth of the Cool	Vinyl	24.99
Empire Records	Hello Nasty	Vinyl	24.99
e-music.com	Aquemini	Digital	8.99
e-music.com	Birth of the Cool	Digital	8.99
e-music.com	Hello Nasty	Digital	8.99

Figure 10.15: A table with multivalued dependencies



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



High Fidelity and *e-music.com* only sell one format (*vinyl* and *digital*, respectively), while *Empire Records* sells all albums on *CD* and *vinyl*. Storing everything in a single table means that *Album* depends on *Format* and *Store*, which are not superkeys. Separating the information into two tables removes the multivalued dependency and satisfies 4NF:

Store Albums		Store Formats		
Store 	Album 	Store 	Format 	Price
High Fidelity	Aquemini	High Fidelity	Vinyl	26.99
High Fidelity	Birth of the Cool	Empire Records	Vinyl	24.99
High Fidelity	Hello Nasty	Empire Records	CD	10.99
Empire Records	Aquemini	e-music.com	Digital	8.99
Empire Records	Birth of the Cool			
Empire Records	Hello Nasty			
e-music.com	Aquemini			
e-music.com	Birth of the Cool			
e-music.com	Hello Nasty			

Figure 10.16: A design that satisfies 4NF

However, this is only true if the rules in the *Store Formats* table hold. If any store could sell any format, the table in *Figure 10.14*, which satisfies BCNF (and *Soprano*), would also satisfy 4NF. As business knowledge is required to determine 4NF, no systematic heuristics can guarantee it. The same holds true for the fifth normal form, as we will now discover.

5NF

The **fifth normal form (5NF)** is satisfied when the following is in place:

- 4NF rules are satisfied
- Tables can not be decomposed further without loss of data

After removing the functional and multivalued dependencies, the only way to reduce further is by analyzing the data as a function of existing business rules. Let us look at the following table, which shows albums in their available formats:

Album Formats	
Album 	Format 
Aquemini	CD
Aquemini	Vinyl
Aquemini	Digital
Birth of the Cool	Digital
Hello Nasty	CD
Hello Nasty	Digital

Figure 10.17: Table of albums and available formats

Decomposing the table further will yield tables (seen in the subsequent diagram) that cannot be joined to recreate the data in *Figure 10.17*:

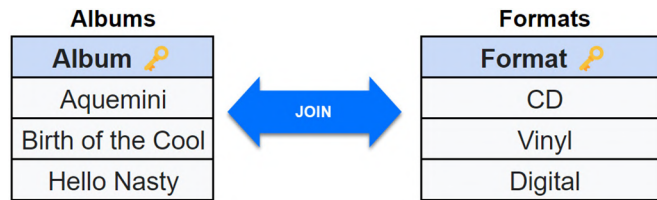


Figure 10.18: A join between unique albums and unique formats

Attempting to join the smaller tables produces a Cartesian product with every combination of possible values, thereby losing the original detail.

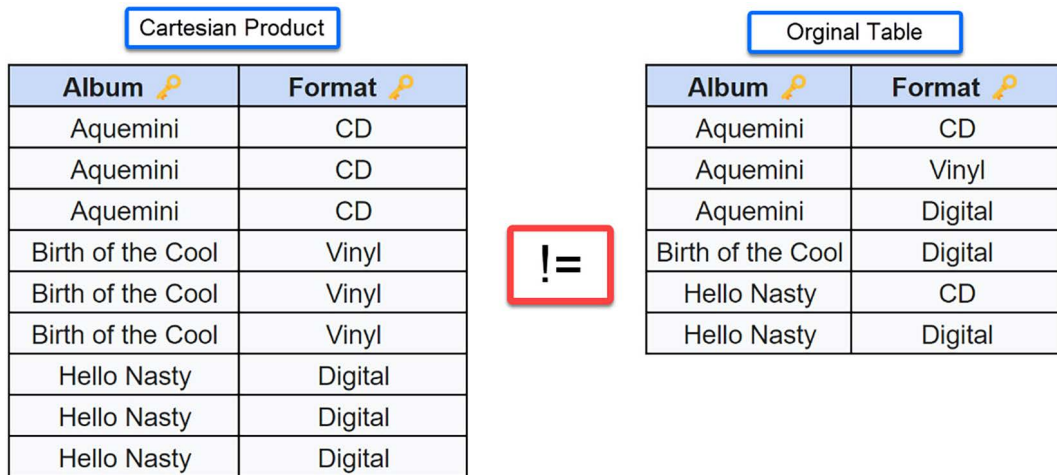


Figure 10.19: A Cartesian product between decomposed tables results in data loss

5NF is not commonly used in practice, as it is difficult to achieve and unnecessary for most databases. Rarely do tables that satisfy 5NF not satisfy 4NF (and, by extension, BCNF).

However, even 5NF fails to guard against domain anomalies, which are common enough to consider avoiding.

DKNF

The **domain-key normal form (DKNF)** is satisfied when the following are in place:

- 5NF rules are satisfied
- Domain constraints do not exist in any table

Various examples throughout this chapter have included an album's RIAA Certification and Certified Sales. In the music industry, RIAA certifications are assigned based on the number of albums sold: 500k units for *Gold*, then 1M increments for *Platinum*, *2x Platinum*, until reaching *Diamond* at 1B copies. This dynamic can be seen in the following table:

Album					
Album 	Artist	Release Year	Length	Certified Sales	RIAA Certification
Aquemini	OutKast	1998	74:47	2,000,000	2× Platinum
Birth of the Cool	Miles Davis	1957	35:29	280,000	None
Hello Nasty	Beastie Boys	1998	67:28	3,000,000	3× Platinum

Figure 10.20: A design that satisfies 5NF but not DKNF

Certified Sales and RIAA Certification are both attributes that depend on the album's primary key, but there is an implicit domain constraint between them as record sales dictate the RIAA status.

As we can see in the following table, it is subject to domain anomalies that may result in nonsensical data, which nevertheless conforms to 5NF:

Albums					
Album 	Artist	Release Year	Length	Certified Sales	RIAA Certification
Aquemini	OutKast	1998	74:47	2,000,000	2× Platinum
Birth of the Cool	Miles Davis	1957	35:29	280,000	100× Platinum
Hello Nasty	Beastie Boys	1998	67:28	3,878,000	3× Platinum

Figure 10.21: A domain anomaly occurs despite 5NF

To avoid the possibility of a domain anomaly, RIAA Certification must be separated into another table, with clearly defined bounds for each certification. This way, the correct certification can be derived logically from the album's Certified Sales column:

Albums				
Album 	Artist	Release Year	Length	Certified Sales
Aquemini	OutKast	1998	74:47	2,000,000
Birth of the Cool	Miles Davis	1957	35:29	280,000
Hello Nasty	Beastie Boys	1998	67:28	3,878,000

RIAA Cert. Requirements			
RIAA Cert. Req. ID 	Minimum Sales	Maximum Sales	Name
1	0	499,999	None
2	500,000	999,999	Gold
3	1,000,000	1,999,999	Platinum
4	2,000,000	2,999,999	2× Platinum
5	3,000,000	3,999,999	3× Platinum
...	...	...	...
12	10,000,000	9,999,999,999	Diamond

Figure 10.22: A design that satisfies DKNF

Now, it's time to get extreme. The final form we will cover is still under debate in the modeling community and is only practicable in rare cases. However, it is not without merit.

6NF

6NF is satisfied when the following are in place:

- 5NF rules are satisfied
- Every table contains a primary key and, at most, one attribute

6NF takes reducing dependencies to an extreme by making them a physical impossibility. In 6NF, tables are allowed no more than one attribute per table. The following example shows a table in 5NF consisting of a primary key and four attributes. Notice that the last record has no value for Price:

Albums				
Album 	Artist	Release Year	Length	Price
Aquemini	OutKast	1998	74:47	10.99
Birth of the Cool	Miles Davis	1957	35:29	24.99
Hello Nasty	Beastie Boys	1998	67:28	

Figure 10.23: A design that satisfies 5NF but not 6NF

To satisfy 6NF, this table must be split into five smaller tables. Notice how the resulting tables conform to the original primary key column (Album), but the Album Price table has fewer records due to Hello Nasty missing that attribute:

Albums		Album Artist		Album Release Year	
Album 		Album 	Artist	Album 	Release Year
Aquemini		Aquemini	OutKast	Aquemini	1998
Birth of the Cool		Birth of the Cool	Miles Davis	Birth of the Cool	1957
Hello Nasty		Hello Nasty	Beastie Boys	Hello Nasty	1998

Album Length		Album Price	
Album 	Length	Album 	Price
Aquemini	74:47	Aquemini	10.99
Birth of the Cool	35:29	Birth of the Cool	24.99
Hello Nasty	67:28		

Figure 10.24: A design that satisfies DKNF



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



The proliferation of tables under 6NF (and the effort required to query and maintain them) makes it impractical for most business purposes. 6NF has gained some notoriety through ensemble modeling frameworks such as Anchor Modeling, which falls outside the scope of this book. However, it is worth mentioning the database features that Anchor Modeling depends on, both of which are supported in Snowflake (and covered in more detail in later chapters).

With the many tables that 6NF requires, joins become exceedingly difficult to write. However, query performance can be significantly increased in 6NF through features such as join elimination compared to larger denormalized tables. With join elimination, the database optimizer can intelligently avoid performing certain joins if attributes from the related tables are not explicitly requested in the query. (More on join elimination in *Chapter 13, Putting Transformational Modeling into Practice*.)

Multi-table insert is another Snowflake feature with broad applications in many modeling scenarios, particularly in Data Vault (covered in *Chapter 18, Scaling Data Models Through Modern Techniques*) and Anchor Modeling. In *Figure 10.24*, we created five tables that share the same primary key. Snowflake's multi-table insert feature would allow us to load all five in a single load operation and even include conditional logic dictating which tables should be updated and when.

Let us summarize what we have learned after seeing the gradual tightening of database rules from 1NF to 6NF.

Data models on a spectrum of normalization

The prior sections demonstrate how, as we move up the normal form scale, redundancy and the potential for data anomalies decrease while raising the number of tables and associated complexity. With this in mind, we can visualize where various data modeling patterns fall on the normal form spectrum to help understand their suitability in design scenarios. While the dimensional modeling process was discussed in *Chapter 8, Putting Conceptual Modeling into Practice*, the associated schema patterns (star and snowflake) are explained in further detail in *Chapter 18, Scaling Data Models Through Modern Techniques*, along with Data Vault (where they are often implemented at the information mart layer). However, visualizing them in the context of normal forms will help lay the groundwork for later understanding their design. Also, bear in mind that the following assertions speak to general tendencies, not hard-set rules.

The following figure displays the various modeling patterns and methodologies mentioned throughout this book on the normalization spectrum:

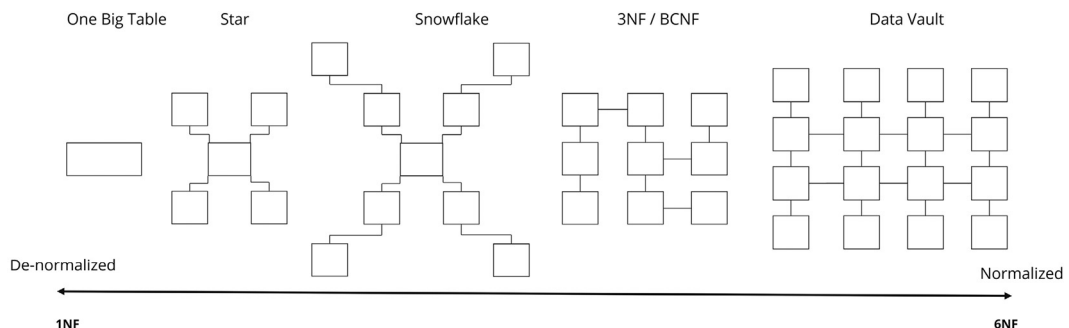


Figure 10.25: Common data models arranged on the normalization spectrum

Arguably, 1NF (*the key*) is the minimum requirement for working with relational data. For its ease of understanding, 1NF is often seen in the **One Big Table (OBT)** approach, where facts and dimension attributes are all stored in a single table. While they are difficult to maintain, OBT is dead simple to query (i.e., no joins required). Business users are comfortable with OBT, provided the tables don't get too big (by size or number of columns) to where usability begins to degrade.

Star and snowflake schemas are patterns often seen when following Kimball's dimensional approach and are good candidates for the data platform's analytical self-service or reporting layers. In both models, the central fact table is typically normalized to 3NF. In the star schema, dimension tables are connected directly to the fact table and denormalized to 2NF—a good trade-off between maintenance and ease of use. In the snowflake schema approach, dimension tables are generally normalized to 3NF, forming a hierarchy and trading on analytical complexity in exchange for easier maintenance with less redundancy.

At the far end of the normalization spectrum are Data Vault models. While Data Vault does not satisfy 6NF in the strict sense, it does incorporate many of its principles in its design.

Now that we have seen how various modeling patterns line up on the normalization spectrum, let's review what we have learned.

Summary

Through various examples in this chapter, we saw how the process of normalization organizes a database in a way that reduces redundancy and dependency within its tables.

Dependency and redundancy in database tables increase the likelihood of data anomalies, which come in many forms. Update anomalies occur due to redundancy, which makes it possible to update some, but not all, of the associated records. Physical (as opposed to logical) dependencies are the root cause of insertion and deletion anomalies. When too many details of varying granularity are bunched into a single table, inserting or deleting records that do not match all the criteria becomes difficult. The domain anomaly is the hardest to spot because it requires functional knowledge of the data in question.

Database normalization can be applied through escalating stages of formal rules called normal forms, ranging from 1NF to 6NF, to avoid such anomalies. The most commonly used normal forms are the first through third, which are sufficient for most databases. The mnemonic *the key, the whole key, and nothing but the key* aids in recalling the considerations for each of the first three forms.

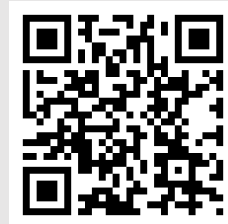
Through practical examples that took data from a completely denormalized format and transformed it through each normal form, a common theme emerged—with each normal form, the possibility for data anomalies decreased while the number of tables grew, encumbering maintenance and analysis.

The examples in this chapter demonstrate that more normalization does not imply a better design—it simply imposes stricter rules on the database. Having seen the specific data issues that each of the normal forms addresses, applying their principles broadly or selectively will help modelers achieve an optimal design with the right balance of rigor and usability.

In the next chapter, rigor and usability will take center stage as we review database naming conventions and organization. Coupled with the insights from this chapter, we will then be ready to tackle the physical model.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

11

Database Naming and Structure

In previous chapters, we took a database design through the necessary steps to transform it into a logical model. While a logical model is database-independent, it is close enough to a physical design that it can easily be adapted and deployed to any database. However, before tackling the Snowflake-specific properties of the data model—which will be covered in the following chapter—we should get acquainted with naming conventions and database organization best practices that govern all database designs.

Naming conventions are the guiding principles with which database objects are constructed. Consistent naming standards reduce uncertainty for developers and help business users orient themselves within a database and find the required data assets. Beyond object naming, this chapter will also cover the structure of the database itself, organizing it into logical groupings by schema, which improves usability. Finally, we will look at database environments and replication to ensure reporting data is isolated from volatile development changes.

A theme that quickly emerges in every section of this chapter is the focus on consistency. Rather than advocating for the *right* way to name or structure a database, it's important to be flexible and adjust standards to organizational and project needs—then stick to those decisions. The main topics that will be covered in this chapter are as follows:

- Choosing a case standard that works best with Snowflake and maximizes cache reusability
- Object naming for tables and relationships for enhanced usability
- Styling and usage considerations to consider at design time
- Organizing a database through standard and managed schemas
- Structuring a database for OLAP and OLTP use cases
- Separating database environments for reporting, testing, and development

Naming conventions

Before creating the physical model, naming conventions that govern its design need to be established. Following consistent naming conventions improves understanding, reduces errors, facilitates collaboration, and generally makes it easier to work with your database. While there are many (often conflicting) theories and standards on the right convention to follow, the most important thing is to choose one that is easy to understand, and then use it consistently throughout your database.

However, there are some general best practices to keep in mind when naming objects in Snowflake. After all, object names are like the API to your data model and should be regarded as a contract between the modeler and the data consumers. Once an object is created, downstream systems, users, and processes will reference it by name, forming dependencies and increasing the cost of future changes.

This section will cover some of the most crucial considerations in database naming. Instead of presenting naming conventions by object type, this section will be organized by importance, starting with those rules that are strongly advised to those merely suggested. Whatever you decide, choose a standard that best aligns with your organization and stick to it.

With that, let's look at the critical considerations.

Case

Internally, Snowflake stores all objects in uppercase unless enclosed in double quotes. For SQL queries, Snowflake's parser treats anything not enclosed in double quotes as uppercase to match storage. In practice, this means all of the following object names are compiled identically:

- `my_table`
- `MY_TABLE`
- `mY_Table`
- `"MY_TABLE"`

Object names in double quotes are case-sensitive and may include spaces. Therefore, any query that references an object in double quotes must precisely match the name (i.e., `"mY Table" != MY_TABLE`) to return the expected result. To avoid having to enclose every object and column in double quotes, it is advised to avoid them and use snake case instead. In snake case, spaces are replaced with underscores (`_`), and every word, including the first letter, is written in lowercase (e.g., `my_table`).

While object names may be case-insensitive, the results cache is not. Observe the following example where the same query is executed consecutively using various casing formats:

```
--upper
SELECT O_ORDERKEY, O_TOTALPRICE FROM ORDERS;

--snake
select o_orderkey, o_totalprice from orders;

--mixed
SELECT o_orderkey, o_totalprice FROM orders;
```



Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button

- (1) to quickly copy code into your coding environment, or click the **Explain** button
- (2) to get the AI assistant to explain a block of code to you.



```
function calculate(a, b) {
  return {sum: a + b};
};
```

Copy**Explain**

1

2



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR visit packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



While all three queries will return the same result, none will leverage the results cache (and will consume credits) because of the difference in case. Whatever convention you settle on, ensure that the entire team is aligned on the chosen standard and stays consistent in object naming and keyword capitalization.

With case standards in place, let's look at object naming.

Object naming

This section will focus on the naming conventions for the objects used most frequently in modeling and data analysis. Because users of the database and downstream tools will reference these objects for reporting and analytics, a consistent naming convention will save time for everyone by ensuring that required data assets are discoverable and self-explanatory.

Tables

Plural or singular for table naming? The database community can't agree. Proponents of plural (i.e., *customers* instead of *customer*) argue that a table is a collection of many singular customer records and should therefore be named in the plural. The singular camp counters that pluralization often introduces unnecessary confusion—how to pluralize the person entity, for example, *persons* or *people*?

E.F. Codd's relational model—the basis for relational databases—is consistent with first-order predicate logic. Tuples become rows, and relations become tables, all referred to in the singular. Another advantage of singular names is that they transfer cleanly from conceptual modeling, where business entities tend to be thought of in the singular form (e.g., *customer*, *product*, and *country*).

At least that is the academic opinion. To see what the data community does in practice, I conducted a LinkedIn poll to take the pulse of table naming standards in 2023 (https://www.linkedin.com/posts/serge-cloud-connected_table-naming-for-databases-just-taking-the-activity-7021854119141257216-mm_2/):

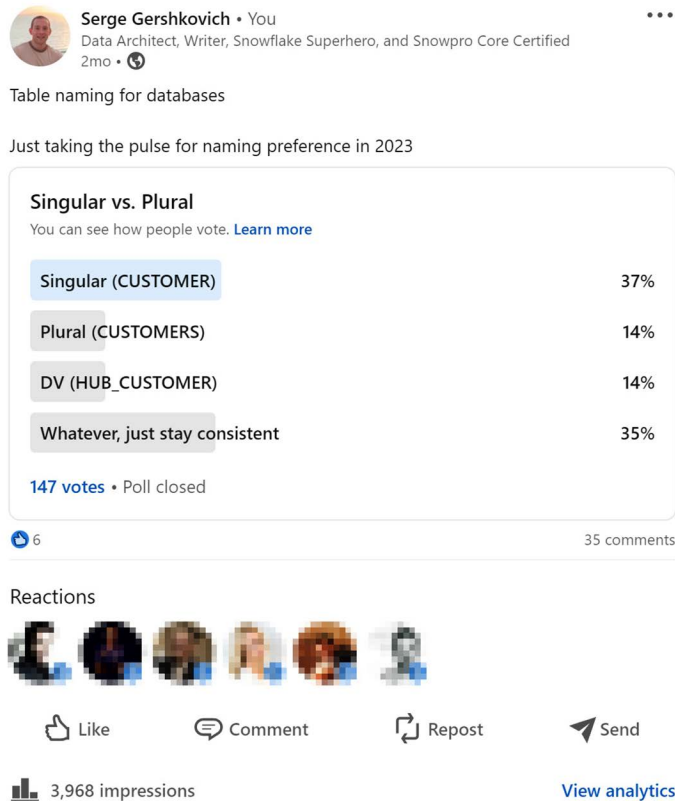


Figure 11.1: LinkedIn poll – singular or plural for database table names

If you are still undecided, go with singular for table names—but more importantly, be consistent. With table names covered, how about their identifiers?

Primary key columns

Unlike tables, there is not much debate on naming their unique identifiers. Primary key columns should be named in singular format, following the table's name, as in `<singularTableName>_id`. Whether your table name is singular or plural (i.e., person or people), the unique identifier should be named `person_id`.

A generically named ID column for all tables is discouraged because it leads to confusion and ambiguity in downstream transformations. But what about additional unique identifiers such as surrogate and business keys?

As long as `primary_id` is present, include additional unique columns following a convention consistent throughout the database, which may include alternate ID suffixes such as `_skey` or `_BKEY`.

Foreign key constraints

Because relationship names are not supported in physical models, a descriptive and consistent naming for foreign keys becomes the best alternative. Take an example that features relationship names between entities:

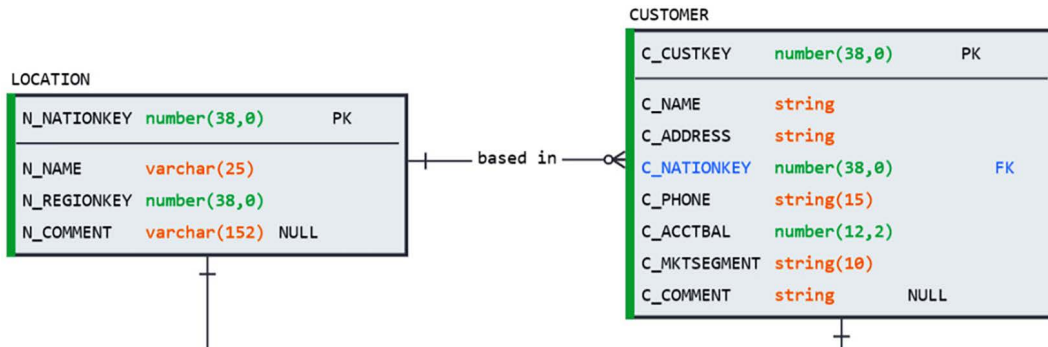


Figure 11.2: A logical model with relationship names and data types

The example describes a many-to-many relationship where CUSTOMER is *based in* LOCATION. Although a physical model does not allow descriptions for foreign key constraints, naming conventions can be used to retain this information. Using a naming pattern of FK_<Child_Table>_<Relationship_Name>_<Parent_Table> (and PK_<Table_Name> for primary keys) results in a physical model that preserves the relationship details:

```
CREATE TABLE CUSTOMER
(
    CUSTOMER_ID number (38,0) NOT NULL,
    NAME varchar NOT NULL,
    ADDRESS varchar NOT NULL,
    LOCATION_ID number (38,0) NOT NULL,
    PHONE varchar NOT NULL,
    ACCOUNT_BALANCE number (12,2) NOT NULL,
    MARKET_SEGMENT varchar NOT NULL,
    COMMENT varchar,
    CONSTRAINT PK_CUSTOMER PRIMARY KEY ( CUSTOMER_ID ),
    CONSTRAINT FK_CUSTOMER_BASED_IN_LOCATION FOREIGN KEY ( LOCATION_ID )
REFERENCES LOCATION ( LOCATION_ID )
)
COMMENT = 'Registered customers';
```

While these are the most important considerations from a modeling perspective, there are a few more that should be considered.

Suggested conventions

Effective and consistent naming for tables and relationships is of utmost importance when designing a physical model. The following suggestions should also be considered for clarity and adaptability in column and object names:

- Ensure consistent column names through conformed dimensions. *Conformed dimensions* are columns whose domain contents and naming do not change when used across multiple tables. As data moves across the data warehouse layers, ensure that column names are descriptive enough in the base layer that they do not need to be renamed for clarity or conflict in downstream tables (i.e., that the ID column does not become CUSTOMER_ID then MAIN_CLIENT_CODE as it moves from source to reporting layers).
- Avoid abbreviations in table and column names. Even if you think you will remember what the abbreviation stands for (you will not), the other users in the system will have a harder time interpreting it. Does CUST_REF imply *customer reference*, *custodial refund*, or *custom refrigeration*?
- Except for the unique identifier, avoid prefixing columns with the table name or initial (e.g., C_NAME in the CUSTOMER table). This naming style guarantees having to rename the columns upstream and makes lineage tracking difficult. The benefit of this naming convention (unambiguous reference when writing queries) can be achieved by using table aliases (C.NAME) or qualifying with the complete table name (CUSTOMER.NAME).
- Avoid naming data sources using their object types (e.g., views as <entity>_V or materialized views as <entity>_MV). Depending on cost and performance considerations, the object type of data sources may change over time, and referring to a static <entity> will prevent errors in the objects that reference it. Instead, use suffixes to identify special-use tables such as historical (_hist), control (_ctrl), and staging (_stg).

The naming suggestions discussed previously are as follows:

- **Case:** Use unquoted snake case (lowercase with an underscore word separator)
- **Table names:** Singular
- **Primary key (columns):** <singular_table_name>_id
- **Primary key (constraint):** pk_<table_name>
- **Foreign key (constraint):** fk_<child_table>_<relationship_name>_<parent_table>

- **Column names:** Be descriptive, stay consistent across tables, and avoid abbreviations
- **Object suffixes:** Avoid adding suffixes such as `_v` and `_mv` to denote object type

Most importantly, remember that these are general guidelines and that you should adapt them to existing company standards and use cases, which may vary from one project or database to another. In short, you can be flexible between databases if you are consistent within databases.

Now, let's cover the structure and logical organization within a database.

Organizing a Snowflake database

The data objects stored by Snowflake—optimized and compressed in an internal columnar format—are not directly visible nor accessible by customers; they are only accessible through SQL query operations. The customer only manages the logical grouping of database objects into schemas and databases. As described in *Chapter 3, Mastering Snowflake's Architecture*, in Snowflake cloud architecture, data is shared virtually without needing to be physically replicated. Therefore, unlike traditional database platforms, the database structure in Snowflake is less concerned with the colocation of physical data and more with the logical grouping of objects—allowing for simple discovery and fine-tuning of access controls.

What does this look like in practice?

Organization of databases and schemas

All Snowflake objects are assigned to a schema upon creation and form a logical hierarchy from object to schema to database. This tiered grouping affords several benefits when organizing a database, including organization, access management, and replication.

The apparent advantage of database/schema tiering is organization. Two-layer classification allows Snowflake users to group objects meaningfully and granularly according to project needs. This logical grouping makes it easy for users to find the tables they are looking for or separate the working layers of a warehouse (as described later in the chapter).

Data sharing and replication also benefit from having a well-organized database with schema grouping. The ability to clone objects individually or at the schema and database level allows users to efficiently replicate datasets for testing and debugging. Sharing data beyond the account is possible, but only at the database level.

Another advantage of grouping objects into schemas and databases is its fine-grained access control. Assigning grants and ownership at the object, schema, or database levels affords Snowflake users great flexibility in managing and fine-tuning access controls to secure data access.

Managing access control within a schema can be handled in one of two ways.

Managed schemas

In addition to traditional role-based access control, Snowflake provides an additional method for managing object access known as **managed schemas**, which delegate privilege management to the schema owner.

In regular schemas, the owner of an object (i.e., the role with the `OWNERSHIP` privilege on the object) can grant further privileges to other roles. In managed schemas, the schema owner manages all privilege grants on objects in the schema.

To create a managed schema, add the `WITH MANAGED ACCESS` parameter to the `CREATE SCHEMA` command like so:

```
create schema my_schema with managed access;
```

In managed schemas, developers can make changes and create new objects; however, only the schema owner can grant privileges to them. This approach is ideal for security-conscious organizations because it allows developers to work unhampered within a schema while delegating access management to a supervisory role.

OLTP versus OLAP database structures

Suggestions for structuring a database will depend heavily on the application or use case that the database is designed for. **Online transaction processing (OLTP)** or transactional systems typically rely on centralized structures, while **online analytical processing (OLAP)** or data warehouses segment their architecture into layers.

As Snowflake has just begun a concerted push into the OLTP space in 2022 with Hybrid Unistore tables, such use cases have had limited applications. However, the unifying theme in such systems is that the database assets are centralized in a single main schema due to their relational dependencies. Because transactions affect multiple related tables, the logical grouping would not benefit usability and would only hamper usage.

However, in OLAP use cases—which make up the majority of Snowflake implementations—properly structuring the database will have a noticeable impact on usability. While there is no one-size-fits-all suggestion, the following schema divisions should be considered, starting with the landing area where data enters the warehouse.

Source layer

All data warehouses should contain a source or raw layer where data is loaded in its original, unaltered state from source systems. Any renaming or transformation should be done downstream in subsequent warehouse layers to prevent having to reload in case of an adjustment in logic or business requirement.

Depending on the amount of data sources the warehouse contains and the number of tables in each, it may even make sense to dedicate an entire database to source data and separate source systems by schema. When source systems are separated by schema, tables can retain their original names. However, when tables from various sources share the same schema, adding a source system prefix to the table name is recommended to identify and separate the sources like so:

- `salesforce_account`
- `oracle_account`

Although column names and their contents should not be modified at this stage, standardized metadata fields may be added as long as this is done consistently for all tables in the schema. Such fields may include the following:

- **Load identifier:** This unique load identifier can be used to remove data from a particular load if data corruption or other data quality issues are discovered. A common technique is using the source system's extract date/time to generate a unique value.
- **Load date/time:** This is a timestamp that indicates when the data was loaded into the data warehouse. If loading or data quality issues occur, this field may differ from the extract date/time of the source system.
- **Source filename:** Because data in Snowflake must be loaded from external file storage, recording the filename will help facilitate traceability and debugging.
- **Loaded by:** When using various integration tools, recording the tool responsible for a given load will help point users to the location of logs, or the interface to retry failed loads is helpful.
- **Load type:** If the loading pattern or method varies, it may help identify the load types, such as daily, scheduled, or ad hoc.

A consistent naming pattern should be used to help distinguish data warehouse metadata fields from source fields (which may include similar metadata fields of their own). Common use cases include prefixing columns with a keyword or underscore as follows:

- `etl_load_id`
- `__load_id`

As a rule, access to the source schema should be limited to the central data or engineering team. Because the data has not yet been cleansed or conformed, business users will struggle to obtain meaningful results. However, if the data science team has sufficient business knowledge, they may benefit from direct access to the data in the source schema and glean meaningful insights without depending on the involvement of the warehouse team.

Once data is landed in the source layer, it's time to clean it up for downstream consumption, and having a designated schema where this takes place is ideal.

Staging layer

The staging or working schema is intended as temporary storage for transformed source data as it makes its way downstream through the data warehouse. The tables in this schema are used for data loading and transformations and should be off-limits to users and reporting tools.

The advantages of grouping temporary tables in their own schema are as follows:

- It cleanly segregates provisional data from trusted data
- It avoids having to prefix tables to make the distinction (i.e., `stg_table_name`)
- It allows schema-level default settings for data retention and transient table types

Once the data has been cleaned up, conformed, and enhanced through the application of business rules and formulas, it can be shared with the rest of the organization through the approaches described in the following sections.

Normalized self-service analytics

A trusted and governed mid-level schema with clean normalized data can be a powerful tool for self-service and analytics. In organizations with high data literacy among analysts and business users (i.e., users capable of joining tables without help from the **business intelligence (BI)** team or ChatGPT), a schema that contains normalized (typically to **third normal form (3NF)**) and conformed dimensions (attributes from multiple source systems unified in individual dimension tables) can accelerate self-service reporting and analysis by removing the bottleneck from the warehousing team.

There is no standard for naming this schema, but common variants include `core`, `self_service`, and `analytics`. Because the data assets in this schema are meant to be analyzed and combined ad hoc by everyone in the organization, due diligence must be applied to naming, documentation, auditing, stewardship, and governance. Therefore, participation from domain experts and data governance teams is encouraged.

The alternative (or add-on) to a lightly curated self-service schema is a strictly governed and denormalized reporting schema built for plug-and-play consumption.

Denormalized reporting analytics

Even in organizations with high degrees of data literacy among analysts and business users, some analyses are too complex or too important to be done individually and must be centralized in a governed reporting schema.

Data sources in this schema—typically named reporting or analytics—are owned and maintained by the data team, which takes responsibility for their accuracy and reliability. Tables in the reporting schema tend to be denormalized (usually to **second normal form (2NF)**) because the emphasis is placed on having curated metrics and attributes in a single table to facilitate reporting.

Although the data warehouse team is responsible for building the reporting schema and keeping its contents up to date, the governance team typically plays a vital role in establishing and enforcing business rules and ensuring that naming and concepts are used consistently throughout the organization (i.e., if a *customer* is defined as someone having made a purchase in the last 12 months, then the definition is applied consistently to every table that includes a *customer* reference).

This completes the overview of the common schemas maintained by the warehouse and engineering teams. However, in larger organizations, business departments may have their own analytics teams and manage their own data sources.

Departmental data

By lowering the technical and operational barrier to entry to using a data platform, Snowflake has opened the door for business teams to take advantage of robust analytical capabilities without having to depend on the (typically) overburdened warehouse team.

At a minimum, a department should have its own database to load and manage the data sources that its team depends on. While some handholding by the data team may be required initially to get started, a well-trained business department will quickly become self-sufficient.

Once this happens, sharing and incorporating trusted data assets from a departmental database into the self-service or reporting schemas mentioned previously will become feasible. Because the data already exists within the same Snowflake account, it can be shared instantly with the rest of the company without duplication or the integration tools required to do so.

Departmental databases can be thought of as miniature data warehouses unto themselves. Therefore, they will often include schemas separating source, stage, and reporting layers and separate schemas for sandbox, development, and testing.

However, unlike a departmental database, a data warehouse should be replicated and maintained across multiple environments to separate trusted data from that being developed.

Database environments

Now that we have covered the layers within the database of a data warehouse, we should address the separation of its development environments. Once more, this is not meant to be a comprehensive guide on database environment segregation, merely a best-practice outline that the users should follow and possibly augment with specialized automation tools.

Separate database environments ensure that developers can make changes and test new projects without any risk of impacting existing data sources that the organization relies on. There should be at least three environments: **development (dev)**, **testing (test)**, and **production (prod)**—some organizations separate testing into various stages such as **quality assurance (QA)** and **user acceptance testing (UAT)**. The purpose of each is summarized here:

- **Prod:** Governed, secured, and (near) up-to-date data used to drive company reports and analytics
- **Test:** A close copy of production where changes are validated before being applied to production
- **Dev:** A sandbox environment where developers are free to make changes and experiment

All environments should aim to be synchronized as closely as possible but vary in the degree to which productive standards are applied to them—from most restrictive in prod to least restrictive in dev. The following are some of the criteria that should be observed and applied according to company needs and data volumes:

- **Access control:** This is listed above all other considerations for good reason. The critical concern in managing product data and ensuring trust and correctness is controlling who has access to consume (users and reporting tools) and make changes (ideally, no one). In a prod setting, human users should not be allowed to make changes or perform **data manipulation language (DML)** operations by hand. All such activities should be orchestrated by tools that handle deployment and execution. This ensures that processes are automated and auditable and require little or no human intervention.
- **Data freshness:** The prod environment should have the most up-to-date information required for operational decision-making. A prod snapshot from a given date may be sufficient in a test environment. Dev may benefit from a reduced sample size (for performance) and dummy data (for security).

- **Data volume and exactness:** For maximum precision in dev and testing environments, the underlying data should match that of prod. However, this would require additional maintenance and may hamper performance for simple changes that do not require a complete dataset. The ideal balance should be struck depending on data size and the type of testing that takes place in each environment.
- **Warehouse size:** Although the warehouse is independent of the database contents, testing and development should be done under similar conditions as those used in the prod environment. For example, even if development and testing are successfully performed with a medium warehouse, performance may prove unacceptable to prod business users who are limited to an extra-small warehouse.

The following diagram illustrates these principles on a spectrum across the database environments:

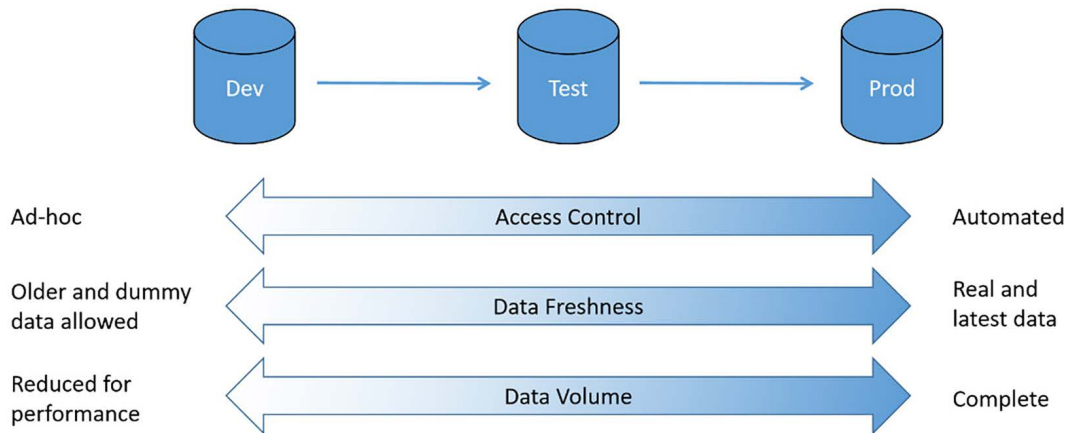


Figure 11.3: Database environments governing principles

With this in mind, let's review what we have covered in this chapter.

Summary

In this chapter, we saw how naming standards and an organized structure make the database easier to use and facilitate maintenance. But every organization is different and must choose the standard that best suits their needs and aligns with existing conventions.

Internally, Snowflake stores object names in uppercase, and its query compiler converts all unquoted names accordingly. It is recommended to use snake case for naming and stick to an established pattern to maximize the results cache utilization and to avoid enclosing every column and table name in double quotes.

For a clean transition between logical and physical models, singular table names are encouraged. The same applies to columns, which should be named consistently across the entire database. Using descriptive naming patterns for foreign keys allows users to preserve logical relationship names within the physical model.

After object naming, the attention turns to database organization through logical schema groupings. Snowflake allows for a two-tier hierarchy (database and schema) to maintain data access and replication across databases.

Following the warehouse data flow, the first step is the source or raw schema, where tables from source systems are replicated by the engineering team exactly as they appear. Following the source schema, preliminary table cleanup and renaming are performed in the staging schema—which is kept off-limits to users and reporting sources.

Once data has been renamed and formatted, it makes its way to the analytical schemas, where it is consumed by users and reporting tools. A normalized self-service schema is encouraged for organizations with knowledgeable and tech-savvy business users, as it lets business users derive analytical insights without depending on the data team. A denormalized reporting schema is used for governed reporting sources and more complex analyses. For larger enterprises, business teams may also have their own Snowflake databases, which function like miniature warehouses and can be integrated with the previously mentioned reporting schemas.

Lastly, different database environments should be used to ensure data quality and separate works in progress from officially validated and governed data. From the least restrictive dev to the production-like test to the securely regulated prod, database environments should follow a spectrum of increasing access control and data quality.

To summarize even further, database administration comes down to two main factors:

- Keep it simple
- Keep it consistent

After covering the essentials of database governance and structure, the next chapter will take our data model from the logical phase to a physical Snowflake architecture.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

12

Putting Physical Modeling into Practice

The modern data warehouse is a fast-paced environment. Multi-source and near-real-time data in Snowflake streams and transforms at the speed that scalable virtual hardware will allow. With potentially limitless computing resources available at trivially low prices, there emerges a tendency to undervalue planning in favor of post hoc adjustment. When this happens, platform and maintenance costs spiral, and suspicion is cast on the platform instead of the data model (or lack thereof).

So tempting is Snowflake's promise of near-zero maintenance and effortless scalability that many take it as an excuse to perform adequate data modeling before diving in. The Snowflake data platform does indeed live up to expectations (and beyond) when the underlying data landscape is built on a pre-planned data model.

Compared to other data platforms, Snowflake handles much of the database administration on the user's behalf. However, as this chapter will demonstrate, some considerations still fall to the database architects to ensure desired levels of performance and scalability while keeping costs in check. The physical model will not only serve to deploy pre-existing conceptual designs but also helps in auditing existing databases.

This chapter will explore these concepts in detail through the following main topics:

- Factors to consider for establishing physical properties
- Translating conceptual modeling elements to physical equivalents
- Creating the relational foundation for transformations and analytics
- Ensuring error-free deployment of a physical model to Snowflake
- Generating a diagram for an existing database

Technical requirements

The **data definition language (DDL)** for the completed physical model created through the exercises in this chapter is available to download and use from the following Git repository: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch12>.

You will arrive at the same result by following the steps in this chapter or by using the code provided. The physical model will be used as the foundation for transformational examples in later chapters.

Considerations before starting the implementation

When transitioning from a conceptual or logical design, where entities, attributes, relationships, and additional context have already been defined, there appears to be little to do at first glance when moving to a physical model. However, the specifics of Snowflake's unique cloud architecture from its variable-spend pricing to time-travel data retention, leave several factors to consider before embarking on physical design. We'll cover these factors in the following sections.

Performance

Query performance in Snowflake is heavily dependent on the clustering depth of the micro-partitions, which, in turn, are influenced by the natural sort order of the data inserted. Apart from Hybrid tables, which allow users to enable indexes, there are few performance tuning options left to the user besides sorting data before inserting and clustering. If the data volume in a given table is expected to reach the multi-terabyte range, Snowflake recommends defining a clustering key, which can be declared on one or more table columns or expressions.

Once data is loaded, the expected query patterns should be evaluated for performance and memory consumption against the intended warehouse size. If aggregations and complex calculations are required, Snowflake materialized views (single-table) or manual materialization via CTAS (multi-table) should be considered. These materializations will improve query performance compared to standard views by pre-aggregating and pre-joining data.

Next, we should consider the cost relative to the data loading and consumption patterns.

Cost

The bulk of the cloud database costs will be incurred by query processing and data transformation, which will be the focus of subsequent chapters. However, data storage costs are directly associated with a physical design and should be considered at this stage before they accumulate unwittingly.

As discussed in *Chapter 4, Mastering Basic Snowflake Objects*, Snowflake tables offer a configurable data retention period to allow time travel. Not all tables benefit from extended time travel periods; some may forgo it altogether, so be sure to set transient and data retention properties as and where needed (table, schema, or database level).

Some data may not require a database table at all. Recall that Snowflake can reference data in external storage, such as **Amazon Simple Storage Service (S3)**, both in the internal and public clouds. If no **data manipulation language (DML)** is required, external tables provide a fully workable option without incurring storage costs. When DML is required for slow-changing datasets such as those used in data lakes, Snowflake is now offering support for external tables in Apache Iceberg format (in private preview as of this writing but expected in 2023).

Data quality and integrity

While there is no guarantee of data quality for data coming into the warehouse, the physical data model should enforce critical business rules and preserve relational metadata. The importance of relational constraints is the central theme of this book but **NOT NULL** and **UNIQUE** constraints, default column values, and proper typing of columns are equally important when designing the physical model.

It is worth repeating that **NOT NULL** is the only constraint enforced by Snowflake. To ensure data integrity, user-enforced logic pre- or post-load should be considered. With the recent (November 2022) announcement of join elimination, the **RELY** property for relational constraints plays an important role in query performance and cost reduction. The use cases and implementation of the **RELY** property will be covered in detail in the following chapter on transformational modeling, but the foundation must be set in the physical model.

The next point to consider is data protection.

Data security

The physical database model should include security measures to protect the data from unauthorized access and to meet compliance regulations. Snowflake's access controls, roles, and object ownership fall outside the scope of this book, but model-related features are mentioned here for reference, along with links to further reading. Bear in mind that the following features are only offered in Snowflake's Enterprise Edition or higher:

- **Column-level security:** Allows the application of a masking policy to a column within a table or view and is available through internal functions or external tokenization (<https://docs.snowflake.com/en/user-guide/security-column.html>).
- **Row-level security:** Allows the application of a row access policy to a table or view to determine which rows are visible in the query result for a given role (<https://docs.snowflake.com/en/user-guide/security-row.html>).
- **Object Tagging:** Allows data tracking for compliance, discovery, protection, and resource usage. Tags can be assigned to higher-level objects such as schemas and tables and inherited by their children (tables and columns, respectively) (<https://docs.snowflake.com/en/user-guide/object-tagging.html>).
- **Tt:** Allows protecting column data by assigning a masking policy to a tag and then setting the tag on a database object or the Snowflake account (<https://docs.snowflake.com/en/user-guide/tag-based-masking-policies.html>).
- **Data classification:** Auto-detects and categorizes (using tags) potentially personal or sensitive data to support compliance and privacy regulations (<https://docs.snowflake.com/en/user-guide/governance-classify.html>).

For all these data governance features, a visual data model becomes vital to their maintenance, management, and tracking. While Snowflake provides metadata on their application in the Account and Information schemas and through internal functions, a navigable diagram of the physical database beats tabular text results for usability and understanding.

For a complete overview of Snowflake access controls, refer to the documentation (<https://docs.snowflake.com/en/user-guide/security-access-control-overview.html>).

After discussing four key considerations before starting physical design, let's review two that are handled natively by Snowflake architecture.

Backup and scalability considerations

Snowflake's cloud-native architecture delivers on the promise of *near-zero maintenance* by removing two critical factors from database design, as follows:

- **Backup and recovery:** Using the `CLONE` command, Snowflake users can nearly instantly create zero-copy snapshots of tables, schemas, or databases. This command can be combined with previously discussed time-travel features to give users even more flexibility to take a snapshot retroactively (see *Chapter 3, Mastering Snowflake's Architecture* for more details).
- **Scalability:** For storage, there is no (practical) limit on the size of individual tables or entire databases. For compute, virtual warehouses are independent of the database model and can be resized instantaneously.

With this in mind, let's continue the modeling journey toward physical design.

Expanding from logical to physical modeling

At this stage in the modeling journey—preparing to transform a logical model into a physical one—the use of a modeling tool will make a marked difference in the effort required to generate the final DDL. While this exercise can be done using anything from a sheet of paper to Excel, using a data modeling tool to accelerate the process is encouraged. (See the *Technical requirements* section of *Chapter 1, Unlocking the Power of Modeling* for a link to a free trial of SqlDBM—the only cloud-based tool that supports Snowflake and offers a free tier.)

Picking up from the finished logical model from *Chapter 8, Putting Conceptual Modeling into Practice*, let's begin the physical transformation.

Physicalizing the logical objects

Logical models contain all the information needed to transform them into a physical design, but they are not one-to-one equivalent regarding the number of elements. Besides the direct translations, such as entities to tables and relationships to constraints, some elements, such as subtypes, may not have a physical counterpart.

Working from the completed logical design from *Chapter 8, Putting Conceptual Modeling into Practice*, where many-to-many relationships have already been resolved, we are left to consider the subtype/supertype relationship between `LOYALTY_CUSTOMER` and `CUSTOMER`:

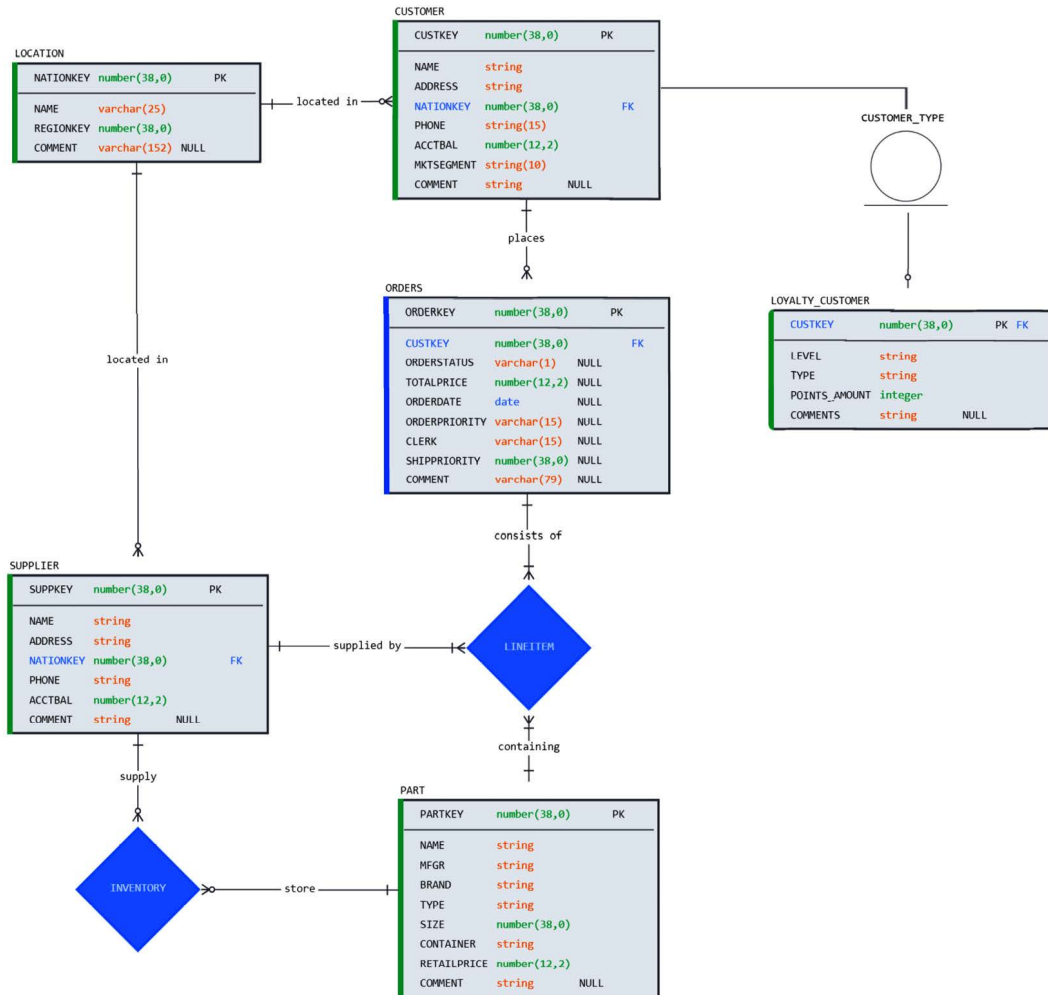


Figure 12.1: A logical model

Recall that subtypes represent inheritance between entity classes. Before defining the physical structure, inheritance rules should be decided.

Chihuahuas and mastiffs, despite obvious size differences, share many common properties of the dog class. Likewise, logical subtype properties are mutually compatible with the supertype (both CUSTOMER and LOYALTY_CUSTOMER have a CUSTOMER_ID). When deciding on a physical design, business requirements and query patterns will dictate whether the two should remain separate tables or fuse into one, and by what means if so.

Perhaps our organization has an entire marketing department dedicated to working with loyalty customers and frequently maintains attributes that are exclusive to them. In that case, a stand-alone LOYALTY_CUSTOMER table makes sense from a maintenance standpoint. If the organization were only concerned with keeping tabs on the percentage of loyalty customers relative to the total, a single CUSTOMER table containing loyalty attributes would be preferable.

There are several options when deciding the physical representation of subtypes based on the business requirements. Recall the scenario of superhero subtypes from *Chapter 2, An Introduction to the Four Modeling Types*:

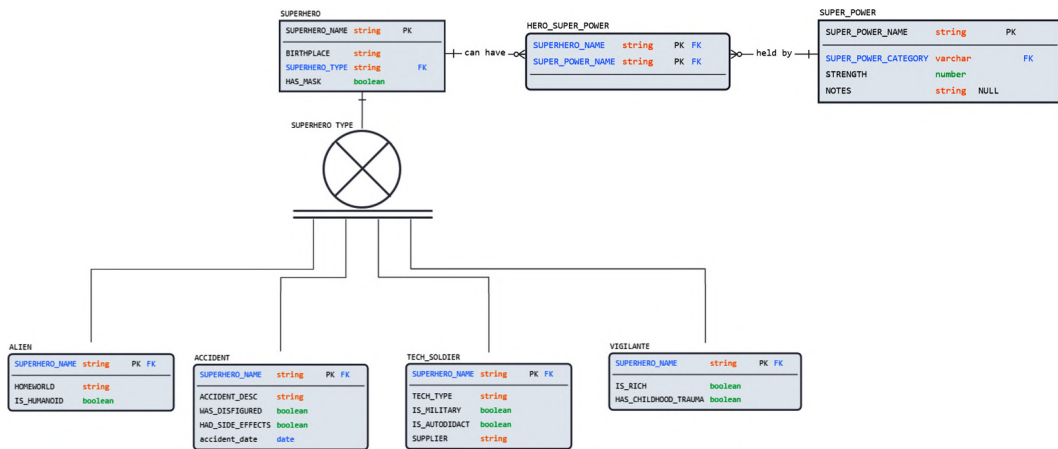


Figure 12.2: A logical subtype/supertype relationship

Converting this arrangement to a physical model can happen in one of the following ways:

- **Maintain logical:** Keep all five tables and existing attributes as defined in the logical model.
- **Inherit from supertype:** Keep all five tables and duplicate supertype attributes (such as BIRTHPLACE and HAS_MASK) to the subtypes. This results in denormalized supertypes that are easier to query but harder to maintain.
- **Rollup:** Keep only the supertype (SUPERHERO) by inheriting the attributes from the four subtypes.
- **Rolldown:** Keep only the four subtypes by inheriting the supertype's properties (such as BIRTHPLACE and HAS_MASK).

For this exercise, we will choose the first option and maintain the logical definition, which results in two physical tables (CUSTOMER and LOYALTY_CUSTOMER).

Now that all the tables are known, it's time to adjust their physical properties.

Defining the tables

It is time to unify the lessons of previous chapters and put them into practice. This is the point in the design process where decisions about naming, such as case, plurality, prefixes, and suffixes, must be finalized.

Naming

This example will follow standard modeling best practices, reiterating the caveat that every organization must choose what works best and aligns with any existing standards. The conversion yields the following changes from the logical model:

- **Case and table naming:** Convert to singular snake case (ORDERS to sales_order)
- **Unique identifiers:** Name unique identifiers consistently <table_name>_id (P_PARTKEY to part_id)
- **Remove prefixes and suffixes:** Applies to table and column names (P_NAME to name)
- **Remove abbreviations:** Make columns unambiguous (P_MFGR to manufacturer)
- **Add units to amounts:** Make amounts unambiguous by adding units (C_ACCTBAL to account_balance_usd and L_DISCOUNT to discount_percent)

With naming in place, it's time to determine the table properties for the entities in our model.

Table properties

Considering storage costs, backup requirements, data sensitivity, and performance, table properties should be set according to business needs. Suppose our example will be used to manage daily operations at the historic *AdventureWorks* company. In this case, we should opt for standard (non-transient) tables that offer a fail-safe and an adjustable retention period.

If 2 weeks of data retention is required for our data, we can set that default at the schema level to be used by all tables unless otherwise specified. Because this is an operational schema, access should be tightly controlled by the owner, so we opt for managed access.

The database and schema declaration would look something like this:

```
create database adventureworks;
create schema operations
    WITH MANAGED ACCESS
    DATA_RETENTION_TIME_IN_DAYS = 14;
```

Although performance and data volume are a concern, Snowflake does not recommend clustering before baseline usage metrics are established. We require two pieces of information before clustering: current query and DML performance and query and loading patterns.

After table properties, column data types are configured. Recall that Snowflake accepts common data-type synonyms such as `string` and `integer` and automatically translates them to their equivalents (`varchar` and `number`, respectively).

The final element that remains is table constraints.

Declaring constraints

Physical constraints translate directly from the logical model to help ensure data integrity and usability. Although (with the exception of hybrid tables) `NOT NULL` is the only constraint enforced by Snowflake, `PK`, `FK`, and `UNIQUE` (`AK` or **alternate key**) constraints provide valuable relational context and aid in query performance when used with the `RELY` property.

The Snowflake query engine uses the `RELY` constraint property to eliminate unnecessary joins. A deep dive into its use cases and implementation will follow in the next chapter, so this discussion will focus only on its declaration. It should be noted that there is no monetary or performance cost to using the `RELY` property in tables that don't violate uniqueness constraints. For nearly all use cases, setting the `RELY` property is recommended.

Following the previous chapter's naming suggestions for relational constraints, we now declare the `PK`, `FK`, and `AK` constraints for our tables. To provide an example of an alternate key, we declare one on the `NAME` column of the `LOCATION` table. To retain relationship names, remember to include them in the foreign key, as discussed previously. In summary, the following conventions are suggested:

- **PK naming:** Include table name (`pk_supplier` for `supplier` table)
- **FK naming:** Include child table, logical relationship name, and parent (`fk_sales_order_placed_by_customer`)
- **AK naming:** Include table name and qualifier as multiple alternate keys can be declared (`ak_location_name` for the alternate key name in the `location` table)

Alternate keys as a foreign key reference



Foreign key relationships can reference alternate keys (UNIQUE) as well as primary keys. This fact is often overlooked in database modeling but opens many more possibilities when establishing relations between tables. In cases where a surrogate key is declared primary, users can still reference the business key as a foreign key if declared as an alternate key.

The resulting physical model is now complete, and can be seen in the following diagram:

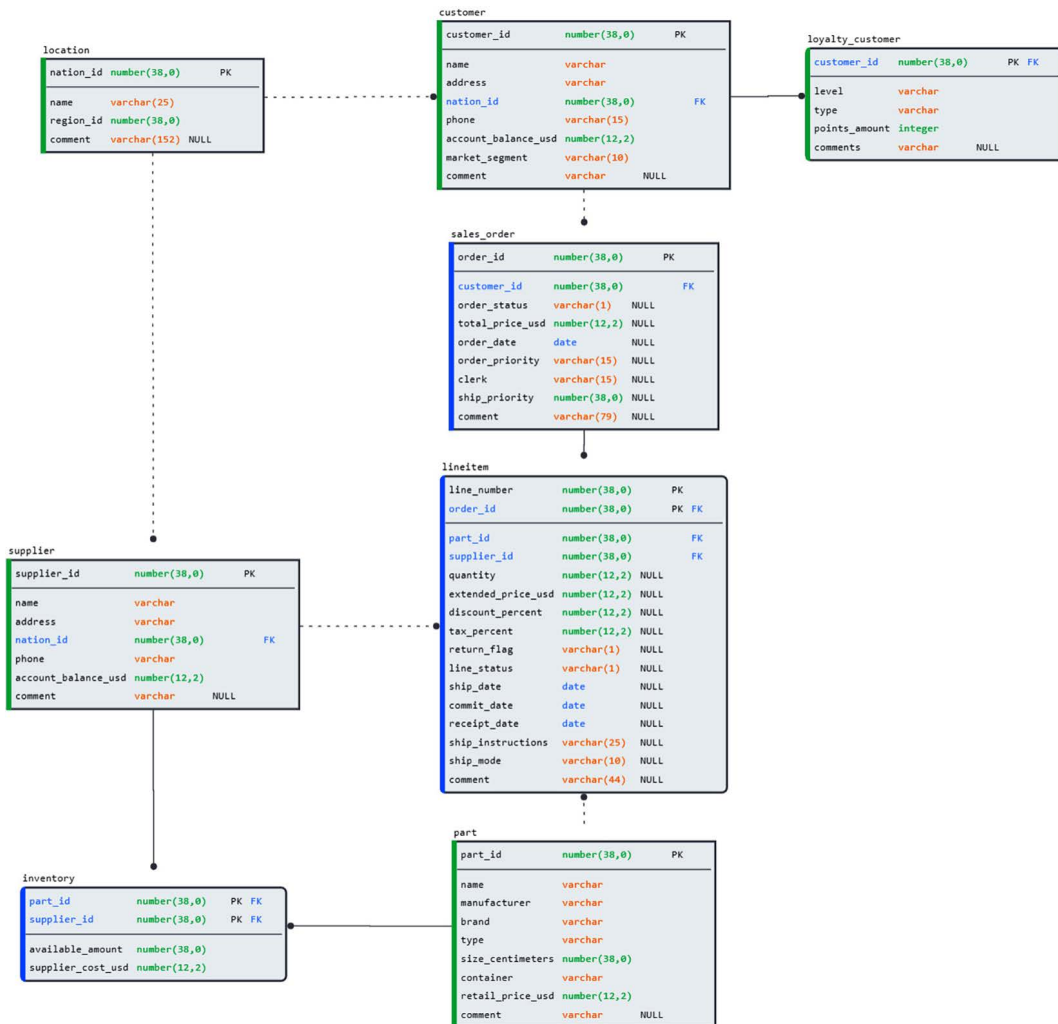


Figure 12.3: A finished physical model

With all the parameters configured, the physical model is ready to be deployed.

Deploying a physical model

At this point, all the tables, relationships, and properties have been defined and are ready to be deployed to Snowflake. If you use a modeling tool, all the DDL will be generated behind the scenes as adjustments are made to the diagram through a process called **forward engineering**. While it's not strictly necessary to use a modeling tool to forward engineer, doing so will make it easier to make adjustments and generate valid, neatly formatted SQL for your data model.

For those following the exercise, the forward-engineered DDL from this exercise is available in a shared Git repository mentioned at the start of this chapter.

With the DDL in hand, pay attention to the database and schema context in the Snowflake UI. Creating a database or schema will automatically set the context for a given session. To switch to an existing database or schema, use the context menu in the UI or the `USE <object> <object name>` SQL expression. Here's an example of this:

```
use database adventureworks;  
use schema operations;
```

Now, run the DDL you've generated or use the example provided.

The physical model has been deployed to the database. But what happens if you want to visualize an existing database schema? Can modeling be used in reverse to help navigate unfamiliar databases?

Creating an ERD from a physical model

As we just demonstrated through the forward engineering deployment process, a physical database model is a one-to-one representation of its relational diagram. This implies that the process of generating a diagram can be run in reverse—from Snowflake DDL to a modeling tool—a process known as **reverse engineering**. Again, it's not strictly necessary to use dedicated modeling tools such as SqlDBM—many SQL IDEs such as Visual Studio Code and DBeaver can generate **Entity-Relationship Diagrams (ERDs)**—doing so will offer greater flexibility in organizing, navigating, and making adjustments to your model.

A similar diagram to the one created in the previous exercise can be generated by connecting to our deployed model through a SQL IDE:

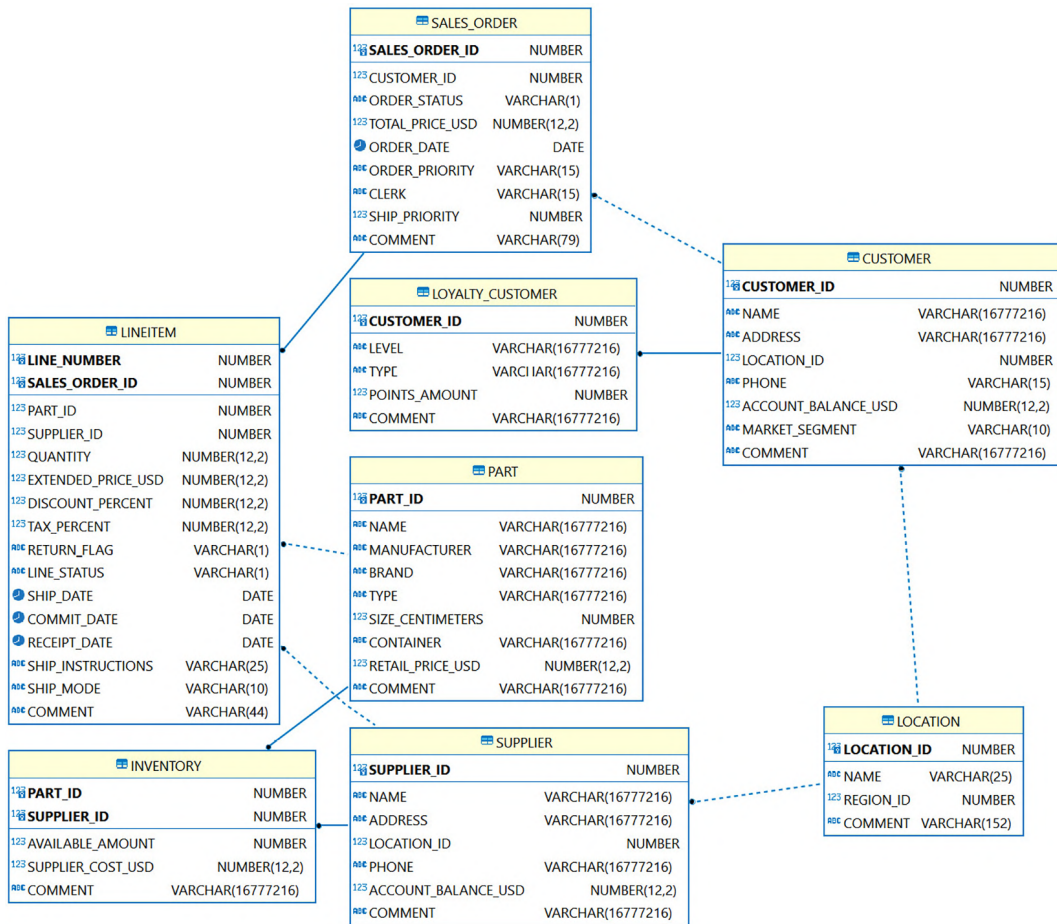


Figure 12.4: Reverse engineering in DBeaver IDE



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



What is evident in this exercise is often overlooked in database designs—the fact that a neat, related, documented ERD is quickly rendered through reverse engineering only when due diligence has been done to properly model its details. While some IDEs can render an ERD from a physical database, they do not have the editing capabilities of a modeling tool. Without following proper modeling guidelines, any generated ERD will be lacking in practical value.

Now that we understand how to transform a logical design into a physical model and reverse engineer it to create a visual representation, we can take the next step into semantic modeling. Semantic models build on physical models by incorporating semantics that allow for unambiguous analytics. These models create accessible, business-friendly layers that accurately reflect real-world concepts and relationships, making it easier to convert them into raw SQL. In the following section, we will explore how to extend the physical model into a semantic model and enable natural language query processing, especially when paired with Snowflake’s native AI capabilities.

Evolving from physical to semantic models

A well-defined physical model with constraints provides a solid foundation for subsequent transformations or analytics using SQL or external dashboarding tools. But Snowflake gives users much more flexibility than traditional BI and allows users to “talk to their data” using natural language. Using **Cortex Analyst**, Snowflake’s fully managed LLM-powered assistant, users bypass SQL altogether and get direct answers to their analytics questions.

However, let's not forget that LLMs, at heart, are non-deterministic probabilistic pattern predictors—and relatively expensive ones at that. The more hints or context we can provide to the model, the less guessing it will have to do, thereby maximizing its accuracy. To achieve this, we transform the physical model into a format that the LLM can interpret: a semantic model.



As described below, a semantic model has wide-ranging benefits beyond Cortex and LLM-related use cases. However, since the feature in Snowflake is directly linked with Cortex Analyst, the examples and exercises will focus on that directly.

A semantic model is a conceptual framework that defines the meaning, relationships, and structure of data within a specific domain. It serves as a bridge between the raw data and how users understand and interact with it. Semantic models are not unique to Snowflake, they are used throughout the industry by data catalogs, ETL tools, and data visualization tools. As an industry standard is yet to emerge, and in the context of this book, we will focus exclusively on Snowflake's semantic model and its specification. The full specification is available in the Snowflake documentation (<https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst/semantic-model-spec>).

A semantic model simplifies data access by allowing users to avoid complex SQL queries and confusing table relationships. Instead, it provides self-service access to reliable, standardized definitions of important business concepts such as “Revenue,” “Active Customer,” and “EMEA.” This democratization of data access empowers business stakeholders to independently explore and analyze information, thereby reducing bottlenecks and accelerating decision-making processes. The true strength of a semantic model lies in its ability to centralize business logic and eliminate inconsistencies in metrics across an organization.

When business rules and KPI calculations are integrated into the semantic layer, teams can consistently reuse these definitions across various dashboards, reports, and analytical tools. This ensures that everyone is literally “speaking the same language” when discussing business performance and prevents conflicting interpretations among teams—building trust and improving data quality. Furthermore, these models provide essential business context that is often lacking in raw databases—such as meaningful groupings, complex formulas, user-friendly synonyms, and hierarchical relationships—making business logic more accessible and human-readable than pure SQL.

Semantic models are domain-specific, reflecting the unique perspectives, terminology, and business logic that different functional areas bring to the same underlying data. For instance, a customer success team might measure “Active Users” based on product engagement metrics, while the billing team defines the same concept based on subscription status. These interpretations aren’t errors—they represent legitimate, contextually appropriate ways of understanding the business from each team’s operational perspective. Rather than forcing artificial consistency that doesn’t reflect business reality, organizations benefit from maintaining separate semantic models that accurately capture each domain’s unique requirements when domain knowledge diverges from the core central definition. The key is to ensure transparency about these differences and provide clear governance on when and why different definitions exist, enabling stakeholders to understand the context behind the varying metrics across various parts of the organization.

Let’s familiarize ourselves with what a semantic model consists of and the format it’s defined in so we can create one and “talk,” using natural language, to the physical model we just created.

Working with a semantic model

Snowflake’s semantic models include standard elements typically found in other industry standards, such as tables, dimensions, and metrics, among other key components. The models are defined using YAML and can also be created as semantic views. Semantic views are database objects that enable users to select named metrics using SQL.

To better understand what constitutes a semantic model, let’s examine its components. A semantic model, defined in a YAML schema, consists of hierarchically nested elements, which include:

- **The model:** Since models are domain-specific, each one needs a name and description to understand its focus and intended purpose.
 - **Tables (logical):** The data sources or entities that comprise the model. The “logical” part serves to abstract the database object that contains the data, which can be physical tables or views (and possibly other database objects in the future). Users can specify properties such as the fully qualified name, primary key, and metadata, including synonyms and sample values.
 - **Dimensions:** As per Kimbal, dimensions refer to columns (or expressions) that are (typically) not dates or numbers. Properties include *data type*, *sample values*, and *unique*.
 - **Time dimensions:** Date or time type columns that provide temporal context for analyzing facts and metrics over time intervals.

- **Facts:** Measurable, quantitative data that provide context for analyses. Facts represent numeric values related to business processes such as sales, costs, or quantities. A fact is an unaggregated, row-level concept.
- **Metrics:** Quantifiable business measures typically calculated by aggregating facts across multiple rows. Metrics can be expressed as a SQL formula combining multiple columns and aggregate functions.
- **Filters:** Named conditions that limit query results. These include groupings of column values or complex expressions.
- **Relationships:** Relationships between (logical) tables that enable joins. These should already be specified as constraints in your physical model to ensure consistency.
- **Verified queries:** Saved queries with accompanying natural language questions that have been reviewed and signed off by subject matter experts.
- **Custom instructions:** Serve as natural language hints to the LLM, providing greater control over SQL generation and accuracy. For example, users can provide instructions such as “round all metrics to 2 decimal places” and “our fiscal year starts in February.”

As you can tell from these components, the foundation of a semantic model depends entirely on the accuracy of the physical model (i.e., the entities, unique identifiers, and relationships). If the physical model is lacking, the semantic model may produce incorrect results or none at all. Let’s observe an example of how a semantic model behaves under these scenarios.

Building a semantic model

In this example, we will start with an already created semantic model that contains all the necessary context based on the physical model we created in this chapter. Seeing the full potential of a semantic model will require data, so we will use a script from a later chapter to populate our DEMO_TPCH schema with sample data.

To get started, run the DDL provided in the following location (https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/blob/main/ch13/create_physical_model_w_data.sql). Then download the semantic model YAML from the GitHub repo at (https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/blob/main/ch12/operations_semantics.yaml).

Next, create a stage to upload the existing semantic model spec.

```
CREATE STAGE myStage;
```

Now perform the following steps to upload the YAML and create the semantic model:

1. Navigate to the Cortex Analyst screen from the Snowsight UI main menu at AI & ML / Cortex Analyst.
2. Select the **DEMO_TPCCH** database created in the previous exercise.
3. Click on **Create new model** and select **Upload your YAML file**.

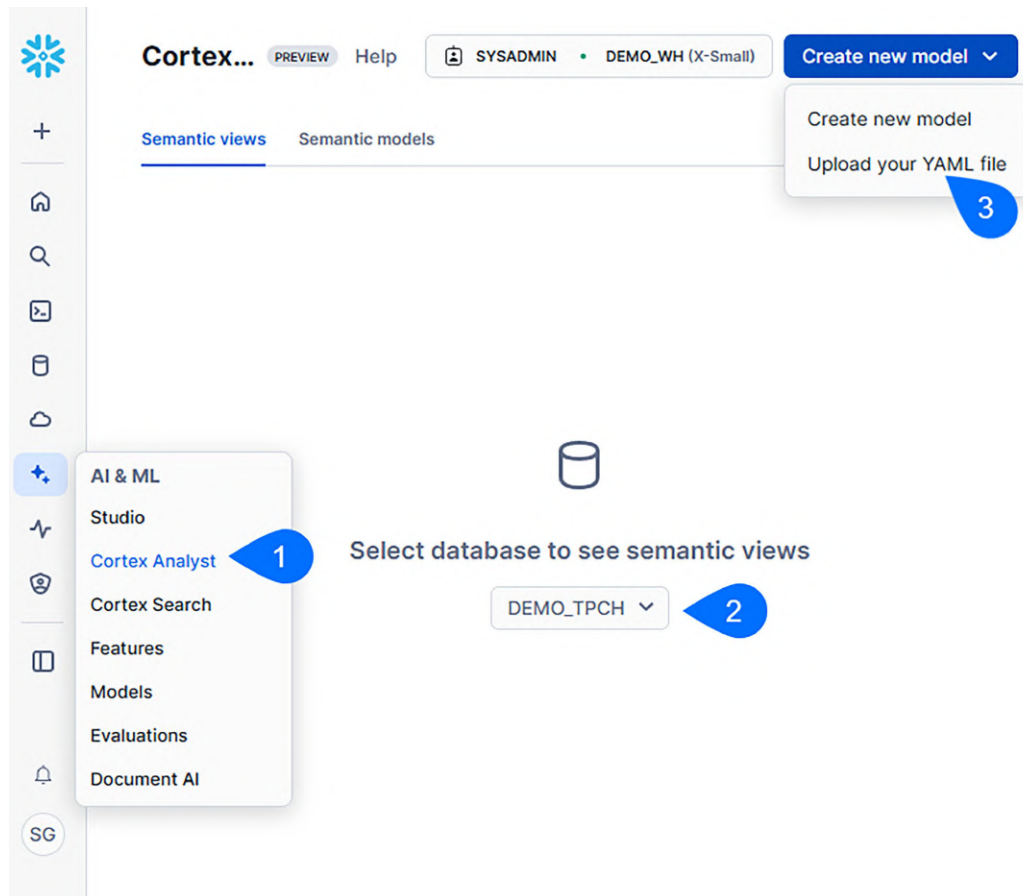


Figure 12.5: DEMO_TPCCH database

On the following screen, upload the `operations_semantics.yaml` file previously downloaded from Git Hub and select **MYSTAGE** as the target.

Upload Your Files

1 file → MYSTAGE

operations_semantics.yaml - 19.1KB

Select database, schema and stage

Schema DEMO_TPCH.OPERATIONS MYSTAGE

Specify the path to an existing folder or create a new one (optional)

Example / Path / To / Folder

Once the files finish uploading, your Directory Table will automatically refresh.

Close Upload

Figure 12.6: Upload your files page

As this semantic model is already complete with entities and relationships matching our physical model (and additional semantics like synonyms and custom instructions), we will work backward: asking questions of a complete model and then removing some of the established semantics to see how the model performs.

Now, let's try asking the following questions:

1. What countries are in North America?
 - a. Answer: CANADA and UNITED STATES
2. What line items exist for sales order id 146402 and what is the order priority?
 - a. Answer: 32 line items with an order priority of "MEDIUM."

3. What client had the highest total sales on their first order date over the entire available time period?
 - a. Answer: Customer#000062836 with a total sales of 701292.75
4. What client in North America had the highest total sales on their first order date over the entire available time period?
 - a. Answer: Customer#000921310 with a total sales of 544855.11

operations_semantics

This semantic model is used by the operations team to analyze sales and customer data.

Custom Instructions

Ensure that all numeric columns are rounded to 2 decimal points in the output if decimals exist.

Logical tables 4

What is a logical table?
A logical table can be thought of as a simple view over a physical database table or view. Each logical table is a collection of logical columns, which are categorized into Dimensions, Time Dimensions, and Facts. [Learn more](#)

CUSTOMER

This table stores information about customers, including their unique identifier, name, address, location, phone number, account balance in US dollars, and market segment, as well as any additional comments.

DEMO_TPCCH.OPERATIONS.CUSTOMER

- Dimensions: 7
- Time Dimensions: 0
- Facts: 1

ACCOUNT_BALANCE_USD

User

What client in North America had the highest total sales on their first order date over the entire available time period?

Cortex Analyst

This is our interpretation of your question:

What customer in North America (location_id IN (3, 24)) had the highest total sales (total_price_usd) on their first order date (earliest order_date) over the entire available time period? Include the full date range of data used in the analysis.

CUSTOMER_NAME	FIRST_ORDER_DATE	TOTAL_SALES	DATA_START_DATE
Customer#0009213	1992-03-22	544855.11	1992-01-01

Semantic query - Physical query (executed)

```
WITH first_orders AS (
  SELECT
    so.customer_id,
    so.order_date,
    ROUND(SUM(so.total_price_usd), 2) AS total_sales,
    RANK() OVER (
      PARTITION BY so.customer_id
      ORDER BY
        so.order_date ASC
    ) AS order_rank
  FROM
    sales_order AS so
  GROUP BY
    so.customer_id,
    so.order_date
)
```

Figure 12.7: operations_semantics.yaml screen

But wait, how did we generate sales for North America when there is no record of that location in the database? Additionally, how did the system accurately identify a “client” as a customer when there isn’t a “client” table present? The answer can be found in the clues provided by the semantic model.

Edit the semantic model and remove the named filter in location.

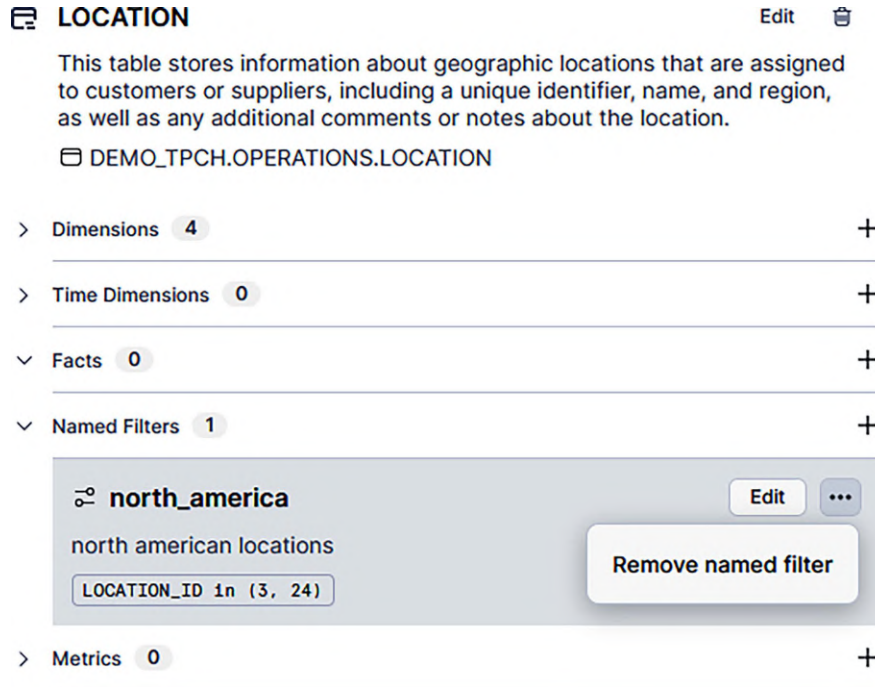


Figure 12.8: Editing semantic model to remove named filter

Now ask the same question:

1. What countries are in North America?
 - a. Answer: ARGENTINA, BRAZIL, CANADA, PERU, UNITED STATES
 - b. If there were multiple interpretations of “North America” in the company, separate semantic models would need to be created to avoid ambiguity, or the named filter would need to be split into something like “North America Sales” and “North America HR.”

Named Filters 0

Metrics 0

LOCATION

This table stores information about geographic locations that are assigned to customers or suppliers, including a unique identifier, name, and region, as well as any additional comments or notes about the location.

DEMO_TPCH.OPERATIONS.LOCATION

Dimensions 4

Time Dimensions 0

Facts 0

Named Filters 0

Metrics 0

SALES_ORDER

This table stores information about individual sales orders, including the order details, customer information, order status, and shipping details, providing a record of each transaction made by a customer.

DEMO_TPCH.OPERATIONS.SALES_ORDER

Dimensions 6

User

What countries are in North America?

Cortex Analyst

Request ID

This is our interpretation of your question:

What countries (location names) are in North America (region_id = 1) over the entire available time period?

NAME
ARGENTINA
BRAZIL
CANADA
PERU
UNITED STATES

Semantic query

Physical query (executed)

```

SELECT
  DISTINCT l.name
FROM
  location AS l
WHERE

```

Enter prompt

Figure 12.9: Updated semantic query result after removing filter

It's important to note that leaving a concept open to interpretation and not clearly defining it can introduce errors into our analysis. This can lead to inconsistencies, disputes, and decreased overall trust in the data. In this exercise, we posed specific questions to the semantic model, but it's worthwhile to remember that they can also be read as documentation for discovering definitions, synonyms, and concepts that are non-obvious (or non-existent) in the underlying data.

Semantic models are powerful tools that enable users across various functions to confidently engage with data and derive insights without requiring deep technical expertise. However, these models are not infallible and may generate incorrect answers or no answers at all if they are not grounded in a trusted and verified physical model.

Summary

The exercises in this chapter demonstrate what is required to transform a logical model into a deployable, physical design. However, before such transformation occurs, each project's use case should be carefully considered. As there is no one-size-fits-all guideline for Snowflake databases, decisions must be made considering performance, cost, data integrity, security, and usability. However, unlike traditional databases, long-standing issues such as backup, recovery, and scalability are handled by Snowflake features and architecture.

Once the physical properties have been decided, users create physical equivalents of all logical objects, including many-to-many and subtype/supertype relationships, yielding a final set of physical tables. Following this, naming standards, database objects, columns, and their relationships are declared before deploying the resulting model.

Deployable Snowflake DDL code is produced from an ERD through a process called forward engineering. As the physical model and its diagram are one-to-one equivalents, the deployment process is straightforward and can be run just as easily in reverse.

Generating ERDs from existing Snowflake databases can be done through reverse engineering, a process supported by modeling tools and many SQL IDEs. However, as the database structure and its physical diagram are equivalent, reverse engineering cannot miraculously create usable diagrams if proper modeling has not occurred—garbage in, garbage out.

We observed a similar phenomenon with semantic models. When they were complete and accurate, we could provide both business users and the Cortext Analyst AI with valuable context. This context helped us define concepts, groupings, and terms that were otherwise absent in the raw data. However, once that context was removed, even the most advanced AI struggled to generate the expected results.

The next chapter will demonstrate just how valuable a good physical model can be as we begin to transform the data using Snowflake queries.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

Part 3

Solving Real-World Problems with Transformational Modeling

This part focuses on advanced transformational modeling techniques and covers the theory and practice of putting them into action in Snowflake. First, we explore slowly changing dimensions—critical for maintaining historical data accuracy and ensuring consistency across different versions of data. Then, we learn how to model facts for rapid analysis, including best practices for optimizing query performance. We then explore semi-structured data formats and hierarchies, which are becoming increasingly prevalent and which Snowflake is so adept at handling.

This part has the following chapters:

- *Chapter 13, Putting Transformational Modeling into Practice*
- *Chapter 14, Modeling Slowly Changing Dimensions*
- *Chapter 15, Modeling Facts for Rapid Analysis*
- *Chapter 16, Modeling Semi-Structured Data*
- *Chapter 17, Modeling Hierarchies*

13

Putting Transformational Modeling into Practice

In the preceding chapters, we went from gathering requirements with business teams to creating and deploying a physical data model to Snowflake, which aligns with our organization's operations. Now it is time to leverage Snowflake's powerful query processing engine and its full-featured library of functions and data manipulation features to creatively transform data to answer business questions.

While physical modeling creates objects by defining the structure, transformational modeling uses logic—*selecting* existing data and creating a new object from the query result. However, query processing in Snowflake comes at the cost of compute credits. This chapter will cover the best practices for writing efficient queries in Snowflake to help control costs and increase performance.

As we build transformational models, we will also learn how to monitor their performance and detect issues using the Snowflake query profile. Most importantly, we will see how the structural foundation of relational modeling helps us create transformations faster and drives down warehouse costs through join elimination and other features.

This chapter will cover the following topics related to transformational modeling:

- Separating logic from objects to improve the database design
- Using the physical model to drive transformational design
- Performance gains through join elimination
- Joins and set operators available in Snowflake
- Monitoring Snowflake queries to identify common issues
- Building a transformational model from a business requirement

Technical requirements

The examples and exercises in this chapter require data to execute. The script used to load the model created in the previous chapter with sample data is `create_physical_model_w_data.sql`, and is included in the following GitHub repository: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch13>. The complete transformational model for the exercise at the end of the chapter is also included (`create_loyalty_customer.sql`), although readers are encouraged to recreate it from scratch.

Separating the model from the object

The ability to instantly scale up warehouses gives Snowflake users easy control over query performance and duration. However, increased warehouse size comes at the price of compute credits. Even keeping the warehouse size constant, changes in data volume and query patterns can cause performant and cost-effective data sources to degrade. To mitigate performance degradation, a view may need to be materialized as a table, or a table may need to become a materialized view.

However, even when converting from a view to a table, the transformational logic stays constant. While traditional modeling advice advocates differentiating views and other objects through suffixes (e.g., `CUSTOMER_V`), Snowflake users are encouraged to avoid such conventions. Orienting object names to their contents (e.g., `CUSTOMER`, `DIM_DATE`) rather than their object type allows modelers to easily pivot between them without breaking downstream dependencies.

Shaping transformations through relationships

The exercises in previous chapters devoted much attention to relational constraints and how they help ensure data integrity, consistency, and quality while making data management easier. Exactly how much easier will be the recurring theme of this chapter.

Even simple business questions, such as *What is the customer and location name for the customer with identifier 775699?*, require us to refer to the relational model to find the tables in which these attributes are stored and the foreign key columns that can be used to join them.



Figure 13.1: Foreign key constraint between location and customer tables

Using the information from the physical model, we can construct a query to obtain customer and location names for customer 775699 as follows:

```

SELECT c.name AS customer_name , l.name AS location_name FROM customer c
JOIN location l ON c.location_id = l.location_id
WHERE customer_id = 775699;

```

And we obtain the result we are interested in:

```

CUSTOMER_NAME      | LOCATION_NAME |
-----+-----+
Customer#000775699| ROMANIA      |

```

Using descriptive aliases

The examples in this book use a single-letter alias style. This is intended to maximize readability on a printed page but is not recommended as a general practice. Recommended alias guidelines are covered later in this chapter.

Paying attention to constraints also allows users to avoid *join explosions*—queries that return more rows than expected. Join explosions distort query results by returning duplicate values and multiplying the amount values returned by the query. However, join explosions are even more insidious at scale because they can multiply the number of records returned by a query by several orders of magnitude—severely impacting performance and needlessly burning through warehouse credits.

Blindly trying to obtain a customer name from the SALES_ORDER table can have unexpected consequences. As the relational diagram indicates, a customer may place multiple orders; therefore, CUSTOMER_ID is not unique in the SALES_ORDER table.

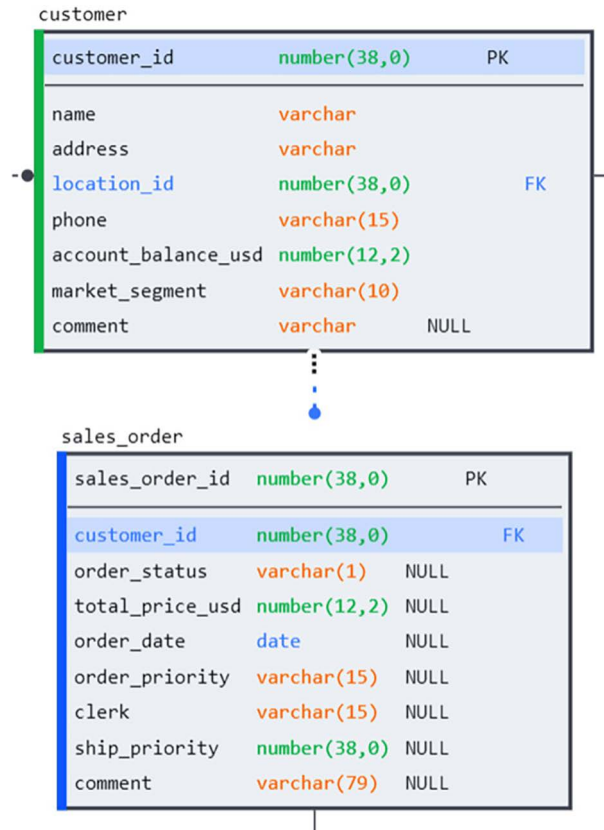


Figure 13.2: Foreign key constraint between customer and sales_order tables

Attempting to join CUSTOMER to SALES_ORDER to obtain the customer name would not be as straightforward as in the previous example, although the resulting query would look similar:

```

SELECT c.name AS customer_name FROM sales_order AS so
JOIN customer AS c ON c.customer_id = so.customer_id
WHERE so.customer_id = 775699;
  
```

However, the query returns multiple records in place of a single name (one per order made by customer 775699):

```
CUSTOMER_NAME |
-----+
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
Customer#000775699 |
```

The constraints in a relational model accelerate analytics by informing users how to link data from multiple data sources to answer business questions and avoid unpleasant surprises. But constraints can play an even more important role in query performance by eliminating certain joins altogether.

Join elimination using constraints

In the previous chapter, we set the RELY property on our constraints to pave the way for the performance gains we will now explore. Snowflake uses the RELY property to perform *join elimination*—avoiding unnecessary or redundant joins in queries. Depending on the table size, joins can be memory-intensive, so avoiding them when possible can significantly improve performance and save compute credits.

Even if a join is specified in the query, but no columns from the joined table are selected as part of the result, the RELY property will tell the Snowflake query engine to avoid performing the join.

If we modify the previous query—joining the CUSTOMER and LOCATION tables—but only request information from CUSTOMER, the RELY property will help us avoid the unnecessary join operation.

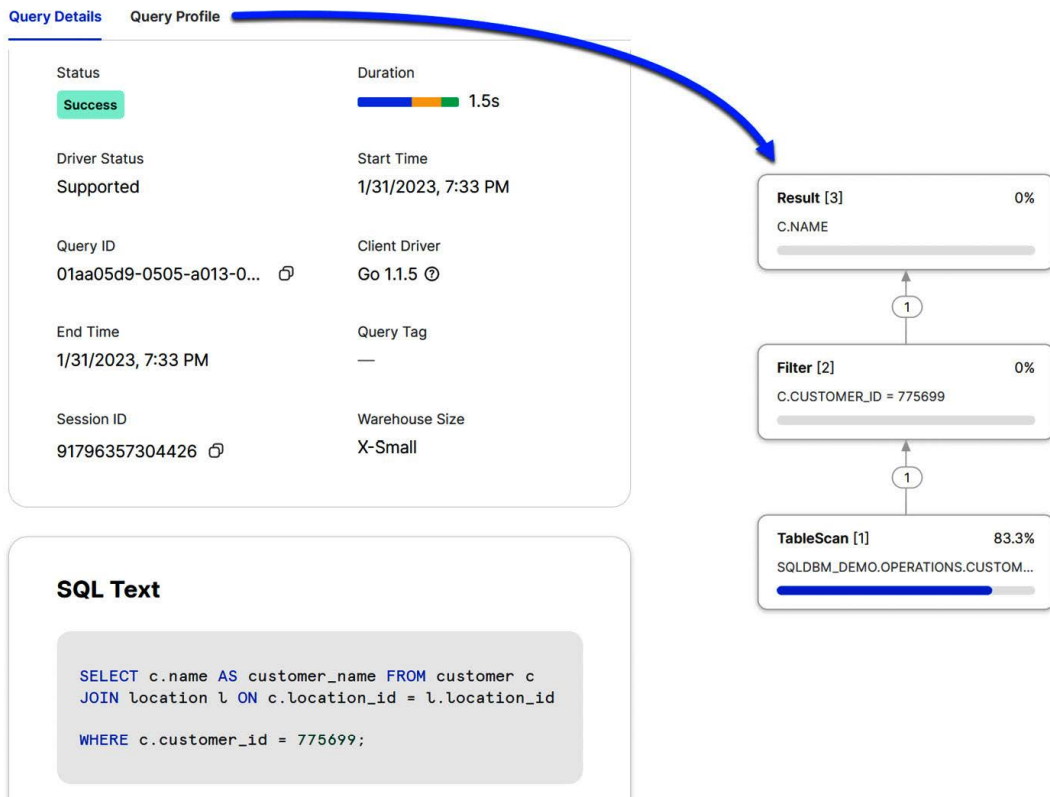


Figure 13.3: A query with join elimination

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



A look at the query profile (which we will explore later in this chapter) confirms that only the `CUSTOMER` table was referenced as part of the query operation—saving us an extra table scan and join.

Here is the same query, pointing at the equivalent tables in the `SNOWFLAKE_SAMPLE_DATA` database that do not use the `RELY` property.

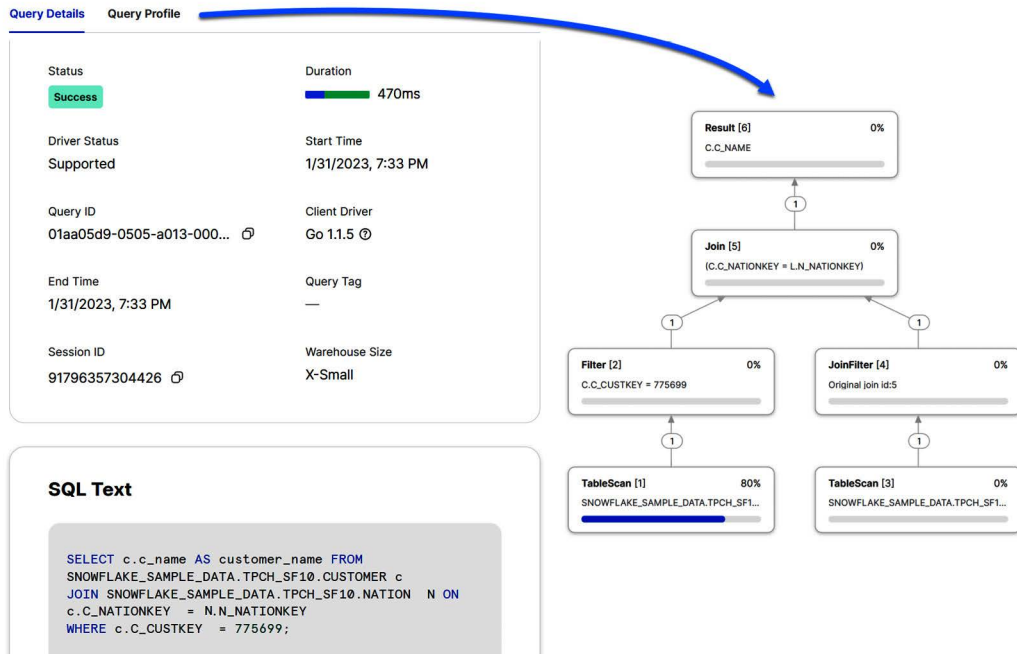


Figure 13.4: A query without join elimination

Notice that without `RELY`, Snowflake scans both the `CUSTOMER` and `NATION` tables. While join elimination can significantly improve query performance in some cases, there are times when it may present unintended consequences.

When to use `RELY` for join elimination

Star and snowflake schemas (discussed in detail in *Chapter 18, Scaling Data Models through Modern Techniques*) are popular modeling choices and offer great opportunities for improving query performance through join elimination. In these scenarios, a central fact table is joined to several dimension tables, which add descriptive attributes. Enabling the `RELY` property on the constraints between the fact table and dimensions allows us to encapsulate all the join conditions and business logic in a multidimensional view and make it available to all users, regardless of the dimensions each is interested in. When a query references the view, Snowflake will only perform the join on the dimensions explicitly requested.

When to be careful using RELY

Not all joins that reference columns from only one table are redundant. Inner joins return matching records between two tables and are often used as a filter instead of returning values from two tables. Often, master data exists as a subset of itself (e.g., VIP_CUSTOMERS as a subset of CUSTOMERS). In such cases, the constraints, if set, are semi-reliable—the primary key contains valid and unique values, but not all of them. Happily, the Snowflake optimizer is smart enough to detect and handle such scenarios when the RELY property is set on the primary key and alternate key constraints.

In the following example, a MAIN_REGION table is created as a subset of REGION. A duly titled LOCATION_BAD_FK table references REGION_MAIN as its foreign key, as seen in the following figure:

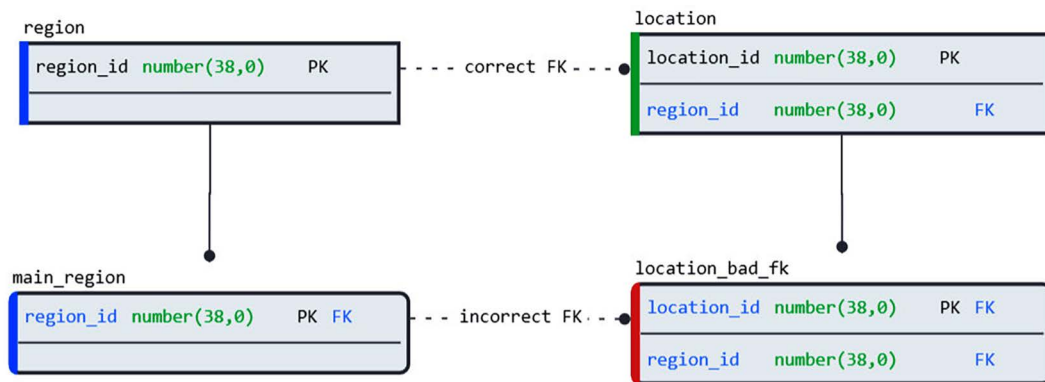


Figure 13.5: A table using an incomplete subset of data as its foreign key

Without an explicit foreign key constraint, join elimination is handled correctly for redundant joins and non-redundant INNER filter joins.

```
418
419     SELECT 'inner join' as method ,count(*) cnt FROM location
420     inner join region_main rm using(region_id)
421     union all
422     SELECT 'left join no filter' as method , count(*) cnt FROM location
423     left join region_main rm using(region_id)
424     union all
425     SELECT 'left join filter' as method , count(*) cnt FROM location
426     left join region_main rm using(region_id)
427     WHERE TRUE
428     and rm.region_id is not null
429     ;
430
```

	METHOD	CNT
1	inner join	20
2	left join no filter	25
3	left join filter	20

Figure 13.6: Join elimination handled correctly on primary key and alternate key constraints

It's only when we create an incorrect foreign key relationship between the duly titled `LOCATION_BAD_FK` and `REGION_MAIN` that we can trick the optimizer into giving us an incorrect result. To perform the experiment yourself, refer to the accompanying code in the file named `join_elimination.sql` in the repository for this chapter.

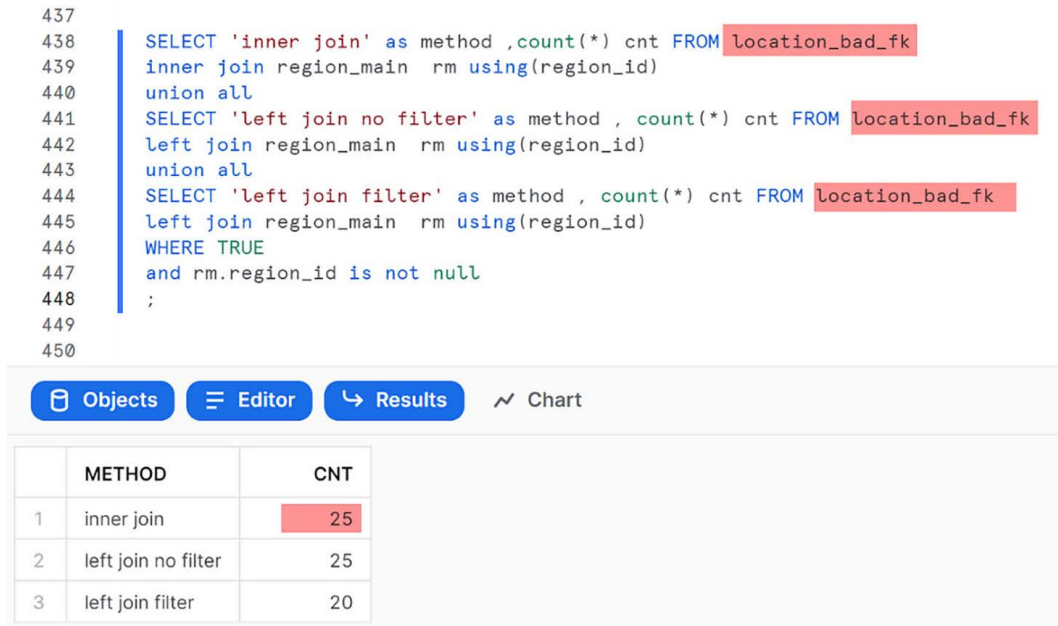


Figure 13.7: Join elimination handled incorrectly with a bad foreign key constraint

The takeaway is that the `RELY` property is an effortless and *nearly* risk-free way to optimize query performance on Snowflake views. (For more on join elimination and using it in multidimensional queries, refer to the extras section of the repository at https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/extras/04_join_elimination.)

Now that we have begun to experiment with bringing together data from multiple tables, it would be a good time to review the complete list of joins and set operators that Snowflake offers to ensure that everyone is familiar with their usage.

Joins and set operators

In a database, there are two ways to bring sources of data together: joins and set operators. A *join* combines rows from two or more tables based on related columns, and a *set operator*—such as `UNION`—combines the results of multiple `SELECT` statements. But, you can also use a join on multiple `SELECT` statements.

An easier way to think about it is that joins combine data horizontally—across related rows—and set operators work vertically. The following example shows how identical records would look in a JOIN or UNION result.

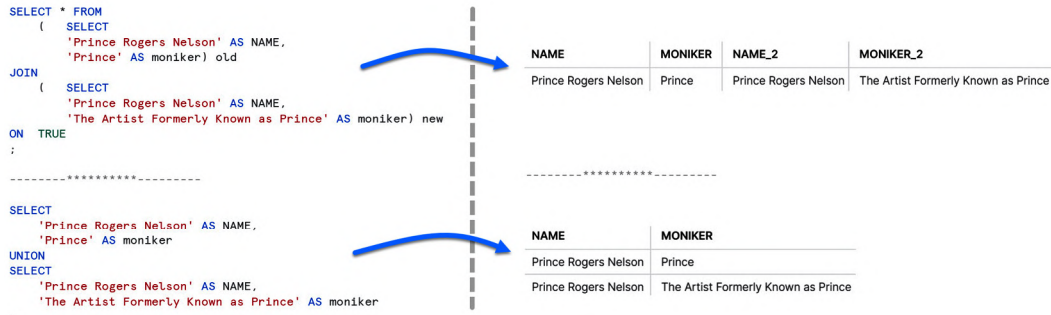


Figure 13.8: An example of a JOIN and a UNION operation

JOIN and UNION are just some of the tools at the developer's disposal. Let's review all the join and set operators that Snowflake provides and briefly cover some of their use cases.

The following diagram shows the join and set operators available in Snowflake.

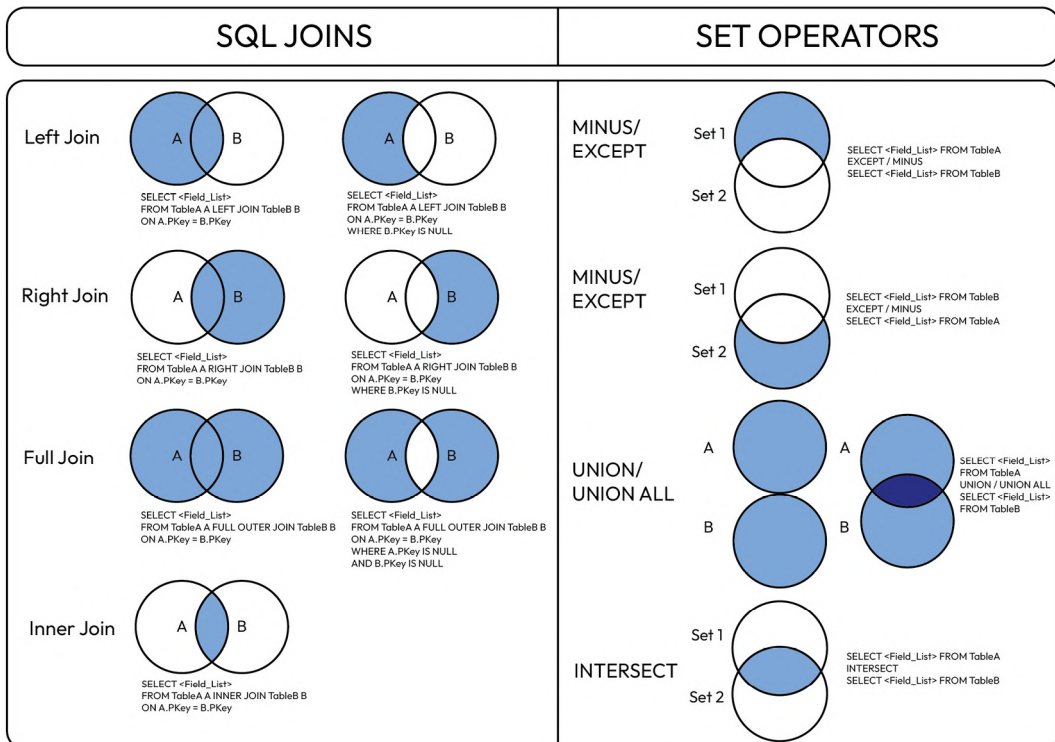


Figure 13.9: Snowflake joins and set operators

Here's an overview of joins:

- **INNER JOIN:** This selects records that have matching values in both tables. By doing so, this join also acts as a filter. Unless data integrity is guaranteed, avoid using the inner join to add related attributes in a query. For example, selecting from a fact table and joining a related dimension to obtain descriptive attributes will cause certain fact records to disappear if no matching dimension records exist. The **INNER** keyword is optional but encouraged for readability.
- **LEFT JOIN:** This returns all records from the left table and the matched records from the right table, thereby avoiding the issue described with **INNER JOIN**.
- **RIGHT JOIN:** This returns all records from the right table and the matched records from the left table.
- **FULL JOIN:** This returns all records when there is a match in either the left or right table. The full join is helpful for data reconciliation and missing data analysis.

Set operators combine the result set of two or more **SELECT** statements as long as the following applies:

- Every **SELECT** statement has the same number of columns
- The columns all have similar data types
- The columns are in the same order

Here's an overview of set operators:

- **UNION ALL:** This combines the result set of two or more **SELECT** statements without performing duplicate elimination. If a record exists in multiple sets, it will be included multiple times.
- **UNION:** This does the same as **UNION ALL** but performs duplicate elimination (using **DISTINCT**). Unless duplicate elimination is required, it is recommended to use **UNION ALL** to avoid the performance penalty of sorting **DISTINCT** records.
- **MINUS/EXCEPT:** This removes rows from one query's result set that appear in another query's result set, with duplicate elimination—often used in data validation and auditing (such as tracking changes in data over time).
- **INTERSECT:** This returns rows from one query's result set that also appear in another query's result set, with duplicate elimination—used in data reconciliation to ensure consistent data across all sources.

Before putting these into practice, let's review some of the performance and query-writing best practices that will help make our queries easier to read and maintain.

Performance considerations and monitoring

Because the Snowflake services layer takes care of query tuning and optimization, writing efficient queries and following best practices should be the developer's only concern. To do this, Snowflake provides a visual query profile to break down query execution into constituent substeps and help identify potential issues. A sample query profile can be seen in the following screenshot:

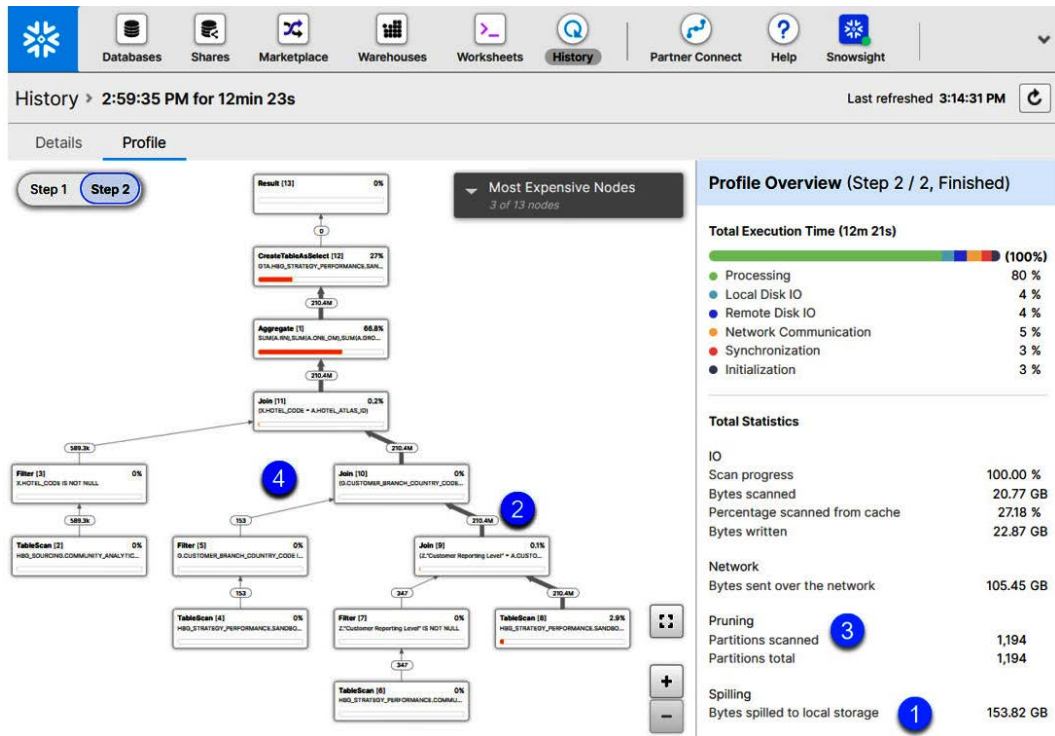


Figure 13.10: Snowflake query profile

A good way to familiarize yourself with the query profile screen is by reviewing the kinds of query issues that it is designed to identify. The numbers in the query profile correspond to the topics listed in the following section.

Common query problems

Most day-to-day query issues can be identified by paying attention to the information displayed in the query profile and following the query best practices described here:

- **Queuing:** When Snowflake determines that further queries in a data warehouse would lead to performance degradation, the query is queued and waits for the necessary system resources to become available. While a query queues, no profile information is visible.

To mitigate queuing issues, users can scale out (add warehouses) or adjust the `MAX_CONCURRENCY_LEVEL` warehouse parameter to accommodate more processes (with less resource allocation for each) (<https://docs.snowflake.com/en/sql-reference/parameters.html#max-concurrency-level>).

- **Spilling:** When a query cannot fit into warehouse memory, it begins *spilling* (writing) data to the local disk and then to network remote storage. Although it causes noticeable performance degradation, spilling is doubly pernicious because users incur credit costs for warehouse memory usage while getting the performance of writing to disk.

If spilling occurs, review the query for potential issues and increase the warehouse size to give the operation more memory to work with. Another strategy is to split the query into multiple steps or process the data in smaller segments.

- **Join explosion:** When a join results in a *Cartesian product* (also known as a *cross join*), reviewing the row counts entering and exiting a join node will reveal the error. The number of records resulting from a join between two tables should be equal to or less than (in the case of `INNER`) the greater of the two.

While record multiplication is desired in some cases, it usually results from an incorrect join condition. If the join condition matches the relational model, the tables should be checked for duplicate records.

- **Insufficient pruning:** As described in *Chapter 5, From Logical Concepts to Snowflake Objects*, Snowflake clusters table data in micro-partitions and maintains metadata and statistics for data stored in each partition. Snowflake must read every partition if a query selects all records from a table. However, if a query filter is applied, it should ideally prune (avoid scanning) some partitions and improve performance. Review the number of partitions scanned versus the total partitions in every table scan node to determine whether the pruning result is adequate.

To improve pruning, the general recommendation is to consider whether clustering can be improved. Review the examples in *Chapter 5, From Logical Concepts to Snowflake Objects* for more details on how to achieve better query pruning.

- **Inefficient join order:** Snowflake's query optimizer generally chooses the optimal join order for queries. However, if, on a rare occasion, you notice that joins are not being performed in the order of most to least restrictive, you can attempt to force the ordering using a CTE. Take the following query as an example:

```
SELECT a,b FROM x           --large fact table
  INNER JOIN y on x.c=y.c    --more restrictive join
  INNER JOIN z on x.d=z.d;    --less restrictive join
```

Reviewing the query profile, you see that tables Y and Z are being joined first, and the result is used to join table X. Knowing the data, you determine that there is a better way and rewrite the query, like so:

```
with optimized_join as (
  SELECT a,b,d FROM x           --large fact table
    INNER JOIN y on x.c=y.c     --more restrictive join
)
select a,b from optimized_join
  INNER JOIN z on optimized_join.d=z.d;  --less restrictive join
```

In addition to these methods, there are other suggestions to ensure clean, readable, and efficient queries.

Additional query considerations

Besides the performance tips mentioned in the previous section, the following guidelines should also be considered to make your queries more readable and easier to test and maintain:

- **Use table aliases:** Aliases are a good way to make queries more readable and avoid naming collisions when joining tables. Table aliases can also make referencing columns from specific tables in the query easier. When choosing an alias, use the Goldilocks principle: not too long to type out but not too short that you forget what they mean—just right. As with the table naming advice in *Chapter 11, Database Naming and Structure*, the guiding principle is to pick a style and be consistent.

- **Use common table expressions (CTEs):** CTEs can simplify reading and debugging complex queries by creating named subqueries using a `WITH` clause. This creates modular code blocks that can be separately debugged. As with all advice, exercise constraint and avoid excessive modularization, which can be as challenging to read as the spaghetti code it is meant to replace. More info on CTEs can be found here: <https://docs.snowflake.com/en/user-guide/queries-cte.html>.
- **Limit the rows returned:** When testing queries, row limits can reduce the number of records that need to be processed. The traditional way to do this is using the `LIMIT` keyword, but this suffers from two disadvantages: `LIMIT` works on the final query result (after all joins and calculations have been performed), and it returns rows in the order they were saved to Snowflake. If new data is written incrementally, `LIMIT` returns the same `N` (older) records every time. A better way is to use the `SAMPLE` function, which picks records randomly from the entire table, giving a much better representation of the data. It can also be customized to return different kinds of samples. More information on how to use and tune the `SAMPLE` command can be found here: <https://docs.snowflake.com/en/sql-reference/constructs/sample.html>. An example of limiting query results is shown here:

```
--Good
SELECT * FROM customer LIMIT 10;

--Better
SELECT * FROM customer SAMPLE (10 ROWS);
```

- **UNION without ALL:** Unless the added `DISTINCT` step is required to de-duplicate the result set, use `UNION ALL`.
- **Use EXPLAIN PLAN:** Generating a query plan requires users to run the query in question. However, you can check the execution plan of a query to see how Snowflake will execute it and check whether any issues need to be addressed. This option will not include statistics on rows returned or spilling but can be useful in validating the join order. The command to do this is `SYSTEM$EXPLAIN_PLAN_JSON`, and more info, including how to convert the output to a table or text, can be found in the documentation: (https://docs.snowflake.com/en/sql-reference/functions/explain_json.html).
- **Continuous monitoring:** Even performant queries can degrade over time as data volumes grow or data quality deteriorates. Consistent monitoring is essential to detect performance drift before it impacts warehouse costs.

The complete guide to using Snowflake's query profile can be found in the documentation: <https://docs.snowflake.com/en/user-guide/ui-query-profile.html>.

Let us put these techniques into practice by using them to create the final table in our operational schema.

Putting transformational modeling into practice

In the last chapter, we deployed our sample physical model. Here, we will populate it with data and create a transformational model to satisfy a business requirement using the techniques and best practices covered in the preceding sections.

Transformational modeling requires data. The script to populate our operational schema with sample data can be found in the Git repository for this chapter. Please run the script titled `create_physical_model_w_data.sql` to recreate the physical model and load it with data from the `SNOWFLAKE_SAMPLE_DATA` database if you have not already done so.

After running the script, all the transactional tables in the schema will have been populated with data. However, the `LOYALTY_CUSTOMER` table is not transactional; it needs to be created through a transformational model. Just as the data model in the previous chapters took shape after getting to know the workings of our company by communicating with the business teams, transformational modeling works to satisfy business questions.

Gathering the business requirements

Knowing the structure of the `LOYALTY_CUSTOMER` table from the physical model tells us little about what constitutes a loyal customer. By sitting down with domain experts from the marketing team, we can glean the business logic.

The marketing team confirms that there are two kinds of loyal customers: the top 400 based on order volume and 5 early supporters. They summarize the business logic in a formal request to define a table that recalculates the results every week. The requirements are as follows:

- The 5 early supporters, identified by `customer_id`, are 349642, 896215, 350965, 404707, and 509986. These are of type *early supporter*.
- The top 400 customers are calculated based on the sum of `total_price_usd` from `sales_order`. These are of type *top 400*.

The loyalty level is calculated as follows:

- **Gold:** early supporters and top 20 customers by `total_price_usd`
- **Silver:** top 21-100 customers by `total_price_usd`
- **Bronze:** top 101-400 customers by `total_price_usd`
- Customers in `location_id` 22 are not eligible for the program

- Customers with a negative account balance are not eligible
- The table should have a `points_amount` column, which the marketing team will maintain

It appears that various tables need to be joined to complete this calculation. Before attempting to write the query, we should validate the join conditions in the physical model.

Reviewing the relational model

Using the physical model, we can analyze the business requirements without relying on other team members. The information we need to get started can be seen in the following figure:

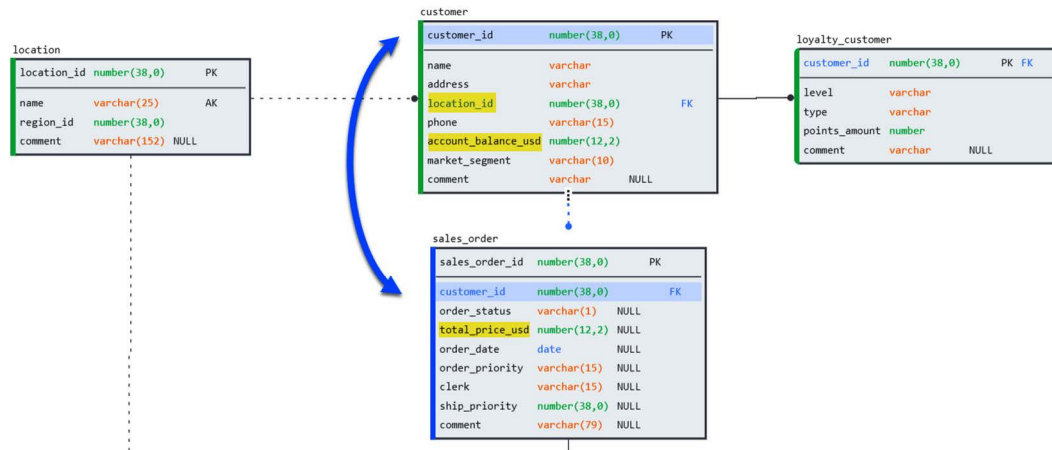


Figure 13.11: Transformational analysis using the physical model



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Reviewing the diagram, it seems that the information needed to determine a loyal customer is in a single table: `SALES_ORDER`. However, we need to join to `CUSTOMER` to exclude those who are ineligible, and the way to do so is through `CUSTOMER_ID`.

Having confirmed the details needed to create the transformational logic, we can write the necessary SQL.

Building the transformational model

The SQL required to write the transformation in this example is straightforward and is included in this chapter's Git repository (`create_loyalty_customer.sql`) for reference. Instead of focusing on its exact implementation, it would be more beneficial to highlight the sections that relate to the principles mentioned in this chapter. The numbered sections in the following figure correspond to the numbered list that follows:

```

CREATE TABLE loyalty_customer
(
  customer_id    number(38,0) NOT NULL,
  level          varchar NOT NULL COMMENT 'loyalty points score',
  type           varchar NOT NULL COMMENT 'early supporter or frequent shopper',
  points_amount  number NOT NULL,
  comment        varchar COMMENT 'customer loyalty status calculated from sales order volume',

  CONSTRAINT pk_loyalty_customer PRIMARY KEY ( customer_id ) RELY,
  CONSTRAINT fk_loyalty_customer FOREIGN KEY ( customer_id ) REFERENCES customer ( customer_id ) RELY
)
COMMENT = 'client loyalty program with gold, silver, bronze status'
AS
WITH cust AS (
  , business_logic AS (
    SELECT *
      , DENSE_RANK() OVER ( ORDER BY total_price_usd DESC ) AS cust_level

      , CASE
        WHEN cust_level BETWEEN 1 AND 20 THEN 'Gold'
        WHEN cust_level BETWEEN 21 AND 100 THEN 'Silver'
        WHEN cust_level BETWEEN 101 AND 400 THEN 'Bronze'
        END AS loyalty_level

    FROM cust_ord
    WHERE TRUE
    QUALIFY cust_level <= 400
    ORDER BY cust_level ASC
  )

  , rename AS (
    SELECT
      customer_id
      , loyalty_level AS level
      , type
      , 0 AS points_amount --will be updated by marketing team
      , '' AS comments
    FROM all_loyalty
  )

  SELECT *
  FROM rename
  WHERE true
;

```

Figure 13.12: Areas of interest in the transformational model

Some of the techniques that have been implemented in this model include the following:

- Although we are creating this table through transformational modeling, declaring relational constraints (primary keys and foreign keys) and updating the model remain equally important for usability. This example contains the constraints and the transformational logic in the same statement. Alternatively, constraints can be declared after the table has been created using an ALTER statement.

- Breaking the logic into modular chunks using CTEs allows for cleaner code that is easier to read and debug. A developer could test the result of `business_logic` by simply calling it from the final `SELECT` statement.
- Column renaming should be reserved until the end of the script. This allows developers to refer to source columns throughout the code and find all renaming in one standard place.
- Modularizing sources, logic, and renaming simplify the final `SELECT` statement, allowing easy filtering and debugging.

After delivering the finished model, the marketing team decides—as business teams often do—that a weekly refresh is insufficient, and the data needs to be updated in real time. Without changing a single line of logic, we can change `LOYALTY_CUSTOMER` to be a view instead of a table by changing the first line of the `CREATE` statement:

```
CREATE VIEW loyalty_customer AS <existing logic>
```

By keeping the object name the same, downstream processes can continue to reference it without requiring updates.

Having created the transformational object and updated the relational model in the process, let's review the takeaways from this chapter.

Summary

While a physical model reflects the company's business model, transformational models typically focus on addressing analytical business questions. Transformational models depend on existing data to perform their functions and, since data grows and changes over time, the object type used to represent the underlying logic may also change. By keeping object names constant, users can make adjustments and pivot between different database objects without breaking downstream processes.

The relationships established in the physical models can inform transformational designs as well as improving performance through join elimination by using the `RELY` property. Users can track performance to spot potential query issues using the query profile in the Snowflake UI. The query profile is a valuable tool for identifying performance issues such as exploding joins and inefficient pruning.

While transformational modeling is performed on top of the physical model, the two should always be kept in sync to provide database users with a complete overview of the system landscape. Keeping the relational model updated ensures that everyone in the organization can locate the data assets they need and identify where they fit in the overall model.

Having seen transformational modeling in action, the next chapter will put it to use in creating dimension entities and tracking their changes over time.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

14

Modeling Slowly Changing Dimensions

In *Chapter 8, Putting Conceptual Modeling into Practice*, we were introduced to database facts and dimensions. While facts capture the transactions of business operations, dimensions help give those transactions meaning by providing descriptive attributes, groupings, and other contextual details. Without careful curation and maintenance of dimension tables, databases would be like 1950s police dramas (just the facts, ma'am), lacking all color and making meaningful analysis impossible.

Dimensions shed light on the nature of entities in a data model, providing details such as a customer's billing address or a product's description. However, entity details are constantly in flux in the fast-paced business world—customers relocate, and products gain new features. A data warehouse must be able to keep up with the steady stream of changes and allow users to quickly pivot between the latest state of the world and a historical perspective.

This chapter will cover the various dimension types used to capture database entities' slowly (or quickly) changing details. Leveraging the unique Snowflake features discussed in previous chapters, we will learn how to create and update historical dimension details in cost-effective ways that have never been possible in other databases.

This chapter will outline the common dimension types and their use cases and then provide the SQL recipes used to create and maintain them.

We will cover the following main topics:

- Historical tracking requirements in dimensional attributes
- The seven types of **slowly changing dimensions (SCDs)**
- The structure and use cases of each SCD type
- Unlocking performance gains through Snowflake-native features
- Handling multiple SCD types in a single table
- Keeping record counts in dimension tables in check using mini dimensions
- Creating multifunctional surrogate keys and comparisons with hashing
- Recipes for maintaining SCDs efficiently using Snowflake features

Technical requirements

The scripts used to instantiate and load the examples in this chapter are available in the following GitHub repository: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch14>. While the key section of each script will be explained in the latter half of this chapter, please refer to the repository for the complete code used to maintain and schedule the loading of the objects discussed, as it is too long to reprint here.

Dimensions overview

A dimension unifies (or conforms) similar attributes from one or various source systems into a single table under a common unique identifier known as a business key. A single surrogate key can also be used in place of multi-column business or primary keys. The unique key of a dimension table plays a critical role in identifying dimension records and allows the database team to track and maintain changes over time. A dimension table can be structured in predetermined ways to allow for different types of change tracking depending on the business requirement.

SCD types

Attributes within a dimension have differing requirements for durability and change tracking. Some attributes are updated directly, while others require historical snapshots, yet others cannot change at all. This section will cover the types of SCDs, or update profiles, that a given attribute in a dimension can have.

It's important to note that the dimension type may not necessarily apply across all dimension attributes equally. Within the same dimension table, some attributes may be overwritten while others may not. By understanding SCD types and when to use them, database developers can implement the proper table structure and update techniques to satisfy the organization's reporting and analytics requirements.

Example scenario

To explain the various SCD types, we will use a simplified CUSTOMER dimension as an example and track the change as it would appear under each configuration.

Suppose our fact table stores order details from customer X, made on the first of every month in 2022. Thanks to X's patronage, their customer status went from *new* to *active* midway through the year. Not only do we want to track when the change occurred but we want to tie the correct status to the recorded sales facts (that is, the customer is *active* today, but half their orders were made with a status of *new*).

The change in customer status is displayed here as it currently appears in the source system and data warehouse landing area:

Source Sys. CUSTOMER (day 2022-01-01)					
CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS	
123	X	US	555-5555	new	

Source Sys. CUSTOMER (day 2022-06-01)					
CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS	
123	X	US	555-5555	active	

RAW.SRC_CUSTOMER						
CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS	ETL_ID	
123	X	US	555-5555	new	2022-01-01	
123	X	US	555-5555	active	2022-06-01	

Figure 14.1: A changed record in the source system and data warehouse raw schema

With this scenario in mind, let's explore the SCD types.

Type 0: Maintain original

Ironically, the first SCD—Type 0—does not change. *Type 0* dimensions are intended for *durable* attributes that cannot change due to their business nature. Examples of Type 0 attributes include birth dates, calendar dates, and any attribute recorded at record creation that needs to be tracked as a baseline, such as original price, weight, or date of first login.

Type 1: Overwrite

Type 1 attributes do not require historical tracking and may be directly overwritten with an UPDATE statement. Sometimes, the latest attribute value is all that the business cares about. For example, our organization demands the latest customer status, and previous values are irrelevant. Maintaining a Type 1 dimension is relatively simple—for example, if the status changes, it is updated directly in the customer dimension, as illustrated here:

CUSTOMER

CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS
123	X	US	555-5555	new
				active

Figure 14.2: New updated for STATUS change in a Type 1 SCD

However, overwriting values is often not enough—a historical value must also be preserved.

Type 2: Add a new row

For some attributes, an organization must register the latest value and maintain prior historical records. *Type 2* attributes generate a new row every time a change is recorded. Generating new rows for a given business key means that uniqueness is violated unless a time dimension (the effective date) is added to the primary key. The effective date of a Type 2 SCD not only separates historical values for a given business key but also allows those records to be tied to fact tables at a given point in time.

Maintaining a Type 2 SCD requires creating new rows when record changes are detected and additional metadata columns to track them. A single record in our example would generate the following change in a Type 2 table:

CUSTOMER

CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS	FROM_DATE	TO_DATE	HASH
123	X	US	555-5555	new	2022-01-01	2022-05-31	4f1f37ea
123	X	US	555-5555	active	2022-06-01	9999-12-31	0c680d79

Figure 14.3: New row generated for a change in a Type 2 SCD

The following metadata fields make working with Type 2 attributes easier:

- **Validity intervals:** Because the business key is being duplicated with each change, another column must be added to the primary key to maintain uniqueness. Validity intervals (also named `valid_from/to`, `start/end_date`) provide the additional unique value for the primary key and timestamp when the change occurred, allowing facts to be linked with the correct point-in-time dimension value. The `TO_DATE` column also provides a flag for identifying the latest record using the standard surrogate high date of 9999-12-31.
- **Hash:** Using a hashing function, such as MD5, provides a quick and standard way to identify when record changes occur. This concept is borrowed from Data Vault (discussed in *Chapter 18, Scaling Data Models Through Modern Techniques*). When there are many Type 2 attributes in a table, instead of checking for changes one by one, hash all of them into a single column and compare them in a single go, as follows:
 - **Create the hash field:** `SELECT MD5 (Col1 || Col2 || ... || ColN) AS hash`
 - **Compare the hash field:** `IFF(hash_new = hash_old, 'same', 'changed')`

Type 3: Add a new column

Type 3 dimensions track changes by adding a new column to store the previous value when a change occurs. The original column is updated and not renamed to avoid breaking any existing downstream references. An effective date metadata column records the time of the change, allowing analytics processes to use the new or historical value based on their validity period.

An example of a status update in a Type 3 attribute is given here:

CUSTOMER						
						
CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS	OLD_STATUS	EFFECTIVE_DATE
123	X	US	555-5555	active	new	2022-06-01

Figure 14.4: New row column created for a change in a Type 3 SCD

Although Type 3 is easier to maintain than Type 2, the limitation is storing multiple changes. While Type 2 attributes can change as often as needed, generating new rows each time, Type 3 can only show one change without creating additional columns—not a scalable design if regular changes occur.

Type 4: Add a mini dimension

When SCDs become quickly changing dimensions—due to rapidly changing attributes—the number of records that Type 2 dimensions generate can cause performance issues. This is especially true with dimensions containing many records—as in millions of rows or more.

In a **Type 4** scenario, the solution is to split the frequently changing attributes into a separate **mini dimension**. To further curtail the number of records, the values in the mini dimension can be banded within business-agreed value ranges that provide a meaningful breakdown for analysis. The mini dimension has its own surrogate key and does not contain the main dimension foreign key—allowing both to retain a relatively low cardinality. However, to tie the main dimension to the mini, the mini dimension foreign key must be included in the fact table (as the main dimension appears at the time of the generated fact).

On a diagram, the arrangement of a Type 4 dimension would look like this:



Figure 14.5: A Type 4 SCD on a relational diagram

For our example, the business wants to track the length in months for how long a customer has been active, as well as their total yearly spend at the time of each sale. To avoid generating a record for each month and order placed, the business teams have agreed to group the MONTHS_ACTIVE attribute into two categories (less than or greater than five months) and band the sales volume into three groups. The mini dimension would need to contain every possible (or allowable by existing business rules) combination of groupings.

Our example would look like this (notice how the profile ID changes throughout the year as a function of the customer's attributes):

CUSTOMER				PROFILE			
CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	PROFILE_ID	STATUS	MONTHS_ACTIVE	YEARLY_SALES_VOLUME
123	X	US	555-5555	1	new	0-4	<\$10,000
				2	new	0-4	\$10,001-100,000
				3	new	0-4	\$100,001-999,999
				4	active	5+	<\$10,000
				5	active	5+	\$10,001-100,000
				6	active	5+	\$100,001-999,999

ORDER				
ORDER_ID	FK (CUSTOMER)		FK (PROFILE)	
	CUSTOMER_ID	PROFILE_ID	ORDER_DATE	ORDER_TOTAL
1	123	2	2020-01-01	\$ 16,668
2	123	2	2020-02-01	\$ 16,668
3	123	2	2020-03-01	\$ 16,668
4	123	2	2020-04-01	\$ 16,668
5	123	2	2020-05-01	\$ 16,668
6	123	3	2020-06-01	\$ 16,668
7	123	6	2020-07-01	\$ 16,668
8	123	6	2020-08-01	\$ 16,668

Figure 14.6: Mini dimension and foreign key in a fact table in a Type 4 SCD



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



While this arrangement satisfies the reporting requirement, bridging dimension tables via a fact encumbers analysis on the dimension itself. To unify the main and mini dimensions into one, a Type 5 SCD is used.

Type 5: Type 4 mini dimension + Type 1

A **Type 5** SCD is an extension of the Type 4 mini-dimension technique—adding the mini-dimension key as a Type 1 attribute in the main dimension (hence the name, $4+1=5$). This approach affords the performance gains of a Type 4 dimension by avoiding the explosive growth of rapidly changing Type 2 records and gives users a simple way to unify the main dimension with the mini dimension through a common join column.

On a diagram, the arrangement of a Type 5 dimension would look like this:

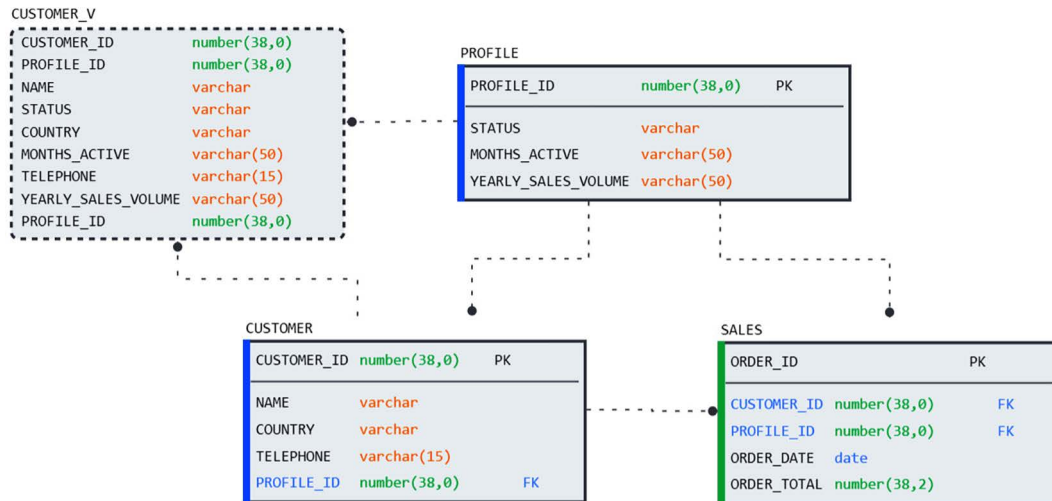


Figure 14.7: A Type 5 SCD and related view on a relational diagram

Notice that to further simplify the user experience, a view is created over the main and mini dimensions to give the users a single entity to work with. Analysis of the fact table becomes more versatile by allowing users to join on one entity (the view) instead of the main and mini dimensions if historical values are not required.

The same scenario described in the section on Type 4 would look like this under Type 5:

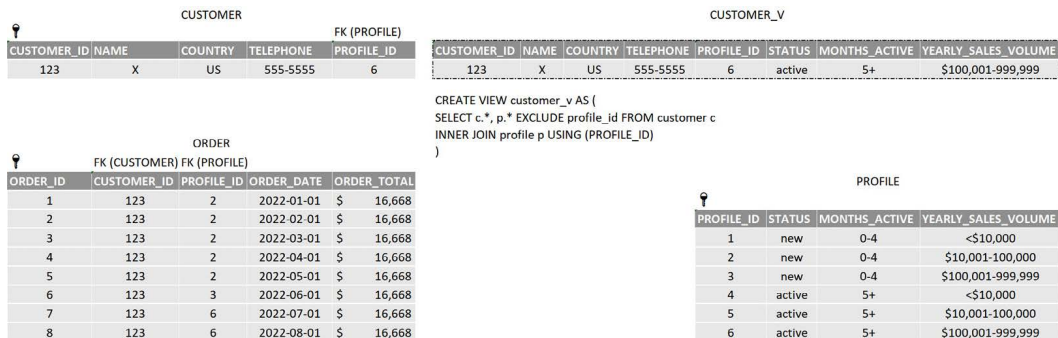


Figure 14.8: Mini-dimension and a related view in a Type 5 SCD

Unfortunately, Type 4, and by extension, Type 5, suffer from the inconvenience of calculating the mini-dimension value to include it as part of each fact. The performance implications involved in adding the mini-dimension foreign key to the fact table should outweigh the performance gain in reducing the number of dimension records through the use of the mini dimension.

Type 6: The Type 1, 2, and 3 hybrid

A *Type 6* SCD is so named because it combines the techniques of Type 1, 2, and 3 ($1+2+3 = 6$) dimensions into one table. Based on business needs, users will demand different levels of historical values to achieve a balance of detail and flexibility in their analytics.

Suppose our customer *X* from previous examples began to relocate—moving headquarters to *Mexico* in 2023, then to *Brazil* in 2024. A Type 6 approach yields a dimension table that gives analysts every possible temporal attribute value in every snapshot: a Type 1 current value, a Type 2 effective dated value, and a Type 3 previous value.

To recap the status and country changes mentioned in this example, a snapshot of the source system over time is presented here:

Source Sys. CUSTOMER

CUSTOMER_ID	NAME	COUNTRY	TELEPHONE	STATUS	
123	X	US	555-5555	new	in H1 2022
123	X	US	555-5555	active	in H2 2022
123	X	MX	555-5555	active	in 2023
123	X	BR	555-5555	active	in 2024



Figure 14.9: Source system showing changes for customer *X*

In a business scenario where the customer status needed Type 2 and the country was presented as Type 1, 2, and 3, the resulting table would look like this (the HASH column is now calculated as a function of STATUS and COUNTRY):

CUSTOMER									
CUSTOMER_ID	NAME	(T2)	(T1)	(T3)	TELEPHONE	STATUS	FROM_DATE	TO_DATE	HASH
		COUNTRY	COUNTRY_CURRENT	COUNTRY_OLD					
123	X	US	BR	US	555-5555	new	2022-01-01	2022-05-31	c0353bd3
123	X	US	BR	US	555-5555	active	2022-06-01	2022-12-31	6bbf42cb
123	X	MX	BR	US	555-5555	active	2023-01-01	2023-12-31	dc851de2
123	X	BR	BR	MX	555-5555	active	2024-01-01	9999-12-31	94402fe5

Figure 14.10: Type 1, 2, and 3 columns combine in a Type 6 SCD

Type 7: Complete as-at flexibility

Business users across all cultures and industries have a penchant for changing their minds. The Type 7 approach gives database modelers a way to deliver the needed historical attribute no matter the criteria or temporal reference point requested.

A Type 7 dimension (unimaginatively named as the number that follows 6) includes a natural key and a surrogate key in a Type 2 table structure and embeds both in the fact table.

A method for generating surrogate keys



An efficient—and data vault-inspired—way to generate a surrogate key for Type 2 records is to use an MD5 hash on the compound primary key (in this example, CUSTOMER_ID and FROM_DATE):

```
SELECT MD5(customer_id || from_date) AS customer_skey
```

In a Type 7 configuration, a surrogate key is added to an otherwise Type 2 structure and is embedded in the fact (the latest SKEY as of the creation of each fact record). Based on the example scenario from the Type 6 section, the tables would look like this:

CUSTOMER								
PK	AK					PK		
CUSTOMER_ID	CUSTOMER_SKEY	NAME	COUNTRY	TELEPHONE	STATUS	FROM_DATE	TO_DATE	HASH
123	4860a035	X	US	555-5555	new	2022-01-01	2022-05-31	c0353bd3
123	d0c90304	X	US	555-5555	active	2022-06-01	2022-12-31	6bbf42cb
123	ea5a543d	X	MX	555-5555	active	2023-01-01	2023-12-31	dc851de2
123	20c53233	X	BR	555-5555	active	2024-01-01	9999-12-31	94402fe5

ORDER							
	FK (CUSTOMER)						
PK							
ORDER_ID	CUSTOMER_ID	CUSTOMER_SKEY	ORDER_DATE	SHIPMENT_DATE	ORDER_TOTAL	LOAD_DATE	
1	123	4860a035	2022-01-01	2022-02-01	\$ 16,668	2022-01-02	
2	123	4860a035	2022-02-01	2022-03-01	\$ 16,668	2022-02-02	
3	123	4860a035	2022-03-01	2022-04-01	\$ 16,668	2022-03-02	
4	123	4860a035	2022-04-01	2022-05-01	\$ 16,668	2022-04-02	
5	123	4860a035	2022-05-01	2022-06-01	\$ 16,668	2022-05-02	
6	123	d0c90304	2022-06-01	2022-07-01	\$ 16,668	2022-06-02	
7	123	d0c90304	2022-07-01	2022-08-01	\$ 16,668	2022-07-02	
8	123	d0c90304	2022-08-01	2022-09-01	\$ 16,668	2022-08-02	
9	123	ea5a543d	2023-08-01	2023-09-01	\$ 16,668	2023-08-02	
10	123	20c53233	2024-08-01	2024-09-01	\$ 16,668	2024-08-02	

Figure 14.11: A Type 7 SCD offers complete analytical flexibility

A Type 7 SCD allows business users to select the appropriate customer attributes based on the following criteria:

- The most recent or current information (that is, `TO_DATE = '9999-12-31'`)
- The primary effective date on the fact record (that is, `LOAD_DATE`)
- When the user changes their mind, any date associated with the fact record (that is, `ORDER_DATE` or `SHIPMENT_DATE`)

Here is how those queries might look:

```
--get current
SELECT < fact and attribute fields >
  FROM order o
  INNER JOIN customer c USING(customer_id)
  WHERE c.to_date = '9999-12-31'

--get dimension values as at the primary effective date on the fact record
SELECT < fact and attribute fields >
  FROM order o
  INNER JOIN customer c USING(customer_skey)

--get dimension values as-at any date on the fact record
--example will use SHIPMENT_DATE
SELECT < fact and attribute fields >
  FROM order o
  INNER JOIN customer c USING(customer_skey)
  AND o.shipment_date BETWEEN c.from_date AND c.to_date
```



Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button

- (1) to quickly copy code into your coding environment, or click the **Explain** button
- (2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {  
  return {sum: a + b};  
};
```

Copy**Explain**

1

2



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR visit packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now that you have a general understanding of the different SCD types, let's recap before detailing the Snowflake recipes used to construct them.

Overview of SCD types

The following screenshot summarizes the seven SCD types covered in the previous section, including their maintenance strategy and usage. While eight (including Type 0) SCDs may seem like a lot, most database designs rarely go beyond Type 3, as the first four SCD types strike an acceptable balance of performance, maintainability, and historical reporting needs:

SCD Type	Strategy	Usage
0	No change	Durable attribute values that cannot change
1	Update / overwrite	No history, current attribute values only
2	Add new row	Change tracking by effective dated value range
3	Add new column	Last historical value available in the same row as current
4	Add mini dimension	Frequently changing attr. values reduced to banded ranges
5	4+1 Add mini dimension and update	Type 4 functionality without relying on a fact table
6	1+2+3 Add and update columns in type 2	Handle multiple historical requirements in a single table
7	Add Surrogate key to fact and type 2	Flexibility to handle any historical rollup on the fly

Figure 14.12: A comparison of SCD types

Now, let's see how to build SCDs with maximal efficiency using Snowflake-specific features.

Recipes for maintaining SCDs in Snowflake

Understanding the structure of an SCD and being able to load it correctly are very different concepts. With a firm grasp of SCD types, we will now cook up the recipes for creating and maintaining them in Snowflake. Unlike generic SQL techniques you may have used in other databases, this book will take full advantage of the cost- and time-saving capabilities of Snowflake's core features, such as streams and zero-copy cloning.

Setting the stage

To give you complete autonomy to construct, experiment, and modify the upcoming exercises, we will first create a base table that will simulate the *day one* snapshot of the data warehouse raw/source schema. The base table will represent the initial *first* load of the source data into the data warehouse. Next, we will construct a routine that simulates a daily load of new and changed records.

For consistency with the first half of this chapter, these examples will use the CUSTOMER table from the snowflake_sample_data.tpch_sf10 schema. Then, we will create a simulated SRC_CUSTOMER table to represent the landing area of the data warehouse which, by default, will contain one-quarter of the 1.5 million records of the sample CUSTOMER table. Finally, we will construct a task, LOAD_SRC_CUSTOMER, which will randomly load 1,000 records into the SRC_CUSTOMER table (approximately 80% new, 10% modifications, and 10% existing unchanged records). The column that will receive changes in this example is account_balance_usd. The parameters for the number of records loaded can be changed directly in the code.

Let's recap the setup here:

Object	Type	Purpose	Record Count
source_system_customer	table	full data set for loading	1,500,000
src_customer	table	simulates DWH landing area	375000
load_src_customer	task	simulates daily changes	1000

Figure 14.13: Summary of the initial setup for creating SCD examples

To get started, instantiate a new schema for running these examples, then create the three base objects as indicated in the accompanying code. The file containing the examples is `create_snowflake_scd.sql`. We will also clone `SRC_CUSTOMER` to create a backup for resetting and re-running the examples:

```
CREATE OR REPLACE SCHEMA ch14_dims;
CREATE OR REPLACE TABLE source_system_customer ... ;
CREATE OR REPLACE TABLE src_customer ... ;
CREATE OR REPLACE TASK load_src_customer ... ;
CREATE OR REPLACE TABLE src_customer_bak CLONE src_customer;
```

This script results in the following objects being created (the backup is not pictured):

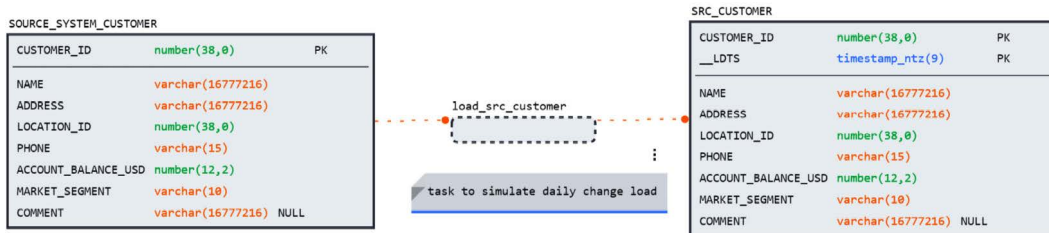


Figure 14.14: Diagram of the initial setup for creating SCD examples



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



With the base objects in place, let's begin with a Type 1 SCD.

Type 1: Merge

The Type 1 table will have a similar structure to SRC_CUSTOMER. It will even include the metadata load date column, __LDTS. However, unlike SRC_CUSTOMER, which captures changes by load date, the Type 1 table will only have one unique record for each entity in the dimension. For this reason, __LDTS cannot be part of the primary key but will be included as metadata to let users know the latest effective date of the record they are seeing.

Another field included in the Type 1 table is the DIFF_HASH column. Although the changes in our example only occur in one column, ACCOUNT_BALANCE_USD, using a DIFF_HASH field can make equality comparisons faster, cleaner, and easier.

Create and populate the Type 1 table with the initial base load from SRC_CUSTOMER by running the following statement:

```
CREATE OR REPLACE TABLE dim_customer_t1... ;
```

This results in the following table structure:

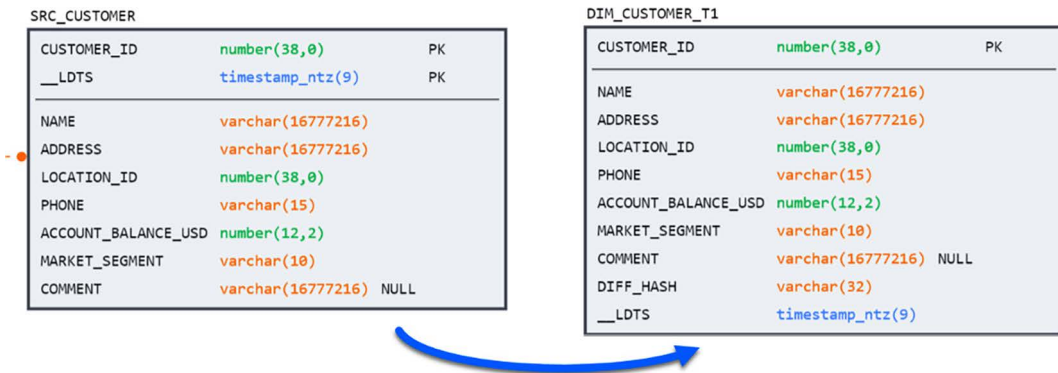


Figure 14.15: A Type 1 table structure next to the source table it's loaded from

Now, prime the SRC_CUSTOMER table by calling the load task:

```
execute task load_src_customer ;
```

Now, we are ready to perform the update. Updating Type 1 attributes requires a MERGE statement—which inserts new records or updates changes. The key sections of the MERGE statement are highlighted here:

```
MERGE INTO dim_customer_t1 dc
  USING ( < SELECT latest snapshot from source > ) sc
  ON dc.customer_id = sc.customer_id --unique identifier
  WHEN NOT MATCHED --new records, insert
  THEN INSERT VALUES ( < source columns > )
  WHEN MATCHED --record exists
  AND dc.diff_hash != sc.diff_hash --only update if changes exist
  THEN UPDATE SET < target columns > = < source columns >
```

A MERGE statement is a relatively expensive database operation since it involves a join, a compare, and writing to disk in the form of inserts or updates. To ensure we are not comparing records that have already been processed, include a filter that only looks at source records that have not yet been processed (latest __LDTS). In a typical data warehouse scenario, this logic can be encapsulated in a view for ease of maintenance. In this example, the logic has been embedded into the merge for ease of understanding:

```
MERGE INTO dim_customer_t1 dc
  USING (SELECT *, MD5(account_balance_usd) AS diff_hash
        FROM src_customer WHERE __ldts =
        (SELECT MAX(__ldts) FROM src_customer) ) sc
```

Run the MERGE statement and observe the impact on the dimension table:

```
MERGE INTO dim_customer_t1;
```

As expected, of the 1,000 records loaded, approximately three-quarters were new records, and 10% were changes:

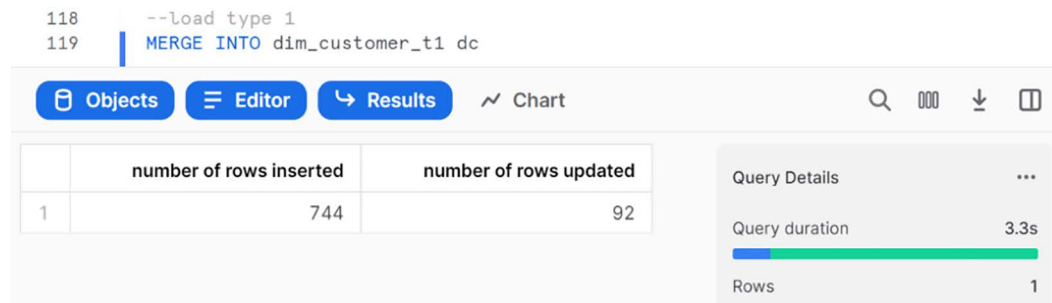


Figure 14.16: A MERGE statement for maintaining a Type 1 SCD

Feel free to experiment by running additional loads and varying the number of records to see the impact on performance. When you're ready, move on to Type 2.

Type 2: Type 1-like performance using streams

Type 2 tables are more performance-intensive than Type 1 because they contain historical changes and, over time, can grow to many times the size of the source table. However, in this section, we will explore a technique that uses Snowflake streams to achieve Type 1-like performance in a Type 2 load. If you are unfamiliar with streams and the meta columns they contain, please revisit *Chapter 4, Mastering Basic Snowflake Objects*, before proceeding with the example.

Since we will be using the same base tables to perform a Type 2 load, remember to reset SRC_CUSTOMER to the original 375,000 rows by cloning it from the backup, like so:

```
CREATE OR REPLACE TABLE src_customer CLONE src_customer_bak;
```

Now, create and instantiate the Type 2 table by running the following statement:

```
CREATE OR REPLACE TABLE dim_customer_t2;
```


Recall that this table contains metadata columns to track the validity of changed records over time. The granularity of these validity intervals (for example, monthly, daily, and millisecond) will depend on the data's load frequency and reporting requirement. Although Snowflake can maintain microsecond splits using the `TIMESTAMP` data type, most reporting scenarios would not benefit from such near-real-time changes. Although daily changes (using the `DATE` data type) are the most commonly used, this example will use `TIMESTAMP` data types to allow users to run back-to-back loads on the same day.

The Type 2 table will look like this:

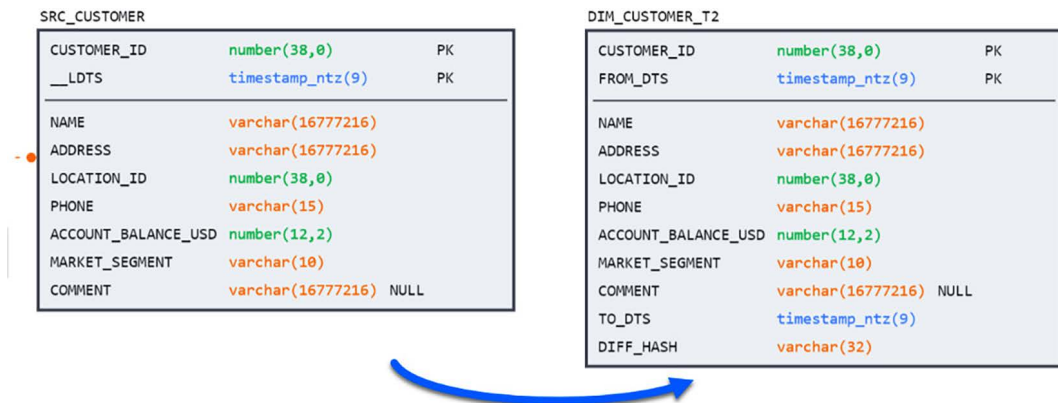


Figure 14.17: The structure of a Type 2 SCD

Note that only the `FROM` date column is required for the Type 2 primary key. In ensuring the data quality in a Type 2 setup, it is essential that for each `BKEY`, `FROM` dates are always unique, and `FROM` and `TO` intervals never overlap.

Now, instantiate the stream for the Type 2 table and kick off a simulated source load to prepare for the `MERGE` statement:

```
CREATE OR REPLACE STREAM strm_dim_customer_t2 ON TABLE dim_customer_t2;
EXECUTE TASK load_src_customer;
```

Updating a Type 2 table is done in two steps. First, we run a `MERGE` statement and update the changes as in a Type 1 load:

```
MERGE INTO dim_customer_t2 dc;
```

Notice here that the results and performance are identical to the Type 1 load so far:

```

201  MERGE INTO dim_customer_t2 dc
202  --get only latest records from src_customer. In a real-world scenario,
203  --create a view to get the latest records to make the logic leaner

```

	number of rows inserted	number of rows updated
1	751	73

Query Details ...

Query duration 2.9s

Rows 1

Figure 14.18: Step 1 of a Type 2 load is identical to Type 1 in Figure 14.16

We have overwritten the current records with the latest values just like in a Type 1 load. Now comes the hard part: figuring out which *original* records were changed so that we can insert the *before* image and apply the correct FROM and TO dates. Luckily, thanks to the previously created stream, this can be done without lookups or window functions. The stream already contains the before and after images!

A sample record from the previous merge operation is displayed next. Bear in mind that under the hood, Snowflake is insert-only—it doesn't delete or update the records directly. Because of this, we can easily see the before image exactly as it appeared before the change:

After-image							
CUSTOMER_ID	ACCOUNT_BALANCE_USD	FROM_DTS	TO_DTS	DIFF_HASH	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
418356	1973.45	2023-02-17	9999-12-31	27a46db0fcfc	INSERT	TRUE	433bfadf6e5a60d2e17342f96113b47c
418356	1881.52	2023-02-16	9999-12-31	47fd855b07c6f	DELETE	TRUE	433bfadf6e5a60d2e17342f96113b47c
Before-image							

Figure 14.19: An updated record creates before and after images in a stream

Knowing this, we can insert the before images of all the changed records into the table in a single operation—without performance-intensive joins or updates:

```

INSERT INTO dim_customer_t2
SELECT < stream columns > FROM strm_dim_customer_t2
WHERE metadata$action = 'DELETE'

```

After the insert, notice that the number of rows inserted matches the rows updated in the previous step.

40% performance gain



The alternative to using streams for a Type 2 load is to use a temporary table as seen in dbt snapshots (snapshots are dbt's version of a Type 2 SCD). This approach, although still requiring two steps, involves slightly more logic and suffers from the performance penalty of writing to two separate tables. However, to demonstrate the effectiveness of the streams technique, a comparison with a dbt snapshot is included in the accompanying code. On the standard load (using 1,000 records) the performance of both methods was identical. However, when the record limit was removed and the full 1.5 million rows were processed, the streams technique was 40% faster. The dbt-generated DML and the results are included in the repository for this chapter.

Note that another alternative involves doing a Type 2 merge that includes all changes in a single operation. This is accomplished using independent CTEs that select new records, Type 2 inserts, Type 2 expires, and Type 1 updates, and unioning them together to perform a single merge. While this approach is significantly more performant than using streams (40%–60% faster because reads and writes are consolidated), it is also considerably more complex, requiring more than twice the amount of code.

To simplify the daily loading activity of a Type 2 table, the two-step loading process can be strung together as a series of sequential tasks that can be kicked off with a single command. The instructions for doing so are provided in the accompanying code.

Now, for a variation, let's attempt to recreate a Type 2 SCD using the declarative dynamic tables approach and see how the two compare in terms of performance and effort. Remember to reset the SRC_CUSTOMER table to baseline before moving forward.

Type 2 with a dynamic twist

An alternative to using streams to create a Type 2 dimension is the declarative approach with dynamic tables discussed in *Chapter 6, Mastering Advanced Snowflake Objects*. Dynamic tables abstract the imperative DML operations and orchestration logic, resulting in fewer lines of code, fewer moving pieces, and, ultimately, simpler maintenance. The trade-off is that the entire Type 2 operation must be expressed as a single select statement (even though incremental loads will be used under the hood), which requires window functions.

Window functions enable calculations across sets of rows related to the current row. For example, we can select the test score for each student in a class and, using a window function, also display the class average in the same record. We could also select a record from an ordered set and look at records in front or behind—unlocking the key to a Type 2 dimension. This additional set of records relative to a given row is the “window” that gives window functions their name, and it comes at a cost to performance.

The anatomy of a window function

To understand how Type 2 dimensions can be solved using a declarative approach, we have to break down the structure of a window function. The format looks like this:

```
SELECT WINDOW_FUNCTION(col1) OVER ([PARTITION BY col2] [ORDER BY col3]) AS  
col4 FROM table_name;
```

The student grade and average class grade example could be written like this:

```
SELECT student_id, student_name, class_id, grade,  
       AVG(grade) OVER (PARTITION BY class_id) AS class_average  
FROM grades;
```

The relevant parts of the function are as follows:

- **WINDOW_FUNCTION:** This is an aggregate or ranking function such as SUM, MIN, MAX, LEAD, LAG, or ROW_NUMBER. Here, it is AVG.
- **col1:** This is the column or expression to be selected. Some ranking functions, such as ROW_NUMBER, do not specify a column. Here, it is grade.
- **OVER:** This specifies the window over which the function operates, further defined by its contents.
- **PARTITION BY:** This groups rows into partitions, and then the function is computed over each partition.
- **col2:** This is the column or expression by which to partition the window. Here, it is class_id.
- **ORDER BY:** When ranking functions such as ROW_NUMBER or LAG are used, this clause determines the sorting of the records for each partition.
- **col3:** This is the column or expression by which to sort the partition records. Here, none is specified because an aggregate (AVG) function is used.
- **col4:** This is the alias for the new column being selected. Here, it is class_average.

With this, we are ready to create a dynamic Type 2 dimension.

Creating a dynamic Type 2 dimension

After discussing window functions, let's recreate the Type 2 customer dimension from the earlier example using a dynamic table. A dynamic table requires us to incorporate the entire Type 2 logic within a single SELECT statement, and window functions play a key role in achieving this.

Although we'll be using various functions for calculating the system columns and comparing record versions, they will all use the same OVER window:

- PARTITION BY: The business key of the dimension. Here, it is customer_id.
- ORDER BY: The system timestamp column used by the source. Here, it is __ldts.

For example, the record version column would look like this:

```
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS __rec_
version
```

First, we create the dynamic table, specifying the required parameters, and select the source columns:

```
CREATE OR REPLACE DYNAMIC TABLE dyn_customer_t2
TARGET_LAG = '10 MINUTE'
WAREHOUSE = demo_wh
AS
SELECT
  customer_id || '|' || __ldts AS dim_skey, - for reference
  customer_id AS customer_id,
  name AS name,
  address AS address,
  location_id AS location_id,
  phone AS phone,
  account_balance_usd AS account_balance_usd,
  market_segment AS market_segment,
  comment AS comment,
  __ldts AS __ldts,
```

Next come the system columns. Although only the from/to dates and hash diffs are required, all conceivable system columns (e.g., `is_latest`, `record_version`, `create_date`, etc.) are included for demonstration purposes:

- **From date** (`__from_dts`): The timestamp indicating when the record version became effective (e.g., was first loaded)
- **To date** (`__to_dts`): The timestamp indicating when the record version ceased to be effective, set to a high date (e.g., 9999-12-31) for the current record
- **Record version** (`__rec_version`): A sequential number representing the order of changes for a given entity, starting from 1 for the first version
- **Latest flag** (`__is_latest`): A Boolean flag indicating whether the record is the most recent version (TRUE) or a historical version (FALSE)
- **Created date** (`__create_dts`): The timestamp when the first version of the record was created in the system
- **Updated date** (`__update_dts`): The timestamp when the current record version was inserted or last updated
- **Last updated date** (`__last_update_dts`): The most recent timestamp indicating when the entity was modified, capturing updates across all versions
- **Hash diff** (`__t2diff_hash`): A hash value generated from Type 2 fields to detect changes in data content, ensuring the creation of new versions when differences are found
- **Previous hash diff** (`__prev_t2diff_hash`): The hash value of the prior record version used to compare and validate changes between versions

Using the required formulas and window functions, we get the following:

```
__ldts AS __from_dts,
IFF(
    LEAD(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) IS
    NULL,
    '9999-12-31'::TIMESTAMP_NTZ,
    DATEADD(NANOSECOND, -1, LEAD(__ldts) OVER (PARTITION BY customer_id
    ORDER BY __ldts ASC))
) AS __to_dts,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS __rec_
version,
IFF(__to_dts = '9999-12-31'::TIMESTAMP_NTZ, TRUE, FALSE) AS __is_latest,
FIRST_VALUE(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS
__create_dts,
__ldts AS __update_dts,
LAST_VALUE(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS
__last_update_dts,
SHA1_BINARY(
    NVL(UPPER(TRIM(name::VARCHAR)), '^') || '|' ||
    NVL(UPPER(TRIM(location_id::VARCHAR)), '^') || '|' ||
    NVL(UPPER(TRIM(phone::VARCHAR)), '^') || '|' ||
    NVL(UPPER(TRIM(address::VARCHAR)), '^') || '|' ||
    NVL(UPPER(TRIM(account_balance_usd::VARCHAR)), '^') || '|' ||
    NVL(UPPER(TRIM(market_segment::VARCHAR)), '^') || '|' ||
    NVL(UPPER(TRIM(comment::VARCHAR)), '^')
)::BINARY(20) AS __t2diff_hash,
IFNULL(
    LAG(__t2diff_hash) OVER (PARTITION BY customer_id ORDER BY __ldts
    ASC),
    SHA1_BINARY('*null*')
) AS __prev_t2diff_hash
Now, we add a filter condition that avoids unchanged records. This
condition uses the QUALIFY clause because it relies on the result of a LAG
function.
FROM src_customer
WHERE TRUE
-- do not include records without changes
QUALIFY __t2diff_hash != __prev_t2diff_hash
;
```

This gives us the completed script for the dynamic table:



Figure 14.20: The breakdown of dynamic Type 2 SCD logic

Now, let's extend this to include Type 1 columns.

Creating a Type 2.1 dimension

For most use cases, a pure Type 2 dimension is sufficient. However, sometimes the high change volume of certain columns (e.g., `last_login`, `account_balance`, etc.) can lead to an overwhelming number of change records. This can complicate meaningful analytics and negatively impact performance.

To solve this, we can use a variant of the Type 6 dimension that only uses Type 2 and Type 1 columns.

To accomplish this, we need to do three things:

- Remove Type 1 columns from the hash diff (since they are no longer change relevant).
- Get the latest value (e.g., `LAST_VALUE`) using a window function for these columns.
- Adjust the sequential system columns, such as to-date and record version, to ignore Type 1 changes.

In the following example, I have modified the script from the previous exercise to change the address, location_id, and phone columns from Type 2 to Type 1:

```
CREATE OR REPLACE DYNAMIC TABLE dyn_customer_t2n1
TARGET_LAG = '10 MINUTE'
WAREHOUSE = demo_wh
AS
SELECT *,
-- system column window functions now need to iterate over the final result set, not the initial select
IFF(
  LEAD(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) IS NULL,
  '9999-12-31':TIMESTAMP_NTZ,
  DATEADD(NANOSECOND, -1, LEAD(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC))
) AS __to_dts,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS __rec_version,
FIRST_VALUE(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS __create_dts,
LAST_VALUE(__ldts) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS __last_update_dts,
IFF(__to_dts = '9999-12-31':TIMESTAMP_NTZ, TRUE, FALSE) AS __is_latest
FROM (
  SELECT
    customer_id || '|' || __ldts AS dim_key,
    customer_id AS customer_id,
    name AS name,
    -- TYPE 1 FIELDS
    LAST_VALUE(address) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS address,
    LAST_VALUE(location_id) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS location_id,
    LAST_VALUE(phone) OVER (PARTITION BY customer_id ORDER BY __ldts ASC) AS phone,
    -- END TYPE 1 FIELDS
    account_balance_usd AS account_balance_usd,
    market_segment AS market_segment,
    comment AS comment,
    __ldts AS __ldts,
    __ldts AS __update_dts,
    SHA1_BINARY(
      NVL(UPPER(TRIM(name::VARCHAR)), '^') || '|' ||
      NVL(UPPER(TRIM(account_balance_usd::VARCHAR)), '^') || '|' ||
      NVL(UPPER(TRIM(market_segment::VARCHAR)), '^') || '|' ||
      NVL(UPPER(TRIM(comment::VARCHAR)), '^')
    )::BINARY(20) AS __t2diff_hash,
    IFNULL(
      LAG(__t2diff_hash) OVER (PARTITION BY customer_id ORDER BY __ldts ASC),
      SHA1_BINARY('null')
    ) AS __prev_t2diff_hash,
    SHA1_BINARY(
      NVL(UPPER(TRIM(location_id::VARCHAR)), '^') || '|' ||
      NVL(UPPER(TRIM(phone::VARCHAR)), '^') || '|' ||
      NVL(UPPER(TRIM(address::VARCHAR)), '^')
    )::BINARY(20) AS __t1diff_hash,
    __ldts AS __from_dts
  FROM src_customer
  WHERE TRUE
  QUALIFY __t2diff_hash != __prev_t2diff_hash
)
;
```

Select over the original logic and calculate system columns there

Take the LAST_VALUE for Type 1 columns

Remember to exclude Type 1 columns from the Type 2 hash diff

Optional. Calculate the Type 1 hash diff

Figure 14.21: Extending a dynamic Type 2 SCD to a Type 2.1



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Now that you're familiar with creating Type 2 dimensions using dynamic and standard tables, the next important question to consider is their performance and associated costs.

Performance considerations

When it comes to Type 2 dimensions, dynamic tables rely on memory-intensive declarative operations. Even before associated serverless compute costs are factored in, the dynamic approach will consume more virtual warehouse resources to accomplish the same task. Using the examples in this chapter, I observed that the streams approach was upward of three times faster. Under extreme conditions, sure: 15M rows from `tpch_sf100` with 5M loaded as changes (Snowflake recommends incremental loads of 5% of total record count). However, credit consumption for loading is not the only factor that contributes to the cost of a Type 2 dimension—there's also maintenance.

The dynamic approach results in much fewer lines of code and completely handles orchestration. In exchange for a few seconds of compute on an XS warehouse, dynamic tables may remove the need for third-party ETL tools and engineering resources required to maintain them. When deciding which approach to choose, it's important to take the total operating cost into account.

As a final consideration, even if dynamic tables don't meet your needs, the declarative logic we created in this section can still be used to initialize a Type 2 SCD with full change history, so keep this approach in mind.

Let's move on to the Type 3 SCD, so reset the `SRC_CUSTOMER` table to the baseline and continue.

Type 3: One-time update

Creating a Type 3 attribute, relatively speaking, is a simple and inexpensive operation. The process involves altering the table to add an empty column and an update to set it equal to a base column. After that, keeping the Type 3 table up to date is identical to the method used in Type 1.

First, add the Type 3 column and set it to the baseline, like so:

```
ALTER TABLE dim_customer_t3 ADD COLUMN original_account_balance_usd
number(12,2);
UPDATE dim_customer_t3
SET original_account_balance_usd = account_balance_usd;
```

With every insert, set the original column equal to the base column, and avoid updating it going forward. You can see the complete process in the accompanying code.

Having completed the exercise for a Type 3 dimension, we will wrap up the demonstration. As we saw in the first half of this chapter, the remaining SCDs either combine the techniques used in Types 1, 2, and 3 or rely on straightforward modeling practices that rely on basic DML commands.

Summary

Due to the constantly changing nature of master data in the source system, the data warehouse must serve two critical functions to allow business users to pivot between current and historical attribute values in their reporting. These functions consist of capturing source system changes in a landing area and creating SCDs that meet the organization's reporting needs. Because master data plays such a key part in organizational analytics—often being tracked and scrutinized independently of fact records—learning how to construct the required SCD structures and load them efficiently is a fundamental task for any data warehouse team.

In this chapter, we reviewed nine different SCD structures for meeting various analytical needs: from durable Type 0 attributes that never change to dynamic Type 7 configurations that can handle any requirement. Although many variations exist—even within SCD types—Types 1 to 3 are the most often used as they strike an acceptable balance between maintainability, performance, and reporting requirements.

Using the recipes provided in the accompanying SQL examples, this chapter explored the best practices for constructing SCDs by leveraging Snowflake-specific features such as streams, cloning, and dynamic tables. As demonstrated, using native features such as streams can result in significant cost and performance savings compared to plain SQL methods.

Having understood dimensions, the following chapter will use the same approach to explore facts. Using best practices and Snowflake-only features, we will cover widely used techniques to keep fact tables up to date and enable change tracking and time-based rollups.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

15

Modeling Facts for Rapid Analysis

Fact tables are used to store the quantitative measurements of business operations or events such as sales, employee headcounts, or website traffic. Because they contain the official record of business transactions, fact tables are a prime target for operational analytics. Fact tables aggregate metrics such as sales totals and active users, as well as historical trends (deltas), such as the margin impact of daily returns or same-day bookings before cancellations.

Because business needs vary by industry and sector, various fact table models exist to fit these different demands. Facts such as product sales and returns are erratic, while others, such as manufacturing or fulfillment, follow a predictable pattern. The fact tables supporting such processes must anticipate not only the nature of the data they aim to capture but also the organization's analytical needs to allow for efficient reporting.

This chapter will cover the various fact table models and the industry use cases to which they are best suited. Most importantly, we will look at several Snowflake-driven patterns that can tackle some of the most common design challenges associated with fact tables in a cost-effective and low-maintenance manner.

In this chapter, we will cover the following main topics:

- Getting to know various fact table types
- Understanding the various types of measures or facts
- Reverse balance, the world's most versatile transactional fact table
- Recovering lost (physically deleted) facts
- Working with facts over points and ranges of time
- Snowflake recipes for building, maintaining, and querying each of these fact tables efficiently

Technical requirements

The scripts used to instantiate and load the examples in this chapter are available in the following GitHub repo: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch15>. While the key section of each script will be explained in the latter half of this chapter, please refer to the repo for the complete code used to maintain and schedule the loading of the objects discussed, as it is too long to reprint here.

Fact table types

By capturing the daily operational transactions of an organization, fact tables tend to contain large amounts of records that are constantly growing. By analyzing the data in fact tables, analysts and business users glean insights into business performance and identify trends and patterns. Considering these demands, fact tables must be designed in such a way that balances data loading efficiency with analytical needs and query patterns.

After nearly 20 years and three editions, the definitive guide to designing fact tables remains *The Data Warehouse Toolkit*. In it, authors Ralph Kimball and Margy Ross expertly cover the fundamentals of dimensional modeling, fact table design, and related industry case studies. This chapter will not attempt to replicate the content to which Kimball and Ross have dedicated nearly a quarter of their book. Instead, we will focus on what *The Data Warehouse Toolkit* does not cover: database-specific transformations for managing and maintaining fact tables in Snowflake in cost-efficient and analytically versatile ways. However, before jumping into Snowflake-specific recipes, an overview of fact table types and their various metrics is required.

Nearly every organization has some common analytical needs, such as obtaining operational totals through aggregations across business and temporal dimensions. However, differences in dimension types, such as time, geography, or product category, will drive design decisions that facilitate user-defined requirements such as drilling down or across or comparing historical records. To meet these needs, five basic fact table types have been identified:

- **Transaction fact tables:** These fact tables store information about individual transactions, such as sales or bookings. They typically have the most detailed level of information and the largest number of rows. Most organizations will use this type of fact table as the baseline for granular analysis, summarized aggregates, and generating the other types of fact tables in this list.

- **Snapshot fact tables:** These fact tables store information about a specific point in time, such as inventory levels or customer metrics at the end of a given period. They are usually updated on a regular basis, such as daily, weekly, or monthly. Snapshot fact tables do not contain granular information such as individual order details. Instead, snapshot tables provide predictably occurring slices of operational data, which can easily be compared due to their identical granularity. Snowflake's materialized views should be considered a maintenance-free way of maintaining snapshot tables using a transaction fact table as a source.
- **Accumulating snapshot fact tables:** These fact tables are similar to snapshot fact tables, but they track the progress of a process with predictable steps over time. They typically have a fixed set of milestones that reflect the stages of a fixed business process (e.g., the customer journey through a call center from receipt to resolution). Unlike other fact table types, in which records are only inserted and not updated (except to correct errors), accumulating fact tables contain milestone, status, and corresponding change-tracking columns, which receive updates as the underlying record changes.

	CALL_ID	OPERATOR_ID	STATUS	CALL_RECEIVED	CALL_ATTENDED	CALL_ENDED
1	8c403af3	123	call ongoing	2023-02-27 12:33:15.118 -0800	2023-02-27 12:34:37.118 -0800	null

PK FK

updating status and milestone columns

Figure 15.1: A fact in progress in an accumulating snapshot table

- **Factless fact tables:** These fact tables do not contain any measures but are used to record events or transactions between dimensions, such as student enrollments in college courses.
- **Consolidated fact tables:** When separate but related facts share the same grain and need to be analyzed together, they are consolidated into a single fact table. Sales and sales forecasts are classic examples of consolidated facts that are analyzed side by side.

The choice of which type of fact table to use depends on the nature of the data being analyzed, the specific business requirements, and the performance and storage constraints of the **data warehouse (DWH)** platform. Just like **slowly changing dimension (SCD)** types, a DWH may combine the different fact table types to support various analytical needs.

Having seen the various types of fact table structures, we should familiarize ourselves with the different categories of measures they can contain. Understanding the kinds of measures will play an important role in creating the transformational logic required for maintaining them.

Fact table measures

Numerical measures associated with a business transaction are called facts, and they fall into three basic categories:

- **Additive facts:** These are measures that can be summed across any dimension. Additive facts are the most common type of fact in a DWH, allowing for a wide variety of analytical calculations and insights. These values can be aggregated across any combination of dimensions, such as time, geography, or product. Examples of additive facts include sales revenue, profit, and quantity sold.
- **Semi-additive facts:** These are measures that can be summed across some dimensions but not all. These measures are usually numeric values that can only be aggregated across certain dimensions, such as customers or products. Examples of semi-additive facts include account balance and inventory levels, respectively. Semi-additive facts require special handling in data analysis to ensure that the aggregation is done correctly and does not spill over to non-compatible entity instances.
- **Non-additive facts:** These are measures that cannot be summed across any dimension. These measures are usually ratios, percentages, or other derived values that cannot be aggregated meaningfully. Examples of non-additive facts include the average price per unit or customer satisfaction score. Non-additive facts require special handling in data analysis and are typically used in more complex analytical calculations, such as forecasting or predictive modeling. In most cases, breaking non-additive facts into their fully additive components (e.g., the price and number of units instead of the average price per unit) is encouraged, as it gives users the flexibility to aggregate at any granularity and re-calculate the derived value.

Having understood the different types of fact tables and the measures they contain, we can look at the challenges involved in designing and maintaining fact tables, keeping in mind that where there's a challenge, there's a Snowflake feature to help overcome it.

Getting the facts straight

The facts in a source system are recorded in real time and updated in case of adjustments. By definition, they are always current. A DWH has a much harder task because it needs to capture and report current facts and track historical changes. Suppose an order was adjusted from containing two items to one. The DWH must find a way to report that a change was made while avoiding the issue of double-counting (as the total quantity is now one, not three).

The task of historical tracking is made even more complicated when the facts are not point-in-time transactions but intervals such as advertising campaigns or employee hires and leavers. In such cases, tabulating the cost of a department can no longer be accomplished by simple aggregation because employees can come and go at various intervals.

Operating a business is messy and complex, and the data that it generates is no exception. Employees come and go, orders are returned, and in some cases, records in the source system may be physically deleted – leaving no artifact for the DWH to detect and reconcile.

In the following sections, we will address these age-old reporting challenges using versatile fact table designs incorporating unique Snowflake features to minimize the costs and required maintenance.

The world's most versatile transactional fact table

When it comes to recording business transactions, capturing changes is often as important as their final states. For example, in online retail, being able to analyze the products that were removed from a shopping cart can yield valuable insight for understanding customer behavior and minimizing such actions in the future. However, tracking changes in a fact table presents a dual challenge: tying a changed record to its previous state and storing both without distorting the overall totals.

The following example shows a sales order placed on day one that suffers a change in the source system on day two and the resulting records that land in the DWH.

Source System		day 1	ORDER								
				ORDER_ID	PART_ID	QUANTITY	EXTENDED_PRICE_USD	DISCOUNT_PERCENT	RETURN_FLAG	ORDER DATE	
				123	1	2	10.99	0.01	N	2020-01-01	
				123	1	3	15.99	0.02	N	2020-01-01	
DWH landing		day 1	RAW.SRC_ORDER								
				LOAD_DATE	ORDER_ID	PART_ID	QUANTITY	EXTENDED_PRICE_USD	DISCOUNT_PERCENT	RETURN_FLAG	ORDER DATE
				2020-01-01	123	1	2	10.99	0.01	N	2020-01-01
				2020-01-02	123	1	3	15.99	0.02	N	2020-01-01

Figure 15.2: Facts in the source and DWH landing area

While the record in the source system is overwritten by the update, the DWH captures both versions. However, it is difficult to answer even simple business questions by looking at the RAW table. What is the total number of parts sold? (Three, not five.) How many parts were sold on day two? (One, not three.)

To help the business answer these (and many other) questions, we can use a fact table design known as reverse balance or mirror image. The reverse balance method yields a fact table that can answer virtually any business question using aggregation (the operation for which OLAP systems are optimized). As the name suggests, in a reverse balance structure, a change is performed by inserting not one but two records; one that contains the new values, and another that negates the original (the reverse balance). The result in our example would appear as follows, containing (from top to bottom) the original value, its reverse balance, and the updated value:

		ORDER_REVERSE_BALANCE							
		(additive measures)					(non-additive measure)		
		ASAT_DATE	IS_AFTERIMAGE	ORDER_ID	PART_ID	QUANTITY	EXTENDED_PRICE_USD	DISCOUNT_PERCENT	RETURN_FLAG
DWH reporting	day 1	2020-01-01	TRUE	123	1	2	10.99	0.01	N
		2020-01-02	FALSE	123	1	-2	-10.99	0.01	N
	day 2	2020-01-02	TRUE	123	1	3	15.99	0.02	N

Figure 15.3: A reverse balance fact table used for reporting

The reverse balance table resembles the raw table with a few key differences:

- The landing table load date becomes an as-at date, indicating when a given version of a fact is valid.
- A before/after-image indicator (IS_AFTERIMAGE, in this example) is added. The after-image flag distinguishes the reverse image (middle row) from the after-image (third row) and is therefore included in the primary key.

This approach makes it easy to answer the prior business questions:

```
--total number of parts sold
SELECT SUM(quantity) FROM lineitem_reverse_balance
-- parts sold on day two (delta)
SELECT SUM(quantity) FROM order_reverse_balance
WHERE asat_date = '2020-01-02'
```

In addition, the as-at date makes it possible to observe historical facts as they appeared at that point in time. For example, what is the number of parts sold as is on day one versus day two?

```
-- parts sold on first of Jan as at day one
SELECT SUM(quantity) FROM order_reverse_balance
WHERE order_date = '2020-01-01'
AND asat_date= '2020-01-01'
AND is_afterimage = TRUE
-- parts sold on first of Jan as at day two
```

```
SELECT SUM(quantity) FROM order_reverse_balance
WHERE order_date = '2020-01-01'
AND asat_date= '2020-01-02'
AND is_afterimage = TRUE
```

Using this method, analysts can quickly aggregate and compare totals, deltas, point-in-time snapshots, and year-end totals as they appear today or at any point in the past. Later in this chapter, we will review an efficient technique for constructing and maintaining a reverse image fact table in Snowflake. The only thing we require is the updated record to be loaded in the landing area. Unfortunately, the data quality in some source systems is less than ideal and records are physically deleted instead of being canceled or corrected. Luckily, the DWH can look back in time and recover deleted records to align them with the source.

The leading method for recovering deleted records

In a perfect world, records are never physically deleted in the source system. Instead, they should be nullified *logically* by marking them with a *deleted* flag and setting the measures to zero, but as we know, the world is not perfect, and records are occasionally removed. Deletions are not a grave problem in dimension tables because a quick aggregation from the landing area table can produce and identify the latest active and deleted entries:

```
SELECT < dimension PK >
, MAX(load_date) AS latest_load_date
, IFF(latest_load_date = current_date, false, true) AS is_deleted
FROM src_dimension
group by < dimension PK >
```

This is to say: in a full daily load, any records that don't exist but have been loaded previously must have been deleted.

Facts pose a greater challenge because they occur at a precise point in time and are typically too numerous to be loaded and compared using full loads. Instead, facts are typically loaded and processed as delta snapshots – containing only changes and updates that occurred since the previous load.

Whether using a reverse balance fact table or any alternative, the issue of orphaned records arises when the DWH is not informed of a deleted fact. In the following example, an order was placed for two parts on day one. On day two, the number of units for part one was increased to 20, but part 2 was (improperly) canceled by a physical deletion of the record.

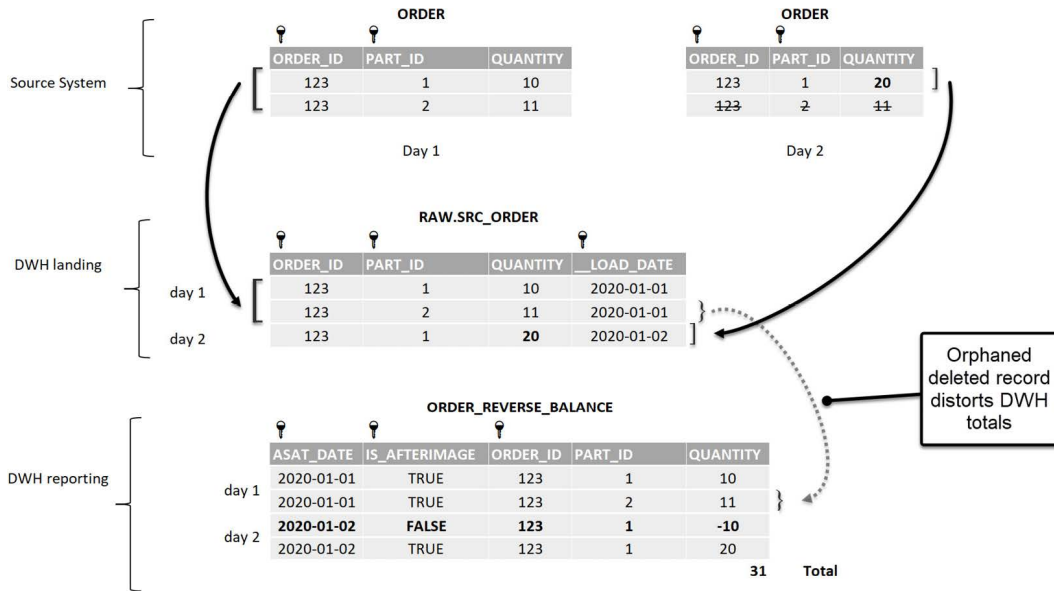


Figure 15.4: A physical deletion in the source system

Physical deletion is a quick (and tempting) way to adjust data in the source system because there, it poses no risk to data accuracy (as of day two, the system correctly reflects the order for 20 units of part 1). However, the DWH is not informed of the change and is now out of sync with the official source of truth.

Unlike dimension entities, looking backward to detect and rectify deleted records in a DWH is a resource-intensive task that usually involves aggregates, temporary tables, window functions over large datasets, and comparisons. However, by turning the process on its head and looking forward, we can drastically reduce the processing resources necessary to identify and revert deleted changes.

The basic concept of the forward-leading-insertion method is to insert (and logically delete) the physically deleted record into the next load date in the raw/landing table. Let's say that an order is created on day one containing three parts. A part is subsequently deleted for days two and three. Figure 15.5 illustrates the result of running the process on the DWH landing area table.

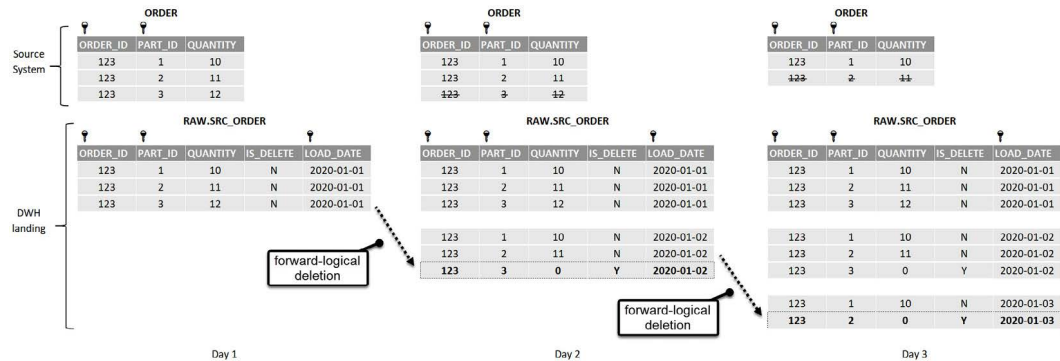


Figure 15.5: Logically recreating deleted records in the DWH landing area

The logical deletion process aims to mark the physical deletion as it happened in a point-in-time snapshot. Therefore, the logically deleted record is inserted once and not passed down to future load dates, as can be seen with PART_ID 3 on day *two*.

In the second half of this chapter, we will write the script that will allow us to perform this operation for an entire fact table or restrict it to a single load date to economize DWH credits for a daily load. However, there is one more fact table scenario that we must learn to handle first: range-based facts.

For range-based facts such as employee active periods or promo campaigns, a single as-at date is insufficient, and a mix of intervals is required. Knowing how to construct a fact table to handle such records efficiently can save developers from writing some truly grisly analytical queries down the line.

Type 2 slowly changing facts

While some business operations occur at a given moment, such as sales, others, such as contracts, endure for a length of time. Complex business data and factless fact tables often blur the line between facts and dimensions. Take, for example, the EMPLOYEE table, which may be used as a dimension when reporting store sales, but also as a fact when analyzing the **operational expenses (OPEX)** of a given department (e.g., a salary multiplied by the percentage of active service during a given period). In the latter scenario, analytics can be tricky.

For example, a start-up hires two employees at the start of 2020 – a contractor, Larry, due to leave in 4 months, and a full-time employee, Lisa. In June, as the lone active employee in the company, Lisa is given a raise. The source and DWH snapshots in January and June would look as follows:

Source System		EMPLOYEE						EMPLOYEE						
		EMPLOYEE_ID	Name	START_DATE	END_DATE	Salary		EMPLOYEE_ID	PRODUCT	START_DATE	END_DATE	Salary		
		123	Larry	2020-01-01	2020-03-31	50,000		123	Larry	2020-01-01	2020-03-31	50,000		
		456	Lisa	2020-01-01	9999-12-31	50,000		456	Lisa	2020-01-01	9999-12-31	50,000 100,000		
DWH landing		SRC_EMPLOYEE							SRC_EMPLOYEE					
		EMPLOYEE_ID	Name	START_DATE	END_DATE	Salary	LOAD_DATE		EMPLOYEE_ID	Name	START_DATE	END_DATE	Salary	LOAD_DATE
		123	Larry	2020-01-01	2020-03-31	50,000	2020-01-01		123	Larry	2020-01-01	2020-03-31	50,000	2020-01-01
		456	Lisa	2020-01-01	9999-12-31	50,000	2020-01-01		456	Lisa	2020-01-01	9999-12-31	50,000	2020-01-01
January						June								

Figure 15.6: Records with duration intervals in the source system and DWH landing area

With an eye toward tracking costs, the company's owners wish to know the exact OPEX costs of keeping the company running. They need a fact table allowing them to generate the figures as they appear in the following figure:

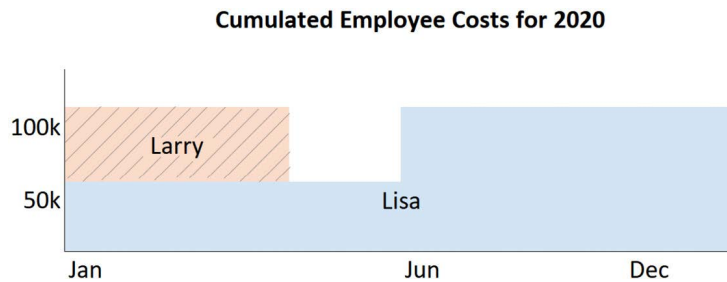


Figure 15.7: Cumulated Employee Costs for 2020

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



The solution is to use the Type 2 SCD structure covered in the previous chapter. Storing employee data in a Type 2 table allows you to query date intervals at a point in time and at any effective date or date range.

A Type 2 fact table for this example would contain the following data:

SRC_EMPLOYEE							
EMPLOYEE_ID	Name	START_DATE	END_DATE	Salary	FROM_DATE	TO_DATE	DIFF_HASH
123	Larry	2020-01-01	2020-03-31	50,000	1900-01-01	9999-12-31	sd9f87gr
456	Lisa	2020-01-01	9999-12-31	50,000	1900-01-01	2020-05-31	9sbf89g6
456	Lisa	2020-01-01	9999-12-31	100,000	2020-06-01	9999-12-31	h3ghm2vf

Figure 15.8: A Type 2 table for employee data

A surrogate low-date such as the one in the FROM_DATE column is often used, as it allows users to identify original/unmodified versions of a record. Otherwise, the creation date can be used instead. Having the data in this format allows us to write a query to answer the OPEX question without having to use window functions or compare records:

```
SELECT name
, IFF (start_date > from_date, start_date, from_date) AS query_from_date
, IFF (end_date < to_date, end_date, to_date) AS query_to_date
, salary
FROM employee
WHERE TRUE
AND start_date <= '2020-01-01'
AND end_date >= '2020-12-31'
```

This simple example uses only one time band (employee start/end date) besides the meta from_date/to_date columns. However, in the real world, entities may contain multiple date ranges and sets of additive, semi-additive, and non-additive measures. For example, a contract may have the following date attributes: service start/end, creation, modification, signing, authorization, release, renewal, review, and many others, which the business teams may wish to interrogate. The Type 2 structure, as we will see in the practice section in this chapter, allows us to easily obtain counts, aggregates, and snapshots for any date range or measure required by the business.

Having seen the approaches that need to be taken to meet some of the most frequent fact table challenges, let us dive into and explore the code that will allow us to meet these with minimal effort and WH credit usage.

Maintaining fact tables using Snowflake features

In this section, we will practice creating and maintaining the fact table techniques discussed previously using available Snowflake features. As in previous chapters, we will load data from the `snowflake_sample_data.tpch_sf10` schema, which will serve as the sample set. We will then simulate source system updates by randomly loading records from this sample.

The first two exercises will use data from the `LINEITEM` table. To continue, create a schema to house the exercises from this chapter and instantiate the source system table, as well as the DWH landing area.

Open the first file in this chapter's repository, `reverse_balance_fact.sql`, and run the first three steps:

```
CREATE OR REPLACE SCHEMA ch15_facts;
CREATE OR REPLACE TABLE source_system_lineitem ...;
CREATE OR REPLACE TABLE src_lineitem ...;
```

These examples require `lineitem` orders to be loaded and processed in their entirety (containing all constituent line items for a given order). To accomplish this, order IDs are first selected at random from the sample set, and then all related line items are loaded:

```
WITH complete_orders AS (
  SELECT DISTINCT sales_order_id
  FROM source_system_lineitem SAMPLE (< N > rows)
)
SELECT < source columns > FROM source_system_lineitem src
  INNER JOIN complete_orders co ON src.sales_order_id = co.sales_order_id
```

Let's begin by constructing the model for a reverse balance fact table and building a script to load it using a merge/insert strategy using Streams.

Building a reverse balance fact table with Streams

While querying a reverse balance fact table is simple, traditional databases have often run into performance constraints when attempting to maintain one. This is due to the difficulty of isolating the previous version of a change (to create the reverse balance). This operation typically requires temporary tables and window functions to lag and compare the same business key across various load dates. However, using Snowflake streams, we can generate the reverse balance by simply negating the additive measures in the before-image of an updated record – a massive performance improvement over the traditional method. This is a similar strategy to the one used in the previous chapter for Type 2 SCDs.

Let's continue with the exercise from the file (`reverse_balance_fact.sql`) we started with.

After cloning `src_lineitem` as a backup for re-running the exercise, let's create the structure of the reverse balance table and populate it with an initial dataset (before any changes occur):

```
CREATE OR REPLACE TABLE lineitem_rb ...;
```

Note the characteristic columns that this type of table requires:

- **Load type:** An optional but useful metadata column that identifies the type of change in a record. The values in this column are user-defined to fit the nature of the data and the types of changes that occur. Our example uses values of *initial* and *update* to distinguish between newly created and updated records. The exercise on recovering deleted records will use this field to mark *deletion* records.
- **As-at date:** Although its value is equivalent to the load date, the term as-at paints a clear picture of the column's function – identifying the state of a fact at any point in time. Although facts are typically loaded daily, requiring a date type for this column, our example is meant to be run multiple times per day, so a timestamp is used instead.
- **Before/after-image flag:** This field is used to split out an update into the new version and its reverse balance. As such, it must be included in the primary key of the table (in addition to the as-at date and the business key).

With the reverse balance table created, create a stream to handle the upcoming changes:

```
CREATE OR replace STREAM strm_lineitem_rb ON TABLE lineitem_rb;
```

Now, simulate a load from the source system, which will include new records, as well as updates to ones already loaded:

```
INSERT INTO src_lineitem ...;
```

Now, merge the newly loaded records into the reverse balance table using its primary key. Our example assumes only new and updated records and can't contain existing line items with no changes. If this were the case, you would use `diff_hash` to determine whether an existing record does indeed constitute a change:

```
MERGE INTO lineitem_rb rb ...;
```

The result of the merge will insert new orders and update existing orders, and it is the update that concerns us. Because we overwrote the original order values, we now have to insert the previous version, and its reverse balance. The process is illustrated in the following diagram:

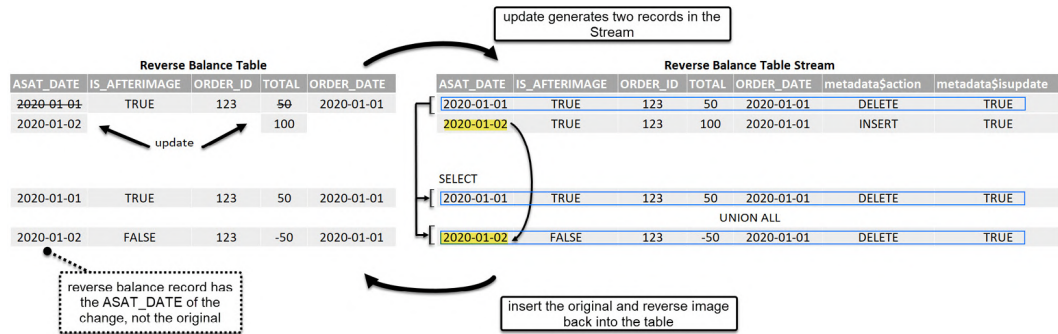


Figure 15.9: A reverse balance is created from a stream update action

As we need to generate two records (the original and reverse-balanced one) from a single record in the stream, we can use a CTE to select the row once and insert it twice using `UNION ALL`. As demonstrated in Figure 15.9, the original record is inserted as it appears in the stream, while the following modifications must be made to the reverse image:

- The as-at date is that of the latest record, not the original
- The before/after-image flag is marked as before
- Additive and semi-additive measures are multiplied by -1

When you run the insert from the stream (`INSERT INTO lineitem_rb ... ;`), observe where these operations take place:

```
INSERT INTO lineitem_rb
  WITH before_records AS (
    SELECT *, asat_after FROM strm_lineitem_rb
    WHERE metadata$action = 'DELETE'
  )
  --insert the original after image that we updated
  --no changes required to column values
  SELECT < stream fields > FROM before_records
  UNION ALL
  --insert the before image as at after dts, but negate additive measures
  SELECT < stream fields >
    , asat_after as ASAT_DTS --USE the asat OF the AFTER image
    , FALSE as IS_AFTERIMAGE --because this IS the BEFORE image
    , -1 * quantity
    , -1 * extended_price_usd
```

```
, discount_percent --do NOT negate, non-additive measure
FROM before_records;
```

Now that the reverse balance table has been fully updated, select an *update* record to verify and inspect the result. A sample lineitem update should generate three records, as in the following screenshot:

```
347 SELECT * FROM lineitem_rb
348 WHERE TRUE
349 AND line_number = 53491302
350 ;
```

	_LOAD_TYPE	ASAT_DTS	IS_AFTERIMAGE	LINE_NUMBER	SALES_ORDER_ID	PART_ID	SUPPLIER_ID	QUANTITY
1	initial	2023-02-26 12:39:44.086	TRUE	53,491,302	1573710	73711	5	36
2	initial	2023-02-26 12:41:37.881	FALSE	53,491,302	1573710	73711	5	-36
3	update	2023-02-26 12:41:37.881	TRUE	53,491,302	1573710	73711	5	41

This is the reverse balance row

Figure 15.10: An update to a reverse balance record results in two new records

The reverse balance fact table is a powerful tool because it allows us to query the state of truth as it appeared on any date. For example, in sales or reservations, a registered transaction may still be canceled in the future, but the as-at date gives us a simple filter with which to travel back in time to observe historical records as they appear today, or at any point in the past. In the following example, we can query last year's bookings as they appear today (with cancelations) and as they appeared last year (before this year's cancelations):

```
--Last year bookings with future cancelations
SELECT sum(sales)
FROM bookings
WHERE booking_date = <LAST_YEAR>
AND asat_date <= <TODAY> --implicit
--Last year bookings without future cancelations
SELECT sum(sales)
FROM reservations
WHERE booking_date = <LAST_YEAR>
AND asat_date <= <LAST_YEAR>
```

Now that we have a working fact table, we will demonstrate how to detect and recover physically deleted records to set the correct balance in the landing area, so that the fact table perceives the event.

Recovering deleted records with leading load dates

This exercise will build on the objects created in the reverse balance example. Open the `deleted_records.sql` file from the repository to get started.

First, use the backups from the previous example to reset the source objects to their initial state:

```
CREATE OR REPLACE TABLE src_lineitem CLONE src_lineitem_bak;
CREATE OR REPLACE TABLE lineitem_rb CLONE lineitem_rb_bak;
CREATE OR REPLACE STREAM strm_lineitem_rb ON TABLE lineitem_rb;
```

Now, we will simulate order updates that contain deleted records by filtering out line numbers ending in any number less than three. Run `INSERT INTO` for the source table:

```
INSERT INTO src_lineitem ...;
```

Now comes the task of detecting and recovering the deletions. Although this can be done as a single operation, this example has split the process out using temporary tables to allow for testing and debugging.

The first step is taking the orders in the current load and building a structure that contains the primary key of the parent object (`sales_order_id`) and the corresponding load dates and using the `LEAD` window function to obtain the next parent object load date. Build the temporary table using the following command and observe the result:

```
CREATE OR REPLACE TEMPORARY TABLE line_load_hist ...;
```

This generates a structure of today's order IDs with corresponding and leading load dates:

Today's orders		Corresponding load date	LEAD to get the next load date
	SALES_ORDER_ID	__LDTS	NEXT_ORDER_LOAD_DT
1	1958222	2023-02-26 12:39:44.086	2023-02-27 07:02:33.630
2	130962	2023-02-26 12:39:44.086	2023-02-27 07:02:33.630
3	154079	2023-02-26 12:39:44.086	2023-02-27 07:02:33.630
4	623246	2023-02-26 12:39:44.086	2023-02-27 07:02:33.630
5	1182475	2023-02-26 12:39:44.086	2023-02-27 07:02:33.630

Figure 15.11: A temporary structure of parent primary keys and corresponding load dates

Knowing the current and leading load dates for the order IDs, we can now do the same for the associated line numbers. Run the temporary `line_deletions` table by running the following command:

```
CREATE OR REPLACE TEMPORARY TABLE line_deletions ...;
```

By inner-joining the line number with the previously created list of order IDs and load dates, we are able to dramatically reduce the number of records that need to be processed using the LEAD window function. Having done so, deleted line items can be identified because they do not have a leading load date when we look forward. A sample deleted record would look like this:

	NEXT_ORDER_LOAD_DT	NEXT_LINE_LOAD_DT	...	__LDTS	SALES_ORDER_ID	LINE_NUMBER	__LOAD_TYPE
1	2023-02-27 07:02:33.630	null		2023-02-26 12:39:44.086	1958222	3,676,900	initial

If it existed previously, but not currently, it is a deleted record

Figure 15.12: A temporary structure of full primary keys and corresponding load dates

Now that we have identified the deleted records and when they were last loaded, we can insert them from the source table, zeroing out the additive measures. Run the following statement to *logically* delete the missing records:

```
INSERT INTO src_lineitem...;
```

Observe that during this process, we zero out the additive measures and use the LEAD load date that we obtained for the missing record. Also, it is essential to create a flag that identifies a logically deleted record for the following reasons:

- We need to use it to avoid re-inserting logically deleted records in the future (as they won't exist in any subsequent load either)
- We need to have a way to reconcile with the source system if asked why there are more records in the DWH

This is all handled during INSERT, as seen here:

```
WITH deleted AS (
  SELECT del.next_order_load_dt, tgt.* FROM src_lineitem tgt
  INNER JOIN line_deletions del
  USING (sales_order_id, line_number, __ldts)
  WHERE TRUE
  AND del.__load_type != 'deletion'
)
--"logically" delete the record by inserting it with 0-value measures
--remember to treat non-additive measures as attributes
SELECT
  line_number
  , sales_order_id
  ...
  , 0 --quantity
  , 0 --extended_price_usd
  , discount_percent
  , tax_percent
  ...
  , next_order_load_dt --the load date when the deletion happened
  , 'deletion'
FROM deleted
```

Figure 15.13: Inserting a logically deleted record

Now that the missing records have been re-inserted as logical deletions, we can observe the impact on the fact table by loading it as in the previous example. Note that although this example used a reverse balance fact table, logical deletions must be performed in the landing area to avoid distorting the totals regardless of the type of fact table used. Load the fact table by running the two commands discussed previously:

```
MERGE INTO lineitem_rb rb...;
INSERT INTO lineitem_rb...;
```

The fact table has now been updated to allow us to aggregate as-at totals correctly. A sample deletion would generate two records to net out the original as seen here:

	__LOAD_TYPE	ASAT_DTS	IS_AFTERIMAGE	LINE_NUMBER	SALES_ORDER_ID	PART_ID	QUANTITY	DISCOUNT_PERCENT
1	initial	2023-02-26 12:39:44.086	TRUE	3,676,900	1958222	8261	6	0.05
2	deletion	2023-02-27 07:02:33.630	TRUE	3,676,900	1958222	8261	0	0.05
3	initial	2023-02-27 07:02:33.630	FALSE	3,676,900	1958222	8261	-6	0.05

Figure 15.14: A logically deleted record in the fact table



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Adjust the example parameters to repeat the exercise to see how new records and deletions on top of already deleted line items impact the final fact table.

Having covered point-in-time fact tables and how to treat possible deletions, we will move on to an example that teaches us how to maintain and interrogate interval-based facts.

Handling time intervals in a Type 2 fact table

In this exercise, we will experiment with employee data that simulates the daily operations of an active business: hiring/terminating employees, adjusting salaries, and balancing headcounts between fixed-term contractors and full-time employees. Open the `type2_facts.sql` file from the chapter repo to get started.

As the sample data contains records from within the range 1992-1998, we will use a variable to set *today's* date to `1995-12-01` to allow us to load the historical data for *existing* employees, and also give us plenty of sample data for *future* changes:

```
SET today = '1995-12-01';
```

Now, instantiate the sample dataset for the simulated source system and the initial DWH landing area:

```
CREATE OR REPLACE TABLE source_system_employee...;
CREATE OR REPLACE TABLE src_employee...;
```


In creating the initial tables, we can use the today variable, just as we would use the current_date() function in a real-world scenario.

Next, we create the Type 2 fact table using the method described in the previous chapter:

```
CREATE OR REPLACE TABLE employee_t2 ...;
```

Note that we will use the surrogate low/high dates of 1900-01-01 and 9999-12-31 for the meta from_date/to_date fields, respectively.

With the Type 2 table created, we will increment the today variable by one and start to insert changes into the landing area: new hires, terminations, and promotions for some existing employees:

```
SET today = $today::date+1;  
INSERT INTO src_employee...
```

Unlike the reverse balance fact table, a Type 2 fact table does not discriminate between additive and non-additive measures – it updates all changes accordingly and creates a new effectively dated record.

Update the fact table with the latest records:

```
MERGE INTO employee_t2 tgt...;  
INSERT INTO employee_t2...
```

Now, increment the today variable and simulate several more days of changes to generate some meaningful data to query. Once several load dates have been added, we can begin to ask business questions about our organization's headcounts.

The surrogate high date will always return the latest version of a fact, so we can use it as a filter when asking questions about the current state of the company, such as *how many employees are currently active?*

```
--currently active employees  
SELECT COUNT(*) cnt FROM employee_t2  
  WHERE TRUE  
  AND is_active  
  AND to_date = '9999-12-31'; --currently
```

What about at a point in the past? Here, we must consider a range of dates: records with an effective date valid before the target date, and after. For example, a contractor with a start-to-end range of 1994-12-01 to 1995-12-31 should match the target date that we're interested in (1995-12-01):

```
--active employees on day 1995-12-01
SELECT COUNT(DISTINCT employee_id) cnt
  FROM employee_t2
 WHERE TRUE
    AND is_active
    AND from_date <= '1995-12-01'
    AND to_date >= '1995-12-01' ;
```

Instead of a point in time, the query can use a range as well – for example, to calculate (without double-counting changes) the number of active employees for the entire year of 1995:

```
--active employees in all of 1995
SELECT COUNT(DISTINCT employee_id) cnt FROM employee_t2
 WHERE TRUE
    AND is_active
    AND YEAR(from_date) <= 1995
    AND YEAR(to_date) >= 1995 ;
```

Using various time criteria, we can mix current and historical values to ask targeted questions such as *who was hired in Q1 1994 last year and was still active on the date of 1995-12-01?*

```
--active employees on day 1995-12-01
--who were hired in Q1 of 1994
SELECT COUNT(DISTINCT employee_id) cnt
  FROM employee_t2
 WHERE TRUE
    AND is_active
    AND hire_date BETWEEN '1994-01-01' AND '1994-03-31'
    AND from_date <= '1995-12-01'
    AND to_date >= '1995-12-01';
```

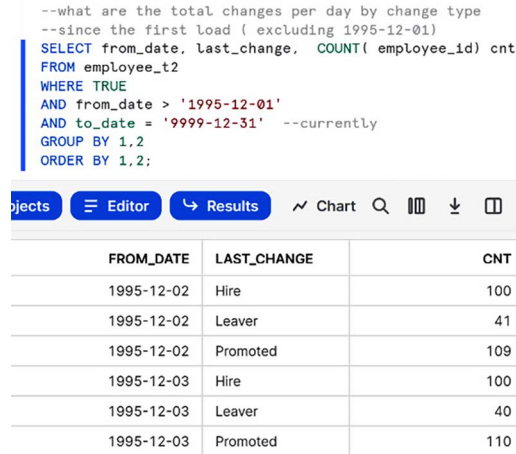
Yes, but now, only show me those who received a promotion. We can query this table, adding ever more complex criteria:

```
--active employees on day 1995-12-01
--who were hired in Q1 of 1994
--and received a promotion
WITH promotions AS (
    SELECT DISTINCT employee_id FROM employee_t2
    WHERE TRUE
    AND last_change = 'Promoted' )
SELECT COUNT(DISTINCT employee_id) cnt
FROM employee_t2
INNER JOIN promotions USING (employee_id)
WHERE TRUE
AND is_active
AND hire_date BETWEEN '1994-01-01' AND '1994-03-31'
AND from_date <= '1995-12-01'
AND to_date >= '1995-12-01';
```

Whether it's capturing distinct groupings by date – as seen in the following example – or aggregating totals over a range of dates, the Type 2 fact table can handle any range-based query that users throw at it:

```
--what are the total changes per day by change type
--since the first load ( excluding 1995-12-01)
SELECT from_date, last_change, COUNT( employee_id) cnt
FROM employee_t2
WHERE TRUE
AND from_date > '1995-12-01'
AND to_date = '9999-12-31' --currently
GROUP BY 1,2
ORDER BY 1,2;
```

The following screenshot shows the result of daily headcount movements since we started the exercise:



```
--what are the total changes per day by change type
--since the first load ( excluding 1995-12-01)
SELECT from_date, last_change, COUNT( employee_id) cnt
FROM employee_t2
WHERE TRUE
AND from_date > '1995-12-01'
AND to_date = '9999-12-31' --currently
GROUP BY 1,2
ORDER BY 1,2;
```

FROM_DATE	LAST_CHANGE	CNT
1995-12-02	Hire	100
1995-12-02	Leaver	41
1995-12-02	Promoted	109
1995-12-03	Hire	100
1995-12-03	Leaver	40
1995-12-03	Promoted	110

Figure 15.15: Querying a range of dates from a Type 2 fact table

When facts pivot around several different dates and date ranges, a Type 2 configuration is the ideal table structure to take advantage of the performance of aggregate functions for a columnar database such as Snowflake.

Summary

In a DWH, fact tables present an additional challenge on top of merely capturing the latest values – they must also be able to capture and reconcile historical changes in a way that allows users to flexibly and cost-effectively query them to resolve business questions, because when it comes to operational analytics, analyzing changes, variations, and what didn't happen can be just as valuable as the current state of truth.

Various types of fact tables exist to help an organization meet these demanding analytical needs, such as transactional, snapshot, and accumulating snapshot fact tables, among others. These fact tables must differentiate between the various kinds of measures they store (e.g., additive, semi-additive, and non-additive) because each is treated differently when updating or recording changes.

To help data teams construct and maintain these tables in Snowflake, this chapter dissected some of the toughest challenges in maintaining fact tables in a DWH and came up with cost-effective recipes for learning to tackle the challenges using Snowflake-native features. Even if you don't encounter these exact scenarios in your work, the concepts and features demonstrated in these examples will surely provide a sound first-principles framework that can be applied to similar situations.

Having tackled the traditional elements of relational databases, such as facts and dimensions, we will focus on semi-structured data in the next chapter, to see how elegantly Snowflake allows users to handle it with table-like performance.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

16

Modeling Semi-Structured Data

So far, this book has focused on modeling structured data, the kind used in relational databases since the early 70s. However, with the rise of the internet, a different style of data became prevalent: semi-structured. Semi-structured data, such as website traffic and social media feeds, contain some organizational structure but do not conform to the formal structure of a relational database.

New file formats also emerged to support this new type of data, starting with the advent of **Ex-tensible Markup Language (XML)** in the early 2000s, followed by **JavaScript Object Notation (JSON)**, and, with the rise of distributed computing, formats such as Avro, ORC, and Parquet. These formats offered a lightweight and flexible way to structure data, making them ideal for web-based and mobile app data.

The popularity of semi-structured data can be attributed to its flexibility, adaptability, and ability to handle data sources that do not fit neatly into traditional relational databases. As a modern cloud data platform, Snowflake is capable of natively ingesting semi-structured data, storing it efficiently, and accessing it using simple extensions to standard SQL.

However, to consumers of semi-structured data, the challenge of wrangling nested entities and establishing meaningful relationships remains. This chapter will explore the tools that Snowflake offers to make working with semi-structured data easy. Using these tools, we will learn how to analyze and deconstruct semi-structured data into a relational model that BI tools and traditional analytics can consume.

In this chapter, we will cover the following main topics:

- The benefits of semi-structured data
- How Snowflake makes working with semi-structured data easy
- The flexibility of schema-on-read
- Techniques for flattening semi-structured data into rows
- The benefits of transforming semi-structured data into a relational schema
- A rule-based method for transforming semi-structured data into relational data

Technical requirements

The scripts used to instantiate and load the examples in this chapter are available in the following GitHub repo: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch16>. While key sections of this script will be highlighted in this chapter, please refer to the `semistruct.sql` file for the complete code required to load, query, and transform semi-structured data, as it is too long to reprint here in full.

The benefits of semi-structured data in Snowflake

Semi-structured data formats are popular due to their flexibility when working with dynamically varying information. Unlike relational schemas, in which a precise entity structure must be known and fixed ahead of time, semi-structured data is free to include or omit attributes as needed, as long as they are properly nested within corresponding parent objects.

Think of the contact list on your phone. It contains a list of people and their contact details but does not capture those details uniformly. For example, some contacts may contain multiple phone numbers while others have one. Some entries contain information such as an email address and street address, while others have only a number and a vague description in lieu of a name (seriously, who is *Green Vespa Laura Friend*, and who is Laura?).

To handle this type of data, Snowflake uses the `VARIANT` data type, which allows semi-structured data to be stored as a column in a relational table. As with all column types, Snowflake optimizes how `VARIANT` data is stored internally, ensuring better compression and faster access. Not only can semi-structured data sit next to relational data in the same table but users can also access it using basic extensions to standard SQL and achieve similar performance.

Another compelling reason to use the `VARIANT` data type for semi-structured data is its adaptability to change. If columns are added or removed from semi-structured data, there is no need to modify **extract, load, and transform (ELT)** pipelines. The `VARIANT` data type does not care whether the schema changes – even read operations won't fail for an attribute that no longer exists.

Let's load some semi-structured data to see these features in action.

Getting hands-on with semi-structured data

Although we will query semi-structured JSON data as part of this exercise, its storage still conforms to modeling best practices such as naming and standard columns. In this example, we will use semi-structured data containing information about pirates – such as details about the crew, weapons, and their ship – all stored in a single `VARIANT` data type. With relational data, a row represents a single entity; in semi-structured data, a row is an entire file (although the file itself can contain a single or countless entities). For this reason, metadata columns to mark individual loads and source filenames are stored alongside `VARIANT`.

pirate_json			
__load_id	number	IDENTITY	PK
__load_name	varchar		
__load_dts	timestamp_ntz		
v	variant		

```

CREATE OR REPLACE TABLE pirate_json
(
  __load_id  number NOT NULL AUTOINCREMENT START 1 INCREMENT 1,
  __load_name varchar NOT NULL,
  __load_dts timestamp_ntz NOT NULL,
  v          variant NOT NULL,
)
CONSTRAINT pirate_json__load_id PRIMARY KEY ( __load_id )
COMMENT = 'table w. a variant for pirate data, with meta ELT fields'

```

Figure 16.1: A table with ELT meta columns and `VARIANT` for storing semi-structured data

This example uses `AUTOINCREMENT` (a.k.a. `IDENTITY`) as the default to generate a sequential unique ID for each load/record inserted.

In a real-world scenario, semi-structured data would be loaded into external stages and then into Snowflake tables using `Snowpipe` (streaming) or `COPY INTO` (bulk).

`Snowpipe` instructions can be found on the Snowflake website: <https://docs.snowflake.com/en/user-guide/data-load-snowpipe-intro>.

However, in this example, we will learn how to create JSON data on the fly and work with that instead. But first, an overview of semi-structured data in a JSON file.

JSON files contain nested objects, which consist of name-value pairs (a.k.a key-value pairs) and have the following properties:

- Data is stored in name-value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays
- Object and array values may also be (nested) objects and arrays
- Values (including object and array values) can be one of the following data types:
- An object
- An array
- A Boolean
- A string
- A number
- NULL

The following figure shows a simple example of data in JSON format:

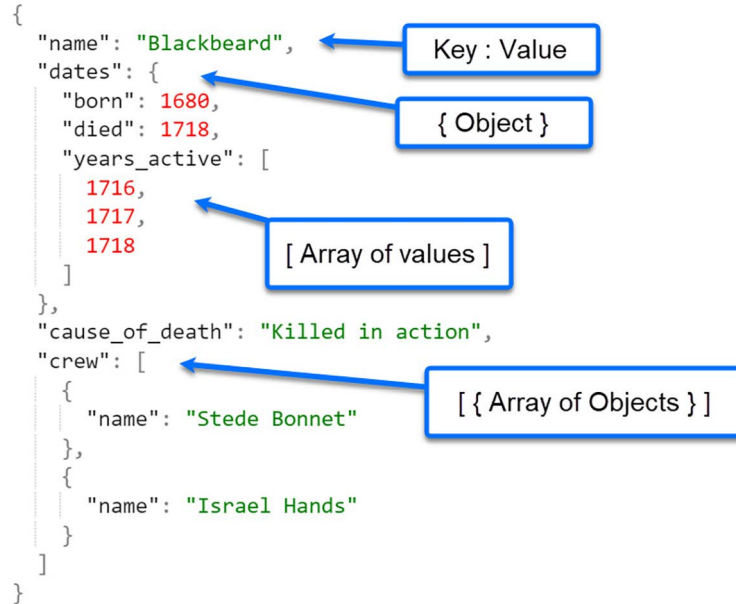


Figure 16.2: Elements in semi-structured JSON data

For this exercise, we will use a slightly more advanced example:

```
{
  "name": "Edward Teach",
  "nickname": "Blackbeard",
  "years_active": [
    1716,
    1717,
    1718
  ],
  "born": 1680,
  "died": 1718,
  "cause_of_death": "Killed in action",
  "crew": [
    {
      "name": "Stede Bonnet",
      "nickname": "Gentleman pirate",
      "weapons": [
        "blunderbuss"
      ],
      "years_active": [
        1717,
        1718
      ]
    },
    {
      "name": "Israel Hands",
      "nickname": null,
      "had_bird": true,
      "weapons": [
        "flintlock pistol",
        "cutlass",
        "boarding axe"
      ],
      "years_active": [
        1716,
        1717,
        1718
      ]
    }
  ],
  "ship": {
    "name": "Queen Anne's Revenge",
    "type": "Frigate",
    "original_name": "La Concorde",
    "year_captured": 1717
  }
}
```

Figure 16.3: JSON data for this exercise

Load this data into the previously created table using the code provided:

```
INSERT INTO pirate_json...;
```

Now, we are ready to perform some analysis.

Schema-on-read != schema-no-need

With the rising popularity of semi-structured data, schema-on-read also entered the lexicon of big data. **Schema-on-read** is the idea that, unlike in relational modeling, the schema definition for semi-structured data can be delayed until long after the data has been loaded into the data platform. Delaying this task means there are no bottlenecks within the ETL process for generating and ingesting semi-structured data. However, implicit in the design is that a knowable schema exists underneath the flexible semi-structured form.

In this section, we will learn how to query JSON data and infer details about its contents using SQL and Snowflake-native functions. Let's begin by extracting some basic attributes for our pirate:

```
SELECT * FROM pirate_json;
```

Although we can query a table containing semi-structured data in a VARIANT column, a simple SELECT * statement does not return meaningful results, as you can see in the following figure:

	_LOAD_ID	_LOAD_NAME	_LOAD_DTS	V	...
1	1	ad-hoc load	2023-03-05 03:51:15.049	{ "born": 1680, "cause_of_death": "Killed in action", "crew": { { "name": "St	

Figure 16.4: The result of standard SQL operators on semi-structured data

To access information stored inside VARIANT, Snowflake uses the colon operator (a period would have been ideal, but SQL claimed that). Using this method, let's find out what dreaded pirate we are dealing with. Recall that v is the name of the variant column in the pirate_json table:

```
SELECT v:name AS pirate_name_json
, v:name::STRING AS pirate_name_string
, v:nickname::STRING AS pirate_name_string
FROM pirate_json;
-----result-----
PIRATE_NAME_JSON|PIRATE_NAME_STRING|PIRATE_NAME_STRING|
-----+-----+-----+
"Edward Teach" |Edward Teach      |Blackbeard      |
```

Because Snowflake returns JSON data in quotes and often contains similar attribute names for nested objects, casting and aliasing the results is encouraged for readability. Now, let's move down a level and query a sub-column using a familiar dot notation:

```

SELECT v:name::STRING AS pirate_name
, v:ship.name::STRING AS ship_name
FROM pirate_json;
-----result-----
PIRATE_NAME |SHIP_NAME      |
-----+-----+
Edward Teach|Queen Anne's Revenge|

```

Remember the claim that semi-structured data is flexible enough to handle new and deleted attributes? Let's see what happens when we query a column that does not exist (spoiler alert, there's no error):

```

SELECT v:name::STRING AS pirate_name
, v:loc_buried_treasure::STRING AS pirate_treasure_location
FROM pirate_json;
-----result-----
PIRATE_NAME |PIRATE_TREASURE_LOCATION|
-----+-----+
Edward Teach|                          |

```

Now, let's learn how to interrogate an array. As in other programming languages, arrays in Snowflake represent an indexed set of objects. Individual elements can be selected using familiar square-bracket notation, and common functions such as `ARRAY_SIZE/CONTAINS/ADD` are available. Let's get some information regarding Blackbeard's reign of terror:

```

SELECT v:name::STRING AS pirate_name
, v:years_active AS years_active
, v:years_active[0] AS active_from
, v:years_active[ARRAY_SIZE(v:years_active)-1] AS active_to
FROM pirate_json;
-----result-----
PIRATE_NAME |YEARS_ACTIVE      |ACTIVE_FROM|ACTIVE_TO|
-----+-----+-----+-----+
Edward Teach|[1716, 1717, 1718]|1716      |1718     |

```

Now, let's learn how to dynamically handle multiple repeating values. Although arrays can be queried directly, their nested values remain in `VARIANT` format and are not treated as rows. Let's try and get to know two of Blackbeard's crew members:

```
--query multiple elements
SELECT v:name::STRING AS pirate_name
, v:crew::VARIANT AS pirate_crew
FROM pirate_json;
-----result-----
PIRATE_NAME |PIRATE_CREW
-----+-----
Edward Teach|[ { "name": "Stede Bonnet", .....
```

Unfortunately, the result is not user-readable. If we would like to pivot the elements of an array into intelligible rows, we must use two Snowflake features in conjunction: lateral and flatten.

- **Lateral join:** A LATERAL join behaves similarly to a loop in a correlated subquery and can reference columns from a table expression:

```
SELECT ...
FROM <left_hand_table_expression>, LATERAL (<inline_view>)
```

- **Flatten:** This is a table function that takes a VARIANT, OBJECT, or ARRAY column and produces a lateral view (i.e., an inline view that contains correlation referring to other tables that precede it in the FROM clause). More information on FLATTEN and its parameters can be found in the Snowflake documentation (<https://docs.snowflake.com/en/sql-reference/functions/flatten>). Please review this documentation and familiarize yourself with its list of output columns before continuing.

With these tools, let's transform those crew members into individual rows:

```
SELECT v:name::STRING AS pirate_name
, c.VALUE:name::STRING AS crew_name
, c.VALUE:nickname::STRING AS crew_nickname
FROM pirate_json, LATERAL FLATTEN(v:crew) c;
-----result-----
PIRATE_NAME |CREW_NAME |CREW_NICKNAME |
-----+-----+-----+
Edward Teach|Stede Bonnet|Gentleman pirate|
Edward Teach|Israel Hands|
```

In this example, c is the output of the FLATTEN function, which is joined to the columns selected from the table. We use the dot operator to reference its VALUE property, which contains a VARIANT data type (where a colon is used to extract contents). (Note that NULL values are returned as empty strings.)

Using the same technique, we can handle multiple nested arrays. For example, what weapons did each of Blackbeard's crew mates employ?

```
SELECT v:name::STRING AS pirate_name
, c.VALUE:name::STRING AS crew_name
, w.VALUE::STRING AS crew_weapons
FROM pirate_json, LATERAL FLATTEN(v:crew) c
, LATERAL FLATTEN(c.VALUE:weapons) w;
-----result-----
PIRATE_NAME | CREW_NAME   | CREW_WEAPONS |
-----+-----+-----+
Edward Teach|Stede Bonnet|blunderbuss   |
Edward Teach|Israel Hands|flintlock pistol|
Edward Teach|Israel Hands|cutlass       |
Edward Teach|Israel Hands|boarding axe  |
```

Remember that working with information inferred from a semi-structured schema is like running standard SQL on relational tables. For example, if we wish to know how many different weapons Israel Hands employed, we can turn to familiar SQL filters and aggregates:

```
SELECT COUNT(crew_weapons) AS num_weapons FROM (
SELECT c.VALUE:name::STRING AS crew_name
, w.VALUE::STRING AS crew_weapons
FROM pirate_json, LATERAL FLATTEN(v:crew) c
, LATERAL FLATTEN(c.VALUE:weapons) w
WHERE crew_name = 'Israel Hands' );
-----result-----
NUM_WEAPONS |
-----+
3 |
```

Mind your Hands



If the name Israel Hands sounds familiar, it is the basis for the treacherous sidekick in Robert Louis Stevenson's novel *Treasure Island*, but Hands was indeed part of Blackbeard's crew and is remembered for having been shot by Teach during a game of cards, who then remarked that "if [he] did not know and then kill one of them, they would forget who [he] was." Interestingly, the injury saved Hands from the gallows, as he was convalescent on shore when Blackbeard was captured. He died a few years later, a beggar on the streets of London.

As these examples demonstrate, Snowflake makes schema-on-read easy thanks to simple SQL extensions and the VARIANT data type, which is optimized for semi-structured data. However, inferring a schema and transforming semi-structured data into cleanly formatted rows and columns takes time. In more advanced cases, inferring schema-on-read requires input and guidance from the business users who specialize in it – just as in logical and conceptual modeling.

Remember that schema-on-read only delays the work of relational modeling, which must be done eventually to allow the organization to query it through traditional methods. The next section will examine a simple method for converting semi-structured data into a structured, relational schema.

Converting semi-structured data into relational data

As we saw in the previous exercise, semi-structured data is flexible and can accommodate any amount of densely or sparsely nested elements. However, in nested objects, it can be inferred that lower-level elements are attributes of their immediate parents.

Observe the following simplified example of semi-structured data with three levels of nesting and use the indentation to count the depth.



Figure 16.5: Observing the number of nested levels in a semi-structured object

Here, we can see that a (*ship*) *type*, of depth 2, is an attribute of ship, which is a level-one attribute of the root object. By this logic, if we follow the levels of a semi-structured object to its maximum depth N, those elements become attributes of an entity N-1. Then, N-1 entities become attributes of N-2, repeating recursively until arriving at the root.

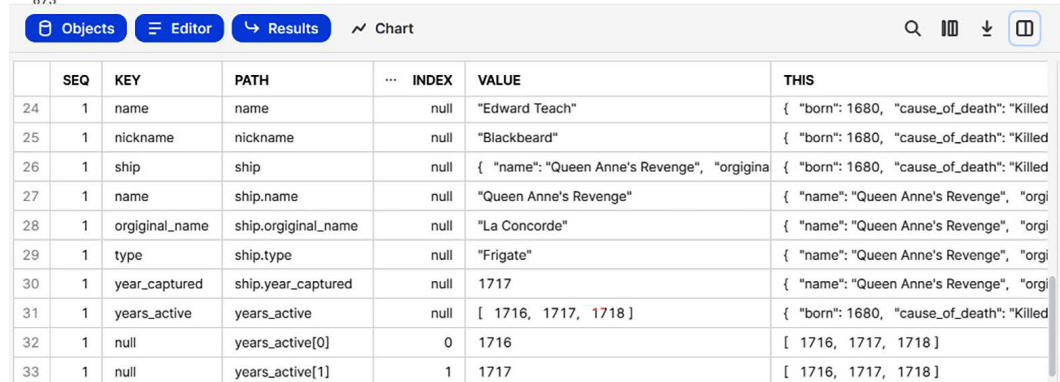
In the current example, `boarding_axe` and `blunderbuss` become instances in the weapon entity, which is an attribute of the crew member, which is an attribute of pirate. However, there are several nuances to consider. Due to its dynamic nature, any object in a semi-structured file is free to vary and include whatever attributes or nested objects it needs. This means, unlike in structured data, looking at one instance of an entity, such as the crew member, tells us nothing about the attributes of another. Notice how `Stede Bonnet` has a `nickname` attribute while `Israel Hands` has a Boolean `parrot` indicator. The only way to fully know all the levels and attributes of a semi-structured file is to scan it to the very end. Even then, there is no guarantee that new attributes won't appear tomorrow.

The good news is that the process of determining the depth of a semi-structured file can be automated through Snowflake functions. Using the `FLATTEN` function covered in the previous section, we can set the `RECURSIVE` parameter to automatically expand every element to its ultimate depth. The output of recursive flattening can be seen in the following figure:

```

671 SELECT f.*
672 FROM pirate_json p, LATERAL FLATTEN( v , RECURSIVE => TRUE ) f ;
673

```



SEQ	KEY	PATH	INDEX	VALUE	THIS
24	1	name	null	"Edward Teach"	{ "born": 1680, "cause_of_death": "Killed"
25	1	nickname	null	"Blackbeard"	{ "born": 1680, "cause_of_death": "Killed"
26	1	ship	null	{ "name": "Queen Anne's Revenge", "original_name": "La Concorde", "type": "Frigate", "year_captured": 1717, "years_active": [1716, 1717, 1718] }	{ "born": 1680, "cause_of_death": "Killed"
27	1	name	null	"Queen Anne's Revenge"	{ "name": "Queen Anne's Revenge", "original_name": "La Concorde", "type": "Frigate", "year_captured": 1717, "years_active": [1716, 1717, 1718] }
28	1	original_name	null	"La Concorde"	{ "name": "Queen Anne's Revenge", "original_name": "La Concorde", "type": "Frigate", "year_captured": 1717, "years_active": [1716, 1717, 1718] }
29	1	type	null	"Frigate"	{ "name": "Queen Anne's Revenge", "original_name": "La Concorde", "type": "Frigate", "year_captured": 1717, "years_active": [1716, 1717, 1718] }
30	1	year_captured	null	1717	{ "name": "Queen Anne's Revenge", "original_name": "La Concorde", "type": "Frigate", "year_captured": 1717, "years_active": [1716, 1717, 1718] }
31	1	years_active	null	[1716, 1717, 1718]	{ "born": 1680, "cause_of_death": "Killed"
32	1	null	0	1716	[1716, 1717, 1718]
33	1	null	1	1717	[1716, 1717, 1718]

Figure 16.6: Recursively flattening an entire JSON file



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Notice that this output does not select from the VARIANT column `v` but from the result of the lateral FLATTEN operation. Using the PATH column, we could calculate the depth by counting the number of dots and end-level array elements to give us a complete list of elements and their level in the semi-structured hierarchy.

```

675     SELECT ARRAY_SIZE(
676         STRTOK_TO_ARRAY( IFF (
677             STARTSWITH( RIGHT(path,3), '[' ),
678             LEFT(PATH ,LENGTH(path)-3) || ' ' || SUBSTR(path,LENGTH(PATH)-1,2),
679             path), '.'
680         )
681     ) AS depth
682     , f.key
683     , f.path
684     , f.value
685 FROM pirate_json p, LATERAL FLATTEN( v , RECURSIVE => TRUE ) f
686 ORDER BY 1 ASC ;

```

Objects Editor Results Chart				
	DEPTH	KEY	PATH	VALUE
6	1	nickname	nickname	"Blackbeard"
7	1	name	name	"Edward Teach"
8	1	died	died	1718
9	2	null	years_active[2]	1718
10	2	null	years_active[1]	1717
11	2	null	years_active[0]	1716
12	2	year_captured	ship.year_captured	1717
13	2	type	ship.type	"Frigate"

Figure 16.7: All elements of a semi-structured file and their depth

Now that we have the structure, we can deconstruct the semi-structured data into normalized relational tables. Starting with the maximum depth (weapons at depth 3), we can create a dimension with metadata columns and a surrogate key (some elements, such as account numbers, contain natural keys, which can be used instead).

Start by creating a table for the weapon dimension. We will use a sequence to generate a surrogate key, but a hash or a natural key can be used instead:

```
CREATE OR REPLACE TABLE weapon
(
    weapon_id number(38,0) NOT NULL AUTOINCREMENT START 1 INCREMENT 1,
    name varchar NOT NULL,
    __load_name varchar NOT NULL,
    __load_dts timestamp_ntz NOT NULL,
    CONSTRAINT pk_weapon_weapon_id PRIMARY KEY ( weapon_id ),
    CONSTRAINT ak_weapon_name UNIQUE ( name )
)
COMMENT = 'weapons used by pirates';
```

Now, merge the weapon values from the JSON in the latest load and insert them if they don't already exist:

```
MERGE INTO weapon w...;
```

Have a look at the table contents:

```
SELECT weapon_id, name FROM weapon;
-----result-----
WEAPON_ID|NAME          |
-----+-----+
         1|blunderbuss    |
         2|flintlock pistol|
         3|cutlass        |
         4|boarding axe   |
```

Now, repeat the process for the elements at depth 2. The ship dimension poses no challenge because it has no child relationships; however, some questions need to be answered before being able to model crew and years_active. Is a crew member a separate entity from a pirate captain or is it a subtype? Does years_active refer to how long a crew member has worked under the current captain or how long they have been active over their entire pirating career? Here, just as with modeling relational entities, only business experts can help determine the nuances of what the data represents and how it is used.

Let's suppose that our domain experts confirm that a crew member is a subtype of pirate and all attributes, including years_active, are shared as part of a single pirate dimension (this is the logical-to-physical rollup scenario described in *Chapter 12, Putting Physical Modeling into Practice*). As part of the pirate rollup, we must make sure to include an FK reference to itself to store the relationship between the crew and captain. First, we create the table structure:

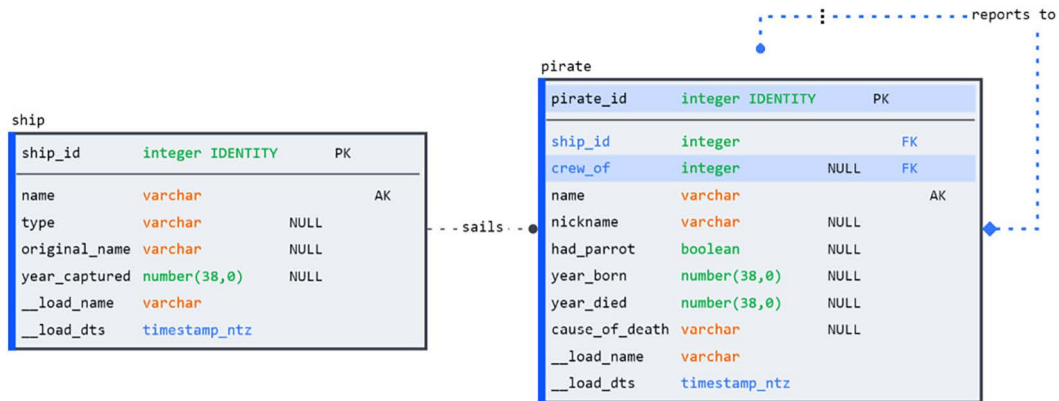


Figure 16.8: A rollup of the crew into the supertype pirate dimension

As we have fused a level-2 entity with its supertype, we must load the data starting with the top-level dimension (as crew will need an existing pirate_id to report to). First, load the level-1 pirate entities (i.e., Blackbeard):

```
MERGE INTO pirate p USING (< select level 1 pirates>)...;
```

Once the top-level object has been loaded, we can load the crew details, referencing their captain's surrogate key in the crew_of column. Load the crew details and verify the result:

```
MERGE INTO pirate p USING ( < select level 2 crew > )...;
```

The output will look something like this:

```
707 | SELECT * FROM pirate;
```

```
708
```

	PIRATE_ID	SHIP_ID	CREW_OF	NAME	NICKNAME	HAD_PARROT	YEAR_BORN	YEAR_DIED	CAUSE_OF_DEATH
1	1	1	null	Edward Teach	Blackbeard	null	1,680	1,718	Killed in action
2	2	1	1	Stede Bonnet	Gentleman pirate	null	null	null	null
3	3	1	1	Israel Hands	null	TRUE	null	null	null

Figure 16.9: Converted semi-structured pirate attributes in 3NF

Look carefully and you will notice that one of the attributes, `years_active`, is missing from the table. Because this data is multi-valued, it cannot be included in the pirate dimension without violating 1NF. For this, we must create and load a separate `pirate_years_active` entity.

The last missing attribute is pirate *weapons*. In this scenario, we have a many-to-many relationship between the pirate and weapon dimensions. As described in *Chapter 9, Putting Logical Modeling into Practice*, modeling many-to-many relationships requires an associative table in the physical layer. This table holds the PKs for the associated entities (i.e., pirate and weapon), as well as the metadata fields that tell us when a certain relationship was first loaded. With this, the relational model is complete, as seen in the following diagram:

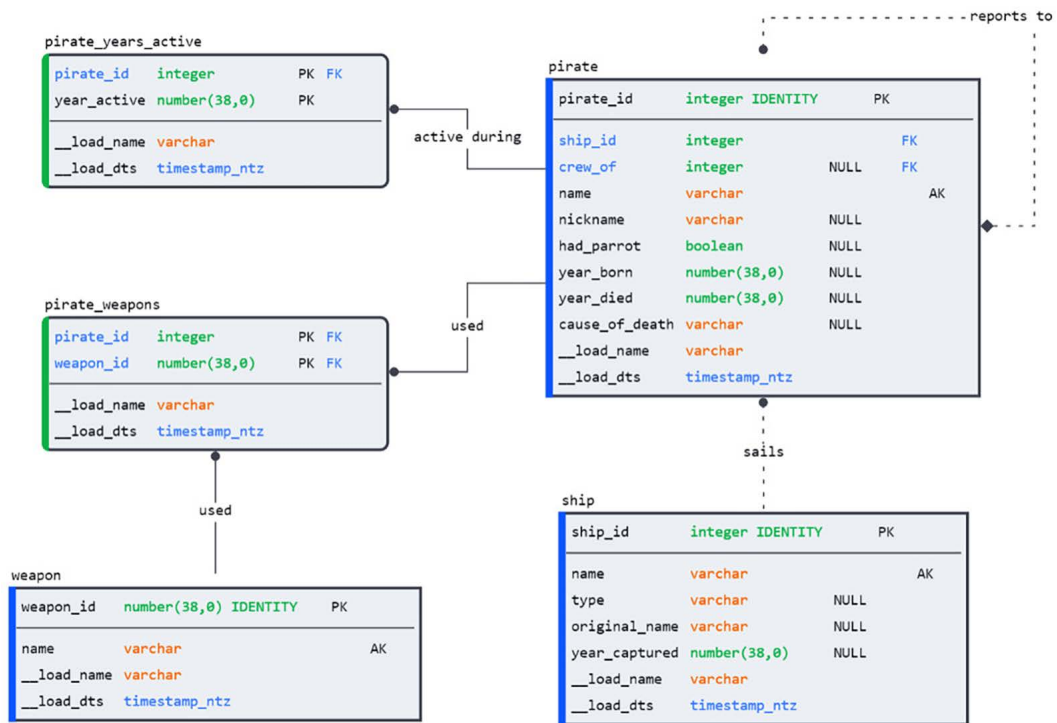


Figure 16.10: A semi-structured schema converted to a relational physical model

Follow the steps in the accompanying code to load the `pirate_weapons` and `pirate_years_active` tables. Once loaded, we are free to analyze the data using traditional relational methods. Here's an example:

```
SELECT p.NAME AS pirate_name
, nvl(p.nickname, 'none') AS nickname
, s.type AS ship_type
, nvl(w.NAME , 'none') AS weapon_name
FROM pirate p
INNER JOIN ship s USING (ship_id)
LEFT JOIN pirate_weapons pw USING (pirate_id)
LEFT JOIN weapon w USING (weapon_id) ;
```

-----result-----

PIRATE_NAME	NICKNAME	SHIP_TYPE	WEAPON_NAME
Stede Bonnet	Gentleman pirate	Frigate	blunderbuss
Israel Hands	none	Frigate	flintlock pistol
Israel Hands	none	Frigate	cutlass
Israel Hands	none	Frigate	boarding axe
Edward Teach	Blackbeard	Frigate	none

This exercise demonstrates that although Snowflake features ease the technical burden of schema-on-read, the process is not trivial. The fact that Snowflake can easily handle semi-structured data does not guarantee that your organization will have the functional or technical capacity to make sense of it. However, by using a repeatable step-by-step process to transform semi-structured data into relational data, as we have done in this chapter, Snowflake users benefit from better organization and data consistency.

Summary

With the rising popularity of web applications and IoT data, semi-structured data has gained prominence for its flexibility in creating and loading dynamically changing objects without affecting ELT pipelines. Semi-structured formats, such as JSON, can handle any amount of variable nested data, which doesn't need to conform to a pre-defined structure. Snowflake makes working with semi-structured formats easy thanks to its `VARIANT` data type – optimized for storage and analytical queries using easy-to-learn extensions to ANSI-standard SQL.

Querying a `VARIANT` data type provides the same performance as standard relational data types without needing to analyze the structure ahead of time – an approach known as schema-on-read. This means Snowflake users can work with semi-structured and relational data on the same platform using familiar SQL commands. However, although Snowflake gives users all the tools necessary for analyzing semi-structured data, schema-on-read only delays but does not eliminate the need for relational-style modeling.

Once the elements of semi-structured data have been understood, they can be converted into a relational schema by looking at element depth to determine its attributes and relationships. As with traditional modeling, engaging business teams for their expertise will be required to create a normalized schema that meets an organization's analytical needs.

Having familiarized ourselves with Snowflake's semi-structured features, we can use many of the same techniques to handle another kind of variable structure: the hierarchy. In the next chapter, we will learn how to model various types of hierarchies using functions provided by Snowflake to make the job easier.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

17

Modeling Hierarchies

In the previous chapter, we learned about the methods and techniques for exploring semi-structured data in Snowflake. While not everyone works with operational web or app data such as JSON, there is another type of (semi-)structured data that exists across all organizations: hierarchies. Every company operates with hierarchical entities such as org levels (tiers of management and the reporting relationships between managers and employees) or calendar dimensions (rollups of days, months, fiscal periods, and years).

Whether formally maintained or naturally occurring, hierarchies are used within an organization to organize entities into meaningful groups and subgroups to facilitate rollups or drill-downs in data analysis. Besides aiding in the analysis of facts, hierarchies themselves can be examined to help organizations understand how they are structured, how they function, and how they can improve their performance by eliminating operational bottlenecks.

This chapter will cover the various ways hierarchical data is modeled in relational databases and Snowflake-specific techniques that enable its rapid analysis and conversion. Drawing upon the techniques covered in previous chapters, we will learn how to anticipate changes in hierarchical data and create models that allow for easy maintenance and impact analysis.

This chapter will cover the following main topics:

- Understanding hierarchies and how they are used in data warehouses
- Distinguishing between the various hierarchy types
- Modeling techniques for maintaining each type of hierarchy
- Snowflake features for traversing a recursive tree structure
- Handling changes in hierarchy dimensions

Technical requirements

The scripts used to instantiate and load the examples in this chapter are available in the following GitHub repo: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch17>. While key sections of this script will be highlighted in this chapter, please refer to the `hierarchies.sql` file for the complete code required to load, query, and transform semi-structured data, as it is too long to reprint here in full.

Understanding and distinguishing between hierarchies

A *hierarchy* is a system in which people, objects, or concepts are organized into a tree-like structure, with each level representing a different category or grouping of data. In modeling, hierarchies can be thought of as a series of descending one-to-many relationships.

At the top of the hierarchy sits the root node, which contains child nodes. Each child node represents a subcategory of the data contained in the parent node and may, in turn, have its own child nodes. This arrangement of nodes and their relationships is often called a tree structure or diagram. If we were to proverbially chop down such a tree and observe it laterally, we would see the hierarchy in its relational form.

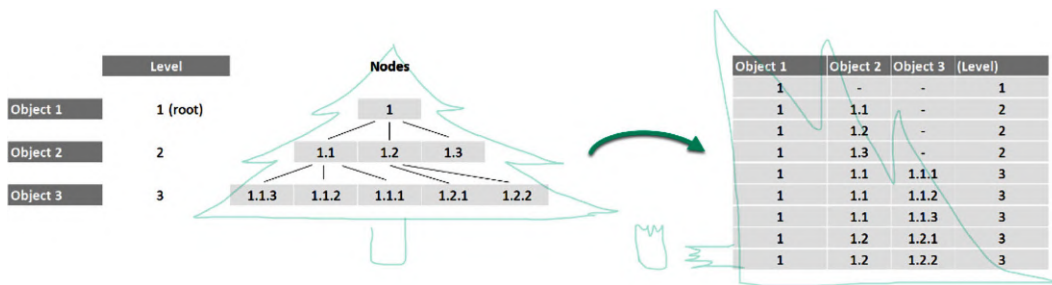


Figure 17.1: A hierarchy seen in a tree (left) and relational format (right)

Hierarchies fall into three general categories depending on the variability in their levels. Let's look at each of these categories.

A fixed-depth hierarchy

A *fixed-depth* hierarchy has a pre-defined and agreed-upon number of levels. Such a hierarchy is simple to model and analyze because it is just a relational table with individual attributes for each level. Take a calendar dimension as an example: the day attribute rolls up into a month, which rolls up into a year. It can be expressed as a simple table with three attributes, each corresponding to a separate level.

Calendar			
(pk)	Level 1	Level 2	Level 3
Date	Year	Month	Day
2020-01-01	2020	1	1
2020-01-02	2020	1	2
...	...	...	...
2022-12-31	2022	12	31

Figure 17.2: An example of a fixed-depth hierarchy

In some cases, though, hierarchy levels are not completely fixed.

A slightly ragged hierarchy

A *slightly ragged* (or slightly variable) hierarchy also has a fixed depth, but not all members occupy all the levels. Think of a region hierarchy that contains cities of various sizes. A city such as *New York* is big enough to be broken down into smaller zones for analysis, while its neighbor, *Jersey City*, is tiny, and further segmentation is not useful.

Because slightly ragged hierarchies have a fixed depth, they can be modeled using the previously discussed structure. The variable attribute levels can then be populated based on business rules, as seen in *Figure 17.3*.

Region				
(pk)	Level 1	Level 2	Level 3	Level 4
Region ID	Country	State	City	Zone
1	USA	NY	New York	A
2	USA	NY	New York	B
3	USA	NY	New York	C
4	USA	NJ	Jersey City	none

Use not-applicable default

OR

Region				
(pk)	Level 1	Level 2	Level 3	Level 4
Region ID	Country	State	City	Zone
1	USA	NY	New York	A
2	USA	NY	New York	B
3	USA	NY	New York	C
4	USA	NJ	Jersey City	A

Use lowest level default

Figure 17.3: Examples of a slightly ragged hierarchy

The following business rules are commonly used for fitting slightly ragged hierarchies into a fixed depth:

- Mark inapplicable values:** When a node does not have a value for a given level of the hierarchy, use a default value (e.g., *none* or *N/A*) to label it accordingly. This approach ensures that aggregates and analytical filters treat not applicable values differently from those that are labeled.

- **Use the lowest level:** When a node does not have a value for a given level of the hierarchy, label it with the lowest value of that level by default. This method allows all nodes in the hierarchy to be grouped under each level's corresponding rank.

Fixed and slightly ragged hierarchies can be analyzed using standard relational methods. However, there are hierarchies whose nodes can vary wildly in the number of levels and children, and a standard relational approach is no longer viable.

A ragged hierarchy

A *ragged* (a.k.a. variable-depth, unbalanced, uneven) hierarchy contains levels or categories that do not have a uniform number of members. One that most people will recognize is an org hierarchy. In an org hierarchy, the highest level may have only one member (CEO), while the next level may have several vice presidents with their own direct reports and the CEO's lone secretary.

Ragged hierarchies are difficult to analyze because they are of indeterminate depth and do not fit neatly into a relational structure. A ragged hierarchy does not have separate attributes corresponding to fixed levels but instead uses a self-referential PK/FK relationship to link parent and child nodes. While the parent/child relationship can be recorded in a single record, as seen in the following example of the relationship between an employee and a manager, analyzing such a hierarchy takes some effort.

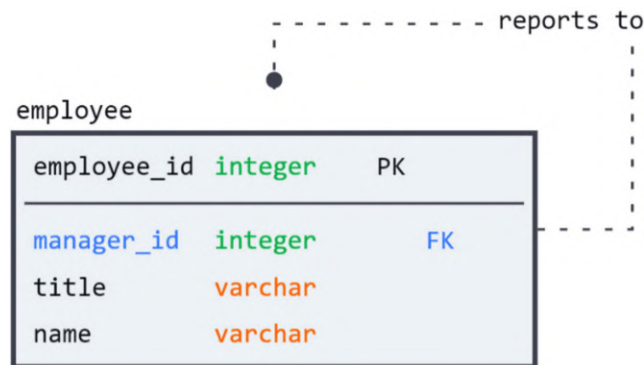


Figure 17.4: A ragged hierarchy with a recursive relationship



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



Snowflake provides several nifty extensions to ANSI SQL that allow users to traverse a ragged hierarchy's recursive parent/child relationships at runtime. These same features can also aid in generating a flattened representation of the hierarchy, known as a bridge table. A **bridge table** contains a row for every possible path in the ragged hierarchy and, as a standalone dimension, allows for alternate versions using familiar SCD patterns. By removing `manager_id` from `employee` and moving it to the bridge table, both dimensions can change without impacting one another. Most importantly, the hierarchy can be analyzed or joined to corresponding entities using traditional relational methods. An example can be seen in the following figure:

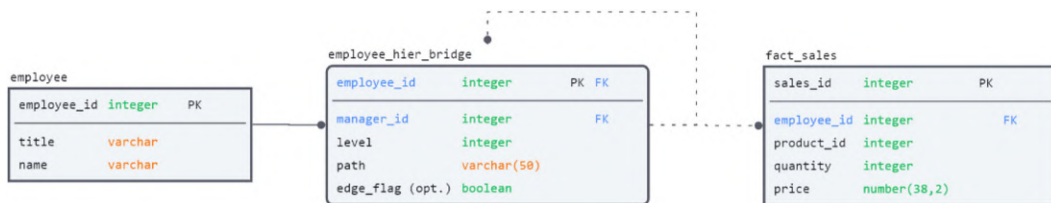


Figure 17.5: A ragged hierarchy with a bridge table

A bridge table makes it easy to read the paths and levels of a ragged hierarchy. The model in this example also includes a column to identify **edges** (nodes without any children). An edge flag can simplify the queries required to answer questions such as *do all VPs have direct reports?*

The following is an example of a simple tree hierarchy and its representation as a bridge table (a denormalized version is provided for easier understanding):

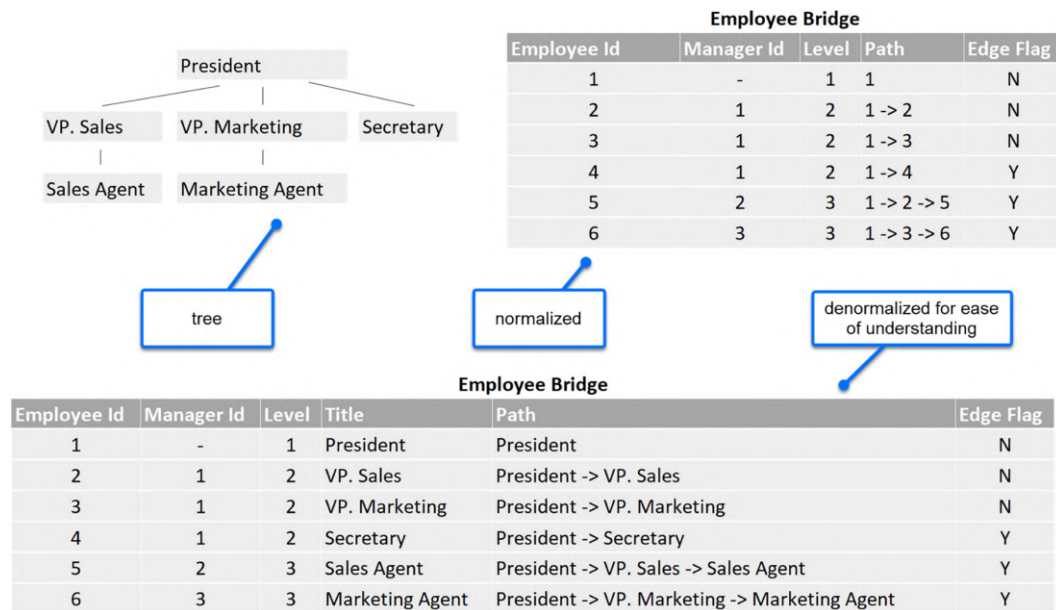


Figure 17.6: Sample data in a ragged hierarchy bridge table

Now that we are familiar with the types of hierarchies and their structure, let's learn how to handle changes and create a bridge table by traversing a recursive relationship.

Maintaining hierarchies in Snowflake

As we have seen, a hierarchy is a grouping of classifications and relationships. It is an adjacent but separate dimension to the entity that it organizes. As a standalone dimension, hierarchies are subject to the same changes in business rules as any master data. However, before we attend to change-tracking, we must get comfortable traversing a ragged hierarchy's recursive relationships.

Once again, Snowflake has us covered with native features that allow us to easily map out the paths and levels of a ragged hierarchy.

Recursively navigating a ragged hierarchy

You have been put in charge of the pirate frigate Queen Anne's Revenge. Its logbook shows all crew members, ranks, and direct reports. The details are listed as follows:

	PIRATE_ID	NAME	RANK	SUPERIOR_ID
1	1	Blackbeard	Captain	null
2	2	Calico Jack	First Mate	1
3	3	Anne Bonny	Second Mate	1
4	4	Mary Read	Navigator	2
5	5	Israel Hands	Boatswain	2
6	6	John Silver	Carpenter	3
7	7	Long John	Gunner	3
8	8	Billy Bones	Cook	4
9	9	Tom Morgan	Sailor	5
10	10	Harry Hawkins	Sailor	5
11	11	Black Dog	Sailor	6
12	12	Dick Johnson	Sailor	6
13	13	Roger Pew	Sailor	7
14	14	Dirk van der Heide	Sailor	7
15	15	Ned Low	Sailor	9
16	16	Edward England	Sailor	9
17	17	Stede Bonnet	Sailor	10
18	18	Charles Vane	Sailor	10
19	19	James Kidd	Sailor	11
20	20	William Kidd	Sailor	11

PIRATE

PIRATE_ID	number(38,0)	PK
NAME	varchar(50)	
RANK	varchar(50)	
SUPERIOR_ID	number(38,0) NULL	FK

Figure 17.7: A ragged hierarchy contained in the pirate dimension

As you review the records, you realize that this list makes it difficult to make sense of the complex web of the variable-depth relationships of the crew. To understand the ship's chain of command, you need a way to traverse the hierarchical tree structure. To accomplish this, Snowflake offers the `CONNECT BY` clause.

The `CONNECT BY` clause follows a parent-child relationship between rows in a table. Each row has a parent identified by a foreign key in the same table. The `CONNECT BY` clause uses a recursive query approach to traverse this hierarchical structure. It starts with a root row (which can be specified using the `START WITH` predicate) and then recursively traverses the structure by finding all the children of the current row and repeating the process until it reaches the bottom of the tree.

Within the `CONNECT BY` clause, Snowflake uses a function called `SYS_CONNECT_BY_PATH` to generate the path. The output is a string column, which shows the path from the root row to each row in the tree. This column is a concatenated string of the values in the column that is used to establish the hierarchy.

Use the code provided in the repository for this chapter to load the pirate table and observe the parameters in the `CONNECT BY` statement that generate the pirate hierarchy in the following figure:



Figure 17.8: The `CONNECT BY` clause using `PATH`

Using the `CONNECT BY` query result, we can easily determine how many levels and branches exist in the crew hierarchy. The pirate crew is as complex as any organization – containing five levels of depth and comprising two distinct branches (those of `First Mate` and `Second Mate`). If we wanted to label these branches to make future analysis easier, we could use the `CONNECT_BY_ROOT` function to return the top-level root node of the branch we are traversing.



Figure 17.9: Split the hierarchy into multiple root branches

Now that we have multiple branches, we can add an edge flag to identify members without direct reports. To do this, use LEFT JOIN for a distinct list of SUPERIOR_ID values (e.g., pirates with direct reports) from the query result we obtained previously. Any NULL value returned by the right table identifies an edge, as seen in the following figure:

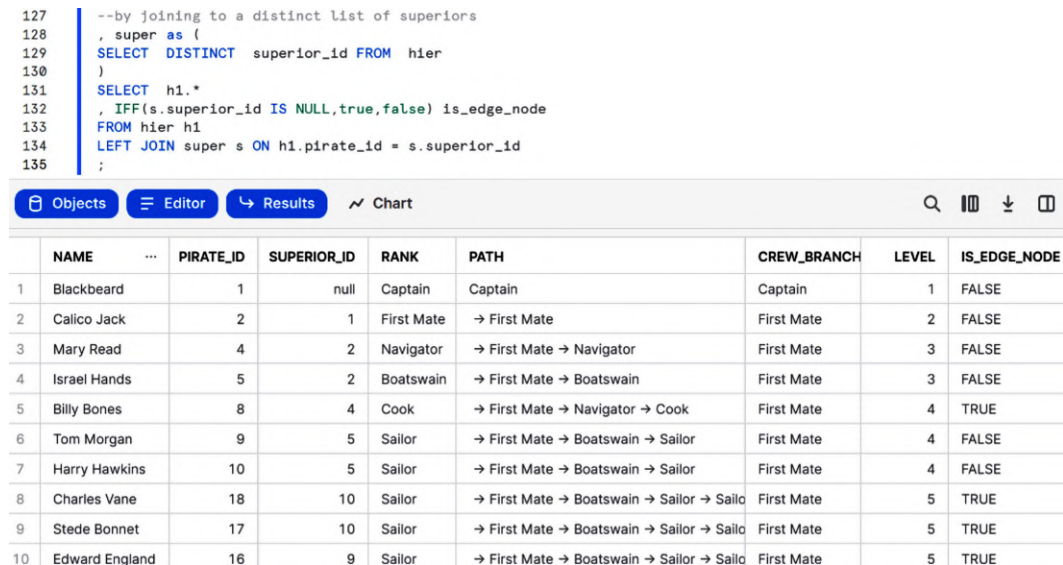


Figure 17.10: Add an indicator for edge nodes

Now that we have all the details needed for the bridge table, we can create it as a standalone dimension for future analysis and subsequent changes. Keeping the hierarchy separate from the source entity ensures that each can be maintained without impacting the other. Also, we can use familiar modeling techniques to track changes in hierarchies of any kind.

Handling changes

A hierarchy can be updated and modified as a standalone dimension without impacting the underlying entities. Imagine if hierarchy details were stored alongside an already large Type 2 employee dimension – every title or manager change would generate a massive cascade of new records. Instead, we can use the SCD models covered in *Chapter 14, Modeling Slowly Changing Dimensions*, to maintain changes to hierarchy dimensions efficiently and according to business needs.

Suppose that region groupings or pirate stations changed regularly. Our organization may be interested in the historical (or proposed) changes for planning and comparison purposes. The following are some of the common historical tracking scenarios that can occur and how to treat them:

- **History not required:** Use Type 1 SCD. If tracking historical changes to the hierarchy is not required, then a Type 1 strategy of *overwrite* is the simplest way to handle changes.
- **Compare last:** Use Type 3 SCD. When a change is made (or proposed) to an existing hierarchy, an organization may wish to compare the impact of the new grouping on the existing facts. For example, what if, in *Figure 16.3*, we grouped *New York City Zones A* and *B* but wanted to compare sales volumes to the previous classification? Adding a Type 3 *last* value column to the hierarchy dimension allows the business to analyze the before-and-after impact using the same join they would normally use to tie the fact to the hierarchy dimension.
- **Compare point-in-time:** Use Type 2 SCD. When historical versions of the hierarchy need to be tied back to the facts at various effective dates, the Type 2 model of *add new row* is used. This approach allows the hierarchy dimension to be joined to a fact table using any effective date to obtain the classification or assignment at a given moment. This allows organizations to answer questions such as *Who did employee X report to last year versus currently?*

Now that we have familiarized ourselves with the types of hierarchies that exist in our organization and learned how to model and analyze them, let's review what we've learned.

Summary

From how employees are organized to how customers or products are segmented, hierarchies exist in every organization. By learning how to model hierarchies as business dimensions, organizations can analyze and maintain them as they would any other entity. However, not all hierarchies behave the same way, and learning to understand their differences is the key to modeling them accordingly.

The biggest difference between hierarchies is the degree of variance between their members. Fixed and slightly ragged hierarchies have a set number of levels corresponding to their attributes' natural grouping. Ragged hierarchies, such as org levels, can vary in depth and the number of branches for each node. Because of their tree-like structure, ragged hierarchies present a challenge for relational databases that rely on a columnar structure.

Using native extensions to standard SQL, Snowflake allows users to easily perform complex operations, such as recursively traversing a tree structure and determining its branches and depth levels. The result can be stored in a separate dimension known as a bridge table, which allows the hierarchy to change independently of the associated entity. Once hierarchy details are separated into dimension tables, they can be historified according to business needs using the SCD techniques covered previously.

Now that we are familiar with the structures commonly found in a data warehouse and the tools that Snowflake provides to maintain them, we can discuss more advanced modeling patterns. In the next chapter, we will explore a popular data warehouse modeling methodology and discover the properties that make it one of the fastest-growing and scalable solutions to date.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

Part 4

Data Modeling for Enterprise Teams

This part covers the emerging trends and leading methodologies in data modeling to meet the demands of the modern data-driven enterprise. Here, we review the basics of Data Vault methodology for building agile, scalable, and auditable architectures, and Data Mesh, which is becoming an increasingly popular approach to managing data domains at scale across large organizations. We also cover the data lakehouse approach for working with semi-structured and unstructured data and explore the topic of AI and the role data modeling plays in ensuring success in machine learning and inference training initiatives. Most importantly, we'll learn to speak the language of the business and learn to defend data modeling using financial and value-based metrics. Finally, you will experiment hands-on with fun and effective SQL recipes that couldn't be included in the rest of the chapters.

This part has the following chapters:

- *Chapter 18, Scaling Data Models through Modern Techniques*
- *Chapter 19, Unlocking Value for Enterprise Organizations*
- *Chapter 20, Appendix*

18

Scaling Data Models through Modern Techniques

After covering theory, architecture, terminology, methodology, and Snowflake-centered transformation strategies throughout the book, this chapter will build upon that foundational knowledge to address common data management challenges in large, complex environments. Specifically, this chapter will explore Data Vault 2.0 and Data Mesh methodologies—popular solutions that have emerged in response to some of the biggest challenges facing large organizations today. Despite their similar naming, Data Vault and Data Mesh attempt to tackle very different challenges, and are often used together.

Data Vault is a methodology that focuses on the efficient and flexible storage of data, with a primary focus on auditing and effortless scalability. It is made up of three pillars: modeling, methodology, and architecture. Its standardized, repeatable design patterns can be applied regardless of the complexity of the data or how many source systems are used.

Data Mesh exists for a very different purpose: to facilitate data discovery and sharing among distributed enterprise teams. Data Mesh is not a specific technology or tool, but rather a set of principles and best practices that can be applied to any architecture or platform. It emphasizes the importance of domain-driven design, self-service, federated data governance, and the use of data products and APIs to facilitate data discovery and sharing.

Data Mesh consists of principles and technical practices that enable effective communication and collaboration between domains. Many of these principles should be considered general best practices whether or not an organization decides to take up the Data Mesh banner wholeheartedly. As such, an overview of Data Mesh and the various modeling guidelines covered throughout this book will close out the final chapter as a general summary.

With this in mind, let's dive in and familiarize ourselves with Data Vault and how it can help organizations overcome some of the challenges associated with traditional data warehousing and analytics.

In this chapter, we're going to cover the following main topics:

- An introduction to Data Vault and its utility in modern data platforms
- Addressing the challenges of managing large, complex, and rapidly changing data environments with Data Vault
- The functional layers of the Data Vault architecture and core elements of the Raw Vault
- Efficiently loading Data Vault with multi-table inserts
- Modeling techniques for data marts
- An introduction to Data Mesh and managing data in large, complex organizations
- Reviewing Data Mesh and modeling best practices mentioned throughout the book

Technical requirements

The scripts used to instantiate and load the examples in this chapter are available in the following GitHub repo: (<https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/ch18>). While key sections of this script will be highlighted in this chapter, please refer to the `data_vault.sql` file for the complete code required for following the Data Vault exercise, as it is too long to reprint here in full.

Demystifying Data Vault 2.0

Data Vault was introduced in the early 2000s by Dan Linstedt as a response to the extensibility limitations of warehouses built using 3NF and star schema (discussed later in the chapter) models. Data Vault overcame these limitations while retaining the strengths of 3NF and star schema architectures by using a methodology especially suited to meet the needs of large enterprises. Around 2013, Data Vault was expanded to accommodate the growing demand for distributed computing and NoSQL databases, giving rise to its current iteration, Data Vault 2.0.

Data Vault uses a pattern-based design methodology to build an auditable and extensible data warehouse. When most people refer to Data Vault, they are referring to the **Raw Vault**, which consists of Link, Hub, and Satellite tables. Atop the Raw Vault, sits the **Business Vault**—designed to be a business-centric layer that abstracts the technical complexities of the underlying data sources and uses constructs such as **Point-in-Time (PIT)** and **Bridge** tables. Data cleansing and additional business rules are performed in separate data mart—or information mart—layers based on department and organizational reporting requirements.

The Business Vault and information mart layers are then exposed to end users for curated and self-service analytics.

Auditability is a major focus in Data Vault. Data warehouses are often more than just sources of historical data—as source systems are updated or decommissioned, a data warehouse becomes the system of record. Depending on where and how data is modified in a data warehouse, traceability to the source system may be permanently severed. To overcome this problem, Data Vault uses an insert-only strategy—no updates, no data cleansing, and no business rules in the base layers (this is handled downstream of the data marts and information delivery layers). As Data Vault practitioners often say: *Data Vault is a source of facts, not a source of truth* (since truth, in the data world, can vary based on whose business rules you follow).



Architectural alignment with Snowflake

Under the hood, Snowflake architecture is insert-only. Data stored in Snowflake micro partitions is never physically modified. Rather, modifications such as deletions or updates are performed logically, based on the record ID. This design makes Snowflake architecture an ideal fit for the insert-only methodology of Data Vault.

While the Data Vault methodology is simple to understand, it requires training and experience to execute correctly. This chapter will examine the core elements of the Raw Vault and highlight Snowflake-specific features that aid in efficient design and data loading. However, before undertaking a Data Vault implementation, consider obtaining a certification or enlisting the help of an expert to help decide whether and how Data Vault should be implemented in your organization.

Remember, Data Vault is scalable because it is source-system-agnostic and business-focused. To ensure the success of your Data Vault endeavor, start the same way as you would with any design: by understanding business semantics and building a conceptual business model as described in *Chapter 8, Putting Conceptual Modeling into Practice*. By doing so, you will preemptively address many of the design questions to come.

This exercise will use only one source system and a small subset of tables to demonstrate the core elements of the Raw Vault. The conceptual model in *Figure 18.1* will guide the design. Even though only one source system is used in this example, an accurate conceptual model will ensure that business entities and their relationships are modeled correctly (no matter where they are sourced from, referred to as *country* or *nation*, or identified by columns labeled *key* or *id*). If the business model is not well understood in advance, the Data Vault model will be built on a shaky foundation and fail to deliver the expected outcomes.

The conceptual business model and the physical source system model in this example consist of two dimension tables and a fact table, as shown here:

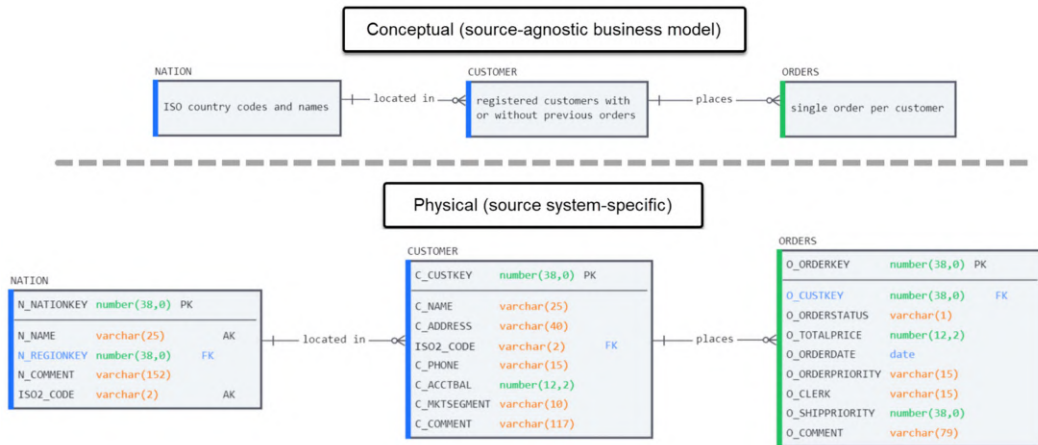


Figure 18.1: ERD of the source system that serves as the base for the Data Vault exercise

For this exercise, we will again source data from Snowflake’s TPCB sample data. Use the accompanying code to instantiate the environments and create the landing area tables and associated objects by running the code in the sections titled as follows:

- Setting up environments
- Setting up the landing area
- Simulating data loads from the source system
- Starting with this foundation, let’s get to know the core elements of the Raw Vault: links, hubs, and satellites.

Building the Raw Vault

In this section, we will familiarize ourselves with the building blocks of the Data Vault by constructing a basic Raw Vault. Data Vault is a rule-based methodology and also highlights the importance of understanding general modeling standards like the ones covered throughout this book. Data Vault standards mandate the structure, transformation rules, and naming conventions. The latter can vary as long as it is consistent across the entire implementation and follows suggested guidelines (a list of suggested naming standards can be found on the Data Vault Alliance website: <https://datavaultalliance.com/news/dv/dv-standards/data-vault-2-0-suggested-object-naming-conventions/>).

To begin the exercise, we will create a landing area for the source tables we'll work with using the code provided in this chapter. Once the source data is loaded, we can begin modeling the core of the Data Vault design: hubs.

Hubs

The first building block of Data Vault is the hubs. **Hubs** are a collection of business keys belonging to a business entity. In modeling terms, this would be a natural key—not a surrogate with no intrinsic meaning.

Take the LOCATION table as an example:

```
SELECT n_nationkey, iso2_code, n_name FROM src_nation LIMIT 5;
```

```
--
```

N_NATIONKEY	ISO2_CODE	N_NAME
0	AL	ALGERIA
1	AR	ARGENTINA
2	BR	BRAZIL
3	CA	CANADA
4	EG	EGYPT



Quick tip: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button

(1) to quickly copy code into your coding environment, or click the **Explain** button

(2) to get the AI assistant to explain a block of code to you.

```
function calculate(a, b) {
  return {sum: a + b};
};
```

Copy

Explain

1

2



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR visit packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



N_NATIONKEY, the PK in the source system, is a surrogate key unrelated to any business concept. ISO2_CODE, which is also unique and contains meaningful values, is better suited to act as the hub key.

A hub table is a simple structure consisting of the following:

- **Hash key:** A calculated field consisting of a hash (e.g., SHA Binary or MD5) of the Business Key columns that serves as the PK for the hub. This technique has been used to generate unique IDs in previous chapters. This example will use the naming format <dv_object>_<entity_name>_hk (e.g., hub_customer_hk).

- **Business key column(s):** Natural keys (not surrogates) that the business understands and commonly refers to. This can be a single or compound key and uses the same column names as the source table.
- **Load date:** The timestamp of when the record was first loaded into the Data Vault. This example will use the naming format `load_dts`.
- **Source system:** An identifier for the source system from where the data is being loaded. This identifier should be consistent across the entire Data Vault. This example will use the naming format `rec_src`.
- **Hub table naming convention:** Hub tables should follow a uniform naming convention that identifies them as hubs and points to the source business entity (not necessarily the table name). This example will use the naming format `hub_<entity>` (e.g., `hub_customer`).

A note on repeating keys and multiple source systems



Hubs, like all Raw Vault objects, record only the first instance and source system that introduced a business key to the Data Vault. Once a business key exists in the hub, subsequent loads, even those originating in other source systems, are not inserted or updated. By incorporating the timestamp into their PK, Satellite (sats) track historical changes to business attributes, but, like other Data Vault objects, only do so the first time a record enters the Data Vault.

An example of a hub based on the CUSTOMER source table would look as follows:

L1_RDV. HUB_CUSTOMER		
MD5_HUB_CUSTOMER	varchar(32)	PK
C_CUSTKEY	number(38,0)	
LOAD_DTS	timestamp_ntz(9)	
REC_SRC	varchar(16777216)	

Figure 18.2: An example of a hub table structure



 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



While a hub may be simple in nature, it can already answer business questions, such as counting the number of entity instances and the sources that provide data on each. Now, we will learn how to record transactions between entities using link tables.

Links

Data Vault owes much of its versatility to its links. **Links** store the intersection of business keys, or the FKs, from related hubs. In this regard, links can be thought of as fact tables without facts or attributes—they contain nothing but business keys and must be connected to two or more hubs.

A link structure resembles that of a hub in its simplicity. It contains the following:

- **A link PK:** A calculated field consisting of a hash of the Business Key columns. This example will use the naming format `<dv_object>_<entity_names>_hk` (e.g., `lnk_customer_order_hk`).
- **Associated hub PKs:** The hashed PK columns used in the associated hubs, declared here as FKs.
- **Load date:** The timestamp of when the record was first loaded into the Data Vault.
- **Source system:** An identifier for the source system from where the data is being loaded.
- **Link table naming convention:** Link tables should follow a uniform naming convention that identifies them as such. This example will use the naming format `<dv_object>_<entity_names>` (e.g., `lnk_customer_order`).

Just like hubs, links record only the first instance of a relationship loaded into the Data Vault. An example of a link between CUSTOMER and ORDER entities would look as follows:

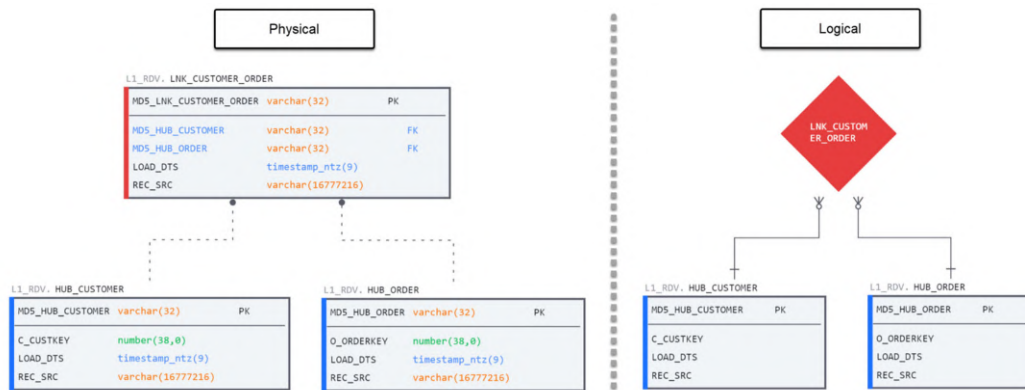


Figure 18.3: An example of a link table structure

Notice that, logically, a link is identical to an M:M associative table. Even though CUSTOMER happens to have a one-to-many (optional) relationship with ORDER, links are designed with flexibility in mind and can store relationships of any cardinality with no changes required. If business rules changed to allow joint orders from multiple customers, the Data Vault could continue loading without changing its structure or ETL processes.

Now that we have a way to store entity business keys and interactions between them, we can learn how to record attributes and their changes using satellites.

Satellites

Satellites store the attributes in a Data Vault and provide change history. Sats function very much like Type-2 SCDs except, like all Data Vault tables, they are insert-only. Because sats can be split to accommodate scenarios such as multiple source systems, rate of change, and security, it is a good practice to include the scenario abbreviation in the sat table name (in this basic example, we use a single source system called *system 1*).

A sat has the following structure:

- **A hub PK column:** A sat uses the associated hub or link PK column (name and value) as its PK (e.g., `hub_customer_hk`).
- **Load date:** The timestamp of when each set of attribute values was first loaded into the Data Vault.

- **Hash diff:** A hash of all attribute values to allow for easy comparison of changes on new loads. This technique was used in *Chapter 14, Modeling Slowly Changing Dimensions (SCDs)*, for its convenience, as it is much faster to compare one column than write out comparisons for every attribute in the table. Due to the name, beginners often mistake this column as a diff between multiple records, but it is only the concatenation of the attributes in one—the diff occurs (or doesn't) when two hash diffs are compared.
- **Source system:** An identifier for the source system from where the data is being loaded. This value may not match that of the hub (e.g., if the business key was first loaded from Sys A but later modified in Sys B).
- **Non-PK source attribute columns:** All the non-PK columns that did not make it into the hub (or link) are included in the sat.
- **Sat table naming convention:** Sat tables should follow a uniform naming convention that identifies them as sats and points to the source business entity (not necessarily the table name). This example will use the naming format `sat_<src system>_<entity>` (e.g., `sat_sys1_customer`).

Sats use a two-part PK consisting of the associated hub key and load date. On every load, the records are compared using the business key and hash diff. When changes are detected, the new records are inserted and given a timestamp (this implies that Data Vault is real-time compatible).

An example of a sat based on the CUSTOMER source table would look as follows:

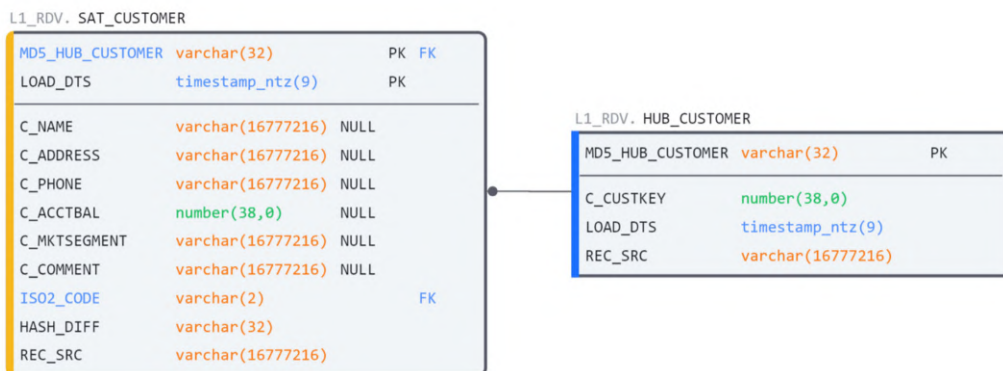


Figure 18.4: An example of a satellite table structure

Every column that didn't make it into the hub or link ends up as an attribute in the sat. Although hubs, links, and sats comprise the Raw Vault's core, an additional object type can also be found at this level.

Although the loads in a Data Vault are insert-only, they are not trivial—validations for new records and CDC checks must be performed to keep the data consistent. As we saw with the `ORDER` table, which generated three objects (hub, link, sat) in the Raw Vault, the challenge of loading multiple objects from a single source also exists. Snowflake provides a **multi-table insert** operator to allow users to perform such an operation efficiently.

In its basic (unconditional) form, the multi-table insert loads data from a single subquery into a list of target tables:

```
INSERT ALL
  INTO t1
  INTO t1 (c1, c2, c3) VALUES (n2, n1, DEFAULT)
  INTO t2 (c1, c2, c3)
  INTO t2 VALUES (n3, n2, n1)
SELECT n1, n2, n3 from src;
```

However, in Data Vault, we must define conditions that only insert records (business keys, transactions, and attributes) when the Data Vault *sees* them for the first time. For this, we will use the conditional multi-table insert. Besides the `WHEN` condition, the multi-table insert allows users to specify an operator that determines whether all `WHEN` clauses are executed for a record or only the first that evaluates to `TRUE`:

```
-- Conditional multi-table insert
INSERT [ OVERWRITE ] { FIRST | ALL }
  { WHEN <condition> THEN intoClause [ ... ] }
  [ ... ]
  [ ELSE intoClause ]
<subquery>
```



Pull over to the side of the load

Outside of the Data Vault framework, the multi-table insert is a tool that can aid in tactical data quality enforcement. When the type of data anomaly is known in advance (e.g., out of range or not permitted values), the multi-table insert can divert anomalous records to a staging table for review while allowing the rest of the load to proceed as planned to the intended target.

Before writing the multi-table insert, we define the outbound views that contain all the source fields and Data Vault required columns to simplify downstream loads. The view is constructed over the stream object created in the previous exercise because it allows us to identify new records without added logic. To follow along with the accompanying code, execute the scripts in the sections titled as follows:

- Creating views for loading the Raw Vault
- Setting up the Raw Vault

The outbound view for the ORDER table would look as follows:

```

CREATE OR REPLACE VIEW src_order_strm_outbound AS
SELECT
-- source columns
*
-- business key hash
    , MD5(UPPER(TRIM(o_orderkey)))          md5_hub_order
    , MD5(UPPER(TRIM(o_custkey)))          md5_hub_customer
    , MD5(UPPER(ARRAY_TO_STRING(ARRAY_CONSTRUCT( NVL(TRIM(o_orderkey)          , 'x')
    , NVL(TRIM(o_custkey)          , 'x')
    ), '^')))) AS md5_lnk_customer_order

-- record hash diff
    , MD5(UPPER(ARRAY_TO_STRING(ARRAY_CONSTRUCT( NVL(TRIM(o_orderstatus)      , 'x')
    , NVL(TRIM(o_totalprice)        , 'x')
    , NVL(TRIM(o_orderdate)         , 'x')
    , NVL(TRIM(o_orderpriority)     , 'x')
    , NVL(TRIM(o_clerk)             , 'x')
    , NVL(TRIM(o_shippriority)      , 'x')
    , NVL(TRIM(o_comment)           , 'x')
    ), '^')))) AS order_hash_diff

FROM src_orders_strm
;

```

Figure 18.6: A view with all required columns for the outbound loads

Using these views, which contain all the columns needed to load the Raw Vault objects, we can configure the multi-table insert statement—allowing us to take a single subquery that selects from the source table and load the associated links, hubs, and satellites in parallel. The multi-table insert would have the following structure:

```

INSERT ALL
-- condition to check if BKEY exists in the hub
WHEN (SELECT COUNT(1) FROM hub_customer tgt WHERE tgt.hub_customer_hk =
src_hub_customer_hk) = 0
-- if it's a new BKEY, insert into the hub table
THEN INTO hub_customer
( < hub columns> )
VALUES
( < hub columns> )
-- condition to check if BKEY exists in the sat
WHEN (SELECT COUNT(1) FROM sat_sys1_customer tgt WHERE tgt.hub_customer_hk
= src_hub_customer_hk

```

```

-- and only insert if changes based on
-- hash diff are detected
AND tgt.hash_diff = src_customer_hash_diff) = 0
-- if it's a new BKEY, or changes to attribute values
-- are detected, insert into the hub table
THEN INTO sat_sys1_customer
( < sat columns> )
VALUES
( < sat columns> )
-- subquery
SELECT < columns AS src_columns> --aliased source columns
FROM l0_src.src_customer_strm_outbound src;

```

Complete (and optionally repeat) the Raw Vault load by running the script in the section titled *Loading the Raw Vault using multi-table insert*.

Each simulated load will process 1,000 customer records and approximately ten times as many related order records. Due to the new-records-only insert nature of Data Vault, repeating the same batch of customer records into the landing area tables will result in no records being loaded into the Vault.

Now that we are familiar with the Raw Vault and the fundamentals and applications of the Data Vault methodology in conjunction with relevant Snowflake features, it is worth spending some time on the modeling patterns often used in information marts (or data marts, as they are called outside of Data Vault) to serve as a springboard for architecting the reporting layer.

Modeling the data marts

This section will explore the Star and Snowflake schemas—popular options for architecting user-facing self-service schemas and data marts due to their efficiency and ease of understanding. Both approaches are designed to optimize the performance of data analysis by organizing data into a structure that makes it easy to query and analyze. But first, a quick overview of what a data mart is.



Data mart versus data warehouse

A data warehouse and a data mart are repositories for storing and managing data, but they differ in scope, purpose, and design. A **data warehouse** is a large, centralized repository of integrated data used to support decision-making and analysis across an entire organization. Data warehouses are optimized for complex queries and often use Kimball's dimensional modeling technique or Inmon's 3NF approach (described in his book *Building the Data Warehouse*). On the other hand, a **data mart** is a subset of a data warehouse designed to serve a specific department or function within an organization. Data marts are typically designed using a star or snowflake schema model.

With this in mind, let's get to know the first of these two architectures: the star schema.

Star schema

A **star schema** is a database architecture that consists of a central fact table surrounded by several dimension tables radiating out in a star-like pattern. The fact table contains the measures (e.g., sales or revenue), while the dimension tables represent the attributes (e.g., time, geography, and product).

A star schema is highly denormalized, making it efficient for querying large datasets and allowing for fast aggregations and drill-downs. The advantage of a star schema is that it is simple to understand and easy to query. The downside, as with any denormalized design, is that integrity issues and data anomalies can occur. While the star schema may be simple to create, it becomes difficult to maintain as the schema grows due to redundancy concerns, and the inability to create entity hierarchies or many-to-many relationships. Such concerns are addressed by the snowflake schema approach.

Snowflake schema

A **snowflake schema** is a variation of the star schema where the dimension tables are further normalized into sub-dimension tables. For example, a product dimension might be split into product, product category, and product sub-category dimensions—forming a hierarchy.

In a snowflake schema, the dimension tables are connected through a series of one-to-many relationships, creating a snowflake-like pattern. While this approach increases data normalization, it makes writing queries more complex and harder to understand than a star schema. The snowflake schema is generally considered less performant than a star because of the number of joins required to query it. However, that may not be true once Snowflake's performance enhancements, such as join elimination through RELY (*Chapter 12, Putting Transformational Modeling into Practice*), are considered.

The following table summarizes the differences between the star and snowflake schemas, highlighting their respective benefits in **bold**:

Comparison of star and snowflake schemas	
Star	Snowflake
Simple and easy to understand	More complicated than a star
Denormalized—increasing redundancy and anomaly risk	More normalized with less possibility of anomalies
Less flexible and harder to scale	More flexible and scalable than a star
Easier to write and understand queries	More complex queries due to number of joins required
Faster performance due to fewer joins	Faster performance if leveraging join elimination
Efficient for querying large datasets	Better suited for analysis of individual dimensions
Can be difficult to handle dimensions with many attributes	Better suited for handling dimensions with many attributes

Figure 18.7: A comparison of star and snowflake modeling approaches

Choosing between a star and snowflake schema will ultimately depend on the organization's specific needs and goals. A star schema is simpler and more efficient for querying large datasets but may not be as flexible for complex table relationships. A snowflake schema offers greater data normalization and flexibility but at the cost of increased complexity.

While star and snowflake schemas are often presented as distinct alternatives, they are actually part of the same dimensional modeling philosophy introduced by Ralph Kimball. Kimball did not discourage snowflaking outright. He simply emphasized that you should snowflake intentionally, only when the normalization benefits outweigh the simplicity and performance of a denormalized structure.

While data marts and structured modeling schemas have long powered analytics, they're built around predefined questions and rigid schemas. In the modern world, when organizations grapple with high-volume, multi-format data from across the business, dimensional modeling struggles to keep pace. Here, a data lake architecture emerges as a viable alternative.

Diving into data lakes

Starting in the early 2000s, the explosion of big data and web-driven workloads such as click-stream analytics, IoT, and sensor readings began to generate volumes of semi-structured data and established conventions. Structured relational datasets are simpler to model and govern, following the techniques and best practices outlined in this book. With its constantly evolving feature set, Snowflake allows users to utilize various architectures within a single interoperable platform. The key takeaway is that when transitioning to Snowflake, users are not required to choose exclusively between a lakehouse or warehouse; instead, they can benefit from both and apply elements of each where they are most appropriate and unstructured data that traditional warehouses were unprepared to handle.

The five V's of big data

Big data refers to extremely large, diverse, and rapidly changing datasets that are difficult or impossible to process with traditional relational methods. The following five factors are generally considered to be its defining characteristics:

Volume: Big data deals with massive amounts of data, often measured in terabytes, petabytes, and even exabytes.

- **Velocity:** This describes the speed at which data is generated and processed. Big data use cases such as social media feeds and sensor data require continuous streaming and near real-time processing.
- **Variety:** Big data deals with different data types and formats. It includes structured (i.e., columnar), semi-structured (e.g., XML and JSON), and unstructured data (e.g., text, images, audio, and video).
- **Veracity:** Big data can be noisy, inconsistent, and contain errors or biases. Ensuring the accuracy and reliability of the data is a significant challenge in downstream analytics.
- **Value:** Big data has no value until it is transformed into useful insights. From a business perspective, this is considered the most important “V.”



A data architecture pattern that emerged to meet the demands of big data was **data lake**: a centralized repository to store structured and unstructured data at any scale. Unlike traditional databases or data warehouses, data lakes don't require data to be cleaned or transformed before loading. Instead, they embrace a schema-on-read approach discussed in *Chapter 16*, where raw data is ingested as is into cloud storage such as S3, analyzed on read, and transformed only when it's queried.

The flexibility to hold structured data (such as tables from relational databases), semi-structured data (such as JSON, XML, Parquet), and unstructured data (such as images, video, audio, or log files) makes data lakes ideal for diverse analytical workloads such as machine learning and real-time analytics. As we saw in previous chapters, Snowflake offers many native features to facilitate working with semi-structured and unstructured data.

Objects such as external, Iceberg, and directory tables make it easy to manage data hosted on external storage, and native query syntax simplifies the analysis of semi-structured data in `VARIANT` or `OBJECT` formats, among others. This hybrid approach of marrying the flexibility of a data lake with the security, access control, and governance of a data warehouse has given rise to a new paradigm known as a *data lakehouse*.

The flexibility of a lake house architecture enables use cases that are difficult to achieve with the rigidity of traditional data warehouse schemas. Here are several examples that draw upon the flexibility of a lakehouse to overcome warehousing bottlenecks:

- **Raw data ingesting and staging:** Teams can ingest data in its native format and then process and transform it downstream when needed. While this doesn't remove the burden of modeling, cleansing, and transforming the data, it does push it down the road until requirements demand it, alleviating the pressure on the engineering team and allowing them to capture more sources of information.
- **Data science and ML:** Data scientists can experiment freely with large volumes of semi-structured data and combine it with existing curated and governed sources. This autonomy means data scientists and ML engineers can operate independently, reducing their dependence on the data team.
- **Multi-cloud and multi-platform:** Organizations can operate in multiple clouds and regions while centralizing their data through an open format such as Iceberg. Teams across organizations can work with any tech stack that suits their needs without worrying about vendor lock-in, physical data movement, or duplication.

Of course, there are drawbacks and downsides to using a lakehouse approach that make it less suitable for other workloads. As the exercise in *Chapter 16* demonstrated, the schema-on-read approach is not a substitute for a governed data model. It requires a fair amount of wrangling and guesswork and introduces risks of data inconsistencies if misinterpreted. The following drawbacks are inherent in this approach:

- **Decreased performance:** Although Snowflake can work with semi-structured data without performance limitations, the transformations required during query execution for schema-on-read typically result in more processing overhead than curated data sets.

- **Data governance challenges:** Ensuring data quality, consistency, and security in a lake-house can be challenging due to the diverse nature of data and decentralized storage. A lack of conformity across dimensions and dimension values can make accurate analysis difficult without a governed schema.

Workloads that focus on structured data sets, even when there are additional semi-structured or unstructured data sources, are generally more suitable for a data warehouse architecture. Data warehouses are easier to manage, thanks to their native SQL support.

When organizations become large enough to warrant multiple data marts, data lakes, or warehouses, coordinating them without creating data siloes becomes challenging. In the last few years, a new data management framework, Data Mesh, has emerged to facilitate sharing and collaboration among organizational data domains.

Discovering Data Mesh

Data Mesh (DM) is an approach to organizing and managing data in large, complex organizations, introduced in 2019 by Zhamak Dehghani, a thought leader in the field of data architecture.

The DM approach advocates for decentralized data ownership and governance, with data treated as a product owned and managed by the teams using it. This contrasts with the traditional centralized (or, as Zhamak calls it, *monolithic*) approach to data management, where a single team or department is responsible for all data-related activities.

In a DM architecture, data is organized into self-contained domains, each responsible for its own data curation and sharing. These domains are often organized around business capabilities or processes and are staffed by cross-functional teams that include technical and business experts.

DM consists of four principles that aim to enable effective communication and collaboration between domains: domain-driven design, self-service, and data product thinking. These practices help ensure that each domain can operate independently while still being able to share data and insights with other domains as needed.

The four principles of DM are as follows:

- **Domain-oriented decentralized data ownership:** In DM, data is owned by the domain that produces it, and the domain is responsible for making it available to the rest of the organization. This means that each domain has the autonomy to choose its technology stack, data models, and data storage.

- **Data as a product:** Data is treated as a product designed and built for consumption by other teams. Data is managed with the same rigor and discipline as software products, focusing on delivering customer value.
- **Self-serve data infrastructure as a platform:** DM promotes the idea of building self-serve data infrastructure that provides a set of core data services that different domains can consume. This helps reduce the complexity of data integration and allows domains to focus on their core competencies.
- **Federated governance:** DM recognizes that governance is important for ensuring data quality, compliance, and security. However, instead of centralized governance, DM promotes federated governance, where each domain has the autonomy to govern its own data while adhering to organization-wide standards and policies.

While DM attempts to tackle the biggest data challenges that enterprises face, it has also been criticized for its complexity and the skillset required to execute it correctly because, when mismanaged, the decentralization of DM can lead to a proliferation of data siloes—the very thing it intends to mitigate. But, on the face of it, DM is a collection of best practices that are hard to argue with. Best of all, DM guidelines are perfectly aligned and reinforced by Snowflake architecture and innate functionality. As a fitting end to this book, we will close with a rundown of best practices for DM and the broader modeling context.

Start with the business

A successful data model must accurately reflect the business model. As the understanding of business rules and processes does not sit with the data team, attempting to build any system without the guidance and express approval of business teams is folly. In larger organizations with multiple business lines, identifying domain boundaries is key to segmenting the modeling task into smaller independent chunks.

Depending on the depth and breadth of the organizational hierarchy, a segmented data model can be separated logically into schemas, databases, or even Snowflake accounts. Snowflake sharing and securitization features are then used to establish appropriate access rights for anyone within the organization.

Adopt governance guidelines

Governance must be woven throughout the development process to ensure end-to-end consistency. Define a development workflow that starts with modeling (iterated through all its stages) and provides the appropriate levels of checks and approvals throughout. Ensure that naming conventions are clearly defined and enforced at every stage of the process.

Be sure that all the work within a team, from the model to the transformational logic, is versioned and source controlled. Most people are familiar with code repositories such as Git and should also be aware that modern modeling tools allow for version control and parallel development of the same kind. Version control allows teams to compare and recover prior states of their modeling structure or transformational logic.

Once checked into a repository and validated, physical deployments must also be accompanied by the requisite documentation and metadata collateral to ensure usability by others in the organization, including other departments. Such materials include ER diagrams, table and column-level descriptions, and table constraints. Following these guidelines will accelerate self-service and data discovery.

Emphasize data quality

Data quality is driven by the design of the systems that generate it, and good design starts with an accurate model. When the model and the business it supports are in alignment, anomalous data, such as values that fall outside the accepted bounds or violate established cardinality, will be less likely to occur. However, if the data is inconsistent by nature, integrity checks must be built as far upstream in the loading process as possible. Some anomalies, such as null values, can be caught by Snowflake table constraints. However, integrity checks on FKs, PKs, and AKs must be performed manually during loading. As Data Vault rightly suggests, truth is subjective, while facts are not. Transitive business rules (e.g., active customers) may change over time or vary by department, but a customer entity remains a cornerstone of the business model. Pushing transitive business rules downstream to mutable reporting layers while keeping data in its original form at the lower levels will ensure the model can handle changes or re-interpretations of business rules without requiring an integral redesign.

One of the pillars of DM and a best practice that all data teams stand to benefit from is the concept of data as a product. This simple change of perspective helps domain and centralized data teams embrace data quality holistically and take responsibility for the data they generate or maintain.

Encourage a culture of data sharing

Whether facilitating sharing or reviewing data assets within or across teams, data sharing is the foundation of Snowflake's cloud architecture—allowing organizations to convene on a single source of truth without moving or copying data through cloning and time travel. The documentation and governance guidelines mentioned previously will enable everyone in the organization to use and interpret the data assets in a straightforward manner.

The same principles of sharing and collaboration that Snowflake has baked into its architecture also apply to the data modeling assets. Data modeling is an essential tool for engineering and deploying to a physical database, but its utility extends much further when shared with the broader organization. Once a data model is deployed and operational, the diagrams, definitions, and functional details must be made available to the organization through a collaborative interface that keeps everyone aligned and allows for iterative enhancement. Modern modeling solutions provide the previously mentioned features and integrations with other tools in the BI stack.

Of course, we couldn't talk about data modeling in the modern era without addressing the topic of **artificial intelligence (AI)**.

Data modeling in the dawn of AI

In a rapidly evolving landscape where AI systems are increasingly mastering human-level tasks such as coding and drawing inferences, one pressing question emerges: Is data modeling still relevant?

Actually, yes. In fact, the demands of AI have made it more relevant than ever.

If this seems counterintuitive, think back to before the early 2000s when data modeling was a ubiquitous and inseparable part of database and data warehouse development. The reason was simple: the costs of not getting data quality and consistency right on the first go were staggeringly prohibitive due to limited storage and processing capacity.

Then came the internet and the era of big data, where two fundamental changes took place:

1. Due to the five V's, relational systems could no longer meet all business demands
2. Technological advances such as cheap storage, distributed computing, and in-memory processing have lowered costs and data processing time

Suddenly, the relevance of data modeling came into question.

1. In certain big data scenarios, data modeling only delays development and adds unnecessary costs.
2. Even in a data warehousing context, it was often faster and cheaper to transform data and reprocess it to correct inconsistencies than to spend time formally modeling and validating it beforehand. See *Chapter 19* for why this fallacy persists.

Then, somewhere around 2023, advances in generative AI and **Large Language Models (LLMs)** meant that there was a new and powerful way to unlock the coveted big “V”—value—buried in the volumes of big data that organizations had collected. However, training AI models to learn complex patterns requires vast data sets and massive computational power, bringing costs back into focus.

While the processing behind artificial neural networks and deep learning models is complicated, the “garbage in, garbage out” paradigm still applies.

Like all data initiatives, ML and predictive analytics rely on clear business objectives, consistent and accurate data, domain alignment, and explainability. Bad data or misaligned business logic can derail a dbt model just as easily as an ML model. The difference is that a dbt model can be reprocessed for a few dollars, while retraining a predictive model could cost millions.

This book has outlined many benefits that data modeling brings to organizational data initiatives, so let’s explore how they relate to AI specifically.

- **Data quality and consistency:** Data modeling enforces standards and consistency and ensures that data is clean and well-understood. When data is messy or poorly organized, AI workflows such as training or inference can suffer from inaccuracies and biases.
- **Scalability and futureproofing:** A robust data model ensures that additional data sources can be easily integrated and scale more gracefully because data definitions are formalized rather than haphazard. Without a data model, duplicate and unconformed data can slow down training and inference. As the organization expands, a data model helps ensure faster iteration cycles and reliability.
- **Governance and compliance:** Incorporating compliance rules into the data model, such as tag-based and dynamic masking policies, helps ensure that sensitive data is handled properly and applied consistently rather than piecemeal. For AI projects that involve sensitive data, a robust data model can assist in meeting compliance standards for regulatory frameworks such as GDPR, CCPA, or HIPAA.
- **Domain alignment:** The value in data is only unlocked when insights are derived that help move business objectives forward. Data models provide a common language for data scientists, engineers, and business stakeholders, making it easier to define goals and work with the same understanding of the data. This clarity is essential for aligning AI initiatives with real-world needs because, without it, teams can end up building powerful models that solve the wrong problems.

While technology continues to improve and redraw the boundaries of what we can do with data, cost and efficiency remain important factors. The transformations that the data industry witnessed during the big data era, and now with AI, support the idea—known as **Jevons' paradox**—that as technology increases the efficiency of a resource's use, overall consumption of that resource tends to rise instead of falling.

While per-unit costs (e.g., dollar per gigabyte stored or processed) have plummeted over the past few decades, overall data expenditures have remained steady or even increased due to higher volumes of data and the need for data-driven insights for organizations to stay competitive.

Yet, amidst all this change, data modeling continues to deliver on its promise of simplifying complexity, enabling a shared understanding, and making data management more efficient. In conclusion, the two main benefits of data modeling have held constant since databases were first introduced and continue to do so even under the demands of today's complex workloads:

- Help architect clean, concise, and conformed data.
- Help understand the data and communicate it without ambiguity.

This is not likely to change for the foreseeable future.

Summary

Amidst the ever-increasing data volume, variety, and velocity, new technologies and methodologies continue to emerge to navigate the complexities of modern data landscapes.

When use cases call for a scalable architecture capable of handling many source systems, we can look to Data Vault—a robust framework for constructing scalable and auditable data warehouses, emphasizing efficient, insert-only data loading. For data marts, we compared the star and snowflake schemas, highlighting their distinct advantages in shaping them for optimized self-service and analytics.

Furthermore, we addressed the organizational challenges of managing data in distributed enterprise teams by introducing Data Mesh, a paradigm-shifting framework that advocates for decentralized data ownership, treating data as a product, and fostering a culture of collaboration and self-service.

For big data use cases that go beyond traditional warehousing capabilities due to variable data formats, we explored data lake architectures. Snowflake's robust feature set allows users to interoperate between data lake and data warehouse features under one managed platform, unlocking a powerful hybrid paradigm known as lakehouse.

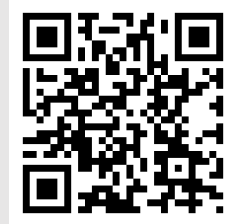
Throughout this exploration, we've underscored the critical role of data modeling, governance, and a business-aligned perspective in building data architectures that are technically sound and strategically viable from a business perspective. Even in the age of AI, data modeling and the methodologies and best practices discussed here provide a roadmap for simplifying complexity, fostering a shared understanding of data, and enhancing data management efficiency.

Whether it's Data Vault's structured approach, the flexibility of data lakehouses, or the distributed governance of Data Mesh, the key takeaway is that data modeling remains a cornerstone of effective data strategy.

However, understanding the value of data modeling and effectively promoting it from a business perspective are separate skills. In the following chapter, we will explore data modeling from the standpoint of business metrics. We will learn why, without these skills, many data modeling initiatives fail to get off the ground and learn how to convince budget and business stakeholders using metrics they understand—ensuring that data modeling efforts are embraced by tying them to tangible business outcomes.

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

19

Unlocking Value for Enterprise Organizations

This book began by highlighting that data modeling is a collaborative effort—a team sport—that reaches its full potential when it incorporates the expertise and insights of the entire organization, from the business side to the data team. Then, throughout many chapters, we unpacked the theory, semantics, and technical approaches to creating and evolving a data model. However, despite having practical knowledge and good intentions, many data modeling initiatives fail in large enterprises. Some never progress beyond the initial idea stage.

This chapter will examine some of the most common obstacles hindering enterprise-wide data modeling adoption. Whether due to misconceptions or a lack of experience, many initiatives aimed at introducing data modeling are often dismissed before they receive proper consideration.

We will also go beyond common misconceptions to examine the genuine and significant challenges that hinder effective data modeling in data-driven organizations. By learning to recognize the obstacles that impede the adoption of data modeling, you will be better equipped to face them in the wild.

Like all data initiatives, the work put into establishing data modeling within an organization will be assessed using well-established metrics such as **Return on Investment (ROI)** and user adoption. We will even simulate a project budget proposal to demonstrate the important role that data modeling can play. Supported by business metrics, we will identify actionable strategies that will help ensure the success of a data modeling project.

In this chapter, we will cover the following topics:

- Recognize and dispel common data modeling misconceptions
- Learn to speak to business teams using metrics they recognize
- Create a project proposal that highlights business value
- Prepare a project plan that demonstrates the value of data modeling
- Learn strategies to overcome challenges to implementing a unified data model
- Tactics to help the data team win friends and influence stakeholders

Misconceptions

Before we can begin planning and justifying data modeling efforts, it's important to dismiss some common misconceptions that can hinder these initiatives from starting. Unlike the barriers we will cover later in this chapter, which require some actions to overcome, misconceptions can often be dispelled outright if we are prepared to address them. Below are some misconceptions about data modeling that you might encounter, typically framed as questions from senior decision-makers when reviewing a data modeling proposal. Let's ensure we have the correct responses ready when they do.

Data modeling is just for the data team

The misconception that data modeling is a technical activity with limited application outside the data team likely stems from the legacy tooling that data modeling has been confined to for decades. When the data model exists solely in specialized software on the desktop of the data architect and can, at best, be viewed in static format on a print-out, such a claim is not unwarranted.

However, in the modern era of cloud enterprise software, with easy access, real-time updates, and an intuitive presentation layer, there is no reason why the data model could not be leveraged by the entire organization. The variety of solutions that exist today offer everything from simple conceptual diagrams to native database models to fully-featured data catalogs. To the claim that data modeling is too complex to have utility beyond the data team, this book serves as a rebuttal—demonstrating that a relational model is not the same as a relational database. Rather, a relational model depicts the company's operating model using simple semantics. When the operating model is mutually understood, it accelerates data-related discussions and **Business Intelligence (BI)** initiatives that help move the company forward.

This is certainly no trivial task, yet, for some reason, another common myth holds that data modeling is a one-and-done ordeal. Nothing could be further from the truth.

Data modeling is a one-time effort

If businesses didn't grow or expand, the data model would never have to change. But, organizations are constantly evolving—opening new markets or merging with other industry players—and generating plenty of demand for insights and analytics. Fortunately, the advantages of data modeling only increase as the size of the data landscape grows, provided that a minimum amount of effort is invested in keeping it up to date.

As the data landscape expands, it becomes increasingly challenging for individuals to keep track of every entity, attribute, and business rule. Without a data model for reference, data teams will gradually see analysis and investigation consume more of their delivery time, just as growing amounts of rework and **Business as Usual (BAU)** gain ground over value-added work.

In certain situations, such as a regulatory audit or during an acquisition/consolidation, having a robust data model is essential. Without one, the data team risks being caught with its pants down right when its efforts are needed most. Just like it's easier to perform regular auto maintenance than deal with expensive repairs later, devoting small but constant attention to maintaining a data model ensures that organizations can reap its benefits throughout its lifecycle instead of being left stranded in a time of need.

If the data model's value grows in proportion to the organization's size, it would be logical to assume that data modeling is only relevant for large organizations—but that's another misconception.

Data modeling is only for large projects and organizations

The belief that only larger organizations have the resources to do data modeling effectively or that only larger projects justify the effort is another fallacy that often hinders its adoption. This belief only holds when the data model is trivially simple and the organizational headcount equals the number of legs a flamingo stands on (when it's showing off). Smaller organizations usually work with tighter budgets and fewer resources and must find efficiencies wherever possible. Without the luxury of large data teams to manage messy data and the rework and adjustments that result from lacking a solid foundation, smaller teams must focus on high-value tasks by ensuring that a robust data model exists to keep them on course. When the business model is small enough for one individual to understand, it is often tempting to forgo the effort of data modeling. However, as businesses grow, relying on personal knowledge alone is insufficient. By the time data modeling becomes essential, starting from scratch will be significantly more challenging than having maintained it (and enjoyed its benefits) all along.

As we have seen, data modeling offers organizations plenty of benefits in exchange for a modest effort. However, with short time horizons and tight deadlines, decision-makers can mistake data modeling for a burden rather than a boon, perpetuating another common misconception.

Data modeling slows down project timelines

The belief that data modeling delays project timelines stems from one key factor: investment is easier to measure than return. The effort involved in data modeling can be measured in developer hours against project budgets. The reward typically comes in the form of undesirable outcomes that *never* happen: data inconsistencies avoided, meetings saved, rework not needed, and so on.

In the early stages of a project, teams tend to over-index on the initial deliverables and discount the long-term maintenance costs. The average project lifecycle is about eight years, while implementations are typically measured in months. By skimping on data modeling at the outset, teams trade delivery costs for maintenance, and over eight years, those costs have plenty of time to compound. But even more dangerous is when those costs are normalized and accepted as BAU, or the cost of doing business.

Like the proverbial frog in boiling water, the pain associated with not having a data model is felt long before it's recognized. Something is always greater than nothing, and a project estimate with data modeling is longer than one without—at least on paper. While a project plan that omits data modeling may seem faster and cheaper up front, it will ultimately lead to higher costs over its lifecycle due to inefficiency and increased maintenance.

In the following section, we will explore this scenario and others and learn to disprove them using business metrics that decision-makers know and understand.

Barriers

Understanding data modeling and effectively leading a company through its adoption are very different skills. While a data model yields far-reaching benefits across an organization, the expertise required to build it resides in the data team. Thus, the engineer must learn to speak through business metrics, demonstrate ROI, and form strategic alliances to champion this initiative.

Data modeling is a team sport that can't be won by engineering alone. However, recruiting the star business players means convincing them to take time out of their busy schedules. When asked to collaborate, it is natural for a person to question what's in it for them. And business collaborators will not be the only ones who need convincing. As a project line item, data modeling will also require budget and executive commitment.

After everything you’ve learned about data modeling—from implementation to impact—you must temporarily adopt a beginner’s mindset to connect with your colleagues at their level of familiarity. Each of them will likely be encountering data modeling for the first time and will need to be convinced of its potential to address their concerns and frustrations. Moreover, many of these individuals will evaluate your proposal through the lens of the business metrics with which they are used to working. So, let’s familiarize ourselves with what data modeling looks like to the company’s bottom line.

Demonstrating value with business metrics

The engineering team wants to invest in data modeling, while business stakeholders want faster delivery dates. There is a path to satisfying them both.

In any data project, it is important to consider the entire lifecycle, not just the initial delivery costs. Although data modeling may initially seem like an expense, it provides significant benefits that can justify the investment throughout the project’s lifecycle. Let’s learn to deliver this message using language that resonates with business users and the budget committee.

In planning a data or IT project, the team first needs to estimate and break down the effort required into line items ranging from development to hardware and software expenses—in total, this is called the **initial investment**. Then, we have to estimate the operating costs (e.g., **Full-time Employees (FTEs)**, licenses, cloud infrastructure, etc.) and multiply those by the expected lifetime of the solution. For reference, a small project like a dashboard targeted at a concrete business initiative might live for 1-2 years, a data mart for 3-5, and an enterprise data platform could last 5-10 years or more. When you add initial investment and expected lifetime operating costs, you get the **Total Cost of Ownership (TCO)**.

On the other side of the scale is the **Expected Benefit** that delivering the project is designed to achieve. Business stakeholders own this metric, so we must take it at face value. Together, Expected Benefit and TCO give us the necessary components for the key metric to determine whether the project is deemed viable: the **Return on Investment (ROI)**.

The formula for calculating ROI looks like this:

$$ROI = \frac{\text{Expected Benefit} - TCO}{TCO}$$

Whether or not a project is green-lighted to proceed will depend on external factors and opportunity costs that are outside the scope of this book, so we will limit ourselves to getting that ROI number as high as possible through the strategic use of data modeling.

Let's start with a straightforward example of a typical medium-scale data project to explore how the metrics vary with and without data modeling. The project involves the development of a data mart for our business stakeholders. It is estimated to take one year to complete and is expected to provide productivity gains for five years thereafter. The business stakeholders are responsible for evaluating the benefits/returns, while it falls to us to determine the expenses and efficiencies contributing to the TCO.

The following table shows the breakdown of the costs of delivering and supporting this project. Here, two scenarios with associated cost columns are included: one that includes data modeling (B) and one that does not (A). Some line items have been split along similar lines to illustrate the difference. Let's review the comparison and highlight several key factors.

Category	Line Item	Description	Scenario A	Scenario B
			No Data Modeling Cost	With Data Modeling Cost
Development costs (year 1)	Development and consulting no modeling	Cross-functional team (Project Manager, Data Architect, Data Engineers, BI Developers, QA) Expect the project to complete 15% faster due to less rework and reloading	600,000	
	Development and consulting with modeling			510,000
	Snowflake platform no modeling	100 credits/month x 12 months at \$2.50	3,000	
	Snowflake platform with modeling	Expect the project to complete 15% faster due to less rework and reloading		2,550
	Infrastructure and licensing	Data integration, reporting, etc.	100,000	100,000
	Data modeling	Training, development, tooling, etc	-	30,000
			703,000	642,550
Operational costs platform (years 2-6)	Snowflake platform ETL	Small WH 2 hours/day x 22 days/month at \$2.50	13,200	13,200
	Snowflake platform analytics	2 XS WH 8 hours/day x 22 days/month at \$2.50	52,800	52,800
	Snowflake platform seasonal spikes	Additional XS WH 4 hours/day x 22 days/month for 2 months at \$2.50	2,200	2,200
	Additional software and licenses	Data integration, reporting, data quality	500,000	500,000
	Additional software and licenses	Data modeling	-	125,000
			568,200	693,200
Data and engineering team no data modeling (years 2-6)	4 FTEs at \$100k avg. salary	\$100k x 4 FTEs x 5 years = \$2M. This will be split between fixes and incremental updates	-	
	Maintenance and bug fixes	45% of the teams time is spent on data quality issues, rework, and performance adjustments	{ 900,000 }	
	Updates and enhancements	55% of the remaining time is spent on delivering incremental enhancements and new functionality		1,100,000
			2,000,000	
Data and engineering team with data modeling (years 2-6)	4 FTEs at \$100k avg. salary	\$100k x 4 FTEs x 5 years = \$2M. This will be split between fixes and incremental updates	-	
	Maintenance and bug fixes	25% of the teams time is spent on data quality issues, rework, and performance adjustments	{ 500,000 }	
	Updates and enhancements	75% of the remaining time is spent on delivering incremental enhancements and new functionality		1,500,000
				2,000,000
Risk Management and contingency (years 2-6)	Contingency Budget	Set aside 10% of annual maintenance labor costs for unforeseen issues	{ 90,000 }	50,000
			90,000	50,000
Total Cost of Ownership (years 1-6)	Development costs (year 1)		703,000	642,550
	Operational costs platform (years 2-6)		568,200	693,200
	Data and engineering team		2,000,000	2,000,000
	Risk Management and contingency		90,000	50,000
			3,361,200	3,385,750

Figure 19.1: Project cost breakdown



Quick tip: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.



The next-gen Packt Reader is included for free with the purchase of this book. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



1. Scenario B includes the extra costs and effort associated with data modeling: training, workshops with business stakeholders, software and tooling, and design.
2. The development costs and timelines have been improved by 15% in scenario B due to better requirements and business alignment, helping avoid rework, reloading, and re-testing. This is a conservative estimate but illustrates how even a modest improvement in efficiency can deliver outsized returns.
3. Curiously, the total cost of the data and engineering team doesn't change under each scenario—only the breakdown. It is a secret that hides in plain sight, masking deep-rooted problems and making them appear commonplace. The data team typically does not expand to address the demand of data quality issues. Instead, they steal the bandwidth from incremental improvements and value-added work to address these challenges. (Note that in this example, the delta between maintenance and bug fixes between the two scenarios is roughly equal to one FTE—a quarter of the data team working exclusively on fixes under scenario A).
4. Project plans typically include a contingency budget as a percentage of estimated maintenance, which, as we expect to be 25% lower under scenario B, we realize additional savings here.

Now that costs have been calculated, let's turn our attention to the benefits. Although this is exclusively determined by the business stakeholders, data modeling can still make an impact.

Category	Line item	Description	Scenario A	Scenario B
			No Data Modeling Benefit	With Data Modeling Benefit
Expected Benefits	Main benefit of deliverable	Efficiency gains, improved decision-making, operational savings over 5 year lifetime	4,000,000	4,000,000
	Benefit of future updates	Added value in incremental updates and new features. 25% relative to main benefit x updates & enhancements capacity	550,000	750,000
			4,550,000	4,750,000
ROI	Total ROI	(Expected Benefit - TCO) / TCO	35%	40%

Figure 19.2: Project benefit breakdown

Although the project's main benefit will be realized through its delivery after year one, incremental changes and enhancements, which we have already budgeted, will provide further value as a percentage thereof. As demonstrated in point 3 of the cost breakdown, the total bandwidth of data team time allocated to enhancements is greater in scenario B due to less time spent on fixes. Therefore, more time can be spent realizing value-added work, thus maximizing this benefit.

Finally, we combine the Expected Benefits and TCO to calculate the ROI. Here, scenario B yields a 5% greater return thanks to the efficiencies and added value through data modeling. If it appears that some costs and benefits have been double-counted to reach this figure, it's because they have—this is called **opportunity cost**. Proper planning and alignment that results in not having to redo a task means those resources are at once A, not wasted, and B, invested in a meaningful alternative. Let that momentum build over the lifecycle of a project, and organizations will see truly astonishing gains in productivity and achievement.

Now, we arrive at the most important factor that data modeling offers—and one that cannot be quantified on any balance sheet. What is not captured directly on any line item in this example is what it *feels* like to work with or for the data team in the data modeling scenario B. With better planning and alignment and less time fixing data quality issues, business teams have no cause to blame, argue, or harangue the data team to help them obtain accurate information. With data quality comes trust, respect, and a collaborative spirit that drives organizations forward in lock-step.

As the example shows, operating and employee costs don't change in the presence of bad data—the warehouse consumes the same credits, and employees receive the same salaries—regardless of the workload. What changes is the quality, purpose, and effective value of the data being delivered, as every warehouse credit or FTE dollar spent on fixes and rework is not spent on meaningful value-added endeavors.

But, if we are speaking the language of business, let's highlight one final metric: the ROI of data modeling itself. By comparing the total expected benefits of the project with and without data modeling, we can determine the difference and perform the ROI calculation using only its associated costs. In this example, the results reveal that although the overall difference in ROI is only 5% between each scenario, every dollar spent on data modeling in scenario B generates a return of nearly 30%. This impressive figure is sure to motivate business teams to get on board with the data modeling proposal.

This Excel worksheet is available in Git to allow you to adjust the numbers or adapt them to existing project estimates.

Armed with an understanding of business metrics, let's examine another possible barrier that has the potential to disorient attempts at unified data modeling.

Decentralized data ownership

In many organizations, siloed and decentralized data ownership creates a significant challenge when implementing a unified data model. When different teams manage their data without a centralized framework, it results in inconsistencies, redundancies, and conflicting definitions. This fragmented approach hinders effective data sharing and collaboration across departments, leading to duplicated efforts and misaligned business insights. The lack of a unified view impedes strategic decision-making, as leadership struggles to fit disparate (and often conflicting) data points into a coherent narrative.

While it may seem easier to address the problem one data silo at a time, the key to achieving unification lies in identifying common touchpoints and aligning where necessary. By diving into teams and meeting with stakeholders in parallel, the team can understand each department's specific data requirements and, most importantly, identify commonalities that can serve as building blocks for a unified model and eliminate conflicting metrics. This approach helps gather detailed insights, builds trust, and encourages buy-in from the individual teams, making the process more collaborative and transparent.

Once the foundational elements are identified, the data team can develop a unified data model that serves the entire organization. This integrated model standardizes data definitions, establishes clear ownership, and sets protocols for data sharing, thereby breaking down the existing silos. By fostering a culture of data consumption and collaboration between teams, the unified model enhances consistency, reduces redundancies, and ultimately leads to more informed decision-making across the company.

Now that we have learned to address the important barriers that may hinder our progress, let's explore some targeted actions to maximize the chances of our organization successfully adopting data modeling.

Actions

When attempting to tackle a challenge as formidable as company-wide data modeling adoption, we must give ourselves every advantage possible. Having discussed the methods used to “sell” data modeling internally, let's review some additional actions that can give the data team a tactical advantage for this initiative.

Build momentum through quick wins

Adopting data modeling won't always coincide with a large-scale data migration or project where its merits take center stage. However, a data team can still deliver significant value by identifying the company's most pressing pain points and focusing on small, targeted data modeling exercises to address them. By pinpointing areas where data inconsistencies or misalignments hinder decision-making—such as conflicting metrics between departments, redundant customer records, or unused datasets—the team can design focused models that provide clarity and uniformity.

The key is to prioritize projects with high-impact potential, enabling the team to generate immediate improvements in data quality and operational efficiency. These quick wins serve as tangible proof of concept for the broader benefits of data modeling—building stakeholder confidence and support for future initiatives. As the data team collaborates closely with affected business units, they can showcase measurable improvements, such as faster report generation, enhanced data reliability, and reduced manual reconciliation efforts.

But how do we get the initial commitment from teams that are already busy pursuing other objectives? Sometimes, when the promise of data clarity isn't enough, a clever way to start the conversation is by asking to confirm something in their domain that you know to be incorrect or contentious. For example, asking to confirm that the formula for a certain metric is X when you know their version to be Y. Nothing ensures the participation of an expert than giving them a chance to correct the narrative. Once the conversation starts, let the business experts demonstrate the full extent of their domain knowledge.

Once you successfully engage several teams and assist them in addressing their critical data issues, identify influential and high-profile colleagues to act as your champions. They will likely be eager to share their success stories and showcase operational wins that reflect positively on their team. Leverage this enthusiasm when approaching new teams or proposing data modeling initiatives for future projects. Use the momentum generated from these quick wins and the lasting partnerships you've built to expand and unify the evolving company data model. Once you establish a proven formula, repeat the process until data modeling becomes part of the company culture, allowing everyone to enjoy the benefits of improved data quality and effective collaboration.

Another proven tactic is to tie data modeling to existing company objectives.

Link data modeling to business objectives

As described in previous sections, our main challenge is getting business stakeholders and company decision-makers to believe in the value of something they may not be familiar with. However, we can maximize our chances of success by relating this effort to something everyone in the company will recognize: stated business objectives.

At any given moment, an organization will likely promote operational initiatives such as cost optimization, data modernization, or customer 360 (master data consolidation). Further down the chain, individual departments will also have performance metrics such as response times and **Service Level Agreements (SLAs)**. The data team can tap into these initiatives by advocating the benefits of data modeling in relation to them. This approach mirrors the strategy outlined in the earlier section on “Demonstrating value with business metrics” but is applied on a targeted level. For example, for cost optimization, focus on the benefits of operational alignment, such as fewer and more productive meetings, fewer bugs, and less rework. If a picture is worth a thousand words, what better way to bring attention to duplicate data than presenting conflicting datasets in a diagram? Tying data modeling to business metrics is crucial because it directly links technical efforts to tangible business outcomes. By aligning data models with business metrics such as ROI, operational efficiency, and customer satisfaction, organizations can demonstrate a clear path to value and justify their investment in data initiatives.

There is, however, one initiative that deserves special attention: data literacy.

Data literacy discourages data siloes

In the age of AI and data-driven company culture, the biggest challenge for **Chief Data Officers (CDOs)** and **Chief Information Officers (CIOs)** isn't modernizing technology but modernizing employee thinking. In many data modernization efforts, the bottleneck is poor data literacy and its knock-on effects: **Shadow BI** and siloed data.



Shadow BI refers to the unofficial creation and use of reports, dashboards, or analyses by employees using unsanctioned tools, often Excel or other self-service software, outside of formal BI systems. This practice can lead to inconsistencies, data silos, and challenges in data governance, as it bypasses the organization's centralized data management processes.

Like all tech skills, data literacy is a spectrum that runs from basic filters in Excel to parsing data frames in Python. Within this spectrum, data modeling (e.g., learning to read an ER diagram, understanding the concept of a unique identifier) is one of the simplest skills to acquire. It is a gateway to more advanced concepts, enabling non-technical team members to operate and self-serve using existing data sources and facilitating higher-quality communication among departments and the data team.

The more comprehensive a data model becomes, the more it functions as a magnet for company-wide adoption, drawing more teams and departments into its sphere. Its adoption discourages the proliferation of Shadow BI by centralizing data governance and providing a reliable source of truth. Including metrics such as user adoption or department coverage can help track the effectiveness of the data modeling effort. With well-defined data models, teams can rely on consistent, sanctioned datasets rather than resorting to proprietary spreadsheets due to a lack of awareness. This reduces inconsistencies and errors and ensures that analytics and reporting align with organizational standards, leading to more reliable decision-making and efficient use of resources.

The data team plays a crucial role in helping the company achieve its data literacy goals by offering training and upskilling opportunities when they engage with business teams or through on-demand seminars. The same principles that contribute to the success of enterprise data modeling—accessibility, a single source of truth, and collaboration—should also apply to its adoption among business teams. It is essential to ensure that everyone in the company has access to the necessary software (e.g., data modeling, data governance, and data quality) with appropriate permission levels, allowing them to lead or contribute to the data modernization effort and discouraging siloed initiatives.

Now that we have our playbook for maximizing the chances of company-wide adoption of data modeling, let's review the salient points.

Summary

In this chapter, and throughout this book, we have emphasized that data modeling is a collaborative effort that benefits from the integration of diverse insights from both the business side and the data team. The true power of data modeling lies in its ability to serve as a shared language for the entire organization, facilitating better communication and more informed decision-making. But despite its potential, many enterprise data modeling initiatives falter, often stalling at the idea stage due to misconceptions and a lack of organizational buy-in.

The first myth that often needs to be debunked is that data modeling is a purely technical task, overlooking the value that communication and synergistic collaboration between business and data teams bring to the entire organization—often uncovering hidden technical debt that has laid dormant for years. Another misconception is that data modeling is a one-time effort or only necessary for large-scale projects. In reality, as organizations expand and their data environments become more complex, maintaining a continuously updated data model becomes ever more important, as it avoids inefficiencies, rework, and increased maintenance costs—issues that often become accepted as the “cost of doing business” when proper modeling is lacking.

Of course, real challenges exist when adopting a unified data modeling culture, particularly the issues stemming from decentralized data ownership and the emergence of shadow BI. In a federated environment, teams crave autonomy but often lack the modeling expertise of an experienced central governance team. To effectively address these challenges, it’s crucial to connect data modeling initiatives to measurable business outcomes, such as ROI, operational efficiency, and data modernization efforts, and demonstrate positive results. Doing so can help alleviate resistance from business stakeholders and C-suite executives.

By showcasing quick wins through targeted modeling exercises that tackle key pain points, organizations can build momentum and promote a trusted, centralized approach to data quality. This approach transforms fragmented and siloed data into a unified asset that supports strategic decision-making and fosters long-term collaboration.

I wish you much success on your data modeling journey!

Unlock this book's exclusive benefits now

Scan this QR code or go to packtpub.com/unlock, then search for this book by name.



Note: Keep your purchase invoice ready before you start.

20

Appendix

Wouldn't it be fitting if a book on modeling was itself modeled after something? Well, it has been! This book follows a spiral model, starting from a high-level overview of modeling and rising in winding loops, revisiting its core elements in ever-greater depth and complexity. This may seem like an unorthodox approach, but I believe that the difficulty of the technical *how* is alleviated by the understanding of the conceptual *why*. As such, I made it a point not to presuppose any modeling knowledge beyond a basic understanding of SQL and to never lean on a term or concept before formally introducing it.

Following a helical structure while writing this text allowed me to carve out a clear path from the theoretical to the technical and kept me from drifting into areas that were not core to the central theme. Inevitably, some exercises had to be excluded because they did not cleanly fit within the framework of the chapters. Rather than leave them on the proverbial cutting room floor, I have decided to include them as extras in the associated GitHub repository of this book.

While the book must be finalized for printing, I plan to continue to add interesting examples to the extras folder of the repo. The following is a summary of the types of content that can already be found there.

Technical requirements

The scripts used to instantiate and load the examples in this chapter are available in the following GitHub repo: <https://github.com/PacktPublishing/Data-Modeling-with-Snowflake-2E/tree/main/extras>.

The exceptional time traveler

What changed? This a question that's been asked since the first **data manipulation language (DML)** operations were performed on a database. While Snowflake's built-in table change tracking and streams can help answer this question, they are not enabled on tables by default. However, even in Snowflake's Standard Edition, all tables have a default time travel data retention period of one day.

The Time Travel feature can be combined with the **EXCEPT** set operator to isolate and compare any changes made. The exercise uses a randomly generated filter when selecting which records to update to make things interesting. The only way to solve the mystery is to use Snowflake's exceptional time-traveling powers.

The exercise is explained in the file titled `01_exceptional_time_travel.sql`.

The secret column type Snowflake refuses to document

Snowflake's co-founders and chief architects, Benoit Dageville and Thierry Cruanes, spent many years working at Oracle. In fact, Oracle's influence can be seen in many of the SQL constructs and functions that Snowflake supports. One such example is the concept of the virtual column.

Virtual columns straddle the line between physical and transformational modeling—between table and view. Virtual columns look like normal table columns, but their values are derived rather than stored on disc. They are an efficient way to embed simple business rules and transformational logic in a table without the overhead of maintaining views and incurring storage costs. Virtual columns can be defined through constants or transformational expressions such as the **DEFAULT** column operator. Strangely, they are not mentioned in the **CREATE TABLE** documentation at the time of writing (<https://docs.snowflake.com/en/sql-reference/sql/create-table>). However, references to Snowflake users taking advantage of them can be found in the community forums as far back as 2018 (<https://community.snowflake.com/s/question/0D50Z00008ixGQKSA2/does-snowflake-supports-computed-columns-in-a-table-while-creating-table>).

For an overview of the advantages of using virtual/computed columns in tables and how they differ from column defaults, see the example in the file titled `02_secret_virtual_columns.sql`.

Read the functional manual (RTFM)

Snowflake's technical documentation is among the clearest and most informative I have ever encountered (virtual columns aside). Paying attention to the usage notes and best practices or simply scrolling down the list of available functions helped me elevate my SQL game and discover new features when I first started, and it continues to pay dividends. One such discovery came—putting aside my pride—while reading the usage notes for `ORDER BY` (<https://docs.snowflake.com/en/sql-reference/constructs/order-by>). There, I learned about the `NULLS FIRST` and `NULLS LAST` keywords and how they override the default ordering of `NULL` when arranging in `ASC` or `DESC`. It's a feature that has come in handy many times since. Most importantly, it serves as a reminder to check the documentation periodically to help spot new features and functionality.

See the example in the file titled `03_order_by_nulls.sql`.

Summary

If you have made it this far, thank you. Truly. Writing this book could not have been possible without input from my peers throughout the data community and their thought-provoking modeling questions and suggestions. I hope this book and its further reading materials have given you a solid base from which to expand your modeling practice in Snowflake and beyond. If you have any questions, nifty tricks to include in the extras section, or just want to say “Hi,” find me on LinkedIn (<https://www.linkedin.com/in/serge-cloud-connected/>).

Here's to taking modeling from a *lost art* to an *essential skill* in the world of data together!



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



The Ultimate Guide to Snowpark

Shankar Narayanan SGS, Vivekanandan SS

ISBN: 978-1-80512-341-5

- Harness Snowpark with Python for diverse workloads
- Develop robust data pipelines with Snowpark using Python
- Deploy mature machine learning models
- Explore the process of developing, deploying, and monetizing native apps using Snowpark
- Deploy and operate containers in Snowpark
- Discover the pathway to adopting Snowpark effectively in production



Data Engineering with dbt

Roberto Zagni

ISBN: 978-1-80324-628-4

- Create a dbt Cloud account and understand the ELT workflow
- Combine Snowflake and dbt for building modern data engineering pipelines
- Use SQL to transform raw data into usable data, and test its accuracy
- Write dbt macros and use Jinja to apply software engineering principles
- Test data and transformations to ensure reliability and data quality
- Build a lightweight pragmatic data platform using proven patterns
- Write easy-to-maintain idempotent code using dbt materialization

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packt.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Data Modeling with Snowflake, Second Edition*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

A

- ACID properties** 107
- alternate key (AK)** 227
 - using, as unique constraints 94
- American National Standards Institute (ANSI)** 7, 169
- Apache Iceberg Open Table Format (OTF)** 106
- artificial intelligence (AI)** 372
- associative** 173
- Atomic Consistent Isolated Durable (ACID)** 106
- atomic grain** 156
- attributes** 25
 - adding 169, 170
- attributes, as columns** 84, 94
 - semi-structured data, storing 85, 86
 - Snowflake data types 84, 85
- automatic clustering** 83
- Avro** 60

B

- big data**
 - value 367
 - variety 367
 - velocity 367
 - veracity 367
 - volume 367
- BI tools**
 - functionality, automating 100

- Boyce-Codd Normal Form (BCNF)** 188, 192, 193
- bridge table** 341, 352
- Business as Usual (BAU)** 379
- Business Event Analysis and Modeling (BEAM)** 152
 - business process, engaging 152, 153
- business intelligence (BI)** 35, 100, 213, 378
- business key** 91
- Business Vault** 352
- bus matrix** 155

C

- capital expenditure (CapEx)** 40
- Cartesian product** 256
- change data capture (CDC)** 62
- Chief Data Officers (CDOs)** 387
- Chief Information Officers (CIOs)** 387
- cloud-native architecture**
 - backup and scalability considerations 223
- clustering**
 - automatic clustering 83, 84
 - statistics, storing 80
 - used, for data storage 80-83
- clustering key** 83
- column-level security**
 - reference link 222
- common table expressions (CTEs)** 258
- compound key (composite)** 88
- compute layer** 45

- conceptual context**
 - adding, to Snowflake architecture 143
- conceptual modeling** 24-28
- conformed attributes** 163
- conformed dimensions** 163, 209
- consolidated fact tables** 297
- constraints** 86
 - used, for join elimination 247-249
- Cortex Analyst** 231
- costs, considerations** 48
 - cache, used for saving cash 50
 - data storage, cost 48
 - data transfer costs 50
 - serverless compute cost 49
 - services layer 50-52
 - services layer compute cost 49
 - storage layer 52
 - virtual warehouse compute cost 49
- create, read, update, delete (CRUD)** 35
- CREATE TABLE AS SELECT (CTAS)** 34, 60, 114
- cross join** 256
- Crow's foot notation** 141, 142

D

- database modeling**
 - benefits 8-10
- database normalization**
 - anomalies 185
 - data models, on spectrum of 200, 201
 - overview 184, 185
 - through examples 188
- database normalization, anomalies**
 - deletion anomaly 187
 - domain anomaly 188
 - insertion anomaly 186, 187
 - update anomaly 186

- database normalization, with examples**
 - 1NF 189, 190
 - 2NF 190, 191
 - 3NF 191, 192
 - 6NF 198-200
 - BCNF 192, 193
 - domain-key normal form (DKNF) 197, 198
 - fifth normal form (5NF) 195, 196
 - fourth normal form (4NF) 193-195
- data classification**
 - reference link 222
- data definition language (DDL)** 32
- data egress** 50
- data ingress** 50
- data lake** 367, 368
- data lakehouse** 368
- data manipulation**
 - language (DML) 76, 215, 221, 392
- data marts** 365
 - modeling 364
- Data Mesh** 351, 369
 - business model 370
 - culture of data sharing, encouraging 371
 - data quality, emphasizing 371
 - governance guidelines, adopting 370, 371
 - principles 369
- data modeling**
 - actions 386-388
 - barriers 380-385
 - in modern era 372-374
 - misconceptions 378-380
- Data Vault** 351-354
 - hubs 355-358
 - links 358, 359
 - reference tables 361
 - satellites 359, 360
- Data Vault 2.0** 153, 352

- data warehouse** 365
 - versus transactional database 13
- data warehouses** 11, 121, 297
- Decision Support Systems (DSSs)** 11
- deletion anomaly** 187
- dependency** 185
- dimensional modeling (DM)** 151, 152
 - bus matrix, visualizing in
 - conceptual model 158-161
 - conceptual model, creating 153-155
- dimensional modeling (DM), conceptual model**
 - business process, defining 155, 156
 - dimensions, determining 157
 - facts, identifying 157, 158
 - grain, determining 156
- dimensions**
 - overview 266
- domain anomaly** 188
- domain-key normal form (DKNF)** 197, 198
- dynamic tables** 110
 - cost considerations 117
 - declarative statement 111
 - error handling 115-117
 - failed refreshes 115-117
 - imperative statement 111
 - monitoring 115
 - orchestrating, with precision 118-120
 - refresh types 114
 - target lag 113
 - tips and tricks 117
 - use cases 120
 - versus, materialized views 121
 - versus, streams and tasks 121
- dynamic Type 2 dimension**
 - creating 286-289

E

- edges** 341
- enforcement** 87
- Enterprise Edition**
 - features 222
- entities**
 - using, as tables 76
- entity** 24
- entity granularity** 25
- entity instances** 24
- entity-relationship diagram (ERD)** 10
 - creating, from physical model 229-231
 - versus relational model (RM) 135-138
- entity-relationship (ER)** 155
- event tables** 128
 - benefits 129, 130
 - costs 129, 130
- exceptional time traveler** 392
- Expected Benefit** 381
- Extensible Markup Language (XML)** 319
- external stages**
 - cloud storage services 56
- Extract, Transform, Load (ETL)** 14, 321
 - analytical modeling systems 14-16
- eye-dee-fix (IDEFIX) notation** 142, 143

F

- factless fact tables** 297
- fact table**
 - deleted records, recovering with leading load dates 310-313
 - leading method, for recovering deleted records 301-303
 - measures 298
 - Type 2 SCD structure 303-305

fact tables 155
 maintaining, with Snowflake features 306
 types 296, 297

Fail-safe 58

fifth normal form (5NF) 195, 196

file formats 57

first normal form (1NF) 184, 189, 190

fixed-depth hierarchy 338

FLATTEN function 326
 reference link 326

foreign key (FK) 95, 170
 benefits 98
 data, visualizing 98, 99
 joins, informing 99, 100
 relationship, using as 95-98

forward engineering 229

fourth normal form (4NF) 193-195

Full-time Employees (FTEs) 381

H

Hex Base64 58

hierarchies 338
 distinguishing between 338
 fixed-depth hierarchy 338
 ragged hierarchy 340-342
 slightly ragged hierarchy 339

hierarchies, maintaining in Snowflake 342
 changes, handling 346
 ragged hierarchy,
 navigating recursively 342-346

hubs 355-358

hybrid tables 47, 121
 beyond structured data 47
 constraints, enforcing 101
 costs 125, 126

 functionality 123
 indexes, working with 123-125
 limitations 127
 referential integrity 122
 versus, standard tables 127, 128
 working with 126, 127

hybrid transactional and analytical processing (HTAP) 47, 121

I

Iceberg tables 106
 managed approach 108, 109
 unmanaged approach 108, 109
 use cases 109, 110
 working 107

identifiers
 using, as primary keys 87-89

indexes
 secondary indexes 123

Information Engineering (IE) 22

inheritance 178-181

initial investment 381

insertion anomaly 186, 187

internal stages 56

J

JavaScript Object Notation (JSON) 319

Jevons' paradox 374

join 252-254

join elimination
 constraints, using 247-249
 RELY, usage considerations 249-252

join explosions 245

junction entity 173

L

Large Language Models (LLMs) 373

lateral join 326

links 358, 359

logical modeling 28-30

conceptual, expanding from 168

logical to physical model transformation 223

logical objects, physicalizing 223-225

tables, defining 226

M

managed schemas 211

Management Information Systems (MISs) 11

mandatory columns

using, as NOT NULL constraints 101, 102

many-to-many (M:M)

relationship 30, 31, 138, 144, 173-175

materialized views 66

metrics (numerical attributes) 89

micro-partitions 76

modeling

purpose 5, 6

modeling toolkit

natural language semantics 6

technical semantics 7

using 6

visual semantics 7

multi-table insert operator 361-364

multivalued dependency 194

N

naming conventions 204

case 204

object naming 206-209

suggested conventions 209, 210

natural language semantics 6

normal forms 184

O

Object Tagging

reference link 222

offset storage technique 67

OLTP database structure

versus OLAP database structure 211-215

One Big Table (OBT) approach 201

online analytical

processing (OLAP) 11, 35, 47, 121, 211

online transactional

processing (OLTP) 11, 35, 47, 121, 211

Open Table Format (OTF) 106

operational and analytical modeling scenarios 11, 12

operational expenses (OPEX) 303

opportunity cost 384

ORC formats 60

P

Parquet 58, 60

pay-as-you-go 48

permanent tables 60

physical model 32

deploying 229

ERD, creating from 229-231

semantic model, evolving from 231-233

physical model implementation, considerations

cost 221

data quality and integrity 221

data security 222

performance 220

physical modeling 32-34

physical tables 59, 60

Point-in-Time (PIT) 352

primary key (PK) 162, 170

benefits 89

business key 91

correct join results, ensuring 89, 90

duplicate values, avoiding 90

granularity, determining 89

identifiers, using as 87, 88

sequences 92-94

specifying 90, 91

surrogate key 92

taxonomy 91

Q

quality assurance (QA) 215

R

ragged hierarchy 340, 341

Raw Vault 352

building 354

read the functional manual (RTFM) 393

redundancy 185

reference tables 361

referential integrity (RI) 121

relational data

semi-structured data,
converting into 328-334

Relational Database Management System (RDBMS) 7

relational modeling

history 134, 135

versus entity-relationship

diagram (ERD) 135-138

relationship 25, 95

cementing 170-173

Crow's foot notation 141, 142

eye-dee-fix (IDEFIX) notation 142, 143

relationship granularity 26

RELY

usage, considerations for

join elimination 249, 252

Return on Investment (ROI) 377, 381

reverse balance fact table

building, with Streams 306-309

reverse engineering 32, 161, 229

business processes, validating 164

facts and dimensions, identifying 161, 162

relationships, identifying 162-164

row-level security

reference link 222

S

satellites 359, 360

SCDs, maintaining in Snowflake 277

stage, setting 277, 278

Type 1 279-281

Type 2 281-284

schema-on-read 8, 324-328

second normal form (2NF) 190, 191, 214

semantic model

building 234-239

evolving, from physical model 231-233

working with 233, 234

semi-structured data 321-323

benefits, in Snowflake 320, 321

converting, into relational data 328-334

storing 85, 86

- sequence** 92
- Service Level Agreements (SLAs)** 387
- services layer** 46
 - metadata cache 50
 - query results cache 51
 - warehouse cache 52
- set operator** 252-254
- Shadow BI** 388
- shared-disk architecture** 41
- shared-nothing architecture** 42, 43
- siloed data** 388
- Simple Storage Service (S3)** 221
- Single source of truth (SSOT)** 13
- sixth normal form (6NF)** 184, 198-200
- slightly ragged hierarchy**
 - examples 339, 340
- slowly changing dimension (SCD)** 297
 - example scenario 267
 - Type 0 267
 - Type 1 268
 - Type 2 268, 269
 - Type 3 269, 292
 - Type 4 270, 271
 - Type 5 271, 272
 - Type 6 273
 - Type 7 274, 275
 - types 266, 276
- snapshot fact tables**
 - accumulating 297
- Snowflake**
 - database environments 215, 216
 - database, organizing 210
 - databases and schemas, organizing 210
 - features 46
 - limitations 12
 - managed schemas 211
 - OLTP, versus OLAP database structures 211
 - reference link 4
 - solution 43
 - string, storing 85
 - three-tier architecture 43
 - used, for data storage 76-80
- Snowflake architecture**
 - many-to-many (M:M) relationship 144
 - subtypes 145, 146
 - supertypes 145, 146
 - used, for adding conceptual context 143
- Snowflake Data Cloud** 40
- Snowflake, features**
 - hybrid tables 47, 48
 - Time Travel 47
 - used, for maintaining fact tables 306
 - zero-copy cloning 46
- Snowflake, limitations**
 - operational modeling systems 13, 14
 - relational and transformational modeling 13
- snowflake schema** 365
 - versus star schema 366
- Snowflake, three-tier architecture**
 - compute layer 45
 - services layer 46
 - storage layer 45
- Snowflake views**
 - advantages 64
 - caching 65
 - change tracking 68-70
 - materialized views 66
 - security 65
 - streams 67
 - streams, loading from 67, 68
 - tasks 70, 71
 - tasks, combining with streams 72

Social Security number (SSN) 14
software as a service (SaaS) model 40
spilling 256
SqlDBM online database modeling tool
 reference link 4
SQL integrated development environments (IDEs) 32
stage metadata tables 62
 directory tables 63
 external tables 62
stages 56, 57
star schema 365
storage layer 45, 52
streams 67
 append/insert-only 68
 loading from 67
 standard 68
 used, for building reverse
 balance fact table 306-309
Structured Query Language (SQL) 4
subtypes 145, 146, 178
 binary properties 145
supertype entity 178
supertypes 145, 146
surrogate key 92
synchronized modeling
 benefit 147

T

tables 58, 59
 constraints, declaring 227
 defining 226
 naming 226
 permanent tables 60
 physical tables 59, 60
 properties 226

 stage metadata tables 62
 temporary tables 61
 transient tables 61
tag-based masking policies
 reference link 222
target lags, for dynamic tables
 specification options 113
 tips and tricks 113
tasks 70, 71
 combining, with streams 72
technical semantics 7
telemetry 128
temporary tables 60, 61
 table types, summary 62
terabytes (TB) 83
third normal form (3NF) 191, 192, 213
Time Travel 47
Total Cost of Ownership (TCO) 381
traditional architectures 40
 shared-disk architecture 41
 shared-nothing architecture 42, 43
 Snowflake's solution 43
transactional database
 versus data warehouse 13
transaction fact tables 296
transformational model
 best practices 259-263
 join elimination, with constraints 247-249
 joins and set operators 252-254
 separating, from object 244
 shaping, through relationships 244-247
transformational modeling 35, 36
transformational model, performance
 additional query considerations 257, 258
 common query problems 256, 257
 considerations and monitoring 255

transformational operations 60

transient tables 61

transitive functional dependency (TFD) 191

Type 2.1 dimension

- creating 289-291
- performance considerations 291

Type 2 fact table

- time intervals, handling 313-317

U

ubiquitous modeling 21-23

Unified Modeling Language (UML) 22

update anomaly 186

user acceptance testing (UAT) 215

UTF-8 58

V

variable spend 48

versatile transactional fact table 299-301

views 33

visual modeling conventions 138, 139

- conceptual context, adding to Snowflake architecture 143
- entities, depicting 139, 140
- relationships, depicting 141

visual semantics 7

W

weak entity 175-177

window function

- anatomy 285

Z

zero-copy cloning 46

